reply. However, the remaining $k+1$ will also produce the same answer, so the client or voter can just believe the majority.

Now suppose that in a *k*-fault tolerant group, a single process fails. The group as a whole is still living up to its specifications, namely that it can tolerate the failure of up to *k* of its members (of which one has just failed). But what happens if more than *k* members fail? In that case, all bets are off and whatever the group does, its results, if any, cannot be trusted. Another way of looking at this is that the process group, in its appearance of mimicking the behavior of a single, robust process, has failed.

8.2.3 Consensus in faulty systems with crash failures

As mentioned, in terms of clients and servers, we have adopted a model in which a potentially considerable collection of clients now send commands to a *group of processes* that jointly behave as a *single, highly robust process*. To make this work, we need to make an important assumption:

> *In a fault-tolerant process group, each nonfaulty process executes the same commands, in the same order, as every other nonfaulty process.*

Formally, this means that the group members need to reach **consensus** on which command to execute. If failures cannot happen, reaching consensus is easy. For example, we can use Lamport's totally ordered multicasting, as described in Section 5.2.1. Or, to keep it simple, using a centralized sequencer that hands out a sequence number to each command that needs to be executed will do the job as well. Unfortunately, life is not without failures, and reaching consensus among a group of processes under more realistic assumptions turns out to be tricky.

Flooding-based consensus

To illustrate the problem at hand, let us assume we have a group of processes $\mathbf{P} = \{P_1, \ldots, P_n\}$ operating under fail-stop failure semantics. In other words, we assume that crash failures can be reliably detected among the group members. Typically, a client contacts a group member requesting it to execute a command. Every group member maintains a list of proposed commands: some which it received directly from clients; others which it received from its fellow group members. We can reach consensus using the following approach, adopted from Cachin et al. [2011], and referred to as **flooding consensus**.

Conceptually, the algorithm operates in rounds. In each round, a process P_i sends its list of proposed commands it has seen so far to every other process in $\mathbf{P}$. At the end of a round, each process merges all received proposed commands into a new list, from which it then will deterministically select the command to execute, if possible. It is important to realize that the selection algorithm is the same for all processes. In other words, if all process have the

same list, they will all select the same command to execute (and remove that command from their list).

It is not difficult to see that this approach works as long as processes do not fail. Problems start when a process P_i detects, during round r, that, say, process P_k has crashed. To make this concrete, assume we have a process group of four processes $\{P_1, \ldots, P_4\}$ and that P_1 crashes during round r. Also, assume that P_2 receives the list of proposed commands from P_1 before it crashes, but that P_3 and P_4 do not (in other words, P_1 crashes before it got a chance to send its list to P_3 and P_4). This situation is sketched in Figure 8.5.

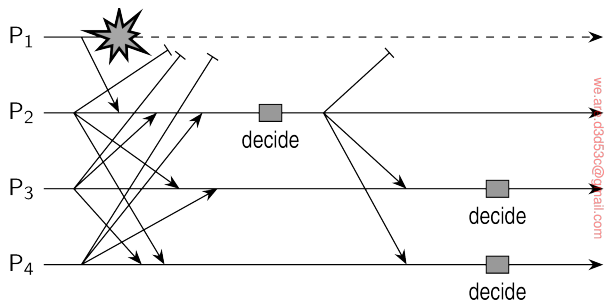


Figure 8.5: Reaching consensus through flooding in the presence of crash failures. Adopted from Cachin et al. [2011].

Assuming that all processes knew who was a group member at the beginning of round r, P_2 is ready to decide on which command to execute when it receives the respective lists of the other members: it has all commands proposed so far. Not so for P_3 and P_4. For example, P_3 may detect that P_1 crashed, but it does not know if either P_2 or P_4 had already received P_1's list. From P_3's perspective, if there is another process that did receive P_1's proposed commands, that process may then make a different decision than itself. As a consequence, the best that P_3 can do is postpone its decision until the next round. The same holds for P_4 in this example. A process will decide to move to a next round when it has received a message from every nonfaulty process. This assumes that each process can reliably detect the crashing of another process, for otherwise it would not be able to decide who the nonfaulty processes are.

Because process P_2 received all commands, it can indeed decide and can subsequently broadcast that decision to the others. Then, during the next round $r + 1$, processes P_3 and P_4 will also be able to decide: they will decide to execute the same command selected by P_2.

To understand why this algorithm is correct, it is important to realize that a process will move to a next round without having made a decision, only when it detects that another process has failed. In the end, this means that in the worst case at most one nonfaulty process remains, and this process can

simply decide whatever proposed command to execute. Again, note that we are assuming reliable failure detection.

But then, what happens when the decision by process P_2 that it sent to P_3 was lost? In that case, P_3 can still not make a decision. Worse, we need to make sure that it makes the same decision as P_2 and P_4. If P_2 did not crash, we can assume that a retransmission of its decision will save the day. If P_2 did crash, this will also be detected by P_4 who will then subsequently rebroadcast its decision. Meanwhile, P_3 has moved to a next round, and after receiving the decision by P_4, will terminate its execution of the algorithm.

A more realistic approach: Raft

Let us now take a look at a consensus protocol that operates under crash-failure semantics, or actually, a fail-noisy failure model: a process will *eventually* correctly conclude that another process has crashed. **Raft**, described by Ongaro and Ousterhout [2014] was developed in reaction to the inherent intricacies of a famous consensus protocol, Paxos (which we discuss further below). In this section, we will stick to the essence of Raft, thereby even simplifying a few matters to focus entirely on understanding how and why it works.

In Raft, we typically have a group of some five replicated servers. We assume the set of servers is fixed (although Raft allows servers to join and leave the group). Each server maintains a **log** of operations, some of which have already been executed (i.e., committed), as well as pending operations. Consensus is expressed in terms of these logs: committed operations have the same position in each of the respective server's logs. One of the servers operates as a **leader** and decides on the order in which pending operations are to be committed. In essence, Raft is a primary-backup protocol, with the primary acting as leader and the backups as **followers**. We discussed Raft's leader-election algorithm in Section 5.4.4.

A client always sends an operation request to the leader (possibly after having been redirected by one of the followers). That means that the leader is fully aware of all pending requests. Each client request for executing an operation o is appended to the leader's log, in the form of a tuple $\langle o, t, k\rangle$ in which t is the **term** under which the current leader serves, and k the index of o in the leader's log. To recall, after electing a *next* leader, the term for new operations will be $t+1$. Let c be the index of the most recently committed operation. Raft guarantees that operations that have been registered as committed, have been performed by a majority of the servers, and that the result has been returned to the original client.

Assume the leader has a log of length n, is operating in term t, and receives a request for executing operation o. In that case, it appends $\langle o, t, n+1\rangle$ and *conceptually* sends its entire log to all the other servers, along with the current value of c. Again conceptually, each following server copies the entire log, and

returns an acknowledgement to the leader, while ensuring that all operations up to and including index c have been executed. Note that operation o cannot yet be committed by a follower.

As soon as the leader receives a majority of acknowledgements, it executes o, returns the result (if any, but at least an acknowledgement) to the client, and sets c to $n+1$. The next time it communicates with the other servers, it also sends c so that each of them can commit operations, in particular, o. It is important to note that in this scheme, the leader is indeed in full control: its state (as expressed by its log), is to be seen as the collective state of the server group. Of course, in reality, the leader will send only the tuple $\langle o, t, n+1 \rangle$, where it will be appended to the local log of the respective followers. In this sense, the protocol essentially follows the one described in Section 7.5.1 (except that clients always send a request to the primary, and not through a follower). More important, however, is that by stating that the leader, conceptually speaking, sends the entire log, it overwrites the log of its followers. If the leader receives no acknowledgement from one of its followers, it will simply repeat sending the log until acknowledged.

Suppose the leader crashes. In that case, a new one is elected as described in Section 5.4.4, and this new leader's log is the collective state of the server group. There is only one problem: if that log misses committed operations, then it is not representative for what the majority of servers has decided on. For this reason, during an election, a server S will *not* vote for a candidate server S' if it turns out that S's log is more up to date, than that of S'. Oversimplifying matters somewhat, a log is more up to date when it contains more committed operations (meaning that S' has missed several commits already done by a majority of other servers), or when it contains operations from more recent terms (meaning that S' has missed elections). Assuming that the new leader will have executed all the committed operations, its state will therefore be the same as that of the previous leader before it crashed. However, do note that the new leader may not necessarily have received all pending requests. From a consensus point of view, this is not a problem. To avoid missing any client requests, Raft does assume that when a client does not receive a response from the server, it will resubmit its original request.

What if the leader crashed after executing o^1 (because there was a majority of servers), yet did not have a chance to inform (all of) the other servers? First, note that the new leader will have a log that is at least a subset of the logs of a majority of the remaining servers, in terms of committed operations (otherwise it would have never been elected as new leader). When handling a new operation o^2, broadcasting its log to the other servers, and receiving an acknowledgement from a majority, it can not only commit o^2, but now also o^1. This situation is shown in Figure 8.6. First, note that server S_5 will be brought up to date about operation o^1 as soon as the new leader tells about o^2. After receiving enough acknowledgements, the leader can go ahead and commit o^1

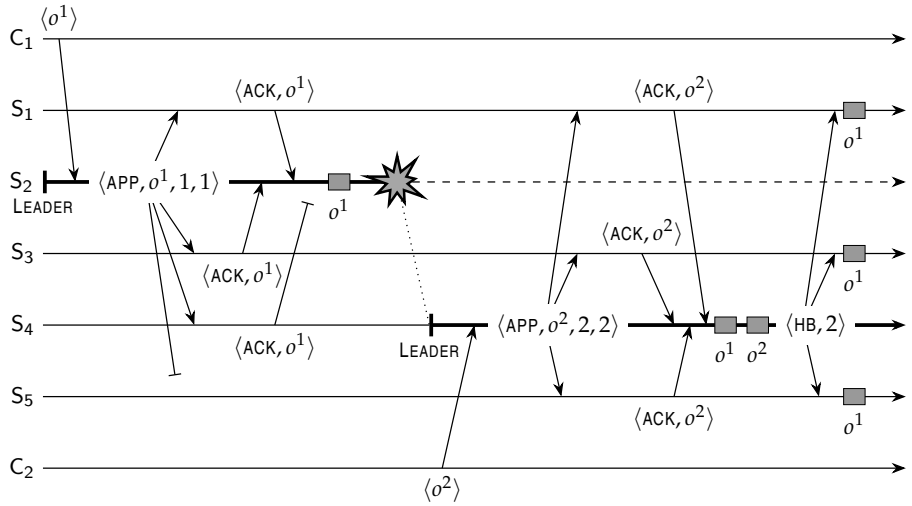


Figure 8.6: The situation when a leader crashes after executing an operation but before being able to tell other servers that the operation has been committed.

(as well as o^2). All other servers can commit o^1 as soon as they know that the leader did so as well.

We have skipped a number of details, notably details related to the fact that the leader does not send entire logs, but only operation requests. For one, this means that after an election, servers may be missing operations, or could have extraneous operations, or both. Bringing the logs of followers into a consistent state with the new leader requires some attention, which can be ignored by assuming that entire logs are sent. These, and other details, are found in Ongaro and Ousterhout [2014] and Ongaro [2014].

8.2.4 Example: Paxos

Discussing consensus in the presence of crash failures cannot be decently done without discussing **Paxos**. It was originally published in 1989 as a technical report by Leslie Lamport, but it took about a decade before someone decided that it may not be such a bad idea to disseminate it through a regular scientific channel [Lamport, 1998]. The original publication is not easy to understand, exemplified by other publications that aim at explaining it [Lampson, 1996; Prisco et al., 1997; Lamport, 2001; van Renesse and Altinbuken, 2015]. In fact, it is generally agreed that Paxos is not only quite difficult to explain and understand, but that these intricacies have led to implementations that actually differ from the original protocol. These intricacies formed an important motivation for developing Raft, which we explained above.

Essential Paxos

The assumptions under which Paxos operates are rather weak:

- The distributed system is partially synchronous (in fact, it may even be asynchronous).
- Communication between processes may be unreliable, meaning that messages may be lost, duplicated, or reordered.
- Messages that are corrupted can be detected as such (and thus subsequently ignored).
- All operations are deterministic: once an execution is started, it is known exactly what it will do.
- Processes may exhibit crash failures, but not arbitrary failures, nor do processes collude.

By-and-large, these are realistic assumptions for many practical distributed systems.

We first roughly follow the explanation given by Lamport [2001] and Kirsch and Amir [2008]. The algorithm operates as a network of *logical* processes, of which there are different types. First, there are **clients** that request a specific operation to be executed. At the server side, each client is represented by a single **proposer**, which attempts to have a client's request accepted. Normally, a single proposer has been designated as being the **leader**, and drives the protocol toward reaching consensus.

What we need to establish is that a proposed operation is accepted by an **acceptor**. If a majority of acceptors accepts the same proposal, the proposal is said to be *chosen*. However, what is chosen still needs to be *learned*. To this end, we will have a number of **learner** processes, each of which will execute a chosen proposal once it has been informed by a majority of acceptors.

It is important to note that a single proposer, acceptor, and learner form a single *physical* process, running on a single machine that the client communicates with, as shown in Figure 8.7. We thus assume that if, for example, a proposer crashes, then the physical process that it is part of will have crashed. By replicating this server, we aim at obtaining fault tolerance in the presence of crash failures.

The basic model is that the leading proposer receives requests from clients, one at a time. A nonleading proposer forwards any client request to the leader. The leading proposer sends its proposal to all acceptors, telling each to accept the requested operation. Each acceptor will subsequently broadcast a learn message. If a learner receives the same learn message from a majority of acceptors, it knows that consensus has been reached on which operation to execute, and will execute it.

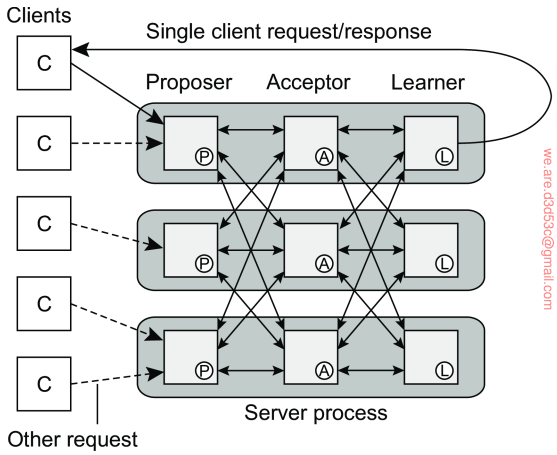


Figure 8.7: The organization of Paxos into different logical processes.

There are at least two specific issues that need further attention. First, not only do the servers need to reach consensus on which operation to execute, we also need to make sure that each of them actually executes it. In other words, how do we know for sure that a majority of the nonfaulty servers will carry out the operation? There is essentially only one way out: have learn messages be retransmitted. However, to make this work, an acceptor will have to log its decisions (in turn requiring a mechanism for purging logs). Because we are assuming globally ordered proposal timestamps (as explained shortly), missing messages can be easily detected, and also accepted operations will always be executed in the same order by all learners.

As a general rule, the server hosting the leading proposer will also inform the client when its requested operation has been executed. If another process had taken over the lead, then it will also handle the response to the client.

This brings us to the second important issue: a failing leader. Life would be easy if the failure of a leader would be reliably detected, after which a new leader would be elected, and later, the recovering leader would instantly notice that the world around it had changed. Unfortunately, life is not so easy. Paxos has been designed to tolerate proposers who still believe they are in the lead. The effect is that proposals may be sent out concurrently by different proposers (each believing to be the leader). We therefore need to make sure that these proposals can be distinguished from one another to ensure that the acceptors handle only the proposals from the current leader.

Note that relying on a leading proposer implies that any practical implementation of Paxos will need to be accompanied by a leader-election algorithm. In principle, that algorithm can operate independently of Paxos, but is normally part of it.

To distinguish concurrent proposals from different proposers, each proposal p has a uniquely associated (logical) timestamp $ts(\mathsf{p})$. How uniqueness is achieved is left to an implementation, but we will describe some of the details shortly. Let $oper(\mathsf{p})$ denote the operation associated with proposal p. The trick is to allow multiple proposals to be accepted, but that each of these accepted proposals has the same associated operation. This can be achieved by guaranteeing that if a proposal p is chosen, then any proposal with a higher timestamp will also have the same associated operation. In other words, we require that

$$\mathsf{p} \text{ is chosen } \Rightarrow \text{ for all } \mathsf{p}' \text{ with } ts(\mathsf{p}') > ts(\mathsf{p}) : oper(\mathsf{p}') = oper(\mathsf{p})$$

Of course, for p to be chosen, it needs to be accepted. That means that we can guarantee our requirement when guaranteeing that if p is chosen, then any higher-timestamped proposal accepted by any acceptor, has the same associated operation as p. However, this is not sufficient, for suppose that at a certain moment, a proposer simply sends a new proposal p', with the highest timestamp so far, to an acceptor A that had not received any proposal before. Note that this may indeed happen according to our assumptions concerning message loss and multiple proposers, each believing to be in the lead. In absence of any other proposals, A will simply accept p'. To prevent this situation from happening, we thus need to guarantee that

> *If proposal* p *is chosen, then any higher-timestamped proposal issued by a proposer, has the same associated operation as* p.

When explaining the Paxos algorithm below, we will indeed see that a proposer may need to adopt an operation coming from acceptors in favor of its own. This will happen after a leading proposer had failed, but its proposed operation had already made it to a majority of the acceptors.

The processes collectively formally ensure *safety*, in the sense that only proposed operations will be learned, and that at most one operation will be learned at a time. In general, a **safety property** asserts that nothing bad will happen. Furthermore, Paxos ensures *conditional liveness* in the sense that if enough processes remain up-and-running, then a proposed operation will eventually be learned (and thus executed). **Liveness**, which tells us that eventually something good will happen, is not guaranteed in Paxos, unless some adaptations are made. We return to liveness in Note 8.4.

There are now two phases, each, in turn, consisting of two subphases. During the first phase, the leading proposer interacts with acceptors to get a requested operation accepted for execution. The first phase is needed to rule out any trouble caused by different proposers, each believing they are the leader. The best that can happen is that an individual acceptor promises to consider the proposer's operation and ignore other requests. The worst is that the proposer was too late and that it will be asked to adopt some other

proposer's request instead. Apparently, a leadership change had taken place and there may be former requests that need to be handled first.

In the second phase, the acceptors will have informed proposers about the promises they have made. The leading proposer essentially takes up a slightly different role by promoting a single operation to the one to be executed, and subsequently telling the acceptors.

Phase 1a (prepare): The goal of this phase is that a proposer P who believes it is the leader and is proposing operation o, tries to get its proposal timestamp *anchored*, in the sense that any lower timestamp failed, or that o had also been previously proposed (i.e., with some lower proposal timestamp). To this end, P broadcasts its proposal to the acceptors. For the operation o, the proposer selects a proposal number m higher than any of its previously selected numbers. This leads to a **proposal timestamp** $t = (m, i)$ where i is the (numerical) process identifier of P. Note that

$$(m, i) < (n, j) \Leftrightarrow (m < n) \text{ or } (m = n \text{ and } i < j)$$

This timestamp for a proposal p is an implementation of the previously mentioned timestamp $ts(\mathsf{p})$. Proposer P sends PREPARE(t) to all acceptors (but note that messages may be lost). In doing so, it is (1) asking the acceptors to promise not to accept any proposals with a lower proposal timestamp, and (2) to inform it about an accepted proposal, if any, with the highest timestamp less than t. If such a proposal exists, the proposer will adopt it.

Phase 1b (promise): An acceptor A may receive multiple proposals. Assume it receives PREPARE(t) from P. There are three cases to consider:

(1) t is the highest proposal timestamp received from any proposer so far. In that case, A will return PROMISE(t) to P stating that A will ignore any future proposals with a lower timestamp.

(2) If t is the highest timestamp so far, but another proposal (t', o') had already been accepted, A also returns (t', o') to P. This will allow P to decide on the final operation that needs to be accepted.

(3) In all other cases, do nothing: there is apparently another proposal with a higher timestamp that is being processed.

Once the first phase has been completed, the leading proposer P knows what the acceptors have promised. Essentially, the leading proposer knows that all acceptors have agreed on the same operation. This will put P into a position to tell the acceptors that they can go ahead. This is needed because although the leading proposer knows on which operation consensus has been reached, this consensus is not known to the others. Again, we assume that P received

a response from a majority of acceptors (whose respective responses may be different).

Phase 2a (accept): There are two cases to consider:

(1) If P does not receive any accepted operation from any of the acceptors, it will forward its own proposal for acceptance by sending ACCEPT(t, o) to all acceptors.

(2) Otherwise, it was informed about another operation o', which it will adopt and forward for acceptance. It does so by sending ACCEPT(t, o'), where t is P's proposal timestamp and o' is the operation with proposal timestamp highest among all accepted operations that were returned by the acceptors in Phase 1b.

Phase 2b (learn): Finally, if an acceptor receives ACCEPT(t, o'), but did not previously send a promise with a higher proposal timestamp, it will accept operation o', and tell all learners to execute o' by sending LEARN(o'). At that point, the acceptor can forget about o'. A learner L receiving LEARN(o') from a majority of acceptors, will execute the operation o'. We now also know that a majority of learners share the same idea on which operation to execute.

It is important to realize that this description of Paxos indeed captures only its essence: using a leading proposer to drive the acceptors toward the execution of the same operation. When it comes to practical implementations, much more needs to be done (and more than we are willing to describe here). An excellent description of what it means to realize Paxos has been written by Kirsch and Amir [2008]. Another write-up on its practical implications can be found in [Chandra et al., 2007].

Understanding Paxos

To properly understand Paxos, but also many other consensus algorithms, it is useful to see how its design could have evolved. We say "could have," as the evolution of the algorithm has never been documented. The following description is largely based on work described by Meling and Jehl [2013][1]. As our starting point, we consider a server that we wish to make more robust. By now, we know that this can be achieved through replication and making sure that all commands submitted by clients are executed by all servers in the same order. The simplest situation is to add one server, thus creating a group of two processes, say S_1 and S_2. Also, to make sure that all commands are executed in the same order, we appoint one process to be a sequencer, which increments and associates a unique timestamp with every submitted command. Servers

[1] Special credits go to Hein Meling for helping us better understand what Paxos is all about.

are required to execute commands according to their timestamp. In Paxos, such a server is referred to as the **leader**. We can also consider it to be a **primary server**, with the other acting as a **backup server**.

We assume that a client broadcasts its requested command to all servers. If a server notices it is missing a command, it can rely on the other server to forward it when necessary. We will not describe how this happens, but silently assume that all commands are stored at the servers and that we merely need to make sure that the servers agree on which command to execute next. As a consequence, all remaining communication between servers consists of control messages. To make this point clear, consider the situation sketched in Figure 8.8. (In what follows, we use subscripts to designate processes, and superscripts to designate operations and states.)

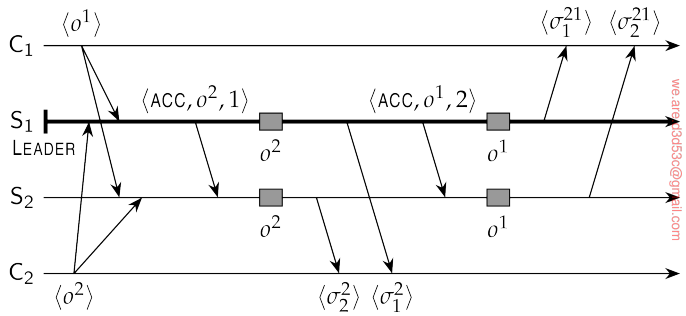


Figure 8.8: Two clients communicating with a 2-server process group.

In this example, server S_1 is the leader and as such will hand out timestamps to submitted requests. Client C_1 has submitted command o^1 while C_2 submitted o^2. S_1 instructs S_2 to execute operation o^2 with timestamp 1, and later operation o^1 with timestamp 2. After processing a command, a server will return the result to the associated client. We designate this using the notation $\langle \sigma_i^j \rangle$, where i is the index of the reporting server, and j the state it was in, expressed as the sequence of operations it has carried out. In our example, client C_1 will thus see the results $\langle \sigma_1^{21} \rangle$ and $\langle \sigma_2^{21} \rangle$, meaning that each server has executed o^1 after executing o^2.

In Paxos, when a leader associates a timestamp with an operation, it does so by sending an **accept message** to the other server(s). As we assume that messages may be lost, a server accepting an operation o does so by telling the leader it has **learned** the operation by returning a LEARN(o) message. When the leader does not notice that operation o has been learned, it simply retransmits an ACCEPT(o, t) message, with t being the original timestamp. Note that in our description, we are skipping the phase of coming to agreement on the operation to be carried out: we assume the leader has decided and now needs to reach consensus on executing that operation.

Compensating for a lost message is relatively easy, but what happens when also a server crashes? Let us first assume that a crash can be reliably detected. Consider the situation sketched in Figure 8.9(a). The issue, of course, is that server S_2 will never have learned (about) operation o^1. This situation can be prevented by demanding that a server may execute an operation only if it knows that the other server has learned the operation as well, as illustrated in Figure 8.9(b).

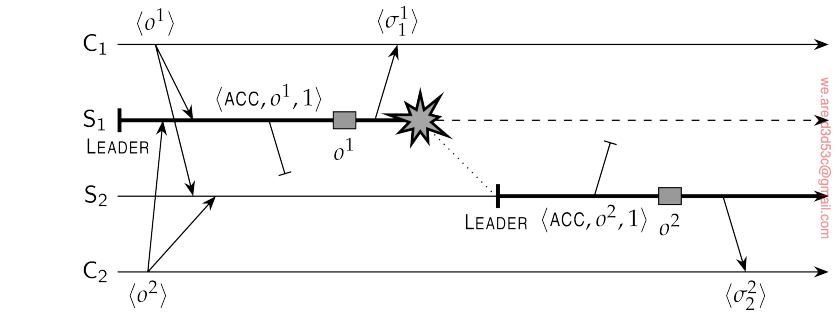


Figure 8.9: (a) What may happen when the leader crashes in combination with a lost accept, and (b) the solution, namely demanding that the other server has learned the operation as well before executing it.

It is not difficult to see that with a larger process group, we can get into the same situation as in Figure 8.9(a). Simply consider a group of three servers $\{S_1, S_2, S_3\}$ with S_1 being the leader. If its ACCEPT(o^1, t) message to S_3 is lost, yet it knows that S_2 has learned o^1, then it should still not execute o^1 until it has also received a LEARN(o^1) message from S_3. This situation is shown in Figure 8.10.

Let us consider the three-server case and imagine what would happen if LEARN(o^1) returned by S_2 would not make it to S_1. Of course, S_1 would

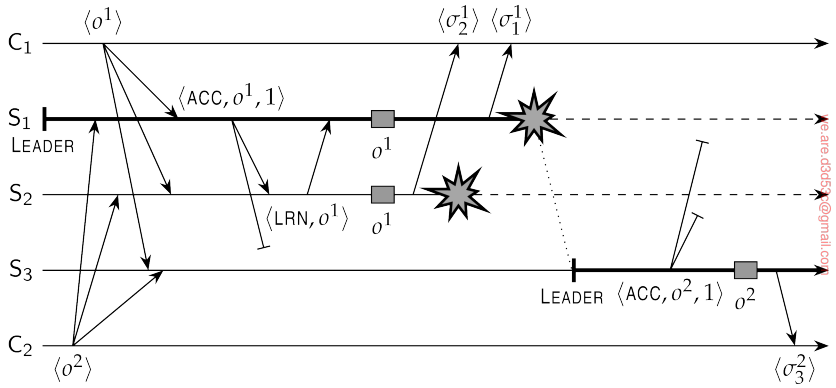


Figure 8.10: The situation when dealing with three servers, of which two crash. In this case, S_1 should have waited with executing operation o^1 until it had received LEARN(o^1) from S_3, but also S_2 should wait until it knows that S_3 has learned about o^1.

not execute o^1, but otherwise we would still be in trouble: S_2 will execute o^1 while S_3 would take over leadership and execute o^2 without ever knowing that o^1 had already been processed. In other words, also S_2 must wait with the execution of o^1 until it knows that S_3 has learned that operation. This brings us to the following:

> *In Paxos, a server* S *cannot execute an operation* o *until it has received a* LEARN(o) *from all other nonfaulty servers.*

Up to this point, we have assumed that a process can reliably detect that another process has crashed. In practice, this is not the case. As we will discuss more extensively shortly, a standard approach toward failure detection is to set a timeout on expected messages. For example, each server is required to send a message declaring it is still alive, and at the same time the other servers set timeouts on the expected receipt of such messages. If a timeout expires, the sender is suspected to have failed. In a partially synchronous or fully asynchronous system, there is essentially no other solution. However, the consequence is that a failure may be falsely detected, as the delivery of such "I'm alive" messages may have simply been delayed or lost.

Let us assume that Paxos has realized a failure detection mechanism, but that the two servers falsely conclude that the other has failed, as shown in Figure 8.11. The problem is clear: each may now independently decide to execute their operation of choice, leading to divergent behavior. It is at this point that we need to introduce an extra server, and demand that a server can execute an operation only if it is certain that a *majority* will execute that operation. Note, that in the three-server case, execution of operation o by

server S can take place as soon as S has received at least one (other) LEARN(o) message. Together with the sender of that message, S will form a majority.

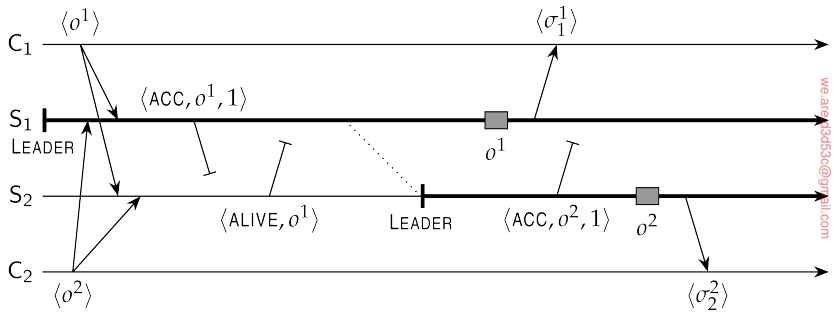


Figure 8.11: The situation in the case of false failure detections.

We have now come to a point where it should be clear that Paxos requires at least three replicated servers $\{S_1, S_2, S_3\}$ to operate correctly. Let us concentrate on the situation that one of these servers crashes. We make the following assumptions.

- Initially, S_1 is the leader.
- A server can reliably detect it has missed a message. The latter can be realized, for example, through timestamps in the form of strictly increasing sequence numbers. Whenever a server notices it has missed a message, it can then simply request a retransmission and catch up before continuing.
- When a new leader needs to be elected, the remaining servers follow a strictly deterministic algorithm. For example, we can safely assume that if S_1 crashes, then S_2 will become leader. Likewise, if S_2 crashes, S_3 will take over, and so on.
- Clients may receive duplicate responses, but besides being required to recognize duplicates, form no further part of the Paxos protocol. In other words, a client cannot be asked to help the servers to resolve a situation.

Under these circumstances, no matter when one of S_2 or S_3 crashes, Paxos will behave correctly. Of course, we are still demanding that execution of an operation can take place only if a server knows that a majority will execute that operation.

Suppose now that S_1 in its role as leader, crashes after the execution of operation o^1. The worst that can happen in this case is that S_3 is completely ignorant of the situation until the new leader, S_2 tells it to accept operation o^2. Note that this is announced through an ACCEPT$(o^2, 2)$ message such that the

timestamp $t = 2$ will alert S_3 that it missed a previous accept message. S_3 will tell so to S_2, who can then retransmit ACCEPT(o^1, 1), allowing S_3 to catch up.

Likewise, if S_2 missed ACCEPT(o^1, 1), but did detect that S_1 crashed, it will eventually either send ACCEPT(o^1, 1) or ACCEPT(o^2, 1) to S_3 (i.e., in both cases using timestamp $t = 1$, which was previously used by S_1). Again, S_3 has enough information to get S_2 on the right track again. If S_2 had sent ACCEPT(o^1, 1), S_3 can simply tell S_2 that it already learned o^1. In the other case, when S_2 sends ACCEPT(o^2, 1), S_3 will inform S_2 that it apparently missed operation o^1. We conclude that when S_1 crashes after executing an operation, Paxos behaves correctly.

So, what can happen if S_1 crashes immediately after having sent ACCEPT(o^1, 1) to the other two servers? Suppose again that S_3 is completely ignorant of the situation because messages are lost, until S_2 has taken over leadership and announces that o^2 should be accepted. Like before, S_3 can tell S_2 that it (i.e., S_3) missed operation o^1, so that S_2 can help S_3 to catch up. If S_2 misses messages, but does detect that S_1 crashed, then as soon as it takes over leadership and proposes an operation, it will be using a stale timestamp. This will trigger S_3 to inform S_2 that it missed operation o^1, which saves the day. Again, Paxos is seen to behave correctly.

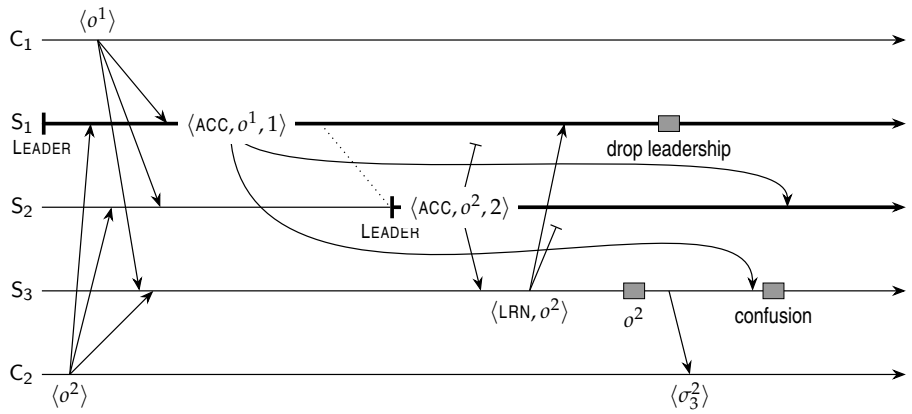


Figure 8.12: Why incorporating the ID of the current leader is needed: S_2 falsely concludes that S_1 has crashed.

Problems may arise with false detections of crashes. Consider the situation sketched in Figure 8.12. We see that the accept messages from S_1 are considerably delayed and that S_2 falsely detects S_1 having crashed. S_2 takes over leadership and sends ACCEPT(o^2, 1), i.e., with a timestamp $t = 1$. However, when finally ACCEPT(o^1, 1) arrives, S_3 cannot do anything: this is not a message it is expecting. Note that, in principle, S_3 does not check who is the current leader. The only thing it knows is that it is itself *not* the leader.

Precisely for this reason, it does not expect an ACCEPT message with the same timestamp as before. Things change, however, if it does know who the current leader is, in combination with a deterministic leader election. In that case, it could safely reject ACCEPT(o^1, 1), knowing that by now S_2 has taken over, and even perhaps retransmit LEARN(o^2) to S_1. A (perhaps previously sent) LEARN(o^2) message received by S_1 will allow S_1 to conclude that leadership had been taken over. We conclude that the leader should include its ID in an accept message.

We have covered almost all cases and have thus far shown that Paxos behaves correctly. Unfortunately, although being correct, the algorithm can still come to a grinding halt. Consider the situation illustrated in Figure 8.13. What we are seeing here is that because the LEARN messages returned by S_3 are lost, neither S_1 nor S_2 will ever be able to know what S_3 actually executed: did it learn (and execute) ACCEPT(o^1, 1) before or after learning ACCEPT(o^2, 1), or perhaps it learned neither operation? A solution to this problem is discussed in Note 8.4.

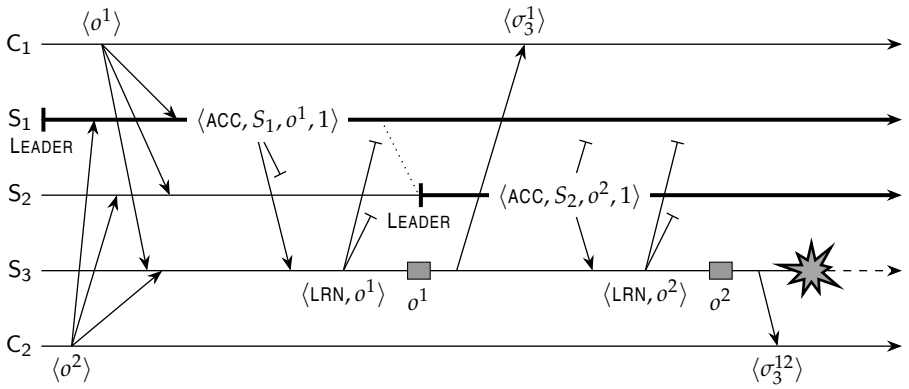


Figure 8.13: When Paxos can make no further progress.

Note 8.4 (Advanced: Making progress in Paxos)
Up to this point, we have discussed the development of Paxos such that **safety** is ensured. Safety essentially means that nothing bad will happen, or, put differently, that the behavior of the algorithm is correct. To also ensure that eventually something good will happen, generally referred to as **liveness** of an algorithm, we need to do a bit more. In particular, we need to get out of the situation sketched in Figure 8.13.

The real problem with this situation is that the servers have no consensus on whom the leader is. Once S_2 decides it should take over leadership, it needs to ensure that any outstanding operations initiated by S_1 have been properly dealt with. In other words, it has to ensure that its own leadership is not hindered

by operations that have not yet been completed by all nonfaulty processes. If leadership is taken over too quickly and a new operation is proposed, a previous operation that has been executed by at least one server may not get a chance to be executed by all servers first.

To this end, Paxos enforces an explicit leadership takeover, and this is where the role of **proposers** come from. When a server crashes, the next one in line will need to take over (recall that Paxos assumes a deterministic leader-election algorithm), but also ensure that any outstanding operations are dealt with. This explicit takeover is implemented by broadcasting a **proposal message**: PROPOSE(S_i), where S_i is the next server to be leader. When server S_j receives this message, it replies with a PROMISE(o^j, t_j) message, containing the most recently *executed* operation o^j and its corresponding timestamp t_j. Note that S_i is particularly interested in the most recent operation o^* executed by a majority of servers. By "adopting" this operation from the (apparently crashed) server S^* that had originally proposed its acceptance, S_i can effectively complete what S^* could not due to its failure.

There are two obvious optimizations to this procedure. The first one is not that servers return the most recently executed operation, but the most recently *learned* operation that is still waiting to be executed, if any. Furthermore, because it may be that the collection of servers has no more pending operations, S_i can also suggest a next operation o^i when initially proposing to take over leadership, giving rise to a PROPOSE(S_i, o^i) message. In essence, this is the situation we described earlier, yet now it should be clear where the idea of proposals actually comes from.

When S_i receives a majority of promises for operation o^*, and the highest returned timestamp is t^*, it broadcasts ACCEPT(S_i, o^*, t^*), which is essentially a retransmission of the last operation proposed before S_i took over leadership. If no such o^* exists, S_i will propose to accept its own operation o^i.

8.2.5 Consensus in faulty systems with arbitrary failures

So far, we assumed that replicas were subject to only crash failures, in which case a process group needs to consist of $2k + 1$ servers to survive k crashed members. An important assumption in these cases, is that a process does not collude with another process, or, more specifically, is consistent in its messages to others. The situations shown in Figure 8.14 should not happen. In the first case, we see that process P_2 is forwarding a different value or operation than it is supposed to. Referring to Paxos, this could mean that a primary tells the backups that not operation o had been accepted, but instead propagates a different operation o'. In the second case, P_1 is telling different things to different processes, such as having a leader sending operation o to some backups, and at the same time operation o' to others. Again, we note that this need not be malicious actions, but simply omission or commission failures.

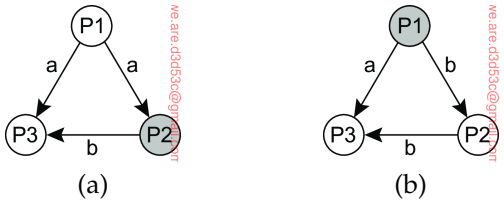


Figure 8.14: A process in a replicated group acting inconsistently: (a) not forwarding properly, and (b) telling different things to different processes.

In this section, we take a look at reaching consensus in a fault-tolerant process group in which k members can fail, assuming arbitrary failures. In particular, we will show that we need at least $3k + 1$ members to reach consensus under these failure assumptions.

Consider a process group consisting of n members, of which one has been designated to be the *primary*, P, and the remaining $n - 1$ to be the *backups* $\mathsf{B}_1, \ldots, \mathsf{B}_{n-1}$. We make the following assumptions:

- A client sends a value $\mathsf{v} \in \{\mathsf{T}, \mathsf{F}\}$ to the primary, where v stands for either *true* or *false*.
- Messages may be lost, but this can be detected.
- Messages cannot be corrupted without that being detected (and thus subsequently ignored).
- A receiver of a message can reliably detect its sender.

To achieve what is known as **Byzantine agreement**, we need to satisfy the following two requirements:

BA1: Every nonfaulty backup process stores the same value.

BA2: If the primary is nonfaulty, then every nonfaulty backup process stores exactly what the primary had sent.

Note that if the primary is faulty, BA1 tells us that the backups may store the same, but different (and thus wrong) value than the one initially sent by the client. Furthermore, it should be clear that if the primary is not faulty, satisfying BA2 implies that BA1 is also satisfied.

Why having 3k processes is not enough

To see why having only $3k$ processes is not enough to reach consensus, let us consider the situation in which we want to tolerate the failure of a single process, that is, $k = 1$. Consider Figure 8.15, which is essentially an extension of Figure 8.14.

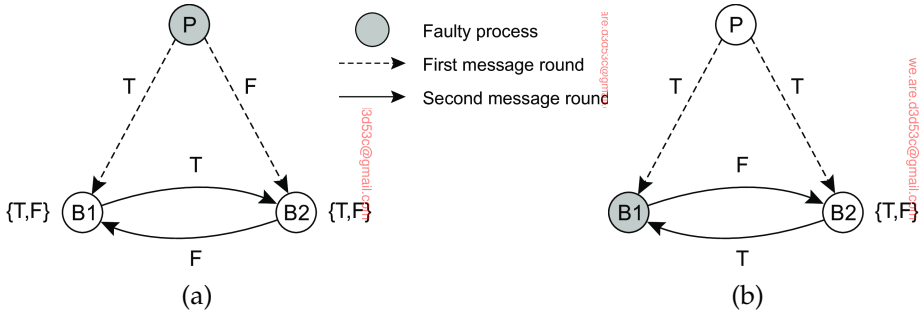


Figure 8.15: Impossibility to reach consensus with 3 processes and trying to tolerate a single arbitrarily failing process.

In Figure 8.15(a), we see that the faulty primary P is sending two different values to the backups B_1 and B_2, respectively. To reach consensus, both backup processes forward the received value to the other, leading to a second round of message exchanges. At that point, B_1 and B_2 each have received the set of values $\{T, F\}$, from which it is impossible to draw a conclusion.

Likewise, we cannot reach consensus when wrong values are forwarded. In Figure 8.15(b), the primary P and backup B_2 operate correctly, but B_1 is not. Instead of forwarding the value T to process B_2, it sends the incorrect value F. The result is that B_2 will now have seen the set of values $\{T, F\}$ from which it cannot draw any conclusions. In other words, P and B_2 cannot reach consensus. More specifically, B_2 can not decide what to store so that we cannot satisfy requirement BA2.

Note 8.5 (Advanced: The case where $k > 1$ and $n \leq 3k$)
Generalizing this situation to other values of k is not that difficult. As explained by Kshemkalyani and Singhal [2008], we can use a simple reduction scheme. Assume that there is a solution for the case where $k \geq 1$ and $n \leq 3k$. Partition the n processes $Q_1, \ldots, Q_n$ into three disjoint sets $\mathbf{S_1}$, $\mathbf{S_2}$, and $\mathbf{S_3}$, together containing all processes. Moreover, let each set $\mathbf{S_k}$ have less or equal than $n/3$ members. Formally, this means that

- $\mathbf{S_1} \cap \mathbf{S_2} = \mathbf{S_1} \cap \mathbf{S_3} = \mathbf{S_2} \cap \mathbf{S_3} = \emptyset$
- $\mathbf{S_1} \cup \mathbf{S_2} \cup \mathbf{S_3} = \{Q_1, \ldots, Q_n\}$
- for each $\mathbf{S_i}$, $|\mathbf{S_i}| \leq n/3$

Now consider a situation in which three processes Q_1^*, Q_2^*, and Q_3^* *simulate* the actions that take place in and between the processes of $\mathbf{S_1}$, $\mathbf{S_2}$, and $\mathbf{S_3}$, respectively. In other words, if a process in $\mathbf{S_1}$ sends a message to another process in $\mathbf{S_2}$, then Q_1^* will send a same message to Q_2. The same holds for process communication within a group. Assume that Q_1^* is faulty, yet Q_2^* and Q_3^* are not. All processes simulated by Q_1^* are now assumed to be faulty, and will thus lead to incorrect

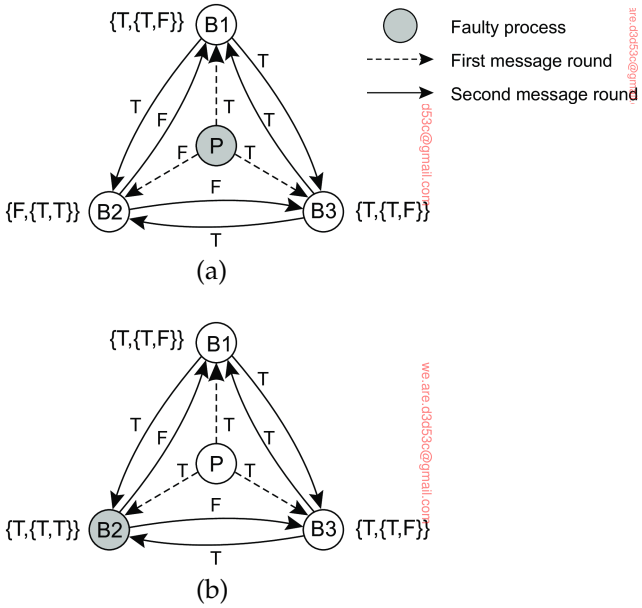


Figure 8.16: Reaching consensus with four processes, of which one may fail arbitrarily.

messages being sent to Q_2^* and Q_3^*, respectively. Not so for Q_2^* (and Q_3^*): all messages coming from processes in $\mathbf{S_2}$ (and $\mathbf{S_3}$, respectively) are assumed to be correct. Because $n \leq 3k$ and for each set $\mathbf{S_i}$ we have that $|\mathbf{S_i}| \leq n/3$, at most $n/3$ of the simulated processes $Q_1, \ldots, Q_n$ are faulty. In other words, we are satisfying the condition for which we assumed that there would be a general solution.

We can now come to a contradiction, for if there would exist a solution for the general case, then the processes Q_1^*, Q_2^*, and Q_3^* could simulate this solution, which would then also be a solution for the special case that $n = 3$ and $k = 1$. Yet, we just proved that this cannot be so, leading to a contradiction. We conclude that our assumption that there is a general solution for $k \geq 1$ and $n \leq 3k$ is false.

Why having 3k + 1 processes is enough

Let us now focus on the case in which we have a group of $3k + 1$ processes. Our goal is to show that we can establish a solution in which k group members may suffer from fail-arbitrary failures, yet the remaining nonfaulty processes will still reach consensus. Again, we first concentrate on the case $n = 4$, $k = 1$. Consider Figure 8.16, which shows a situation with one primary P and three backup processes B_1, B_2, and B_3.

In Figure 8.16(a) we have sketched the situation in which the primary P is

faulty and is providing inconsistent information to its backups. In our solution, the processes will forward what they receive to the others. During the first round, P sends T to B_1, F to B_2, and T to B_3, respectively. Each of the backups then sends what they have to the others. With only the primary failing, this means that after two rounds, each of the backups will have received the set of values $\{T, T, F\}$, meaning that they can reach consensus on the value T.

When we consider the case that one of the backups fails, we get the situation sketched in Figure 8.16(b). Assume that the (nonfaulty) primary sends T to all the backups, yet B_2 is faulty. Where B_1 and B_3 will send out T to the other backups in a second round, the worst that B_2 may do is send out F, as shown in the figure. Despite this failure, B_1 and B_3 will come to the same conclusion, namely that P had sent out T, thereby meeting our requirement BA2 as stated before.

Note 8.6 (Advanced: The case where $k > 1$ and $n = 3k + 1$)
As a sketch toward a general solution, consider the more intricate case in which $n = 7$ and $k = 2$. We let the primary P send out a value v_0. Using a similar notation as found in [Kshemkalyani and Singhal, 2008], we proceed as follows. Note that we effectively use the index 0 to denote the primary P (think of P being equal to a special backup process B_0).

1. We let P send v_0 to the six backups. Backup B_i stores the received value as $v_{i,0}\langle\,\rangle$. This notation indicates that the value was received by process B_i, that it was sent by $P = B_0$, and that the value of v_0 was directly sent to B_i and not through another process (using the notation $\langle\,\rangle$). So, for example, B_4 will store v_0 in $v_{4,0}\langle\,\rangle$.
2. Each backup B_i, in turn, will send $v_{i,0}\langle\,\rangle$ to every one of the other five backups, which is stored by B_j as $v_{j,i}\langle 0\rangle$. This notation indicates that the value is stored at B_j, was sent by B_i, but that it originated from $P = B_0$ (through the notation $\langle 0\rangle$). Looking at B_4 again, it will receive values from B_1, B_2, B_3, B_5, and B_6. For example, B_2 will send $v_{2,0}\langle\,\rangle$, which is then stored as $v_{4,2}\langle 0\rangle$ by B_4. Likewise, B_4 will also store $v_{4,1}\langle 0\rangle$, $v_{4,3}\langle 0\rangle$, $v_{4,5}\langle 0\rangle$, and $v_{4,6}\langle 0\rangle$.
3. Suppose that B_i now has the value $v_{i,j}\langle 0\rangle$. Again, it will send out this value to all processes except $P = B_0$, B_j, and B_i (i.e., itself). If B_k receives $v_{i,j}\langle 0\rangle$ from B_i, it stores this received value in $v_{k,i}\langle j, 0\rangle$. Indeed, by then v_0 will have traveled the path $P \to B_j \to B_i \to B_k$. For example, in the previous round, B_2 will have stored $v_{2,1}\langle 0\rangle$, which it eventually sends to B_4, who, in turn, will store it as $v_{4,2}\langle 1, 0\rangle$. Note at this point, that B_4 can send out *this* value only to processes B_3, B_5, and B_6. There is no use in sending it out to other processes.
4. Continuing this line of thought, assume that B_i has value $v_{i,j}\langle k, 0\rangle$, which it sends out to the three remaining processes not equal to $P = B_0$, B_k, B_j, and B_i (itself). Returning to B_4, eventually, B_4 will also receive a similar message from, say, B_3, such as $v_{3,2}\langle 5, 0\rangle$: the value v_0 initially sent to B_5,

which then forwarded it to B_2, who, in turn, sent it to B_3. B_4 will store that value as $v_{4,3}\langle 2,5,0\rangle$. That value can be sent only to B_1 and B_6, after which only a single round is left.

Once all these messages have been sent, each backup can start moving out of the recursion again. Note that in the above scheme we have effectively constructed a tree, rooted at value v_0 with six child nodes (representing the six backups). Each of these nodes, in turn, will have five children of its own, and so on. In total, there will be $6! = 720$ leaf nodes. Each path from the root to a leaf node is encoded by the value stored at that leaf node, such as $v_{4,1}\langle 6,5,3,2,0\rangle$. This leaf node will have a single parent (namely B_1 storing $v_{1,6}\langle 5,3,2,0\rangle$), in turn with a single sibling (B_4 storing $v_{4,6}\langle 5,3,2,0\rangle$).

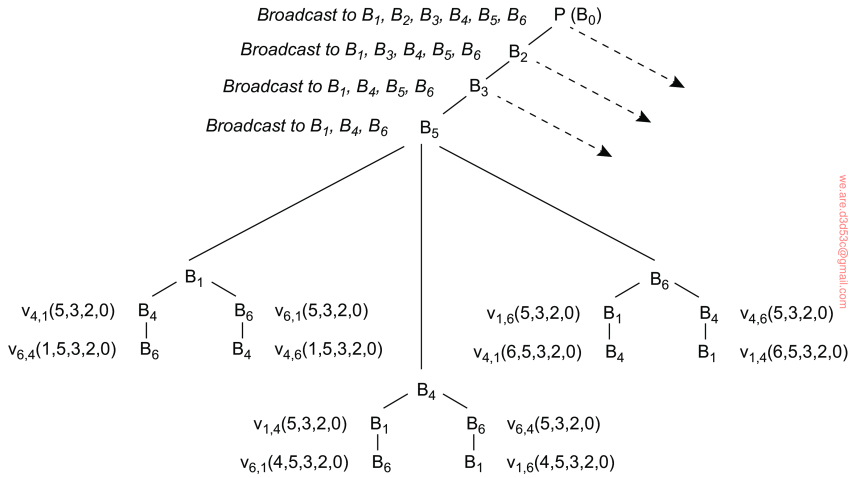


Figure 8.17: Rounds of broadcasting v_0 to the backups, by P and the backups $B_1, \ldots, B_6$.

We can now let each backup start computing *estimates* of v_0, that is, a value that it believes v_0 should be. To this end, we assume that each (nonfaulty) process executes the same procedure *majority*() that selects a unique value from a given set of inputs. In practice, this will be the majority among the input set. If there is no majority, a default value is chosen. To give a few examples, also shown in Figure 8.17:

$$
\begin{aligned}
w_{4,1}\langle 5,3,2,0\rangle &\leftarrow \mathit{majority}(v_{4,1}\langle 5,3,2,0\rangle, v_{6,4}\langle 1,5,3,2,0\rangle)\\
w_{6,1}\langle 5,3,2,0\rangle &\leftarrow \mathit{majority}(v_{6,1}\langle 5,3,2,0\rangle, v_{4,6}\langle 1,5,3,2,0\rangle)\\
w_{1,4}\langle 5,3,2,0\rangle &\leftarrow \mathit{majority}(v_{1,4}\langle 5,3,2,0\rangle, v_{6,1}\langle 4,5,3,2,0\rangle)\\
w_{6,4}\langle 5,3,2,0\rangle &\leftarrow \mathit{majority}(v_{6,4}\langle 5,3,2,0\rangle, v_{1,6}\langle 4,5,3,2,0\rangle)\\
w_{1,6}\langle 5,3,2,0\rangle &\leftarrow \mathit{majority}(v_{1,6}\langle 5,3,2,0\rangle, v_{4,1}\langle 6,5,3,2,0\rangle)\\
w_{4,6}\langle 5,3,2,0\rangle &\leftarrow \mathit{majority}(v_{4,6}\langle 5,3,2,0\rangle, v_{1,4}\langle 6,5,3,2,0\rangle)
\end{aligned}
$$

In turn, B_1, B_4, and B_6 can compute estimates like:

$$\begin{array}{lcl} w_{1,5}\langle 3,2,0\rangle & \leftarrow & \mathit{majority}(v_{1,5}\langle 3,2,0\rangle, w_{4,1}\langle 5,3,2,0\rangle, w_{6,1}\langle 5,3,2,0\rangle \\ w_{4,5}\langle 3,2,0\rangle & \leftarrow & \mathit{majority}(v_{4,5}\langle 3,2,0\rangle, w_{1,4}\langle 5,3,2,0\rangle, w_{6,4}\langle 5,3,2,0\rangle \\ w_{6,5}\langle 3,2,0\rangle & \leftarrow & \mathit{majority}(v_{6,5}\langle 3,2,0\rangle, w_{1,6}\langle 5,3,2,0\rangle, w_{4,6}\langle 5,3,2,0\rangle \end{array}$$

And from there, B_5 can continue, for example, with:

$$w_{5,3}\langle 2,0\rangle \leftarrow \mathit{majority}(v_{5,3}\langle 2,0\rangle, w_{1,5}\langle 3,2,0\rangle, w_{4,5}\langle 3,2,0\rangle, w_{6,5}\langle 3,2,0\rangle)$$

This process continues until, eventually, B_2 from Figure 8.17 will be able to execute

$$w_{2,0}\langle\,\rangle \leftarrow \mathit{majority}(v_{2,0}\langle\,\rangle, w_{1,2}\langle 0\rangle, w_{3,2}\langle 0\rangle, w_{4,2}\langle 0\rangle, w_{5,2}\langle 0\rangle, w_{6,2}\langle 0\rangle)$$

and reach the final outcome.

Let us now see why this scheme actually works. We denote by **BAP(n,k)** the above sketched protocol to reach consensus. **BAP(n,k)** starts by having the primary send out its value v_0 to the $n-1$ backups. In the case the primary operates correctly, each of the backups will indeed receive v_0. If the primary is faulty, some backups receive v_0 while others receive $\overline{v_0}$ (i.e., the opposite of v_0).

Because we assume a backup B_i cannot know whether the primary is working correctly, it will have to check with the other backups. We therefore let B_i run the protocol again, but in this case, with value $v_{i,0}\langle\,\rangle$ and with a smaller process group, namely $\{B_1, \ldots, B_{i-1}, B_{i+1}, \ldots, B_n\}$. In other words, B_i executes **BAP(n-1,k-1)** with a total of $n-2$ other processes. Note that at this point there are $n-1$ instances of **BAP(n-1,k-1)** being executed in parallel.

In the end, we see that these executions result in each backup B_i taking the majority of $n-1$ values:

- One value comes from the primary: $v_{i,0}\langle\,\rangle$
- $n-2$ values come from the other backups, in particular, B_i is dealing with the values $v_{i,1}\langle\,\rangle, \ldots, v_{i,i-1}\langle\,\rangle, v_{i,i+1}\langle\,\rangle, \ldots, v_{i,n-1}\langle\,\rangle$.

However, because B_i cannot trust a received value $v_{i,j}\langle\,\rangle$, it will have to check that value with the other $n-2$ backups: $B_1, \ldots, B_{i-1}, B_{i+1}, \ldots, B_{j-1}, B_{j+1}, \ldots, B_{n-1}$. This leads to the execution of **BAP(n-2,k-2)**, of which a total of $n-2$ instances will be running in parallel. This story continues, and eventually, a backup process will need to run **BAP(n-k,0)**, which simply returns the value sent by the primary, after which we can move up the recursion as described above.

With this general scheme, we can now see why the protocol is correct. Following Koren and Krishna [2007], we use induction on k to prove that **BAP(n,k)** meets the requirements BA1 and BA2 for $n \geq 3k+1$ and for all $k \geq 0$.

First, consider the case $k = 0$. In other words, we assume that there are no faulty processes. In that case, whatever the primary sends to the backups, that value will be consistently propagated throughout the system, and no other value will ever pop up. In other words, for any n, **BAP(n,0)** is correct. Now consider the case $k > 0$.

First, consider the case that the primary is operating correctly. Without loss of generality, we can assume the primary sends out T. All the backups receive the

same value, namely T. Each backup will then run **BAP(n-1,k-1)**. By induction, we know that each of these instances will be executed correctly. This means that for any nonfaulty backup B, all the other nonfaulty backups will store the value that was sent by B, namely T. Each nonfaulty backup receives, in total, $n-1$ values, of which $n-2$ come from other backups. Of those $n-2$, at most k values may be wrong (i.e., F). With $k \leq (n-1)/3$, this means that every nonfaulty backup receives at least $1+(n-2)-(n-1)/3 = (2n-2)/3$ values T. Because $(2n-2)/3 > n/3$ for all $n > 2$, this means that every nonfaulty backup can take a correct majority vote on the total number of received values, thus satisfying requirement BA2.

Let us now consider the case that the primary is faulty, meaning that at most $k-1$ backups may operate incorrectly as well. The primary is assumed to send out any value it likes. There are a total of $n-1$ backups, of which at most $k-1$ are faulty. Each backup runs **BAP(n-1,k-1)** and by, induction, each one of these instances is executed correctly. In particular, for every nonfaulty backup B, all the other nonfaulty backups will vote for the value sent by B. This means that all nonfaulty backups will have the same vector of $n-2$ results from their fellow backups. Any difference between two nonfaulty backups can be caused only by the fact that the primary sent something else to each of them. As a result, when applying *majority*() to those complete vectors, the result for each backup will be the same, so that requirement BA1 is met.

Example: Practical Byzantine Fault Tolerance

Byzantine fault tolerance was for long more or less an exotic topic, partly because it turned out that combining safety, liveness, *and* practical performance was difficult to achieve. It was around 2000 that Barbara Liskov and Miguel Castro managed to come up with a practical implementation of a protocol for replicating servers that could handle arbitrary failures. Let us briefly take a look at their solution, which has been coined **Practical Byzantine Fault Tolerance**, or simply **PBFT** [Castro and Liskov, 2002]. To better understand how the protocol works, it may help to also consult Liskov [2010].

Like Paxos, PBFT makes only a few assumptions about its environment. It makes no assumptions about the behavior of replica servers: a faulty server is assumed to exhibit arbitrary behavior. Likewise, messages may be lost, delayed, and received out of order. However, a message's sender is assumed to be identifiable (which is achieved by having messages signed, as we discuss in Section 9.2.3). Under these assumptions, and as long as no more than k servers fail, it can be proven that PBFT is *safe*, meaning that a client will always receive a correct answer. If we can additionally assume synchrony, meaning that message delays and response times are bounded, it also provides *liveness*. In practice, this means that PBFT assumes a partially synchronous model, in which unbounded delays are an exception, for example, caused by an attack.

To understand the algorithm, let us take a step back and partly review what we have discussed so far on establishing a k-fault-tolerant process group. An essential issue is that such a group behaves as a single, central server. As a consequence, under the assumption of having only crash failures, when a client sends a request, it should expect $k+1$ identical answers. If a server had crashed, fewer responses would be returned, but they would be the same.

The first problem that we need to solve is that concurrent requests are all handled in the same order. To this end, PBFT adopts a primary-backup model with a total of $3k+1$ replica servers. To keep matters simple, let us assume for now that the primary is nonfaulty. In that case, a client C sends a request to execute operation o to the primary (denoted as P in Figure 8.18). The primary has a notion of the current collection of nonfaulty replica servers, expressed in terms of a **view** v, which is simply a number. The primary assigns a timestamp t to o, which is then incremented to be used for a subsequent request. The primary subsequently sends a (signed) *pre-prepare* message PRE-PREPARE$(t, v,$ o) to the backups (denoted as B_i in Figure 8.18). We assume that backup B_2 is faulty, indicated by the dashed line used for its messages.

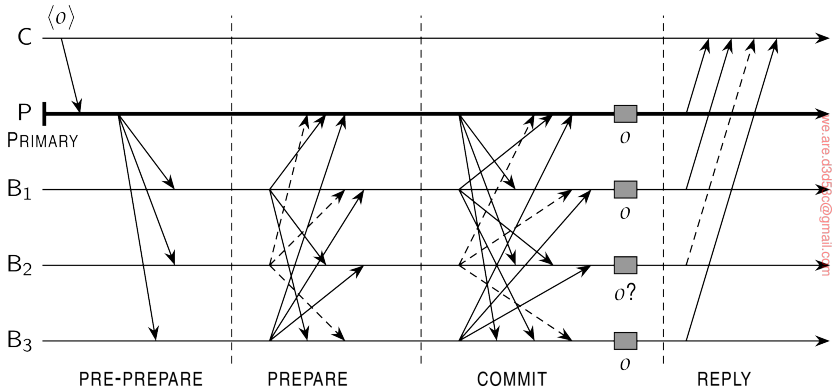


Figure 8.18: The different phases in PBFT. C is the client, P is the primary, and B_1, B_2, B_3 are the backups. We assume that B_2 is faulty.

A (nonfaulty) backup will accept to *pre-prepare* if it is in v and has never accepted an operation with timestamp t in v before. Each backup that accepts to *pre-prepare* sends an (again signed) message PREPARE$(t, v,$ o) to the others, including the primary. A key observation is that when a nonfaulty replica server S has logged $2k$ messages PREPARE$(t, v,$ o) (including its own) that all match the *pre-prepare* message S itself received by the primary (i.e., all have the same value for t, v, and o, respectively), there is consensus among the nonfaulty servers on the order of which operation goes first. To see why, let a **prepare certificate** PC$(t, v,$ o) denote a certificate that is based on such a set of $2k+1$ messages. Let PC$(t, v,$ o$')$ be another prepare certificate with

the same values for t and v respectively, but with a different operation o'. Because each prepare certificate is based on $2k+1$ values from a total of $3k+1$ replica servers, the intersection of two certificates will necessarily be based on messages from a subset of at least $k+1$ servers. Of this subset, we know that there is at least one nonfaulty server, which will have sent the same *prepare* message. Hence, $o = o'$.

In Figure 8.18, notice that, regardless what B_2 sends (or does not send), the two other nonfaulty backups, namely B_1 and B_2, will each have logged three ($2k+1$, with $k=1$) messages for the tuple $\langle t, v, o \rangle$: the original message from P, their own message, and the same message from the other backup. At that point, there is consensus to execute o, and not some other operation. We can then move on to the next phase.

The next phase for a replica server starts when it has a prepare certificate: it *commits* to the operation by broadcasting COMMIT(t, v, o) to the other members in v. Each server S, in turn, collects $2k$ of such commit messages from other servers, leading to a **commit certificate** to execute operation o. At that point, it executes o and sends a response to the client. Again, with its own message and the $2k$ other messages, S knows that there is consensus among the nonfaulty servers on which operation to actually execute *now*. In Figure 8.18, we see that for the primary, as well as the backups B_1 and B_3, there are indeed again three messages that state the same: we can execute o. Each server knows that there are at least two other servers besides itself that will indeed execute o, and it is therefore good to go. What B_2 had sent is not relevant: it will not affect the decision of the others.

The client collects all the results and takes as the answer the response that is returned by at least $k+1$ replicas, of which it knows that there is at least one nonfaulty replica server contributing to that answer. Again notice that in Figure 8.18, regardless what B_2 tells the client about which operation B_2 had executed, the client can safely assume that its requested operation was executed by a majority of the servers.

So far, so good. However, we also need to deal with the situation that the primary fails. If a backup detects that the primary fails, it broadcasts a *view-change* message for view $v+1$. What we wish to establish is that a request that was still being processed at the time the primary failed, will eventually get executed *once and only once* by all nonfaulty servers. To this end, we first need to ensure that there are no two commit certificates with the same timestamp that have different associated operations, regardless the view that each of them is associated with. This situation can be prevented by having a quorum of $2k+1$ commit certificates just as before, but this time based on prepare certificates. In other words, we want to regenerate commit certificates, but now for the new view, and only to make sure that a nonfaulty server is not missing any operation. In this respect, note that we may be generating a certificate for an operation that a server S had already executed (which can be

observed by looking at timestamps), but that certificate will be ignored by S as long as it keeps an account of its execution history.

A backup server will broadcast a (signed) message VIEW-CHANGE$(v+1, \mathbf{P})$, with **P** being the set of its prepare certificates. (Note that we ignore garbage collecting issues.) PBFT includes a deterministic function primary(w) known to all backups that returns who the next primary will be given a view w. This new primary will wait until it has a total of $2k+1$ *view-change* messages, leading to a **view-change certificate X** of prepare certificates. The new primary then broadcasts NEW-VIEW$(v+1, \mathbf{X}, \mathbf{O})$, where **O** consists of *new* pre-prepare messages constructed according to one of the following situations:

- PRE-PREPARE$(t, v+1, o) \in \mathbf{O}$ if the prepare certificate $PC(t, v', o) \in \mathbf{X}$ such that there is no prepare certificate $PC(t, v'', o')$ with $v'' > v'$,
- PRE-PREPARE$(t, v+1, \mathit{none}) \in \mathbf{O}$ if there is no prepare certificate $PC(t, v', o') \in \mathbf{X}$.

What happens is that any outstanding, pre-prepared operation from a previous view is moved to the new view, but considering only the most recent view that led to the installment of the current new view. In this sense, the new primary effectively sends out appropriate new pre-prepare messages, based on what the other backups had already committed to in their prepare phase. Simplifying matters a bit, each backup will check **O** and **X** to make sure that all operations are indeed authentic and broadcast *prepare* messages for all *pre-prepare* messages in **O**. We are then back into the situation shown in Figure 8.18, but now with one of the backups operating as primary, all the other original backups operating as nonfaulty servers, and the old primary operating as a faulty backup.

We have skipped many elements of PBFT that deal with its correctness and above all its efficiency. For example, we did not touch upon garbage collecting logs or efficient ways of authenticating messages. Such details can be found in [Castro and Liskov, 2002]. A description of a wrapper that will allow the incorporation of Byzantine fault tolerance with legacy applications is described in [Castro et al., 2003]. Notably the performance of Byzantine fault tolerance has been subject to much research, leading to many new protocols (see, for example, Zyzzyva [Kotla et al., 2009] and Abstract [Guerraoui et al., 2010]), yet even these new proposals often rely on the original PBFT implementation. That there is still room for improvement when actually using PBFT for developing robust applications is discussed by Chondros et al. [2012]. For example, PBFT assumes static membership (i.e., clients and servers are known to each other in advance), but also assumes that a replica server's memory acts as a stable, persistent storage.

These and other shortcomings along with the inherent complexity of Byzantine fault tolerance have formed a hurdle for widespread use of PBFT. Interestingly, attention for PBFT returned in the advent of **permissioned**

blockhains, being one of the few alternatives that could offer a consenus protocol in a public network. New research was initiated to address shortcomings, notably the relatively small number of participants that PBFT can maximally handle. An important development has been HotStuff [Yin et al., 2019], which effectively provides a much more efficient view change than in PBFT, significantly contributing to the scalability of the protocol.

8.2.6 Consensus in blockchain systems

Coming to this point, let us return to the much debated blockchain systems. In particular, let us zoom in a bit into what consensus entails in these systems. Following the approach presented by Xiao et al. [2020], we can distinguish four requirements for blockchain consensus:

1. **Agreement**: All nonfaulty nodes agree on the acceptance of a block of transactions and its position in the blockchain, or agree that it should be discarded.
2. **Integrity**: Each nonfaulty node sees the same blocks of accepted transactions, and the same positions of these blocks in the blockchain.
3. **Termination**: Every nonfaulty node either discards or accepts a transaction, as contained within a block, to be part of the blockchain.
4. **Validity**: If every node receives the same validated block, it should be accepted for the blockchain.

The first two requirements are at the core of **safety**: the blockchain does what it is supposed to do (i.e., nothing bad happens). The last two can be argued to deal with **liveness**: eventually everything turns out to be good. For our present discussion, reaching agreement is crucial.

For permissionless blockchains, we see that leader election is important: once a leader has been elected, we can essentially ensure agreement by letting the leader decide on the block that is to be appended to the blockchain. As we discuss in Section 9.4.3, guaranteeing the integrity of a blockchain can be established using well-known cryptographic techniques. The only problem with leader elections happens if two or more leaders are elected, which may happen when the declaration of a winner does not reach its competitor in time. In such a case, two processes will start to append blocks to a chain, effectively leading to two branches. A common solution is that once a participant discovers it is reading from the smallest branch, it ignores that branch completely and makes sure that it gets the blocks from the longer branch.

For permissioned blockchains, a common solution is to adopt PBFT. In other words, the decision to append blocks to an existing chain is handled by a relatively small fault-tolerant group of processes (who need not trust

each other). Because PBFT is a relatively costly and nonscalable solution, alternatives have been designed, with HotStuff [Yin et al., 2019] playing a prominent role as experiments showed that the number of replica servers could go up beyond 100.

Xiao et al. [2020] provides an excellent overview of the various consensus protocols for blockchains, along with a discussion on the many trade-offs to be made.

8.2.7 Some limitations on realizing fault tolerance

Organizing replicated processes into a group helps to increase fault tolerance. However, what should have become clear by now is that there is a price to pay, namely a potential loss of performance. In the solutions discussed so far, processes in a fault-tolerant group may need to exchange numerous messages before reaching a decision. The Byzantine agreement protocol is an excellent illustration of how tightly coupled processes may be. The question that comes to mind is whether realizing specific forms of fault tolerance, like being able to withstand arbitrary failures, is always possible.

On reaching consensus

As we mentioned, if a client can base its decisions through a voting mechanism, we can tolerate that k out of $2k + 1$ processes are lying about their result. The assumption we are making, however, is that processes do not team up to produce a wrong result. In general, matters become more intricate if we demand that a process group reaches consensus, which is needed in many cases. There are three requirements for reaching consensus [Fischer et al., 1985]:

- Processes produce the same output value
- Every output value must be valid
- Every process must eventually provide output

Some examples where reaching consensus is necessary include electing a coordinator, deciding whether to commit a transaction, and dividing up tasks among workers. When the communication and processes are all perfect, reaching consensus is often straightforward, but when they are not, problems arise.

The general goal of distributed consensus algorithms is to have all the nonfaulty processes reach consensus on some issue, and to establish that consensus within a finite number of steps. The problem is complicated by the fact that different assumptions about the underlying system require different solutions, assuming solutions even exist. Turek and Shasha [1992] distinguish the following cases:

1. Synchronous versus asynchronous systems. Rephrasing our description somewhat, a system is **synchronous** if and only if the processes are known to operate in a lock-step mode. Formally, this means that there should be some constant $c \geq 1$, such that if any process has taken $c + 1$ steps, every other process has taken at least 1 step.
2. Communication delay is bounded or not. Delay is bounded if and only if we know that every message is delivered with a globally and predetermined maximum time.
3. Message delivery is ordered (in real time) or not. In other words, we distinguish the situation where messages from the different senders are delivered in the order that they were sent in real global time, from the situation in which we do not have such guarantees.
4. Message transmission is done through unicasting or multicasting.

As it turns out, reaching consensus is possible only for the situations shown in Figure 8.19. In all other cases, it can be shown that no solution exists. Note that most distributed systems in practice assume that processes behave asynchronously, message transmission is unicast, and communication delays are unbounded. As a consequence, we need to make use of ordered (reliable) message delivery, such as provided by TCP. And again, in practical situations we assume synchronous behavior to be the default, but take into account that there may be unbounded delays as well. Figure 8.19 illustrates the nontrivial nature of distributed consensus when processes may fail.

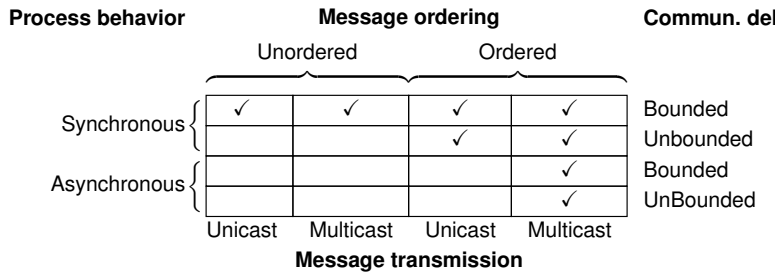


Figure 8.19: Circumstances under which distributed consensus can be reached.

Reaching consensus may not be possible. Fischer et al. [1985] proved that if messages cannot be guaranteed to be delivered within a known, finite time, no consensus is possible if even one process is faulty (albeit if that one process fails silently). The problem with such systems is that arbitrarily slow processes are indistinguishable from crashed ones (i.e., you cannot tell the dead from the living). These and other theoretical results are surveyed by Barborak et al. [1993] and Turek and Shasha [1992].

It should also be noted that the schemes described so far assume that nodes are either Byzantine, or collaborative. The latter cannot always be simply assumed when processes are from different administrative domains. In that case, they will more likely exhibit *rational* behavior, for example, by reporting timeouts when doing so is cheaper than executing an update operation. How to deal with these cases is not trivial. A first step toward a solution is captured in the form of **BAR fault tolerance**, which stands for Byzantine, Altruism, and Rationality. BAR fault tolerance is described in Aiyer et al. [2005] and Clement et al. [2008].

Consistency, availability, and partitioning

Strongly related to the conditions under which consensus can (not) be reached, is when consistency can be reached. Consistency in this case means that when we have a process group to which a client is sending requests, that the responses returned to that client are correct. We are dealing with a **safety property**: a property that asserts that nothing bad will happen. For our purposes, the types of operations we consider are those that seem to be executed in a clearly defined order by a single, centralized server. By now, we know better: these operations are executed by a process group to withstand the failures of k group members.

We introduced process groups to improve fault tolerance, and, more specifically, to improve availability. Availability is typically a **liveness property**: eventually, something good will happen. In terms of our process groups, we aim to eventually get a (correct) response to every request issued by a client. Being consistent in responses while also being highly available is not an unreasonable requirement for services that are part of a distributed system. Unfortunately, we may be asking too much.

In practical situations, our underlying assumption that the processes in a group can indeed communicate with each other may be false. Messages may be lost; a group may be partitioned due to a faulty network. In 2000, Eric Brewer posed an important theorem which was later proven to be correct by Gilbert and Lynch [2002]:

> ***CAP Theorem**: Any networked system providing shared data can provide only two of the following three properties:*
>
> - ***C**: consistency, by which a shared and replicated data item appears as a single, up-to-date copy*
> - ***A**: availability, by which updates will always be eventually executed*
> - ***P**: Tolerant to the partitioning of process group (e.g., because of a failing network).*

In other words, in a network subject to communication failures, it is impossible to realize an atomic read/write shared memory that guarantees a response to every request [Gilbert and Lynch, 2012].

This has now become known as the **CAP theorem**, first published as [Fox and Brewer, 1999]. As explained by Brewer [2012], one way of understanding the theorem is to think of two processes unable to communicate because of a failing network. Allowing one process to accept updates leads to inconsistency, so that we can only have properties $\{A, P\}$. If the illusion of consistency is to be provided while the two processes cannot communicate, then one of the two processes will have to pretend to be unavailable, implying having only $\{C, P\}$. However, only if the two processes can communicate, is it possible to maintain both consistency and high availability, meaning that we have only $\{C, A\}$, but no longer property P.

Note also the relationship with reaching consensus; in fact, where consensus requires proving that processes produce the same output, providing consistency is weaker. This also means that if achieving CAP is impossible, then so is consensus.

For some time, many people thought that the CAP theorem actually told us we were dealing with a practical horrendous situation. However, the CAP theorem is all about reaching a trade-off between safety and liveness, based on the observation that obtaining both in an inherently unreliable system cannot be achieved. Practical distributed systems *are* inherently unreliable. What Brewer and his colleagues observed is that in practical distributed systems, one simply has to make a choice to proceed, although another process cannot be reached. In other words, we need to do something when a partition manifests itself through high latency. What this means, is that we need to look at the distributed application at hand to see how we can salvage the seemingly impossibility implications of the CAP theorem.

The bottom line when it seems that partitioning is taking place, is to proceed (tolerating partitions in favor of either consistency or availability), while simultaneously starting a recovery procedure that can mitigate the effects of potential inconsistencies. Exactly deciding on how to proceed is application-dependent: in many cases having duplicate keys in a database can easily be fixed (implying that we should tolerate an inconsistency), while duplicate transfers of large sums of money may not (meaning that we should decide to tolerate lower availability). One can argue that the CAP theorem essentially moves designers of distributed systems from theoretical solutions to engineering solutions. The interested reader is referred to [Brewer, 2012] to see how such a move can be made.

8.2.8 Failure detection

It may have become clear from our discussions so far that to properly mask failures, we generally need to detect them as well. **Failure detection** is one of the cornerstones of fault tolerance in distributed systems. What it all boils down to is that for a group of processes, nonfaulty members should be able

to decide who is still a member, and who is not. In other words, we need to be able to detect when a member has failed.

When it comes to detecting process failures, there are essentially only two mechanisms. Either processes actively send "are you alive?" messages to each other (for which they obviously expect an answer), or passively wait until messages come in from different processes. The latter approach makes sense only when it can be guaranteed that there is enough communication.

There is a huge body of theoretical work on failure detectors. What it, in the end, all boils down to is that a timeout mechanism is used to check whether a process has failed. If a process P probes another process Q to see if it has failed, P is said to **suspect** Q to have crashed if Q has not responded within some time.

Note 8.7 (More information: On perfect failure detectors)
It should be clear that in a synchronous distributed system, a suspected crash corresponds to a known crash. In practice, however, we will be dealing with partially synchronous systems. In that case, it makes more sense to assume **eventually perfect failure detectors**. In this case, a process P will suspect another process Q to have crashed after t time units have elapsed and still Q did not respond to P's probe. However, if Q later does send a message that is (also) received by P, P will (1) stop suspecting Q, and (2) increase the timeout value t. Note that if Q does crash (and does not recover), P will continue to suspect Q.

In real settings, there are problems with using probes and timeouts. For example, due to unreliable networks, simply stating that a process has failed because it does not return an answer to a probe message may be wrong. In other words, it is easy to generate false positives. If a false positive has the effect that a perfectly healthy process is removed from a membership list, then clearly we are doing something wrong. Another serious problem is that timeouts are just plain crude. As noticed by Birman [2012], there is hardly any work on building proper failure detection subsystems that take more into account than only the lack of a reply to a single message. This statement is even more evident when looking at industry-deployed distributed systems.

There are various issues that need to be considered when designing a failure-detection subsystem (see also Zhuang et al. [2005]). For example, failure detection can take place through gossiping in which each node regularly announces to its neighbors that it is still up and running. As we mentioned, an alternative is to let nodes actively probe each other.

Failure detection can also be done as a side effect of regularly exchanging information with neighbors, as is the case with gossip-based information dissemination (which we discussed in Chapter 4). This approach was essentially also adopted in Obduro [Vogels, 2003]: processes periodically gossip their service availability. This information is gradually disseminated through the

network. Eventually, every process will know about every other process, but more importantly, will have enough information locally available to decide whether a process has failed or not. A member for which the availability information is old, will presumably have failed.

Another important issue is that a failure detection subsystem should ideally be able to distinguish network failures from node failures. One way of dealing with this problem is not to let a single node decide whether one of its neighbors has crashed. Instead, when noticing a timeout on a probe message, a node requests other neighbors to see whether they can reach the presumed failing node. Of course, positive information can also be shared: if a node is still alive, that information can be forwarded to other interested parties (who may be detecting a link failure to the suspected node).

This brings us to another key issue: when a member failure is detected, how should other nonfaulty processes be informed? One simple, and somewhat radical approach is the following. In FUSE [Dunagan et al., 2004], processes can be joined in a group that spans a wide-area network. The group members create a spanning tree that is used for monitoring member failures. Members send ping messages to their neighbors. When a neighbor does not respond, the pinging node immediately switches to a state in which it will also no longer respond to pings from other nodes. By recursion, it is seen that a single node failure is rapidly promoted to a group failure notification.

8.3 Reliable client-server communication

Often, fault tolerance in distributed systems concentrates on faulty processes. However, we also need to consider **communication failures**. Most of the failure models discussed previously apply equally well to communication channels. In particular, a communication channel may exhibit crash, omission, timing, and arbitrary failures. In practice, when building reliable communication channels, the focus is on masking crash and omission failures. Arbitrary failures may occur in the form of duplicate messages, resulting from the fact that in a computer network messages may be buffered for a relatively long time, and are reinjected into the network after the original sender has already issued a retransmission (see, for example, Tanenbaum et al. [2021]).

8.3.1 Point-to-point communication

In many distributed systems, reliable point-to-point communication is established by making use of a reliable transport protocol, such as TCP. TCP masks omission failures, which occur in the form of lost messages, by using acknowledgments and retransmissions. Such failures are completely hidden from a TCP client.

However, crash failures of connections are not masked. A crash failure may occur when (for whatever reason) a TCP connection is abruptly broken so

that no more messages can be transmitted through the channel. In most cases, the client is informed that the channel has crashed by raising an exception. The only way to mask such failures is to let the distributed system attempt to automatically set up a new connection, by simply resending a connection request. The underlying assumption is that the other side is still, or again, responsive to such requests.

8.3.2 RPC semantics in the presence of failures

Let us now take a closer look at client-server communication when using high-level communication facilities such as remote procedure calls (RPCs). The goal of RPC is to hide communication by making remote procedure calls look just like local ones. With a few exceptions, so far we have come fairly close. Indeed, as long as both client and server are functioning perfectly, RPC does its job well. The problem comes about when errors occur. It is then that the differences between local and remote calls are not always easy to mask.

To structure our discussion, let us distinguish between five different classes of failures that can occur in RPC systems, as follows:

1. The client is unable to locate the server.
2. The request message from the client to the server is lost.
3. The server crashes after receiving a request.
4. The reply message from the server to the client is lost.
5. The client crashes after sending a request.

Each of these categories poses different problems and requires different solutions.

Client cannot locate the server

To start with, it can happen that the client cannot locate a suitable server. All servers might be down, for example. Alternatively, suppose that the client is compiled using a particular version of the client stub, and the binary is not used for a considerable period of time. Meanwhile, the server evolves and a new version of the interface is installed; new stubs are generated and put into use. When the client is eventually run, the binder will be unable to match it up with a server and will report failure. While this mechanism is used to protect the client from accidentally trying to talk to a server that may not agree with it in terms of what parameters are required or what it is supposed to do, the problem remains of how should this failure be dealt with.

One possible solution is to have the error raise an **exception**. In some languages, (e.g., Java), programmers can write special procedures that are invoked upon specific errors, such as division by zero. In C, signal handlers

can be used for this purpose. In other words, we could define a new signal type SIGNOSERVER, and allow it to be handled in the same way as other signals.

This approach, too, has drawbacks. To start with, not every language has exceptions or signals. Another point is that having to write an exception or signal handler destroys the transparency we have been trying to achieve. Suppose that you are a programmer, and you are requested to write the append procedure. You smile and tell her it will be written, tested, and documented in five minutes. Then she mentions that you also have to write an exception handler as well, just in case the procedure is not there today. At this point, it is pretty hard to maintain the illusion that remote procedures are no different from local ones, since writing an exception handler for "Cannot locate server" would be a rather unusual request in a nondistributed system.

Lost request messages

The second item on the list is dealing with lost request messages. This is the easiest one to deal with: just have the operating system or client stub start a timer when sending the request. If the timer expires before a reply or an acknowledgment comes back, the message is sent again. If the message was truly lost, the server will not be able to tell the difference between the retransmission and the original, and everything will work fine. Unless, of course, so many request messages are lost that the client gives up and falsely concludes that the server is down, in which case we are back to "Cannot locate server." If the request was not lost, the only thing we need to do is let the server be able to detect it is dealing with a retransmission. Unfortunately, doing so is not so simple, as we explain when discussing lost replies.

Server crashes

The next failure on the list is a server crash. The normal sequence of events at a server is shown in Figure 8.20(a). A request arrives, is carried out, and a reply is sent. Now consider Figure 8.20(b). A request arrives and is carried out, just as before, but the server crashes before it can send the reply. Finally, look at Figure 8.20(c). Again a request arrives, but this time the server crashes before it can even be carried out. And, of course, no reply is sent back.

The annoying part of Figure 8.20 is that the correct treatment differs for (b) and (c). In (b) the system has to report failure back to the client (e.g., raise an exception), whereas in (c) it can just retransmit the request. The problem is that the client's operating system cannot tell which is which. All it knows is that its timer has expired.

Three schools of thought exist on what to do here [Spector, 1982]. One philosophy is to wait until the server reboots (or let the client's middleware transparently rebind to a new server) and try the operation again. The idea is

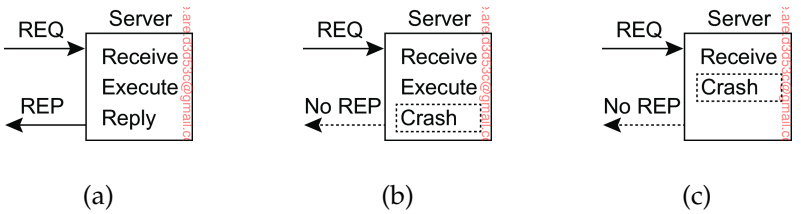


Figure 8.20: A server in client-server communication. (a) The normal case. (b) Crash after execution. (c) Crash before execution.

to keep trying until a reply has been received, then give it to the client. This technique is called **at-least-once semantics** and guarantees that the RPC has been carried out at least one time, but possibly more.

The second philosophy gives up immediately and reports back failure. This approach is called **at-most-once semantics** and guarantees that the RPC has been carried out at most one time, but possibly not at all.

The third philosophy is to guarantee nothing. When a server crashes, the client gets no help and no promises about what happened. The RPC may have been carried out anywhere from zero to many times. The main virtue of this scheme is that it is easy to implement.

None of these are terribly attractive. What one would like is **exactly-once semantics**, but in general, there is no way to arrange this. Imagine that the remote operation consists of processing a document, such as producing a number of PDF files from LaTeX and other sources. The server sends a completion message to the client when the document has been completely processed. Also assume that when a client issues a request, it receives an acknowledgment that the request has been delivered to the server. There are two strategies the server can follow. It can either send a completion message just before it actually tells the document processor to do its work, or after the document has been processed.

Assume that the server crashes and subsequently recovers. It announces to all clients that it has just crashed but is now up and running again. The problem is that the client does not know whether its request to process a document will actually have been carried out.

There are four strategies the client can follow. First, the client can decide to *never* reissue a request, at the risk that the document will not be processed. Second, it can decide to *always* reissue a request, but this may lead to the document being processed twice (which may easily incur a significant amount of work when dealing with intricate documents). Third, it can decide to reissue a request only if it did not yet receive an acknowledgment that its request had been delivered to the server. In that case, the client is counting on the fact that the server crashed before the request could be delivered. The fourth and last strategy is to reissue a request only if it has received an

acknowledgment for the request. With two strategies for the server, and four for the client, there are a total of eight combinations to consider. Unfortunately, as it turns out, no combination is satisfactory: it can be shown that for any combination either the request is lost forever, or carried out twice.

Note 8.8 (Advanced: Why fully transparent server recovery is impossible)
To explain about server recovery, note that there are three events that can happen at the server: send the completion message (M), complete the processing of the document (P), and crash (C). Note that crashing *during* the processing of a document is considered the same as crashing before its completion. These events can occur in six different orderings:

1. $M \rightarrow P \rightarrow C$: A crash occurs after sending the completion message and processing the document.
2. $M \rightarrow C(\rightarrow P)$: A crash happens after sending the completion message, but before the document could be (completely) processed.
3. $P \rightarrow M \rightarrow C$: A crash occurs after sending the completion message and processing the document.
4. $P \rightarrow C(\rightarrow M)$: The document was processed, after which a crash occurs before the completion message could be sent.
5. $C(\rightarrow P \rightarrow M)$: A crash happens before the server could complete the processing of the document.
6. $C(\rightarrow M \rightarrow P)$: A crash happens before the server could even do anything.

The parentheses indicate an event that can no longer happen because the server already crashed. Figure 8.21 shows all possible combinations. As can be readily verified, there is no combination of client strategy and server strategy that will work correctly under all possible event sequences. The bottom line is that the client can never know whether the server crashed just before or after having the text printed.

	Strategy M → P			Strategy P → M		
Reissue strategy	**MPC**	**MC(P)**	**C(MP)**	**PMC**	**PC(M)**	**C(PM)**
Always	DUP	OK	OK	DUP	DUP	OK
Never	OK	ZERO	ZERO	OK	OK	ZERO
Only when ACKed	DUP	OK	ZERO	DUP	OK	ZERO
Only when not ACKed	OK	ZERO	OK	OK	DUP	OK
Client	**Server**			**Server**		

OK = Document processed once
DUP = Document processed twice
ZERO = Document not processed at all

Figure 8.21: Different combinations of client and server strategies in the presence of server crashes. Events between brackets never take place because of a previous crash.

In short, the possibility of server crashes radically changes the nature of RPC and clearly distinguishes single-processor systems from distributed ones. In the

former case, a server crash also implies a client crash, so recovery is neither possible nor necessary. In the latter we can and should take action.

Lost reply messages

Lost replies can also be difficult to deal with. The obvious solution is just to rely on a timer again that has been set by the client's operating system. If no reply is forthcoming within a reasonable period, just send the request once more. The trouble with this solution is that the client is not really sure why there was no answer. Did the request or reply get lost, or is the server merely slow? It may make a difference.

In particular, some operations can safely be repeated as often as necessary with no damage being done. A request such as asking for the first 1024 bytes of a file has no side effects and can be executed as often as necessary without any harm being done. A request that has this property is said to be **idempotent**.

Now consider a request to a banking server asking to transfer money from one account to another. If the request arrives and is carried out, but the reply is lost, the client will not know this and will retransmit the message. The bank server will interpret this request as a new one, and will carry it out too. Twice the amount of money will be transferred. Transferring money is not idempotent.

One way of solving this problem is to try to structure all the requests in an idempotent way. In practice, however, many requests (e.g., transferring money) are inherently nonidempotent, so something else is needed. Another method is to have the client assign each request a sequence number. By having the server keep track of the most recently received sequence number from each client that is using it, the server can tell the difference between an original request and a retransmission and can refuse to carry out any request a second time. However, the server will still have to send a response to the client. Note that this approach does require that the server maintains administration on each client. Furthermore, it is not clear how long to maintain this administration. An additional safeguard is to have a bit in the message header that is used to distinguish initial requests from retransmissions (the idea being that it is always safe to perform an original request; retransmissions may require more care).

Client crashes

The final item on the list of failures is the client crash. What happens if a client sends a request to a server to do some work and crashes before the server replies? At this point a computation is active and no parent is waiting for the result. Such a computation is called an **orphan (computation)**.

Orphan computations can cause a variety of problems that can interfere with normal operation of the system. As a bare minimum, they waste processing power. They can also lock files or otherwise tie up valuable resources. Finally, if the client reboots and does the RPC again, but the reply from the orphan comes back immediately afterward, confusion can result. Note that, in this case, we are essentially dealing with the problem of guaranteeing at-most-once semantics and bringing the client back into a state just before it crashed. As we will explain later, checkpointing just before sending a request, and restoring the client to the checkpointed state will solve many problems with orphan computations. In essence, it boils down to let the orphan do its work, and properly restoring the client.

Yet, what can be done about orphans? Four solutions have been proposed [Nelson, 1981]. First, before a client stub sends an RPC message, it makes a log entry telling what it is about to do. The log is kept on disk or some other medium that survives crashes. After a reboot, the log is checked and the orphan is explicitly killed off. This solution is called **orphan extermination**, an admittedly not very empathically sounding name.

The disadvantage of this scheme is the expense of writing a disk record for every RPC. Furthermore, it may not even work, since orphans themselves may do RPCs, thus creating **grandorphans** or further descendants that are difficult or impossible to locate. Finally, the network may be partitioned, for example because of a failed gateway, making it impossible to kill them, even if they can be located. All in all, this is not a promising approach, and, in general, will need to be avoided due to its complexity.

With the second solution, called **reincarnation**, all these problems can be solved without the need to write disk records. The way it works is to divide time up into sequentially numbered epochs. When a client recovers, it broadcasts a message to all machines declaring the start of a new epoch. When such a broadcast comes in, all remote computations are killed. Of course, if the network is partitioned, some orphans may survive. Fortunately, however, when they report back, their replies will contain an obsolete epoch number, making them easy to detect.

The third solution is a variant on this idea, but somewhat less Draconian. It is called **gentle reincarnation**. When an epoch broadcast comes in, each machine checks to see if it has any remote computations running locally, and if so, tries its best to locate their owners. Only if the owners cannot be located anywhere is the computation killed.

In the fourth solution, called **expiration**, each RPC is given a standard amount of time, T, to do the job. If it cannot finish, it must explicitly ask for another quantum. Of course, this is quite a nuisance. On the other hand, if after a crash the client waits a time T before rebooting, all orphans are sure to be gone. The problem to be solved here is choosing a reasonable value of T in the face of RPCs with wildly differing requirements.

In practice, all of these methods are crude and undesirable. Worse yet, killing an orphan may have unforeseen consequences. For example, suppose that an orphan has obtained locks on one or more files or database records. If the orphan is suddenly killed, these locks may remain forever. Also, an orphan may have already made entries in various remote queues to start up other processes at some future time, so even killing the orphan may not remove all traces of it. Conceivably, it may even have started again, with unforeseen consequences. Orphan elimination is discussed in more detail by Panzieri and Shrivastava [1988].

As mentioned, the best solution is often not to do anything special, but attempt to restore the client to a state in which it can handle any response from its previously orphaned computation. This brings us to checkpointing and message logging, which we discuss in Section 8.6.

8.4 Reliable group communication

Considering how important process resilience by replication is, it is not surprising that reliable multicast services are important as well. Such services guarantee that messages are delivered to all members in a process group. Unfortunately, reliable multicasting turns out to be surprisingly tricky. In this section, we take a closer look at the issues involved in reliably delivering messages to a process group.

8.4.1 Introduction

Let us first define what reliable group communication actually is. Intuitively, it means that a message that is sent to a process group should be delivered to each member of that group. If we separate the logic of handling messages from the core functionality of a group member, we can conveniently make the distinction between *receiving* messages and *delivering* messages, as illustrated in Figure 8.22. A message is received by a message-handling component, which, in turn, delivers a message to the component containing the core functionality of a group member. Informally, a message that is received by process P will also be delivered by P.

As an example, ensuring that messages from the same sender are delivered in the same order as they were sent, is typically taken care of by a message-handling component. Likewise, providing reliable message-passing is a feature that can and should be separated from the core functionality of a group member, and is typically implemented by a message-handling component (if not by the underlying operating system).

With this separation between receiving and delivering messages, we can be more precise about what reliable group communication means. Let us make a distinction between reliable communication in the presence of faulty processes, and reliable communication when processes are assumed to operate

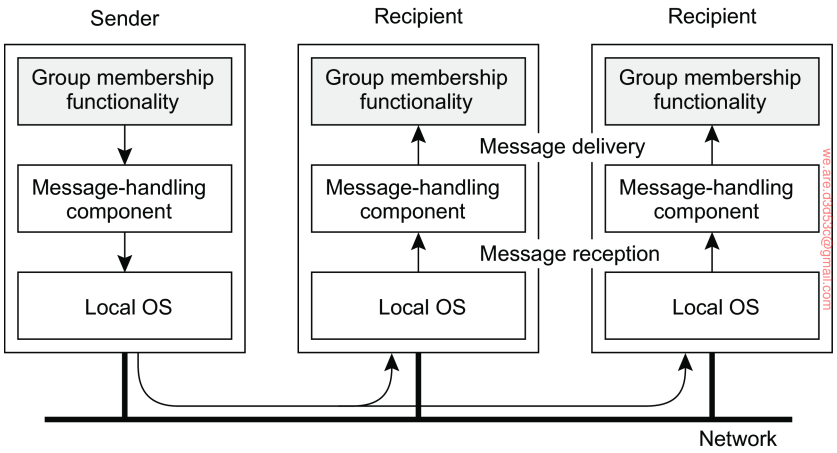


Figure 8.22: The distinction between receiving and delivering messages.

correctly. In the first case, group communication is considered to be reliable when it can be guaranteed that a message is received by and subsequently delivered to all nonfaulty group members.

The tricky part here is that agreement should be reached on what the group actually looks like before a message can be delivered. If a sender intended to have a message delivered by each member of a group $\mathbf{G}$, but that, for whatever reason, at the time of delivery we actually have another group $\mathbf{G}' \neq \mathbf{G}$, we should ask ourselves whether the message can be delivered.

The situation becomes simpler if we can ignore consensus on group membership. In particular, let us first assume that a sending process has a list of intended recipients. In that case, it can simply deploy reliable transport-level protocols such as TCP and, one by one, sends its message to each recipient. If a receiving process fails, the message may be resent later when the process recovers, or ignored altogether (for example, because the sender had left the group). In case a group member is expected to send a response, even if it is just an acknowledgement, communication can be speeded up by separating the sending of a request from receiving a response, as illustrated by the message sequence charts in Figure 8.23.

Most transport layers offer reliable point-to-point channels; they rarely offer reliable communication to a group of processes. The best they offer is to let a process set up a point-to-point connection to each other process it wants to communicate with. When process groups are relatively small, this approach to establishing reliability is a straightforward and practical solution. On the other hand, we can often assume that the underlying communication system does offer unreliable multicasting, meaning that a multicast message may be lost part way and delivered by some, but not all, of the intended receivers.

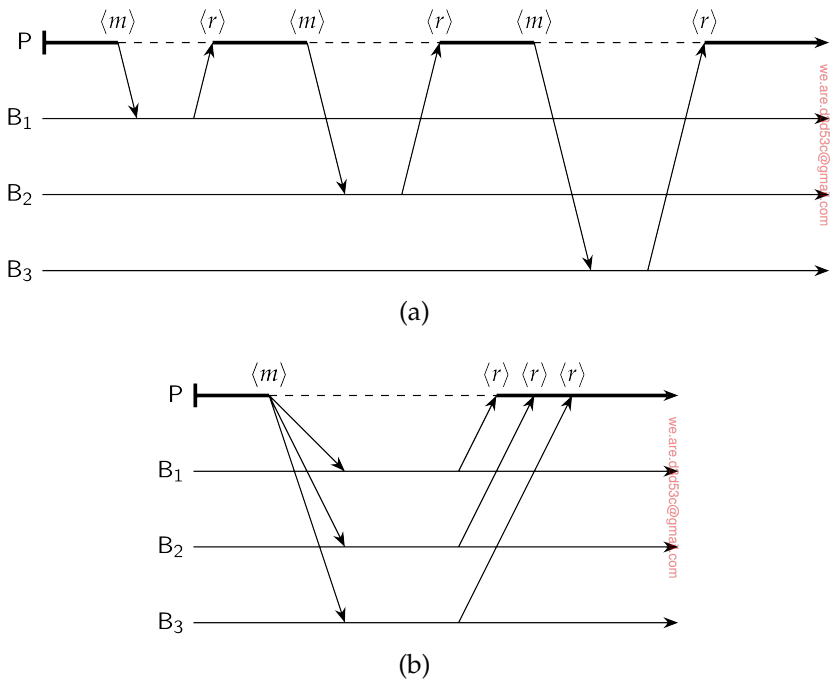


Figure 8.23: (a) A sender sends out requests, but waits for a response before sending out the next one. (b) Requests are sent out in parallel, after which the sender waits for incoming responses.

A simple solution to reach reliable group communication is shown in Figure 8.24. The sending process assigns a sequence number to each message it multicasts and stores the message locally in a history buffer. Assuming the receivers are known to the sender, the sender simply keeps the message in its history buffer until each receiver has returned an acknowledgement. A receiver can suspect it is missing a message m with sequence number s when it has received messages with sequence numbers higher than s. In that case, it returns a negative acknowledgement to the sender, requesting for a retransmission of m.

There are various design trade-offs to be made. For example, to reduce the number of messages returned to the sender, acknowledgements could possibly be piggybacked with other messages. Also, retransmitting a message can be done using point-to-point communication to each requesting process, or using a single multicast message sent to all processes. General issues on reliable multicasting are discussed by Popescu et al. [2007]. A survey and overview of reliable multicasting in the context of publish/subscribe systems, which is also relevant here, is provided by Esposito et al. [2013].

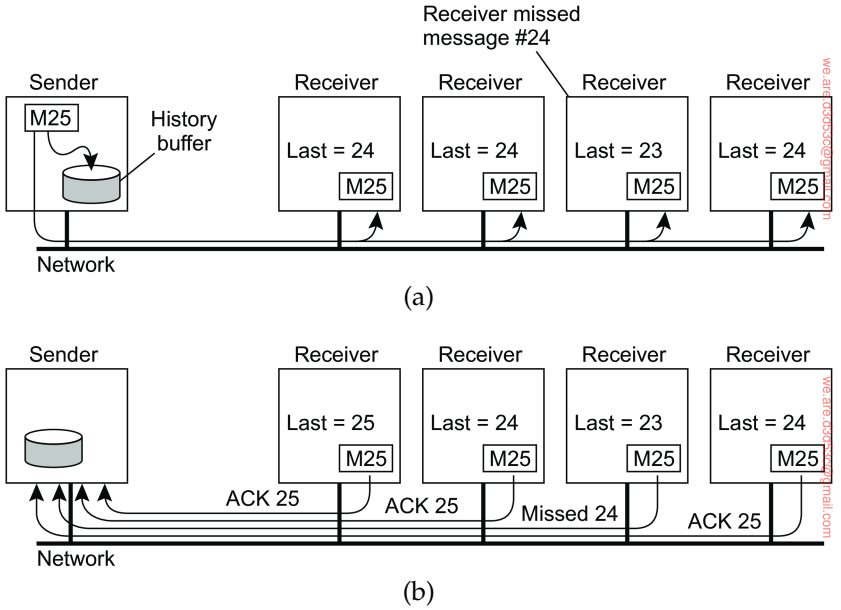


Figure 8.24: A solution for reliable multicasting. (a) Message transmission. (b) Reporting feedback.

8.4.2 Scalability in reliable multicasting

The main problem with the reliable multicast scheme just described is that it cannot support large numbers of receivers. If there are N receivers, the sender must be prepared to accept at least N acknowledgements. With many receivers, the sender may be swamped with such feedback messages, which is also referred to as a **feedback implosion**. When replicating processes for fault tolerance, this situation is not likely to occur as process groups are relatively small. When replicating for performance, we have a different case. Moreover, we may then also need to take into account that the receivers are spread across a wide-area network.

One solution to the problem of a feedback implosion is not to have receivers acknowledge the receipt of a message. Instead, a receiver returns a feedback message only to inform the sender it is missing a message. Returning only such negative acknowledgements can be shown to generally scale better [Towsley et al., 1997], but no hard guarantees can be given that feedback implosions will never happen.

Another problem with returning only negative acknowledgements is that the sender will, in theory, be forced to keep a message in its history buffer forever. Because the sender can never know if a message has been correctly delivered to all receivers, it should always be prepared for a receiver requesting

the retransmission of an old message. In practice, the sender will remove a message from its history buffer after some time has elapsed to prevent the buffer from overflowing. However, removing a message is done at the risk of a request for a retransmission not being honored.

Several proposals for scalable, reliable multicasting exist. A comparison between different schemes can be found in [Levine and Garcia-Luna-Aceves, 1998]. We now briefly discuss two very different approaches that are representative of many existing solutions.

The key issue to scalable solutions for reliable multicasting is to reduce the number of feedback messages that are returned to the sender. A popular model that has been applied to several wide-area applications is **feedback suppression**. This scheme underlies the **Scalable Reliable Multicasting** (**SRM**) protocol developed by Floyd et al. [1997] and works as follows.

First, in SRM, receivers never acknowledge the successful delivery of a multicast message, but instead, report only when they are missing a message. How message loss is detected is left to the application. Only negative acknowledgements are returned as feedback. Whenever a receiver notices that it missed a message, it *multicasts* its feedback to the rest of the group.

Multicasting feedback allows another group member to suppress its own feedback. Suppose several receivers missed message m. Each of them will need to return a negative acknowledgement to the sender, S, so that m can be retransmitted. However, if we assume that retransmissions are always multicast to the entire group, it is sufficient that only a single request for retransmission reaches S.

For this reason, a receiver R that did not receive message m schedules a feedback message with some random delay. That is, the request for retransmission is not sent until some random time has elapsed. If, in the meantime, another request for retransmission for m reaches R, R will suppress its own feedback, knowing that m will be retransmitted shortly. In this way, ideally, only a single feedback message will reach S, which in turn subsequently retransmits m. This scheme is shown in Figure 8.25.

Feedback suppression has shown to scale reasonably well, and has been used as the underlying mechanism for a number of collaborative Internet applications, such as a shared whiteboard. However, the approach also introduces a number of serious problems. First, ensuring that only one request for retransmission is returned to the sender requires a reasonably accurate scheduling of feedback messages at each receiver. Otherwise, many receivers will still return their feedback at the same time. Setting timers accordingly in a group of processes that is dispersed across a wide-area network is not that easy.

Another problem is that multicasting feedback also interrupts those processes to which the message has been successfully delivered. In other words, other receivers are forced to receive and process messages that are useless

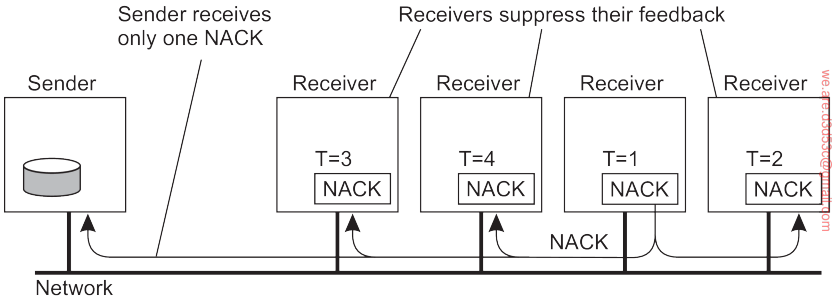


Figure 8.25: Several receivers have scheduled a request for retransmission, but the first retransmission request leads to the suppression of others.

to them. The only solution to this problem is to let receivers that have not received message m join a separate multicast group for m, as explained by Kasera et al. [1997]. Unfortunately, this solution requires that groups can be managed in a highly efficient manner, which is hard to accomplish in a wide-area system. A better approach is therefore to let receivers that tend to miss the same messages team up and share the same multicast channel for feedback messages and retransmissions. Details on this approach are found in [Liu et al., 1998].

To enhance the scalability of SRM, it is useful to let receivers assist in local recovery. In particular, if a receiver to which message m has been successfully delivered, receives a request for retransmission, it can decide to multicast m even before the retransmission request reaches the original sender. Further details can be found in [Floyd et al., 1997] and [Liu et al., 1998].

Feedback suppression as just described is basically a nonhierarchical solution. However, achieving scalability for large groups of receivers requires that hierarchical approaches are adopted. A solution is shown in Figure 8.26. The group of receivers is partitioned into a number of subgroups, which are subsequently organized into a tree. Within each subgroup, any reliable multicasting scheme that works for small groups can be used. Each subgroup appoints a local coordinator, which represents that group in the multicast tree. A link in the tree between two nodes corresponds to a reliable connection between the coordinators of the respective subgroups.

When a process S in group **G** wants to send a message, it simply uses the reliable multicast scheme for **G** to reach all its members, including the group's coordinator, say C. C, in turn, will forward the message to its neighboring coordinators. As a general rule, a coordinator will forward an incoming message m to all its neighboring coordinators, except the one from which it received m. In addition, a coordinator will reliably multicast the incoming message to all members of the subgroup it represents, and notably also handle retransmissions for that group.

In an ACK-based scheme, if coordinator C of group **G** sends a message m to coordinator C′ of another, neighboring group **G**′, it will keep m in its history buffer at least until C′ has sent an acknowledgement. In a NACK-based scheme, only if **G**′ detects it has missed m (and thus also all members of **G**′, and all coordinators to which G' would have forwarded m), it will send a NACK message to C. It is thus seen that a single ACK or NACK message from a coordinator, aggregates many feedback control messages from other processes, leading to a much more scalable, reliable multicasting scheme. Scalability is further improved by letting a coordinator handle the retransmissions to neighboring coordinators, to which it had forwarded a message.

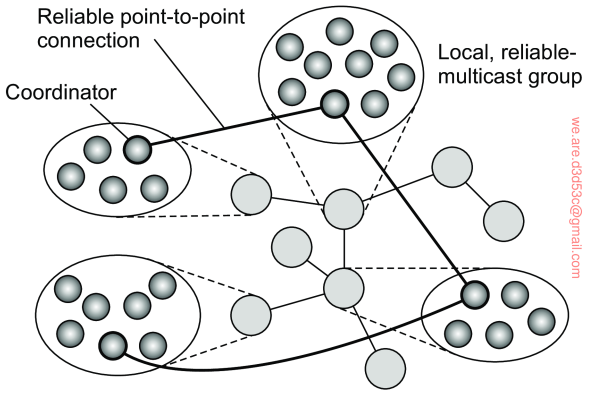


Figure 8.26: The essence of hierarchical reliable multicasting. Each local coordinator forwards the message to its neighboring coordinators in the tree and later handles retransmission requests.

Note that the nonhierarchical feedback control which we discussed before can be used to improve the scalability of a single multicast group. Together with hierarchical feedback control, we would combine relatively large reliable-multicast subgroups into potentially large trees, thus being able to support reliable multicasting for large groups of processes.

The main problem with hierarchical solutions is the construction and management of the tree: how are subgroups formed, which processes are appointed to be coordinator, and how are the subgroups organized in a tree. In many cases, a tree needs to be constructed dynamically. Unfortunately, traditional network-level solutions provide almost no adequate services for tree management. For this reason, application-level multicasting solutions as we discussed in Section 4.4.1 have gained popularity.

Finally, let us briefly consider gossip-based multicasting schemes, in particular the following push-pull **anti-entropy** scheme that we discussed extensively in Section 4.4.3.

In this scheme, a node P picks another node Q at random, and subsequently exchanges updates with Q. In other words, P pushes updates that Q has not

seen before to Q, and pulls in any updates that Q has, but which were missed by P. After the exchange, both processes have the same data. Clearly, this scheme is already inherently robust, for if the communication between P and Q fails for whatever reason, P will simply pick some other node to exchange updates. The net effect is that the speed by which an update propagates through the system slows down, but the reliability is affected only in extreme cases. Nevertheless, this slowdown is considered important for some applications. In this light, the comparison between traditional tree-based multicasting and gossip-based multicasting for the purpose of aggregation as discussed by Nyers and Jelasity [2015] may be of interest.

8.4.3 Atomic multicast

Let us now return to the situation in which we need to achieve reliable multicasting in the presence of process failures. In particular, what is often needed in a distributed system is the guarantee that a message is delivered to either all (nonfaulty) group members or to none. This is also known as the **atomic multicast problem**, to which we return with greater precision below.

To see why atomicity is so important, consider a replicated database constructed as an application on top of a distributed system. The distributed system offers reliable multicasting facilities. In particular, it allows the construction of process groups to which messages can be reliably sent. The replicated database is therefore constructed as a group of processes, one process for each replica. Update operations are always multicasted to all replicas and subsequently performed locally. We are thus assuming that an active-replication protocol is being used.

To keep matters simple, assume a client contacts a replica P and requests it to perform an update. The replica does so by multicasting the update to the other group members. Unfortunately, before the multicast completes, P crashes, leaving the rest of the group in a difficult position: some group members will have received the update request; others will not. If the members who have received the request, deliver it to the database, then obviously we will have an inconsistent replicated database. Some replicas will have processed the update, others will not. This situation needs to be avoided, and we should either have that the update is delivered to all nonfaulty members, or to none. The former case reflects that P crashed *after* completing the multicast, while the latter represents P crashing before it even got a chance to request the update.

Both these situations are fine, and correspond to the case in which a client communicates with a single server that is allowed to crash. If a number of the group members would execute the update, while others would not, distribution transparency is at stake, but even worse, the client would not know what to make of the situation.

Virtual synchrony

Reliable multicast in the presence of process failures can be accurately defined in terms of process groups and changes to group membership. As we did earlier, we make a distinction between *receiving* and *delivering* a message. In particular, we again adopt a model in which the distributed system consists of message-handling components, as was shown in Figure 8.22. A received message is locally buffered in this component until it can be delivered to the application, which is logically placed as a group member at a higher layer.

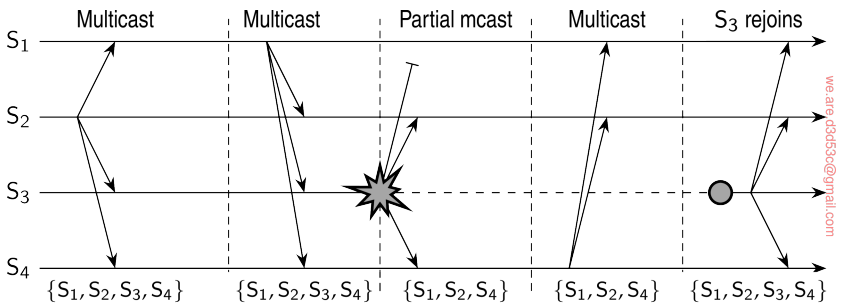
The whole idea of atomic multicasting is that a multicast message m is uniquely associated with a list of processes that should deliver it. This delivery list corresponds to a **group view**, namely, the view on the set of processes contained in the group, which the sender had at the time message m was multicasted. An important observation is that each process on that list has the same view. In other words, they should all agree that m should be delivered by each one of them and by no other process.

Now suppose that message m is multicasted at the time its sender, say P, has group view **G**. Furthermore, assume that while the multicast is taking place, another process Q joins or leaves the group. This change in group membership is naturally announced to all processes in **G**. Stated somewhat differently, a **view change** takes place by multicasting a message vc announcing the joining or leaving of Q. We now have two multicast messages simultaneously in transit: m and vc. What we need to guarantee is that m is either delivered by all processes in **G** before any one executes the view change as specified by vc, or m is not delivered at all. Note that this requirement is comparable to totally ordered multicasting, which we discussed in Section 5.2.1.

A question that quickly comes to mind is that if m is not delivered by any process, how can we speak of a *reliable* multicast protocol? In principle, there is only one case in which delivery of m is allowed to fail: when the group membership change is the result of the sender P of m crashing. In that case, either all remaining (nonfaulty) members of **G** should deliver m before agreeing P is no longer member of the group, or none should deliver m. As mentioned before, the latter corresponds to the situation that P is considered to have crashed before it had a chance to send m.

This stronger form of reliable multicast guarantees that a message multicast to group view **G** is delivered by each nonfaulty process in **G**. If the sender of the message crashes during the multicast, the message is either delivered to all remaining processes, or ignored by each of them. Such a reliable multicast is said to be **virtually synchronous** [Birman and Joseph, 1987].

To illustrate these matters, consider the four processes shown in Figure 8.27. At a certain moment, we have a group consisting of S_1, S_2, S_3, and S_4. After some messages have been multicast, S_3 crashes. However, before crashing, it succeeded in multicasting a message to processes S_2 and S_4, but not to S_1.



we.are.d3d53e@gmail.com

Figure 8.27: The principle of virtual synchronous multicast.

Virtual synchrony in this case guarantees that the message is not delivered at all, effectively establishing the situation that the message was never sent before S_3 crashed.

After S_3 has been removed from the group, communication proceeds between the remaining group members. Later, when S_3 recovers, it can join the group again, after its state has been brought up to date.

The principle of virtual synchrony comes from the fact that all multicasts take place between view changes. Put somewhat differently, a view change acts as a barrier across which no multicast can pass. In a sense, it is comparable to the use of a synchronization variable in distributed data stores, as discussed in the Chapter 7. All multicasts that are in transit while a view change takes place are completed before the view change comes into effect. The implementation of virtual synchrony is not trivial, as we discuss below.

Message Ordering

Virtual synchrony allows an application developer to think about multicasts as taking place in epochs that are separated by group membership changes. However, nothing has yet been said concerning the ordering of multicasts. In general, four different orderings are distinguished:

1. Unordered multicasts
2. FIFO-ordered multicasts
3. Causally ordered multicasts
4. Totally ordered multicasts

A **reliable, unordered multicast** is a virtually synchronous multicast in which no guarantees are given concerning the order in which received messages are delivered by different processes. (Note that reliable multicasting is still about realizing the all-or-nothing property when it comes to delivering

messages to group members.) To explain, assume that reliable multicasting is supported by a library providing a send and a receive primitive. The receive operation blocks the caller until a message can be delivered.

Event order	Process P_1	Process P_2	Process P_3
1	sends m_1	receives m_1	receives m_2
2	sends m_2	receives m_2	receives m_1

Figure 8.28: Three communicating processes in the same group. The ordering of events per process is shown along the vertical axis.

Now suppose a sender P_1 multicasts two messages to a group, while two other processes in that group are waiting for messages to arrive, as shown in Figure 8.28. Assuming that processes do not crash or leave the group during these multicasts, it is possible that the message-handling component at P_2 first receives message m_1 and then m_2. Because there are no message-ordering constraints, the messages may be delivered in the order that they are received. In contrast, the message-handling component at P_3 may first receive message m_2 followed by m_1, and delivers these two in this same order to the higher-level application of P_3.

In the case of **reliable FIFO-ordered multicasts**, the message-handling component layer is forced to deliver incoming messages from the same process in the same order as they have been sent. Consider the communication within a group of four processes, as shown in Figure 8.29. With FIFO ordering, the only thing that matters is that message m_1 is always delivered before m_2, and, likewise, that message m_3 is always delivered before m_4. This rule has to be obeyed by all processes in the group. In other words, when the communication layer at P_3 receives m_2 first, it will wait with delivery to P_3 until it has received and delivered m_1.

Event order	Process P_1	Process P_2	Process P_3	Process P_4
1	sends m_1	receives m_1	receives m_3	sends m_3
2	sends m_2	receives m_3	receives m_1	sends m_4
3		receives m_2	receives m_2	
4		receives m_4	receives m_4	

Figure 8.29: Four processes in the same group with two different senders, and a possible delivery order of messages under FIFO-ordered multicasting.

However, there is no constraint regarding the delivery of messages sent by different processes. In other words, if process P_2 receives m_1 before m_3, it may deliver the two messages in that order. Meanwhile, process P_3 may have received m_3 before receiving m_1. FIFO ordering states that P_3 may deliver m_3 before m_1, although this delivery order is different from that of P_2.

Finally, **reliable causally ordered multicast** delivers messages so that potential causality between different messages is preserved. In other words, if a message m_1 causally precedes another message m_2, regardless of whether they were multicasted by the same sender, then the communication layer at each receiver will always deliver m_2 after it has received and delivered m_1. Note that causally ordered multicasts can be implemented using vector timestamps, as discussed in Chapter 5.

Besides these three orderings, there may be the additional constraint that message delivery is to be totally ordered as well. **Totally ordered delivery** means that regardless of whether message delivery is unordered, FIFO ordered, or causally ordered, it is required additionally that when messages are delivered, they are delivered in the same order to all group members.

For example, with the combination of FIFO and totally ordered multicast, processes P_2 and P_3 in Figure 8.29 may both first deliver message m_3 and then message m_1. However, if P_2 delivers m_1 before m_3, while P_3 delivers m_3 before delivering m_1, they would violate the total-ordering constraint. Note that FIFO ordering should still be respected. In other words, m_2 should be delivered after m_1 and, accordingly, m_4 should be delivered after m_3.

Virtually synchronous reliable multicasting offering totally ordered delivery of messages is called **atomic multicasting**. It encompasses the all-or-nothing property discussed before (i.e., all members have the message delivered to, or none of them do), yet atomic multicasting also requires a totally ordered delivery of messages. With the three different message ordering constraints discussed above, this leads to six forms of reliable multicasting, as shown in Figure 8.30 [Hadzilacos and Toueg, 1993].

Multicast	Basic message ordering	TO delivery?
Reliable multicast	None	No
FIFO multicast	FIFO-ordered delivery	No
Causal multicast	Causal-ordered delivery	No
Atomic multicast	None	Yes
FIFO atomic multicast	FIFO-ordered delivery	Yes
Causal atomic multicast	Causal-ordered delivery	Yes

Figure 8.30: Six different versions of virtually synchronous reliable multicasting and considering totally ordered delivery.

Note 8.9 (Advanced: Implementing virtual synchrony)
Let us now consider a possible implementation of a virtually synchronous reliable multicast. An example of such an implementation appears in Isis, a fault-tolerant distributed system that has been in practical use in industry for several years. We will focus on some of the implementation issues of this technique as described in Birman et al. [1991].

Reliable multicasting in Isis makes use of available reliable point-to-point communication facilities of the underlying network, in particular, TCP. Multicasting a message m to a group of processes is implemented by reliably sending m to each group member. As a consequence, although each transmission is guaranteed to succeed, there are no guarantees that *all* group members receive m. In particular, the sender may fail before having transmitted m to each member.

Besides reliable point-to-point communication, Isis also assumes that messages from the same source are received by a communication layer in the order they were sent by that source. In practice, this requirement is solved by using TCP connections for point-to-point communication.

The main problem that needs to be solved is to guarantee that all messages sent to view **G** are delivered to all nonfaulty processes in **G** before the next group membership change takes place. The first issue that needs to be taken care of is making sure that each process in **G** has received all messages that were sent to **G**. Note that because the sender of a message m to **G** may have failed before completing its multicast, there may indeed be processes in **G** that will never receive m. Because the sender has crashed, these processes should get m from somewhere else.

The solution to this problem is to let every process in **G** keep m until it knows for sure that all members in **G** have received it. If m has been received by all members in **G**, m is said to be **stable**. Only stable messages are allowed to be delivered. To ensure stability, it is sufficient to select an arbitrary live process in **G** and request it to send m to all other processes in **G**.

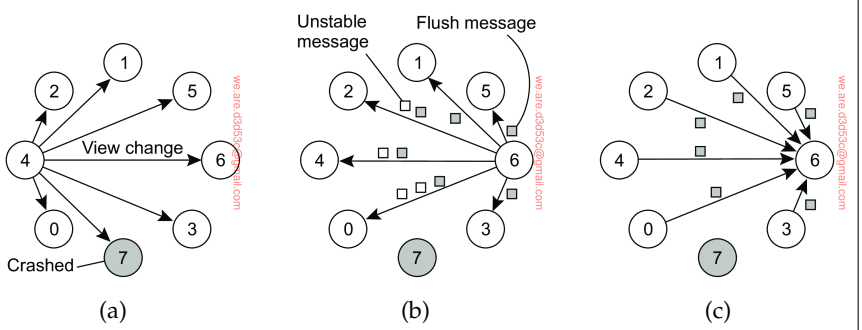


Figure 8.31: (a) Process 4 notices that process 7 has crashed and sends a view change. (b) Process 6 sends out all its unstable messages, followed by a flush message. (c) Process 6 installs the new view when it has received a flush message from everyone else.

To be more specific, assume the current view is $\mathbf{G_i}$, but that it is necessary to install the next view $\mathbf{G_{i+1}}$. Without loss of generality, we assume that $\mathbf{G_i}$ and $\mathbf{G_{i+1}}$ differ by at most one process. A process P notices the view change when it receives a view-change message. Such a message may come from the process wanting to join or leave the group, or from a process that had detected the failure

of a process in $\mathbf{G_i}$ that is now to be removed, as shown in Figure 8.31(a).

When a process P receives the view-change message for $\mathbf{G_{i+1}}$, it first forwards a copy of any unstable message from $\mathbf{G_i}$ it still has to every process in $\mathbf{G_{i+1}}$, and subsequently marks it as being stable. Recall that Isis assumes point-to-point communication is reliable, so that forwarded messages are never lost. Such forwarding guarantees that all messages in $\mathbf{G_i}$ that have been received by at least one process are received by all nonfaulty processes in $\mathbf{G_i}$. Note that it would also have been sufficient to elect a single coordinator to forward unstable messages.

To indicate that P no longer has any unstable messages and that it is prepared to install $\mathbf{G_{i+1}}$ as soon as the other processes can do that as well, it multicasts a **flush message** for $\mathbf{G_{i+1}}$, as shown in Figure 8.31(b). After P has received a flush message for $\mathbf{G_{i+1}}$ from each other process, it can safely install the new view, as shown in Figure 8.31(c).

When a process Q receives a message m while Q still believes the current view is $\mathbf{G_i}$, it delivers m taking any additional message-ordering constraints into account. If it had already received m, it considers the message to be a duplicate and discards it.

Because process Q will eventually receive the view-change message for $\mathbf{G_{i+1}}$, it will also first forward any of its unstable messages and subsequently wrap things up by sending a flush message for $\mathbf{G_{i+1}}$. Note that due to the assumed FIFO-message ordering as provided by the underlying communication layer, a flush message from a process is always received after the receipt of an unstable message from that same process.

The major flaw in the protocol described so far is that it cannot deal with process failures while a new view change is being announced. In particular, it assumes that until the new view $\mathbf{G_{i+1}}$ has been installed by each member in $\mathbf{G_{i+1}}$, no process in $\mathbf{G_{i+1}}$ will fail (which would lead to a next view $\mathbf{G_{i+2}}$). This problem is solved by announcing view changes for any view $\mathbf{G_{i+k}}$ even while previous changes have not yet been installed by all processes. The details are rather intricate, yet the principle should be clear.

8.5 Distributed commit

The atomic multicasting problem discussed in the previous section is an example of a more general problem, known as **distributed commit**. The distributed commit problem involves having an operation being performed by each member of a process group, or none at all. In the case of reliable multicasting, the operation is the delivery of a message. With distributed transactions, the operation may be the commit of a transaction at a single site that takes part in the transaction. Other examples of distributed commit, and how it can be solved, are discussed by Tanisch [2000].

Distributed commit is often established by a coordinator. In a simple scheme, this coordinator tells all other processes that are also involved, called participants, whether to (locally) perform the operation in question. This

scheme is referred to as a **one-phase commit protocol**. It has the obvious drawback that if one of the participants cannot actually perform the operation, there is no way to tell the coordinator. For example, in the case of distributed transactions, a local commit may not be possible because this would violate concurrency control constraints.

In practice, more sophisticated schemes are needed, the most common one being the **two-phase commit protocol**, which we discuss in detail below. The main drawback of this protocol is that it cannot generally efficiently handle the failure of the coordinator. To that end, a three-phase protocol has been developed, which we discuss separately in Note 8.11.

The original two-phase commit protocol (**2PC**) is due to Gray [1978]. Without loss of generality, consider a distributed transaction involving the participation of a number of processes, each running on a different machine. Assuming that no failures occur, the protocol consists of the following two phases, each consisting of two steps (see also Bernstein and Newcomer [2009]):

1. The coordinator sends a VOTE-REQUEST message to all participants.
2. When a participant receives a VOTE-REQUEST message, it returns either a VOTE-COMMIT message to the coordinator, telling the coordinator that it is prepared to locally commit its part of the transaction, or otherwise, a VOTE-ABORT message.
3. The coordinator collects all votes from the participants. If all participants have voted to commit the transaction, then so will the coordinator. In that case, it sends a GLOBAL-COMMIT message to all participants. However, if one participant had voted to abort the transaction, the coordinator will also decide to abort the transaction and multicasts a GLOBAL-ABORT message.
4. Each participant that voted for a commit waits for the final reaction by the coordinator. If a participant receives a GLOBAL-COMMIT message, it locally commits the transaction. Otherwise, when receiving a GLOBAL-ABORT message, the transaction is locally aborted as well.

The first phase is the voting phase, and consists of steps 1 and 2. The second phase is the decision phase, and consists of steps 3 and 4. These four steps are shown as finite state diagrams in Figure 8.32.

Several problems arise when this basic 2PC protocol is used in a system where failures occur. First, note that the coordinator as well as the participants have states in which they block waiting for incoming messages. Consequently, the protocol can easily fail when a process crashes, as other processes may be indefinitely waiting for a message from that process. For this reason, timeout mechanisms are used. These mechanisms are explained next.

When taking a look at the finite state machines in Figure 8.32, it can be seen that there are a total of three states in which either a coordinator or participant

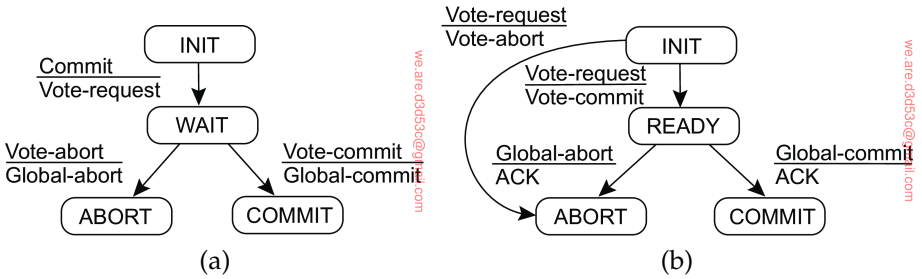


Figure 8.32: (a) The finite state machine for the coordinator in 2PC. (b) The finite state machine for a participant.

is blocked waiting for an incoming message. First, a participant may be waiting in its INIT state for a VOTE-REQUEST message from the coordinator. If that message is not received after some time, the participant will simply decide to locally abort the transaction, and thus send a VOTE-ABORT message to the coordinator.

Likewise, the coordinator can be blocked in state WAIT, waiting for the votes of each participant. If not all votes have been collected after a certain period of time, the coordinator should vote for an abort as well, and subsequently send GLOBAL-ABORT to all participants.

Finally, a participant can be blocked in state READY, waiting for the global vote as sent by the coordinator. If that message is not received within a given time, the participant cannot simply decide to abort the transaction. Instead, it must find out which message the coordinator actually sent. The simplest solution to this problem is to let each participant block until the coordinator recovers again.

A better solution is to let a participant P contact another participant Q to see if it can decide from Q's current state what it should do. For example, suppose that Q had reached state COMMIT. This is possible only if the coordinator had sent a GLOBAL-COMMIT message to Q just before crashing. Apparently, this message had not yet been sent to P. Consequently, P may now also decide to locally commit. Likewise, if Q is in state ABORT, P can safely abort as well.

Now suppose that Q is still in state INIT. This situation can occur when the coordinator has sent VOTE-REQUEST to all participants, but this message has reached P (which subsequently responded with a VOTE-COMMIT message), but has not reached Q. In other words, the coordinator had crashed while multicasting VOTE-REQUEST. In this case, it is safe to abort the transaction: both P and Q can make a transition to state ABORT.

The most difficult situation occurs when Q is also in state READY, waiting for a response from the coordinator. In particular, if it turns out that all

participants are in state READY, no decision can be taken. The problem is that although all participants are willing to commit, they still need the coordinator's vote to reach the final decision. Consequently, the protocol blocks until the coordinator recovers. The various options are summarized in Figure 8.33.

State of Q	**Action by** P
COMMIT	Make transition to COMMIT
ABORT	Make transition to ABORT
INIT	Make transition to ABORT
READY	Contact another participant

Figure 8.33: Actions taken by a participant P when residing in state READY and having contacted another participant Q.

To ensure that a process can actually recover, it is necessary that it saves its state to persistent storage. For example, if a participant was in state INIT, it can safely decide to locally abort the transaction when it recovers, and then inform the coordinator. Likewise, when it had already taken a decision such as when it crashed while being in either state COMMIT or ABORT, it is to recover to that state again, and retransmit its decision to the coordinator.

Problems arise when a participant crashed while residing in state READY. In that case, when recovering, it cannot decide on its own what it should do next, that is, commit or abort the transaction. Consequently, it is forced to contact other participants to find what it should do, analogous to the situation when it times out while residing in state READY as described above.

The coordinator has only two critical states it needs to keep track of. When it starts the 2PC protocol, it should record that it is entering state WAIT so that it can possibly retransmit the VOTE-REQUEST message to all participants after recovering. Likewise, if it had come to a decision in the second phase, it is sufficient if that decision has been recorded so that it can be retransmitted when recovering.

It may thus be possible that a participant will need to block until the coordinator recovers. This situation occurs when all participants have received and processed the VOTE-REQUEST message from the coordinator, while meanwhile, the coordinator crashed. In that case, participants cannot cooperatively decide on the final action to take. For this reason, 2PC is also referred to as a **blocking commit protocol**.

There are several solutions to avoid blocking. One solution is to use a multicast primitive, by which a receiver immediately multicasts a received message to all other processes [Babaoglu and Toueg, 1993]. It can be shown that this approach allows a participant to reach a final decision, even if the coordinator has not yet recovered. Another solution is to use three instead of two phases, as we discuss in Note 8.11.

Note 8.10 (Advanced: 2PC outlined in Python)
An outline of the actions that are executed by the coordinator is given in Figure 8.34. The coordinator starts by multicasting a VOTE-REQUEST to all participants to collect their votes. It subsequently records that it is entering the WAIT state, after which it waits for incoming votes from participants.

If not all votes have been collected, but no more votes are received within a given time interval prescribed in advance, the coordinator assumes that one or more participants have failed. Consequently, it should abort the transaction and multicasts a GLOBAL-ABORT to the (remaining) participants. Likewise, if only a single participant decides to abort the transaction, the coordinator will have to call off the transaction. If all participants vote to commit, GLOBAL-COMMIT is first logged and subsequently sent to all processes. Otherwise, the coordinator multicasts a GLOBAL-ABORT (after recording it in the local log).

```
1  class Coordinator:
2    def run(self):
3      yetToReceive = list(self.participants)
4      self.log.info('WAIT')
5      self.chan.sendTo(self.participants, VOTE_REQUEST)
6      while len(yetToReceive) > 0:
7        msg = self.chan.recvFrom(self.participants, BLOCK, TIMEOUT)
8        if msg == -1 or (msg[1] == VOTE_ABORT):
9          self.log.info('ABORT')
10         self.chan.sendTo(self.participants, GLOBAL_ABORT)
11         return
12       else: # msg[1] == VOTE_COMMIT
13         yetToReceive.remove(msg[0])
14     self.log.info('COMMIT')
15     self.chan.sendTo(self.participants, GLOBAL_COMMIT)
```

Figure 8.34: The steps taken by the coordinator in a 2PC protocol.

After receiving a vote request, the participant does its work. All bets are off if its work failed, but otherwise, it will vote for committing the transaction. It records its decision in a local log and informs the coordinator by sending a VOTE-COMMIT message. The participant must then wait for the global decision. Assuming this decision (which again should come from the coordinator) comes in on time, it is simply written to the local log, after which it can be carried out (the latter is not shown in the code).

Figure 8.35 shows the steps taken by a participant. First, the process waits for a vote request from the coordinator. If no message comes in, the transaction is simply aborted. Apparently, the coordinator had failed.

However, if the participant times out while waiting for the coordinator's decision to come in, it executes a termination protocol by first multicasting a DECISION-REQUEST message to the other processes, after which it subsequently blocks while waiting for a response. When a decisive response comes in (possibly from the coordinator, which is assumed to eventually recover), the participant

writes the decision to its local log and handles accordingly. Any request from another participant for the final decision is left unanswered as long as that decision is not known.

```
class Participant:
  def run(self):
    self.log.info('INIT')
    msg = self.chan.recvFrom(self.coordinator, BLOCK, TIMEOUT)
    if msg == -1:  # Crashed coordinator - give up entirely
      decision = LOCAL_ABORT
    else: # Coordinator will have sent VOTE_REQUEST
      decision = self.do_work()
      if decision == LOCAL_ABORT:
        self.chan.sendTo(self.coordinator, VOTE_ABORT)
        self.log.info('LOCAL_ABORT')
      else: # Ready to commit, enter READY state
        self.log.info('READY')
        self.chan.sendTo(self.coordinator, VOTE_COMMIT)
        msg = self.chan.recvFrom(self.coordinator, BLOCK, TIMEOUT)
        if msg == -1: # Crashed coordinator - check the others
          self.log.info('NEED_DECISION')
          self.chan.sendTo(self.participants, NEED_DECISION)
          while True:
            msg = self.chan.recvFromAny()
            if msg[1] in [GLOBAL_COMMIT, GLOBAL_ABORT, LOCAL_ABORT]:
              decision = msg[1]
              break
        else: # Coordinator came to a decision
          decision = msg[1]
    if decision == GLOBAL_COMMIT:
      self.log.info('COMMIT')
    else: # decision in [GLOBAL_ABORT, LOCAL_ABORT]:
      self.log.info('ABORT')
    while True: # Help any other participant when coordinator crashed
      msg = self.chan.recvFrom(self.participants)
      if msg[1] == NEED_DECISION:
        self.chan.sendTo([msg[0]], decision)
```

Figure 8.35: The steps taken by a participant process in 2PC.

We keep every participant up-and-running after it decided to either commit or abort. It can then assist other participants in need of a decision after detecting that the coordinator had crashed. To this end, a participant blocks on incoming messages and returns its own decision when asked for. Note that we are actually providing an implementation that supports partially synchronous behavior: we assume that timeouts can be applied as a mechanism to detect failures, but take into account that we may have mistakenly concluded that a server crashed.

Of course, the final, now endless loop should eventually terminate. To that end, we need a mechanism to detect that all processes have indeed come to a decision. We leave it as an exercise to the reader for designing such a detector.

Note 8.11 (Advanced: Three-phase commit)
A problem with the two-phase commit protocol is that when the coordinator has crashed, participants may not be able to reach a final decision. Consequently, participants may need to remain blocked until the coordinator recovers. Skeen [1981] developed a variant of 2PC, called the **three-phase commit protocol** (**3PC**), that avoids blocking processes in the presence of fail-stop crashes. Although 3PC is widely referred to in the literature, it is not applied often in practice, as the conditions under which 2PC blocks rarely occur. We discuss the protocol, as it provides further insight into solving fault-tolerance problems in distributed systems.

Like 2PC, 3PC is also formulated in terms of a coordinator and a number of participants. Their respective finite state machines are shown in Figure 8.36. The essence of the protocol is that the states of the coordinator and each participant satisfy the following two conditions:

1. There is no single state from which it is possible to make a transition directly to either a COMMIT or an ABORT state.
2. There is no state in which it is not possible to make a final decision, and from which a transition to a COMMIT state can be made.

It can be shown that these two conditions are necessary and sufficient for a commit protocol to be nonblocking [Skeen and Stonebraker, 1983].

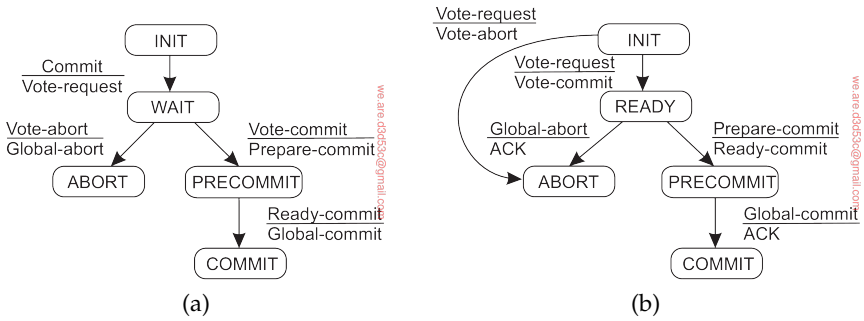


Figure 8.36: (a) The finite state machine for the coordinator in 3PC. (b) The finite state machine for a participant.

The coordinator in 3PC starts with sending a VOTE-REQUEST message to all participants, after which it waits for incoming responses. If any participant votes to abort the transaction, the final decision will be to abort as well, so the coordinator sends GLOBAL-ABORT. However, when the transaction can be committed, a PREPARE-COMMIT message is sent. Only after each participant has acknowledged it is now prepared to commit, will the coordinator send the final GLOBAL-COMMIT message by which the transaction is actually committed.

Again, there are only a few situations in which a process is blocked while waiting for incoming messages. First, if a participant is waiting for a vote request

from the coordinator while residing in state INIT, it will eventually make a transition to state ABORT, thereby assuming that the coordinator has crashed. This situation is identical to that in 2PC. Analogously, the coordinator may be in state WAIT, waiting for the votes from participants. On a timeout, the coordinator will conclude that a participant crashed, and will thus abort the transaction by multicasting a GLOBAL-ABORT message.

Now suppose the coordinator is blocked in state PRECOMMIT. On a timeout, it will conclude that one of the participants had crashed, but that participant is known to have voted for committing the transaction. Consequently, the coordinator can instruct the operational participants to commit by multicasting a GLOBAL-COMMIT message. In addition, it relies on a recovery protocol for the crashed participant to commit its part of the transaction when it comes up again.

A participant P may block in the READY state or in the PRECOMMIT state. On a timeout, P can conclude only that the coordinator has failed so that it now needs to find out what to do next. As in 2PC, if P contacts any other participant that is in state COMMIT (or ABORT), P should move to that state as well. In addition, if all participants are in state PRECOMMIT, the transaction can be committed.

Again analogous to 2PC, if another participant Q is still in the INIT state, the transaction can safely be aborted. It is important to note that Q can be in state INIT only if no other participant is in state PRECOMMIT. A participant can reach PRECOMMIT only if the coordinator had reached state PRECOMMIT before crashing, and has thus received a vote to commit from each participant. In other words, no participant can reside in state INIT while another participant is in state PRECOMMIT.

If each of the participants that P can contact is in state READY (and they together form a majority), the transaction should be aborted. The point to note is that another participant may have crashed and will later recover. However, neither P, nor any other of the operational participants knows what the state of the crashed participant will be when it recovers. If the process recovers to state INIT, then deciding to abort the transaction is the only correct decision. At worst, the process may recover to state PRECOMMIT, but in that case, it cannot do any harm to still abort the transaction.

This situation is the major difference with 2PC, where a crashed participant could recover to a COMMIT state while all the others were still in state READY. In that case, the remaining operational processes could not reach a final decision and would have to wait until the crashed process recovered. With 3PC, if any operational process is in its READY state, no crashed process will recover to a state other than INIT, ABORT, or PRECOMMIT. For this reason, surviving processes can always come to a final decision.

Finally, if the processes that P can reach are in state PRECOMMIT (and they form a majority), then it is safe to commit the transaction. Again, it can be shown that in this case, all other processes will either be in state READY or at least, will recover to state READY, PRECOMMIT, or COMMIT when they had crashed. More on 3PC can be found in [Bernstein et al., 1987].

8.6 Recovery

So far, we have mainly concentrated on algorithms that allow us to tolerate faults. However, once a failure has occurred, it is essential that the process where the failure happened can recover to a correct state. In what follows, we first concentrate on what it actually means to recover to a correct state, and subsequently when and how the state of a distributed system can be recorded and recovered to, through checkpointing and message logging.

8.6.1 Introduction

Fundamental to fault tolerance is the recovery from an error. Recall that an error is that part of a system that may lead to a failure. The whole idea of error recovery is to replace an erroneous state with an error-free state. There are essentially two forms of error recovery.

In **backward recovery**, the main issue is to bring the system from its present erroneous state back into a previously correct state. To do so, it will be necessary to record the system's state from time to time, and to restore such a recorded state when things go wrong. Each time (part of) the system's present state is recorded, a **checkpoint** is said to be made.

Another form of error recovery is **forward recovery**. In this case, when the system has entered an erroneous state, instead of moving back to a previous, checkpointed state, an attempt is made to bring the system to a correct new state from which it can continue to execute. The main problem with forward error recovery mechanisms is that it has to be known in advance which errors may occur. Only in that case, it is possible to correct those errors and move to a new state.

The distinction between backward and forward error recovery is easily explained when considering the implementation of reliable communication. The common approach to recover from a lost packet is to let the sender retransmit that packet. In effect, packet retransmission establishes that we attempt to go back to a previous, correct state, namely the one in which the packet that was lost is being sent. Reliable communication through packet retransmission is therefore an example of applying backward error recovery techniques.

An alternative approach is to use a method known as **erasure correction**. In this approach, a missing packet is constructed from other, successfully delivered packets. For example, in an (n,k)-block erasure code, a set of k *source packets* is encoded into a set of n *encoded packets*, such that *any* set of k encoded packets is enough to reconstruct the original k source packets. Typical values are $k = 16$ or $k = 32$, and $k < n \leq 2k$ (see, for example, Rizzo [1997]). If not enough packets have yet been delivered, the sender will have to continue transmitting packets until a previously lost packet can be constructed. Erasure correction is a typical example of a forward error recovery approach.

By and large, backward error recovery techniques are widely applied as a general mechanism for recovering from failures in distributed systems. The major benefit of backward error recovery is that it is a generally applicable method independent of any specific system or process. In other words, it can be integrated into (the middleware layer) of a distributed system as a general-purpose service.

However, backward error recovery also introduces some problems [Singhal and Shivaratri, 1994]. First, restoring a system or process to a previous state is generally a relatively costly operation in terms of performance. As will be discussed in succeeding sections, much work generally needs to be done to recover from, for example, a process crash or site failure. A potential way out of this problem is to devise very cheap mechanisms by which components are simply rebooted.

Second, because backward error recovery mechanisms are independent of the distributed application for which they are actually used, no guarantees can be given that once recovery has taken place, the same or similar failure will not happen again. If such guarantees are needed, handling errors often requires that the application gets into the loop of recovery. In other words, full-fledged failure transparency can generally not be provided by backward error recovery mechanisms.

Finally, although backward error recovery requires checkpointing, some states can simply never be rolled back to. For example, once a (possibly malicious) person has taken the $1,000 that suddenly came rolling out of the incorrectly functioning automated teller machine, there is only a small chance that money will be stuffed back in the machine. Likewise, recovering to a previous state in most Unix systems after having enthusiastically typed

```
/bin/rm -fr *
```

but from the wrong working directory, may turn a few people pale. Some things are simply irreversible.

Checkpointing allows the recovery to a previous, correct state. However, taking a checkpoint is often a costly operation and may have a severe performance penalty. As a consequence, many fault-tolerant distributed systems combine checkpointing with **message logging**. In this case, after a checkpoint has been taken, a process logs its messages before sending them off (called **sender-based logging**). An alternative solution is to let the receiving process first log an incoming message before delivering it to the application it is executing. This scheme is also referred to as **receiver-based logging**. When a receiving process crashes, it is necessary to restore the most recently checkpointed state, and from there on *replay* the messages that have been sent. Consequently, combining checkpoints with message logging makes it possible to restore a state that lies beyond the most recent checkpoint without the cost of checkpointing.

In a system where only checkpointing is used, processes will be restored

to a checkpointed state. From there on, their behavior may be different from it was before the failure occurred. For example, because communication times are not deterministic, messages may now be delivered in a different order, in turn leading to different reactions by the receivers. However, if message logging takes place, an actual replay of the events that happened since the last checkpoint takes place. Such a replay makes it easier to interact with the outside world.

For example, consider the case that a failure occurred because a user provided erroneous input. If only checkpointing is used, the system would have to take a checkpoint before accepting the user's input in order to recover to the same state. With message logging, an older checkpoint can be used, after which a replay of events can take place up to the point that the user should provide input. In practice, the combination of having fewer checkpoints and message logging is more efficient than having to take many checkpoints.

Elnozahy et al. [2002] provide a survey of checkpointing and logging in distributed systems. Various algorithmic details can be found in Chow and Johnson [1997].

8.6.2 Checkpointing

In a fault-tolerant distributed system, backward error recovery requires that the system regularly saves its state[2]. In particular, we need to record a consistent global state, also called a **distributed snapshot**. In a distributed snapshot, if a process P has recorded the receipt of a message, then there should also be a process Q that has recorded the sending of that message. After all, it must have come from somewhere.

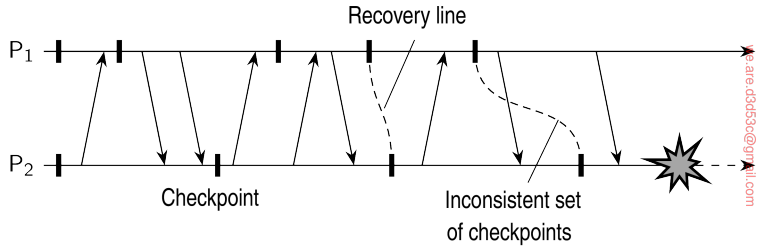


Figure 8.37: A recovery line.

To recover after a process or system failure requires that we construct a consistent global state from local states as saved by each process. In particular, it is best to recover to the *most recent* distributed snapshot, also referred to as a **recovery line**. In other words, a recovery line corresponds to the most recent consistent collection of checkpoints, as shown in Figure 8.37.

[2]We assume that each process has access to a local, reliable storage.

Coordinated checkpointing

In **coordinated checkpointing** all processes synchronize to jointly write their state to local storage. The main advantage of coordinated checkpointing is that the saved state is automatically globally consistent. A simple solution is to use a two-phase blocking protocol. A coordinator first multicasts a CHECKPOINT-REQUEST message to all processes. When a process receives such a message, it takes a local checkpoint, queues any subsequent message handed to it by the application it is executing, and acknowledges to the coordinator that it has taken a checkpoint. When the coordinator has received an acknowledgment from all processes, it multicasts a CHECKPOINT-DONE message to allow the (blocked) processes to continue.

It is easy to see that this approach will indeed lead to a globally consistent state because no incoming message will ever be registered as part of a checkpoint. This is because any message that follows a request for taking a checkpoint is not considered to be part of the local checkpoint. At the same time, outgoing messages (as handed to the checkpointing process by the application it is running), are queued locally until the CHECKPOINT-DONE message is received.

An improvement to this algorithm is to send a checkpoint request only to those processes that depend on the recovery of the coordinator, and ignore the other processes. A process is dependent on the coordinator if it has received a message that is directly or indirectly causally related to a message that the coordinator had sent since the last checkpoint. This leads to the notion of an **incremental snapshot**.

To take an incremental snapshot, the coordinator sends a checkpoint request only to those processes it had sent a message to since it last took a checkpoint. When a process P receives such a request, it forwards the request to all those processes to which P itself had sent a message since the last checkpoint, and so on. A process forwards the request only once. When all processes have been identified, a second multicast is used to actually trigger checkpointing and to let the processes continue where they had left off.

Independent checkpointing

Now consider the case in which each process simply records its local state from time to time in an uncoordinated fashion. To discover a recovery line, requires that each process is rolled back to its most recently saved state. If these local states jointly do not form a distributed snapshot, further rolling back is necessary. This process of a cascaded rollback may lead to what is called the **domino effect** and is shown in Figure 8.38.

When process P_2 crashes, we need to restore its state to the most recently saved checkpoint. As a consequence, process P_1 will also need to be rolled back. Unfortunately, the two most recently saved local states do not form a

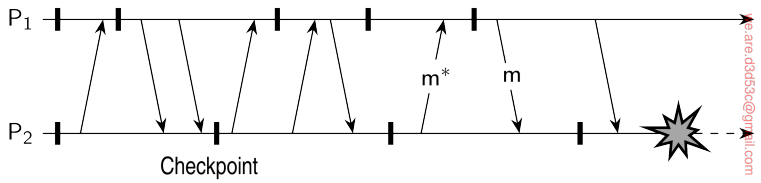


Figure 8.38: The domino effect.

consistent global state: the state saved by P_2 indicates the receipt of a message m, but no other process can be identified as its sender. Consequently, P_2 needs to be rolled back to an earlier state.

However, the next state to which P_2 is rolled back also cannot be used as part of a distributed snapshot. In this case, P_1 will have recorded the receipt of message m^*, but there is no recorded event of this message being sent. It is therefore necessary to also roll P_1 back to a previous state. In this example, it turns out that the recovery line is actually the initial state of the system.

As processes take local checkpoints independent of each other, this method is also referred to as **independent checkpointing**. Its implementation requires that dependencies are recorded in such a way that processes can jointly roll back to a consistent global state. To that end, let $CP_i(m)$ denote the mth checkpoint taken by process P_i. Also, let $INT_i(m)$ denote the interval between checkpoints $CP_i(m-1)$ and $CP_i(m)$.

When process P_i sends a message in interval $INT_i(m)$, it piggybacks the pair (i, m) to the receiving process. When process P_j receives a message in interval $INT_j(n)$, along with the pair of indices (i, m), it records the dependency $INT_i(m) \rightarrow INT_j(n)$. Whenever P_j takes checkpoint $CP_j(n)$, it additionally saves this dependency to its local storage, along with the rest of the recovery information that is part of $CP_j(n)$.

Now suppose that at a certain moment, process P_i is required to roll back to checkpoint $CP_i(m-1)$. To ensure global consistency, we need to ensure that all processes that have received messages from P_i and that were sent in interval $INT_i(m)$, are rolled back to a checkpointed state preceding the receipt of such messages. In particular, process P_j in our example, will need to be rolled back at least to checkpoint $CP_j(n-1)$. If $CP_j(n-1)$ does not lead to a globally consistent state, further rolling back may be necessary.

Calculating the recovery line requires an analysis of the interval dependencies recorded by each process when a checkpoint was taken. It turns out that such calculations are fairly complex. In addition, as it turns out, it is often not the coordination between processes that is the dominating performance factor, but the overhead as the result of having to save the state to local stable storage. Therefore, coordinated checkpointing, which is much simpler than independent checkpointing, is often more popular, and will presumably stay so even when systems grow to much larger sizes [Elnozahy and Plank, 2004]

8.6.3 Message logging

Considering that checkpointing can be an expensive operation, techniques have been sought to reduce the number of checkpoints, but still enable recovery. One such technique is logging messages. The basic idea underlying message logging is that if the transmission of messages can be *replayed*, we can still reach a globally consistent state, but without having to restore that state from local storage. Instead, a checkpointed state is taken as a starting point, and all messages that have been sent since are simply retransmitted and handled accordingly.

This approach works fine under the assumption of what is called a **piecewise deterministic execution model**. In such a model, the execution of each process is assumed to take place as a series of intervals in which events take place. These events are the same as those discussed in the context of Lamport's happened-before relationship in Section 5.2.1. For example, an event may be the execution of an instruction, the sending of a message, and so on. Each interval in the piecewise deterministic model is assumed to start with a nondeterministic event, such as the receipt of a message. However, from that moment on, the execution of the process is completely deterministic. An interval ends with the last event before a nondeterministic event occurs.

In effect, an interval can be replayed with a known result, that is, in a completely deterministic way, provided it is replayed, starting with the same nondeterministic event as before. Consequently, if we record all nondeterministic events in such a model, it becomes possible to completely replay the entire execution of a process in a deterministic way.

Considering that message logs are necessary to recover from a process crash so that a globally consistent state is restored, it becomes important to know precisely when messages are to be logged. Following the approach described by Alvisi and Marzullo [1998], it turns out that many existing message-logging schemes can be easily characterized if we concentrate on how they deal with orphan processes.

An **orphan process** is a process that survives the crash of another process, but whose state is inconsistent with the crashed process after its recovery. As an example, consider the situation shown in Figure 8.39. Process Q receives messages m_1 and m_2 from process P and R, respectively, and subsequently sends a message m_3 to R. However, in contrast to all other messages, message m_2 is not logged. If process Q crashes and later recovers again, only the logged messages required for the recovery of Q are replayed, in our example, m_1. Because m_2 was not logged, its transmission will not be replayed, meaning that the transmission of m_3 may also not take place.

However, the situation after the recovery of Q is inconsistent with that before its recovery. In particular, R holds a message (m_3) that was sent before the crash, but whose receipt and delivery do not take place when replaying what had happened before the crash. This should obviously be avoided.

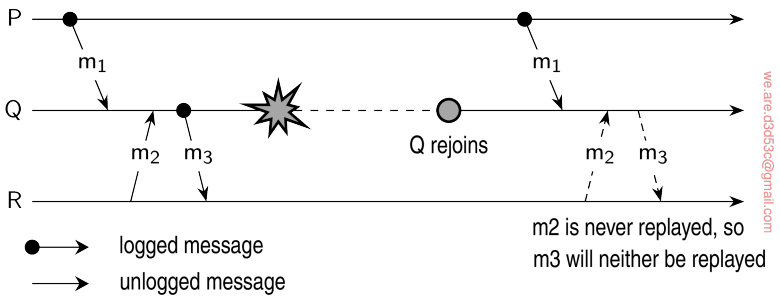


Figure 8.39: Incorrect replay of messages after recovery, leading to an orphan process R.

Note 8.12 (Advanced: Characterizing message-logging schemes)
To characterize different message-logging schemes, we follow the approach described by Alvisi and Marzullo [1998]. Each message m is considered to have a header that contains all information necessary to retransmit m, and to properly handle it. For example, each header will identify the sender and the receiver, but also a sequence number to recognize it as a duplicate. In addition, a delivery number may be added to decide when exactly it should be handed over to the receiving application.

A message is said to be *stable* if it can no longer be lost, for example because it has been written to reliable, local storage. Stable messages can thus be used for recovery by replaying their transmission.

Each message m leads to a set **DEP**(m) of processes that depend on the delivery of m. In particular, **DEP**(m) consists of those processes to which m has been delivered. In addition, if another message m^* is causally dependent on the delivery of m, and m^* has been delivered to a process Q, then Q will also be contained in **DEP**(m). Note that m^* is causally dependent on the delivery of m, if it was sent by the same process that previously delivered m, or which had delivered another message that was causally dependent on the delivery of m.

The set **COPY**(m) consists of those processes that have a copy of m, but have not (yet) reliably stored it. When a process Q delivers message m, it also becomes a member of **COPY**(m). Note that **COPY**(m) consists of those processes that could hand over a copy of m that can be used to replay the transmission of m. If all these processes crash, replaying the transmission of m is clearly not feasible.

Using these notations, it is now easy to define precisely what an orphan process is. Let **FAIL** denote the collection of crashed processes, and assume Q is one of the survivors. Q is an orphan process if there is a message m, such that Q is contained in **DEP**(m), while at the same time every process in **COPY**(m) has crashed. More formally:

$$Q \text{ is orphaned} \Leftrightarrow \exists m : Q \in \mathbf{DEP}(m) \text{ and } \mathbf{COPY}(m) \subseteq \mathbf{FAIL}$$

In other words, an orphan process appears when it is dependent on m, but there

is no way to replay m's transmission.

To avoid orphan processes, we thus need to ensure that if each process in **COPY**(m) crashed, then no surviving process is left in **DEP**(m). In other words, all processes in **DEP**(m) should have crashed as well. This condition can be enforced if we can guarantee that whenever a process becomes a member of **DEP**(m), it also becomes a member of **COPY**(m). In other words, whenever a process becomes dependent on the delivery of m, it will keep a copy of m.

There are essentially two approaches that can now be followed. The first approach is represented by what are called **pessimistic logging protocols**. These protocols take care that for each *nonstable* message m, there is at most one process dependent on m. In other words, pessimistic logging protocols ensure that each nonstable message m is delivered to at most one process. Note that as soon as m is delivered to, say, process P, P becomes a member of **COPY**(m).

The worst that can happen is that process P crashes without m ever having been logged. With pessimistic logging, P is not allowed to send any messages after the delivery of m without first having ensured that m has been written to reliable storage. Consequently, no other processes will ever become dependent on the delivery of m to P, without having the possibility of replaying the transmission of m. In this way, orphan processes are always avoided.

In contrast, in an **optimistic logging protocol**, the actual work is done *after* a crash occurs. In particular, assume that for some message m, each process in **COPY**(m) has crashed. In an optimistic approach, any orphan process in **DEP**(m) is rolled back to a state in which it no longer belongs to **DEP**(m). Clearly, optimistic logging protocols need to keep track of dependencies, which complicates their implementation.

As pointed out by Elnozahy et al. [2002], pessimistic logging is so much simpler than optimistic approaches, that it is the preferred way of message logging in practical distributed systems design.

8.7 Summary

Fault tolerance is an important subject in distributed systems design. Fault tolerance is defined as the characteristic by which a system can mask the occurrence and recovery from failures. In other words, a system is fault tolerant if it can continue to operate in the presence of failures.

Several types of failures exist. A crash failure occurs when a process simply halts. An omission failure occurs when a process does not respond to incoming requests. When a process responds too soon or too late to a request, it is said to exhibit a timing failure. Responding to an incoming request, but in the wrong way, is an example of a response failure. The most difficult failures to handle are those by which a process exhibits any kind of failure, called arbitrary or Byzantine failures.

Redundancy is the key technique needed to achieve fault tolerance. When applied to processes, the notion of process groups becomes important. A

process group consists of a number of processes that closely cooperate to provide a service. In fault-tolerant process groups, one or more processes can fail without affecting the availability of the service the group implements. Often, it is necessary that communication within the group be highly reliable, and adheres to stringent ordering and atomicity properties to achieve fault tolerance.

The real problem is that members of a process group need to reach consensus in the presence of various failures. Paxos is by now a well-established and highly robust consensus algorithm. By using $2k + 1$ servers, it can establish k-fault tolerance. However, we need a total of $3k + 1$ servers if it is needed to deal with arbitrary failures.

Reliable group communication, also called reliable multicasting, comes in different forms. As long as groups are relatively small, it turns out that implementing reliability is feasible. However, as soon as large groups have to be supported, scalability of reliable multicasting becomes problematic. The key issue in achieving scalability is to reduce the number of feedback messages by which receivers report the (un)successful receipt of a multicasted message.

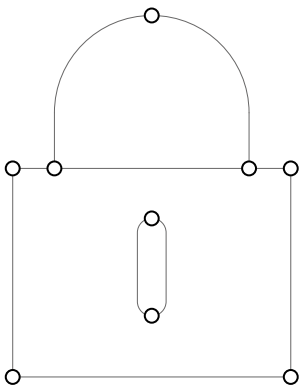
Matters become worse when atomicity is to be provided. In atomic multicast protocols, it is essential that each group member has the same view concerning to which members a multicasted message has been delivered. Atomic multicasting can be precisely formulated in terms of a virtual synchronous execution model. In essence, this model introduces boundaries between which group membership does not change and which messages are reliably transmitted. A message can never cross a boundary.

Group membership changes are an example where each process needs to agree on the same list of members. Such agreement can be reached through a commit protocol, of which the two-phase commit protocol is the most widely applied. In a two-phase commit protocol, a coordinator first checks whether all processes agree to perform the same operation (i.e., whether they all agree to commit), and in a second round, multicasts the outcome of that poll. A three-phase commit protocol is used to handle the crash of the coordinator without having to block all processes to reach agreement until the coordinator recovers.

Recovery in fault-tolerant systems is invariably achieved by checkpointing the state of the system on a regular basis. Checkpointing is completely distributed. Unfortunately, taking a checkpoint is an expensive operation. To improve performance, many distributed systems combine checkpointing with message logging. By logging the communication between processes, it becomes possible to replay the execution of the system after a crash has occurred.

09

SECURITY



What makes a distributed system secure? Answering this question can easily fill entire books, and can certainly not be fully addressed in a single chapter. One of the challenges in making systems secure is that we need to address a negative goal [Saltzer and Kaashoek, 2009]: we need to protect a system against all unauthorized actions. Proving or demonstrating that this goal has been reached is generally (close to) impossible. Much easier is to show that a positive goal has been accomplished, for example, showing that Alice can log in to a system.

In this chapter, we aim to provide a brief introduction into secure distributed systems. We do so by mainly concentrating on two important mechanisms: authentication and authorization. Authentication is all about ensuring that you are dealing with people and devices that have been properly identified before providing access to a system. Authorization is all about ensuring that identified entities, having access to the system, can perform only those operations they are allowed to execute.

Yet, authentication and authorization are not enough. It is equally important to ensure that operations are carried out correctly: have they affected data in a way that may leave the system in an inconsistent state, has the request to execute an operation not been tampered with, etc.? This aspect of ensuring integrity, or at the very least identifying that integrity has been affected, is essential for any secure system and will be discussed as well.

This brings us to perhaps one of the most discussed topics in the last decade: confidentiality, or, to keep it simple, ensuring that information is kept private. Achieving confidentiality time and again turns out to be a difficult subject, not in the least because keeping information private depends so much on the context of that information. For example, in the European's General Data Protection Regulation (GDPR) it is stated that without explicit consent, no data set should allow for the identification of an individual. However, realizing this otherwise indisputable reasonable requirement turns out to be technically a very demanding task in general, yet solutions for specific situations have been developed. Understanding the context turns out to be important, and as we shall see in this chapter, this actually holds for many situations when trying to secure a distributed system.

Realizing security mechanisms and establishing confidentiality and integrity relies heavily on cryptography, which is a huge field in itself. Nevertheless, explaining the basic principles of cryptographic protocols necessary for distributed systems is actually not too difficult and are also discussed in this chapter.

9.1 Introduction to security

We start our description of security in distributed systems by taking a look at some general security issues. First, it is necessary to define what a secure

system is. We distinguish security *policies* from security *mechanisms*. Our second concern is to consider some general design issues for secure systems. Finally, we briefly discuss some cryptographic algorithms, which play a key role in the design of security protocols.

9.1.1 Security threats, policies, and mechanisms

Security in a computer system is strongly related to the notion of dependability. Informally, a dependable computer system is one that we justifiably trust to deliver its services [Laprie, 1995]. Dependability includes availability, reliability, safety, and maintainability. However, if we are to put our trust in a computer system, then confidentiality and integrity should also be considered. **Confidentiality** refers to the property that information is disclosed only to authorized parties. **Integrity** is the characteristic that alterations to a system's assets can be made only in an authorized way, ensuring accuracy and completeness during the lifetime of assets. In other words, improper alterations in a secure computer system should be detectable and recoverable. Major assets of any computer system are its hardware, software, and data (see also Beckers et al. [2015]).

Another way of looking at security in computer systems is that we attempt to protect the services and data it offers against **security threats**. Following Saltzer and Kaashoek [2009] we distinguish the following three broad classes of threats:

1. Unauthorized information disclosure
2. Unauthorized information modification
3. Unauthorized denial of use

In all these cases, we explicitly mention unauthorized actions: actions taken by people, software, or devices (and generally as a combination) that are, simply put, not allowed, or have not been intended to be allowed. To what extent allowing or disallowing actions has been sufficiently expressed or enforced is less important for qualifying a security threat as such: the intent is sufficient for now.

Each of these three classes of threats relate directly to confidentiality, integrity, and availability, respectively.

Simply stating that a system should be able to protect itself against all possible security threats is not the way to actually build a secure system. What is first needed is a description of security requirements, that is, a security policy. A **security policy** describes precisely which actions the entities in a system are allowed to take and which ones are prohibited. Entities include users, services, data, machines, and so on. We will not go into the many aspects of formulating security policies. A good introduction is given by Bishop [2019]. Once a security policy has been laid down, it becomes

possible to concentrate on the **security mechanisms** by which a policy can be enforced. We distinguish the following four important security mechanisms:

1. Encryption
2. Authentication
3. Authorization
4. Monitoring and auditing

Encryption is fundamental to computer security. **Encryption** allows us to transform data into something an attacker cannot understand. In other words, encryption provides a means to implement data confidentiality. In addition, encryption allows us to check whether data have been modified. It thus also provides support for integrity checks.

Authentication is used to verify the claimed identity of a user, client, server, host, device, and so on. In the case of clients, the basic premise is that before a service starts to perform any work on behalf of a client, the service must learn the client's identity (unless the service is available to all). Typically, users are still authenticated through passwords, but there are many other ways to authenticate clients.

After a client has been authenticated, it is necessary to check whether that client is **authorized** to perform the action requested. Access to records in a medical database is a typical example. Depending on who accesses the database, permission may be granted to read records, to modify certain fields in a record, or to add or remove a record.

Monitoring and auditing tools are used to trace accesses to assets, such as which clients are accessing or have accessed what, and in which way. It has become increasingly important to not just record what has happened, but to continuously monitor what is going on in a system with the aim to detect any unauthorized intrusions. With the rise of advanced machine-learning techniques, we are witnessing increasingly sophisticated intrusion detection systems, which we shall discuss further in Section 9.6. Although auditing does not really provide any protection against security threats, audit logs can be extremely useful for the analysis of a security breach, and subsequently taking measures against intruders. For this reason, attackers are generally keen not to leave any traces that could eventually lead to exposing their identity. In this sense, logging accesses makes attacking sometimes a riskier business.

9.1.2 Design issues

Security principles

A distributed system, or any computer system for that matter, must provide security services by which a wide range of security policies can be implemented. There are a number of important design issues that need to be considered

when implementing general-purpose security services. Various **design principles** have been devised over the years, starting with the influential list of eight such principles by Saltzer and Schroeder [1975]. Almost 40 years later, Smith [2012] concluded that many still thrive:

- Fail-safe defaults
- Open design
- Separation of privilege
- Least privilege
- Least common mechanism

Fail-safe defaults are assuming that any set of defaults will generally not be changed. Perhaps the most infamous example is the initial *(user,password)* combination "(admin, admin)" or something similar, for edge devices. In many cases, the password is never changed, making edge devices the source of countless security breaches. This principle states that defaults should already guarantee a good degree of protection. In particular, access decisions should be based on permissions, not exclusions. For edge devices, it could mean that each should be shipped with a default, unique strong password (which has become common practice by now when purchasing and installing a new modem, for example).

The principle of **open design** is all about *not* applying security by obscurity: it is essential that every aspect of a distributed system is open for review. Everyone should be able to see which mechanisms are being used, how they are being used, how they have been implemented, etc. An obvious problem with this principle for modern distributed systems, is that many of them have become so immense complex that a mere inspection of design and implementation is no longer a guarantee for openness. Good documentation next to the continuous focus on keeping systems and their components as simple as possible are key. Unfortunately, simplicity is increasingly becoming more of an ideal state than something that most engineers can realize.

Separation of privilege is about ensuring that truly critical aspects of a system can never be fully controlled by just a single entity. A top-secret file may need to be double encrypted, with keys in the hands of two different people. Likewise, shutting down critical services may need to be controlled by two authorized administrators, each having their own keys to push the (digital) button.

The principle of **least privilege** states that a process should operate with the fewest possible privileges. As a practical example, in Unix systems, most processes (and thus users) cannot execute operations that are intended to be executed by the root. It is therefore needed to explicitly execute the sudo operation, after which one would normally need to provide a password in order to gain administrative rights.

Finally, the **least common mechanism** refers to designing systems in such a way that if multiple components require the same mechanism, then they should all be offered the same implementation of that mechanism. An important motivation for this principle is that maintenance and simplicity is much easier to achieve with a single implementation than having a mechanism spread across the system, possibly with different implementations. A typical example of this approach is that all users have access to the same library (implementation).

Saltzer and Schroeder [1975] also mentioned three other principles, including economy of mechanism, complete mediation, and psychological acceptability. Economy of mechanism means making systems as simple as possible. Although this is still an excellent goal, achieving it is often close to impossible considering how modern distributed systems are, literally, composed. Nevertheless, it also sometimes seems that designers have let go of this principle entirely. Accepting that systems can become complex in an almost uncontrolled fashion does not mean, however, that striving for simplicity should be abandoned.

Complete mediation means that every access to every object must be checked when it comes to whether it is allowed. With the complex, distributed nature of modern systems, it has become extremely difficult to even figure out how and where an adversary has entered a system.

Finally, psychological acceptance often translates to building the appropriate user interfaces so that people can do only the right thing. It has a lot to do with building a mental model of how a system is operating, and subsequently operate according to that model. Smith [2012] may be correct in stating that this aspect is receiving more attention than before, but that further advancements are simply hindered by costs. This may change, however, as organizations are increasingly becoming aware of how vulnerable they are, only because their employees find it so difficult to work with modern information systems.

Layering of security mechanisms

An important issue in designing secure systems is to decide at which level security mechanisms should be placed. A level in this context is related to the logical organization of a system into a number of layers. For example, computer networks are often organized into layers following some reference model, as we discussed in Section 4.1.1. In Section 2.2, we introduced the organization of distributed systems consisting of separate layers for applications, middleware, operating system services, and the operating system kernel. Combining the two organizations leads roughly to what is shown in Figure 9.1.

Typically, what we have shown as low-level protocols encompass general network-security solutions. An often-used solution is to set up a **virtual**

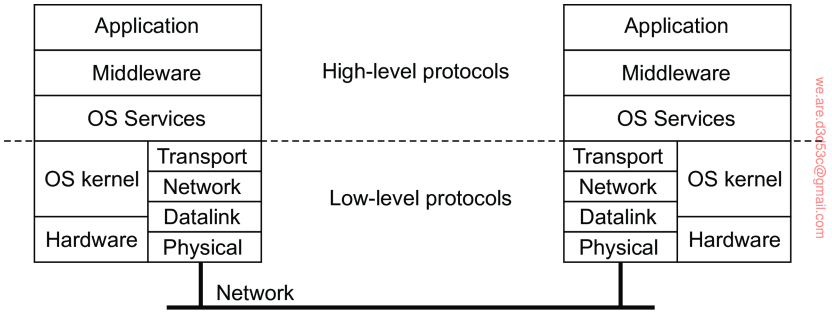


Figure 9.1: The logical organization of a distributed system into several layers.

private network, generally abbreviated to **VPN**. In essence, a VPN sets up a **tunnel** between two remote local networks, or between a host and a remote network. The effect is that a computer on one network seems to be directly connected to the remote network. Although not strictly necessary, a VPN normally encrypts all traffic between its end points and operates at the level of the data link layer or the network layer, as we discussed in Section 4.1.1.

Going higher up the network stack, a well-known example is the **transport-layer security** (**TLS**) service, which can be used to securely send messages across a TCP connection. TLS forms the standard approach for secure communication with Websites employing HTTPS. HTTPS establishes an authenticated connection to a Website through which all data exchanged between a client and the server is encrypted.

Where VPNs and secure Website communication can be considered as operating at the level of networking protocols, this is less obvious for protocols such as SSH. The **Secure Shell Protocol** (**SSH**) is used to establish a secure remote login. As such, one can argue that it operates as an operating-system service. Unlike HTTPS (which formally operates at the application level in terms of networking stacks), SSH comes with its own security protocol and does not rely on a transport-layer security service.

One of the main reasons for introducing middleware was to offer services that are useful for a wide range of distributed applications, while at the same being independent of specific operating systems. One example that we will discuss in detail later is that of an authentication service such as **Kerberos**. The service is indeed independent of a specific operating system (Kerberos has been implemented for a wide range of operating systems, including Windows, many Unix versions, and other non-Unix systems). Other examples of secure middleware services include secure RPC, secure Web servers (think of WordPress), as well as a range of secure databases.

Finally, an approach that is increasingly being adopted is developing **end-to-end application-level security**. In these cases, all secure communication

within the distributed application is handled by the application itself. There is no reliance on middleware security services. A well-known example is formed by modern user-messaging services like WhatsApp, Signal, and Telegram. Likewise, distributed applications that, for example, make use of cloud storage services, may be configured to first encrypt any data before sending it off to the cloud, as is done with BoxCryptor.

What these examples illustrate is that there are choices to be made concerning where and which security mechanisms will be deployed. Obviously, application-level security need generally not rely on lower-level security guarantees, yet the price to pay is that all mechanisms need to be deployed and maintained by the application itself. This is not always the best choice, as it does make sense to separate application functionality from what is needed regarding security. Just imagine automating your own backup services for a collection of remote computers. Making use of available operating-system level services for secure execution of remote commands and file transfers makes life a lot easier.

Trusted Computing Base

Dependencies between services regarding trust lead to the notion of a **trusted computing base** (**TCB**). A TCB is the set of all security mechanisms in a (distributed) computer system that are necessary and sufficient to enforce a security policy, and that thus need to be trusted [Committee on National Security Systems, 2015]. The TCB encompasses firmware, hardware, software, as well as humans. The smaller the TCB, the better. The idea is that if the TCB can be clearly identified, we at least know what to concentrate on when scrutinizing a computer system to state that it is secure.

Identifying the TCB means identifying the modules that need to be trusted, and those that do not. Unfortunately, identifying the trusted modules is not that simple, nor is it even that simple to design a system such that there is always a clear distinction between trusted and untrusted modules.

For example, it is clear that a program needing administrative privileges to be executed will be part of the TCB, for the simple reason that if that program is compromised, an attacker may be able to do much more than what that program was designed for. But what about a Web-based service that facilitates storage on a local server, such as ownCloud? Such a service is typically implemented as a collection of scripts that are executed by a process spawned by the Apache server. The spawned process runs entirely without any special privileges, yet the collection of scripts also include code for adding and removing users (separate from the users known to the underlying operating system). Are the set of scripts that make up this service to be trusted or not? Or more specifically, which scripts from this set are to be trusted? Clearly, the answer to this question depends on the security specifications for, in this case, the ownCloud service. If the policy states that users should be

fully separated, then the scripts for user management and access control will need to be inspected, but also the dependency of those scripts (and perhaps others) on other modules that ownCloud depends on (such as a mysql database and the Apache server).

The ownCloud example illustrates that it may not be that easy to separate trusted from untrusted modules, especially when no security policy has been explicitly specified, or is perhaps otherwise incomplete. The same may hold when designing for security. For example, often we assume a so-called **honest-but-curious** server: a server that behaves according to some protocol, but in the worst case keeps track of all the things it does [Goldreich, 2009]. Making such an assumption may be fine, but what measures have been taken that justify such an assumption? Equally important, are those measures sufficient? When components or modules are complex, it may be very difficult to establish whether a module actually satisfies its specifications and to what extent we may need to classify it as trusted or not.

Privacy

If there is one thing clear when it comes to designing for secure distributed systems, then it is the role of data. In this era of data and notably the effect it has on the personal lives of many, protecting privacy has become a major issue for distributed systems. Privacy and confidentiality are closely related, yet are different. Following Francis [2008], one could state that privacy is *invaded*, whereas confidentiality is *breached*. Important is to recognize that just ensuring confidentiality, that is, data and information is not disclosed in any unauthorized manner, is not enough to guarantee privacy. In this light, one can understand why Nissenbaum [2010] states that the right to privacy is about "a right to *appropriate* flow of personal information." In other words, controlling who gets to see what, when, and how. This also implies, for example, that a person should be able to stop and revoke a flow of personal information.

Discussions on how to design distributed systems that meet such privacy requirements have only recently started to mature. One approach is to concentrate on ownership, such as done in Solid [Sambra et al., 2016], which essentially puts control on personal information at a single person. To what extent this will be sufficient remains to be seen. A point of debate is whether ownership by itself is sufficient to provide the necessary privacy guarantees.

Another approach is to look at what regulations state and to what extent it is possible to design systems that comply to those regulations. Important in this respect is the **General Data Protection Regulation**, or simply **GDPR** [Voigt and Von dem Bussche, 2017], which was installed in 2016. The GDPR is a comprehensive set of regulations aiming to protect personal data. As with many regulations, it is yet unclear to what extent one can develop a truly GDPR-compliant distributed system, but it can be argued that making

a system GDPR-compliant after the fact may be much more difficult, if not impossible.

Like Solid, Schwarzkopf et al. [2019] follow an approach in which a user has a separate, privately owned and controlled database with personal information, denoted as a **shard**. Acknowledging that the real challenge lies in keeping track of how personal information flows into other systems, the authors argue for designing a distributed system by considering it to consist of as being a large dataflow computation. A user can contribute her shard to the system, but equally remove it and along with that removal, revoke participation in all related data flows.

As an example, consider a collection of users, each maintaining their own shard. A shard represents a user's contributions to an online social network with three elements:

- Some personal information, such as name, organization, etc.
- A set of connections, each connection referring to another shard.
- A set of posts, as is common in many social networks, often as a reaction to another post.

An application can be developed that builds a tree consisting of posts and reactions to posts. The development of that tree starts with a single, new post P and implements the function `react` taking the original post P and a (dynamic) list of reactions $R_1, R_2, \ldots, R_n$ as input. Each reaction is, of course, also a post and can be reacted to. When Alice at a certain point decides she wants to completely withdraw from the discussion, she can simply withdraw her post(s), leading to a new tree. All reactions to her posts may necessarily be removed as well, as each of her posts operates as the root of a branch. This can be done automatically, assuming that the function `react` is continuously updated on its inputs, and we assume that removal of the post that led to reactions, recursively leads to the removal of the entire branch.

More intricate examples are described by Agarwal et al. [2022], yet the essence is that removal of an element as part of a dataflow leads to an update of that flow but without the personal information represented by that element. The example above also illustrates that we may not be out of the woods yet, for what happens if Alice is explicitly mentioned in one of the reactions? In other words, more needs to be done to ensure that all references to Alice and her posts are indeed removed.

This example already illustrates that taking a database perspective may help in designing GDPR-compliant systems. Shastri et al. [2020] show that such a perspective may indeed be fruitful, yet the real challenge is maintaining a large amount of metadata, as shown in Figure 9.2. The authors come to the conclusion that extending databases so that a system can meet GDPR requirements is doable from a functional point of view, but has serious adverse effect on performance and, in turn, on the scalability of solutions.

GDPR regulation	Impact on database systems	
	Attributes	Actions
Collect data for explicit purposes	Purpose	Metadata indexing
Do not store data indefinitely	TTL	Timely deletion
Inform customers about GDPR metadata associated with their data	Purpose, TTL, Origin, Sharing	Metadata indexing
Allow customers to access their data	Person id	Metadata indexing
Allow customers to erase their data	TTL	Timely deletion
Do not use data for objected reasons	Objections	Metadata indexing
Allow customers to withdraw from algorithmic decision-making	Automated decisions	Metadata indexing
Safeguard and restrict access to data		Access control
Do not grant unlimited access to data		Access control
Audit operations on personal data	Audit trail	Monitor and log
Implement appropriate data security		Encryption
Share audit trails from affected systems	Audit trail	Monitor and log

Figure 9.2: The effects on (distributed) databases when designing for GDPR compliance (adapted from [Shastri et al., 2020]).

9.2 Cryptography

Let us now draw attention to an essential mechanism in developing secure distributed systems, namely the application of cryptographic solutions. In the following, we first pay attention to some basic cryptographic functions, including symmetric and asymmetric cryptosystems, and secure hashing, to then continue with managing security.

9.2.1 Basics

Fundamental to security in distributed systems is the use of cryptographic techniques. The basic idea of applying these techniques is simple. Consider a sender S wanting to transmit message m to a receiver R. To protect the message against security threats, the sender first **encrypts** it into an unintelligible message m', and subsequently sends m' to R. R, in turn, must **decrypt** the received message into its original form m.

Encryption and decryption are accomplished by using cryptographic methods parameterized by keys, as shown in Figure 9.3. The original form of the message that is sent is called the **plaintext**, shown as P in Figure 9.3. The encrypted form is referred to as the **ciphertext**, illustrated as C.

To describe the various security protocols that are used in building security services for distributed systems, it is useful to have a notation to relate

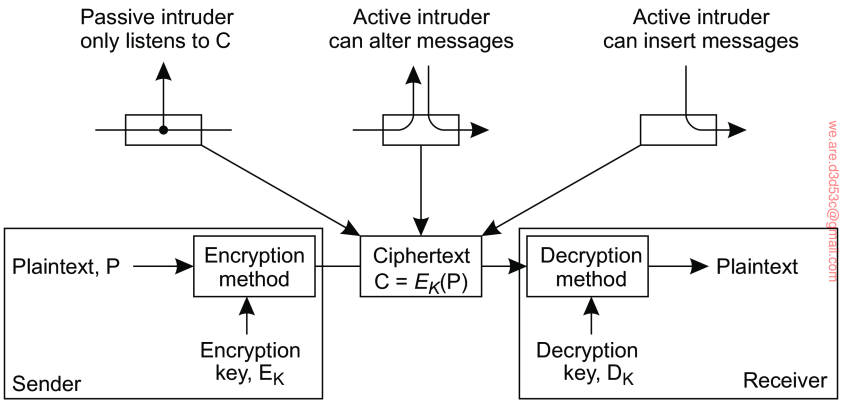


Figure 9.3: Intruders and eavesdroppers in communication.

plaintext, ciphertext, and keys. Following common notational conventions, we will use $C = E_K(P)$ to denote that the ciphertext C is obtained by encrypting the plaintext P using key E_K. Likewise, $P = D_K(C)$ is used to express the decryption of the ciphertext C using key D_K, resulting in the plaintext P.

Returning to our example shown in Figure 9.3, while transferring a message as ciphertext C, there are three different attacks that we need to protect against, and for which encryption helps. First, an **intruder** may intercept the message without either the sender or receiver being aware that eavesdropping is happening. Of course, if the transmitted message has been encrypted in such a way that it cannot be easily decrypted without having the proper key, interception is useless: the intruder will see only unintelligible data. (By the way, the fact alone that a message is being transmitted may sometimes be enough for an intruder to draw conclusions. For example, if during a world crisis, the amount of traffic into the White House suddenly drops dramatically while the amount of traffic going into a certain mountain in Colorado increases, there may be useful information in knowing that.)

The second type of attack that needs to be dealt with is that of modifying the message. Modifying plaintext is easy; modifying ciphertext that has been properly encrypted is much more difficult because the intruder will first have to decrypt the message before it can be meaningfully modified. In addition, the intruder will also have to properly encrypt it again, or otherwise the receiver may notice that the message has been tampered with.

The third type of attack is when an intruder inserts encrypted messages into the communication system, attempting to make R believe these messages came from S. Again, encryption can help protect against such attacks. Note that if an intruder can modify messages, she can also insert messages.

The art and science of devising algorithms for cryptographic systems has a long and fascinating history [Kahn, 1967], and building secure systems is

often surprisingly difficult. It is beyond the scope of this book to discuss any of these algorithms in detail. Information on cryptographic algorithms can be found in [Ferguson et al., 2010] and [Smart, 2016].

9.2.2 Symmetric and asymmetric cryptosystems

There is a fundamental distinction between different cryptographic systems, based on whether the encryption and decryption key are the same. In a **symmetric cryptosystem**, the same key is used to encrypt and decrypt a message:

$$\textit{if } \mathsf{P} = D_K(E_K(\mathsf{P})) \textit{ then } \mathsf{D_K} = \mathsf{E_K}.$$

Symmetric cryptosystems are also referred to as **secret-key** or **shared-key systems** because the sender and receiver are required to share the same key, and to ensure that protection works, this shared key must be kept secret; no one else is allowed to see the key. We will use the notation $\mathsf{K_{A,B}}$ to denote a key shared by A and B.

In an **asymmetric cryptosystem**, the keys for encryption and decryption are different, but together form a unique pair. In other words, there is a separate key $\mathsf{E_K}$ for encryption and one for decryption, $\mathsf{D_K}$, such that

$$\textit{if } \mathsf{P} = D_K(E_K(\mathsf{P})) \textit{ then } \mathsf{D_K} \neq \mathsf{E_K}.$$

One of the keys in an asymmetric cryptosystem is kept private; the other is made public. For this reason, asymmetric cryptosystems are also referred to as **public-key systems**. In what follows, we use the notation $\mathsf{PK_A}$ to denote a public key belonging to A, and $\mathsf{SK_A}$ as its corresponding private (i.e., secret) key.

Which one of the encryption or decryption keys that is actually made public depends on how the keys are used. For example, if Alice wants to send a confidential message m to Bob, she should use Bob's public key to encrypt the message. Because Bob is the only one holding the associated and private decryption key, he is also the only person who can decrypt the message:

$$\textit{if } \mathsf{m} \textit{ is to be kept private: } \mathsf{C} = PK_B(\mathsf{m}).$$

On the other hand, suppose that Bob wants to know for sure that the message he just received actually came from Alice. In that case, Alice can keep her encryption key private to encrypt the messages she sends. If Bob can successfully decrypt a message using Alice's public key (and the plaintext in the message has enough information to make it meaningful to Bob), he knows that message must have come from Alice because the decryption key is uniquely tied to the encryption key:

$$\textit{if } \mathsf{m} \textit{ is to be authenticated: } \mathsf{C} = SK_A(\mathsf{m}).$$

We return to digitally signing messages shortly.

An aspect of asymmetric cryptosystems that is becoming increasingly important is that of **homomorphic encryption**. What it boils down to is that when using homomorphic encryption, mathematical operations on plaintext can also be performed on the corresponding ciphertext. In particular, if x and y are two numbers, then

$$E_K(x) \star E_K(y) = E_K(x \star y)$$

where "$\star$" is some mathematical operation, typically an addition or multiplication. The importance of homomorphic encryption cannot be underestimated. By now, we understand what it takes to store data safely on a remote, untrusted server: simply encrypt it with a public key before sending it to the server. This is what tools such as BoxCryptor do. However, whenever stored data needs to be operated on, servers generally require that the data is available as plaintext. Homomorphic encryption changes that: a server can apply operations on encrypted data and the result (also automatically encrypted with the same key) can be safely stored as well, or used in other computations.

There is only one problem with homomorphic encryption: its general form (known as **full homomorphic encryption** or simply **FHE**) is performance-wise slow. FHE supports addition and multiplication, which is enough to implement any mathematical operation. Currently, FHE implementations are often too slow to be generally applicable. In contrast, there are also **partial homomorphic encryption** (**PHE**) schemes, which support either addition or multiplication, for which efficient implementations exist. However, PHE schemes do require often very specific applications. For a good overview of homomorphic encryption, see Acar et al. [2018].

Note 9.1 (Example: Counting pedestrians over time and space)
As a practical and advanced example of how homomorphic encryption can be applied, consider the following. In many cities, it has become common practice to monitor pedestrian behavior by simply picking up Wi-Fi signals from carry-on devices, notably Wi-Fi–enabled smartphones. Most of these signals are network packets carrying the unique MAC address of the device. By extracting the MAC address as a device identifier, it becomes possible to track movements. That by itself need not necessarily be a problem, but without having explicit consent from a device owner, it is simply a serious infringement of privacy.

Interestingly, many cities are not interested in actually tracking individuals. They are interested in simple questions such as how many people moved from one location to another, or what is the average time that people stayed at a specific location. Such questions are easy to answer when you know the identifier of a device, yet become much more difficult when keeping those identifiers long enough to come to an answer, is simply forbidden.

Stanciu et al. [2020] propose a solution that allows to count devices over time and space without the need to keep device identifiers. A setup consists of a collection of sensors, each sensor essentially being a Wi-Fi–packet sniffer.

During a short epoch (typically a few minutes), a sensor picks up Wi-Fi packets, extracts MAC addresses, and subsequently stores (a pseudonymized version) of each MAC address in a so-called Bloom filter. A Bloom filter is a vector of m bits, initially all set to 0 [Bloom, 1970]. Using a collection of k hash functions, an element x is added to a Bloom filter by setting the position $h_i(x)$ to 1 for each of the k hash functions. Each element is thus represented as an m-bit vector consisting of exactly k bits set to 1. More elements are added by performing a bitwise OR operation each time an element is added.

A Bloom filter has the important property that one cannot retrieve the elements of the set it represents other than by exhaustively testing for all possible elements. In other words, a Bloom filter supports only membership tests. To check if x is in a set $\mathbf{A}$ represented by the Bloom filter BF, one needs to check if every position $h_i(x)$ has been set to 1:

$$x \in \mathbf{A} \text{ only if } \prod_{i=i}^{k} BF[h_i(x)] = 1$$

Counting is possible by a simple estimation n^* of the number of elements in a Bloom filter [Swamidass and Baldi, 2007]:

$$n^* = -\frac{m}{k} \ln(1 - \frac{X}{m})$$

where X is the number of nonzero elements in the Bloom filter.

Counting the number of devices that were at one location and later at another, then resorts to taking the intersection of two Bloom filters and estimating the size of the intersection. Likewise, checking how many people stayed at a location for a specific, long time, amounts to constructing intersections of Bloom filters and estimating the size. In all cases, there is, strictly speaking, no need to test for membership.

However, using Bloom filters in the clear is not safe: a simple brute-force attack can easily give a very good impression of detected devices, and thus also the ability to track devices. Of course, when the Bloom filters are encrypted, detecting actual devices becomes more difficult. Moreover, computing intersections should preferably be done on encrypted Bloom filters. Fortunately, computing an intersection is nothing else but constructing a Bloom filter through a bitwise AND operation, which is equivalent to a multiplication of 0's and 1's. Specifically, if we need to compute the intersection of two Bloom filters A and B, we do so using their homomorphically encrypted versions and compute:

$$\forall i : PK(\mathsf{A} \odot \mathsf{B})[i] = PK(\mathsf{A}[i]) \cdot PK(\mathsf{B}[i])$$

where "$\odot$" represents an elementwise multiplication of two vectors. The basic idea is that a third party, say Alice, who needs the value of a count provides a public key PK by which each sensor homomorphically encrypts the entries of its Bloom filters. Note that if a value p is encrypted, leading to $p_1 = PK(p)$, and that same value is encrypted a next time, leading to $p_2 = PK(p)$, the two encrypted

values will be different: $p_1 \neq p_2$ and an observer will not be able to distinguish the two underlying values to be the same.

An encrypted filter can be handed out to a separate server without disclosing any detections. That server also computes intersections, but completely ignorant of the results, for the simple reason that it cannot decrypt any of the filters. The only thing the server does is shuffle the elements of any (computed) Bloom filter before handing it to Alice, who owns the private key SK associated with PK. This shuffling is important: when Alice decrypts the result, she will have a Bloom filter that has nothing to do with the original, unshuffled Bloom filter. One might say that it represents a completely random set of detections. The only thing in common with the original filter is the number of nonzero elements, from which she can then estimate the size and thus the number of detections. This number is the only information that Alice can get from the server's response.

9.2.3 Hash functions

One final application of cryptography in distributed systems, is the use of **hash functions**. A hash function H takes a message m of arbitrary length as input and produces a bit string h having a fixed length as output:

$$\mathsf{h} = H(\mathsf{m}) \textit{ with length of } \mathsf{h} \textit{ fixed}.$$

A hash h is somewhat comparable to the extra bits that are appended to a message in communication systems to allow for error detection, such as a cyclic-redundancy check (CRC). A well-known application is securely storing passwords: instead of storing a password pw, we keep only $H(\mathsf{pw})$. Then, when Alice logs in with a password pw', we need merely check if $H(\mathsf{pw}) = H(\mathsf{pw}')$.

Hash functions that are used in cryptographic systems, also known as **trapdoors**, have a number of essential properties. First, they are **one-way functions**, meaning that it is computationally infeasible to find the input m that corresponds to a known output h. On the other hand, computing h from m is easy. Second, they have the **weak collision resistance** property, meaning that given an input m and its associated output $\mathsf{h} = H(\mathsf{m})$, it is computationally infeasible to find another, different input $\mathsf{m}' \neq \mathsf{m}$, such that $H(\mathsf{m}) = H(\mathsf{m}')$. Finally, cryptographic hash functions also have the **strong collision resistance** property, which means that, when given only H, it is computationally infeasible to find any two different input values m and m', such that $H(\mathsf{m}) = H(\mathsf{m}')$. In this way, an attacker will need to guess passwords instead of simply stealing or trying to derive them what is already known.

Similar properties must apply to any encryption function E and the keys that are used. For any encryption function E_K, it should be computationally infeasible to find the key $\mathsf{E_K}$ when given the plaintext P and associated

ciphertext $\mathsf{C} = E_K(\mathsf{P})$. Likewise, analogous to collision resistance, when given a plaintext P and a key $\mathsf{E_K}$, it should be effectively impossible to find another key $\mathsf{E_{K'}}$ such that $E_K(\mathsf{P}) = E_{K'}(\mathsf{P})$.

Digital signatures

Let us now take a look at an often-used application of hash functions, namely placing **digital signatures** as part of sending messages. Consider the situation in which Bob has just sold Alice a collector's item of some vinyl record for $500. The whole deal was done through e-mail. In the end, Alice sends Bob a message confirming that she will buy the record for $500. There are at least two issues that need to be taken care of regarding the integrity of the message.

1. Alice needs to be assured that Bob will not maliciously change the $500 mentioned in her message into something higher, and claim she promised more than $500.
2. Bob needs to be assured that Alice cannot deny ever having sent the message, for example because she had second thoughts.

These two issues can be dealt with if Alice digitally signs the message in such a way that her signature is uniquely tied to its content. The unique association between a message and its signature prevents that modifications to the message will go unnoticed. In addition, if Alice's signature can be verified to be genuine, she cannot later repudiate the fact that she signed the message.

There are several ways to place digital signatures. One popular form is to use a public-key cryptosystem. When Alice sends a message m to Bob, she encrypts it with her *private* key $\mathsf{SK_A}$, and sends it off to Bob. If she also wants to keep the message content a secret, she can use Bob's *public* key and send $PK_B(\mathsf{m}, SK_A(\mathsf{m}))$, which combines m and the version signed by Alice:

$$\text{Alice: } \textit{send } \mathsf{C} = PK_B([\mathsf{m}, \mathsf{sig}]) \textit{ with } \mathsf{sig} = SK_A(\mathsf{m}).$$

When Bob receives the ciphertext C, he decrypts it with his private key $\mathsf{SK_B}$ and uses Alice's public key to verify the signature:

$$\text{Bob: } \textit{receive and decrypt } [\mathsf{m}, \mathsf{sig}] = SK_B(\mathsf{C}) \textit{ and verify } \mathsf{m} = PK_A(\mathsf{sig}).$$

If Bob can be assured that the public key is indeed owned by Alice, then decrypting the signed version of m and successfully comparing it to m can mean only that it came from Alice. Alice is protected against any malicious modifications to m by Bob because Bob will always have to prove that the modified version of m was also signed by Alice. In other words, the decrypted message alone essentially never counts as proof. It is also in Bob's own interest to keep the signed version of m to protect himself against repudiation by Alice.

An issue with the scheme just described is that Alice encrypts the entire message with her private key. Such an encryption may be costly in terms of

processing requirements and is actually unnecessary. Recall that we need to uniquely associate a signature with only a specific message. A cheaper and arguably more elegant scheme is to use a **message digest**.

A message digest is a fixed-length bit string h that has been computed from an arbitrary-length message m through a cryptographic hash function H. If m is changed to m', its hash $H(\mathsf{m}')$ will be different from $\mathsf{h} = H(\mathsf{m})$ so that it can easily be detected that a modification has taken place.

To digitally sign a message, Alice can first compute a message digest and subsequently encrypt the digest with her private key. The encrypted digest is sent along with the message to Bob:

$$\text{Alice: } \textit{send } [\mathsf{m}, \mathsf{sig}] \textit{ with } \mathsf{sig} = SK_A(H(\mathsf{m})).$$

When Bob receives the message and its encrypted digest, he need merely decrypt the digest with Alice's public key, and separately calculate the message digest. If the digest calculated from the received message and the decrypted digest match, Bob knows the message has been signed by Alice:

$$\text{Bob: } \textit{receive } [\mathsf{m}, \mathsf{sig}], \textit{ compute } \mathsf{h}' = H(\mathsf{m}) \textit{ and verify } \mathsf{h}' = PK_A(\mathsf{sig}).$$

Note that the message itself is sent as plaintext: everyone is allowed to read it. If confidentiality is required, then the message should also be encrypted with Bob's public key.

There are still a number of problems with both schemes, although the protocols are correct. First, the validity of Alice's signature holds only as long as Alice's private key remains a secret. If Alice wants to bail out of the deal even after sending Bob her confirmation, she could claim that her private key was stolen before the message was sent. We will later describe how blockchains may help in keeping a decentralized account of transactions, such as those between Alice and Bob.

Another problem occurs when Alice decides to change her private key. Doing so may in itself be not such a bad idea, as changing keys from time to time generally helps against intrusion. However, once Alice has changed her key, her statement sent to Bob, becomes worthless. What may be needed in such cases is a central authority that keeps track of when keys are changed, in addition to using timestamps when signing messages. We look at this issue next; Figure 9.4 summarizes the notations and abbreviations we use in the mathematical expressions throughout this book.

9.2.4 Key management

So far, we have described cryptographic protocols in which we (implicitly) assumed that various keys are readily available. For example, in the case of public-key cryptosystems, we assumed that a sender of a message had the public key of the receiver at its disposal so that it could encrypt the message

Notation	Description
$K_{A,B}$	Secret key shared by A and B
PK_A	Public key of A
SK_A	Private (secret) key of A
$E_K(P)$	Encryption of plaintext P using key E_K (or key K)
$D_K(C)$	Decryption of ciphertext C using key D_K (or key K)
$H(m)$	The hash of m computed using function H
$[m]_A$	The message m digitally signed by A

Figure 9.4: Notations used in this chapter.

to ensure confidentiality. Likewise, when verifying the signature of a message, the receiver will need to have the public key of the sender. In both these examples, we also need to be sure that the public key being used actually belongs to the presumed owner.

However, establishing and distributing keys is not a trivial matter. For example, digitally distributing secret keys without using encryption is out of the question and there are cases where we even have to resort to out-of-band methods. Also, mechanisms are needed to revoke keys, that is, prevent a key from being used after it has been compromised or invalidated.

Key establishment

How are keys actually established? Let us take a look at a simple and widely deployed approach for automatically establishing remote secure connections through SSH. As we mentioned, SSH allows a person to securely log in to a remote computer. We can also use SSH to let a user alice at computer A log in as user bob at computer B using a $(public, private)$-key pair. An actual authentication protocol will be described later; here we are interested in establishing and distributing keys in the SSH system using the openSSL framework.

First, Alice needs to generate a $(public, private)$-key pair, which can be done as follows:[1]

```
ssh-keygen -f a
```

This will generate two keys: a private key contained in the file a and a corresponding public key in the file a.pub. The file a will be stored in a special directory (namely in .ssh in Alice's home directory). The file a.pub will have to be transferred to computer B, which can be done as follows:

[1]We are deliberately taking an extremely simple example that works in practice yet is not very sophisticated.

```
ssh-copy-id -i a bob@B
```

We assume that B is the actual computer name that can be used when Alice would normally log in to B as user bob. It is important to note that Alice is assumed to know the password of Bob at B. The ssh-copy-id command will ask for Bob's password at B before completing the transfer. If successful, the file a.pub will be appended to a special file in the .ssh directory in the home directory of user bob at B. From that moment on, Alice can simply log in to B through the command

```
ssh bob@B
```

where she will no longer be asked for a password.

This simple example illustrates two important issues. First, security keys are simply generated through programs such as ssh-keygen and stored in files that can be distributed as any other file (or should be kept secret and properly protected against unauthorized access). Second, we can easily distribute keys *provided we have the authorization to do so*. That authorization can come from knowing a password, but there are other means as well, as we discuss shortly. Interestingly, in lack of authorization (and thus authentication), we would need to rely on the receiving party to *trust* the claimed ownership of the keys. We return to trust below.

Let us move on to a more sophisticated example and consider how session keys can be established. As it name suggests, a **session key** is a *temporary* shared secret key that is used during a single communication session between two parties. A session typically consists of a series of message exchanges. When the session ends, the session key is discarded and never used again. As we shall also explain later, an important reason for using session keys is that one should use long-lasting keys, be they shared, public, or private as little as possible: the more data that is used for encrypting or decrypting data, the easier it becomes to discover the keys used. Two ways immediately come to mind, by which Alice and Bob can agree on establishing a session key. In a symmetric cryptosystem and assuming that Alice and Bob already have a long-lasting shared secret key $\mathsf{K}_{\mathsf{A,B}}$, Alice could decide to generate a session key sk and send it to Bob as the message

$$\text{Alice: } \textit{send } \mathsf{m} = K_{A,B}(\mathsf{sk}).$$

Likewise, in an asymmetric cryptosystem, Alice could encrypt the key with Bob's public key:

$$\text{Alice: } \textit{send } \mathsf{m} = PK_B(\mathsf{sk}).$$

After that, sk can be used for encrypting and decrypting messages for as long as the session lasts.

Both methods require that the communicating parties already have the means available to send and receive encrypted messages. In other words, some form of key establishment and distribution must have taken place before.

An elegant scheme that is widely applied for establishing a shared key across an insecure channel is the **Diffie-Hellman key exchange** [Diffie and Hellman, 1976]. The protocol works as follows. Suppose that Alice and Bob want to establish a shared secret key. The first requirement is that they agree on two large numbers, p and g that are subject to a number of mathematical properties (which we do not discuss here). Both p and g can be made public; there is no need to hide them from outsiders. Alice picks a large random number, say x, which she keeps secret. Likewise, Bob picks his own secret large number, say y. At this point, there is enough information to construct a shared secret key, as shown in Figure 9.5.

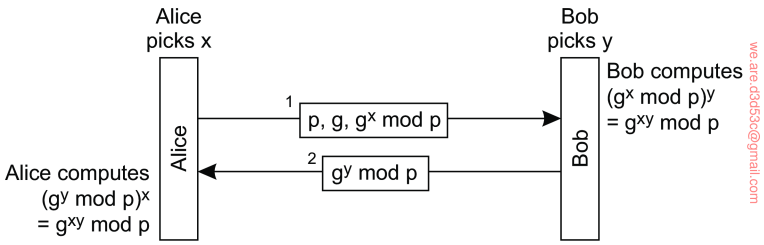


Figure 9.5: The principle of Diffie-Hellman key exchange.

Alice starts by sending $g^x \bmod p$ to Bob, along with p and g. It is important to note that this information can be sent as plaintext, as it is virtually impossible to compute x given $g^x \bmod p$. When Bob receives the message, he subsequently calculates $(g^x \bmod p)^y$ which is mathematically equal to $g^{xy} \bmod p$. In addition, he sends $g^y \bmod p$ to Alice, who can then compute $(g^y \bmod p)^x = g^{xy} \bmod p$. Consequently, both Alice and Bob, and only those two, will now have the shared secret key $g^{xy} \bmod p$. Note that neither of them needed to make their private number (x and y, respectively), known to the other. Diffie-Hellman can also be viewed as a public-key cryptosystem. In the case of Alice, x is her private key, whereas $g^x \bmod p$ is her public key.

In practice, Alice will not send p and g to Bob, but first negotiate which so-called **Diffie-Hellman group** they will use. In essence, the choice of group defines p and g, but also exactly which cryptographic algorithm to use. In our example, we have used $g^x \bmod p$, yet an alternative is to use a so-called elliptic curve.

It is also relevant to note that in practice, Alice and Bob will regularly change their respective private keys. In particular, new values for x and y may be chosen for each new communication session between Alice and Bob, in which case we speak of **ephemeral Diffie-Hellman key exchange**. As we shall see, the Diffie-Hellman key exchange has shown to be a powerful security mechanism throughout many practical situations. In the following note, we discuss another example of its application.

Note 9.2 (Advanced: Computing while keeping data private)
In many distributed systems, we see that security comes almost as an afterthought: designers focus on providing functionality to later adjust the system to meet specific security requirements. For long, it has been thought that functionality may actually suffer from imposing security (or privacy) demands, yet it is unclear to what extent there is actually a tradeoff to consider [Rowe et al., 2012]. Let us look at an increasingly popular technique that makes use of Diffie-Hellman key exchange.

As also mentioned in our discussion on privacy, an important challenge that many distributed systems will need to face is to deal with enormous amounts of data while preserving privacy. A particular case is when private data is essential to compute important statistics. A simple example is computing who has the highest salary among a number of people without anyone having to reveal their salary. Along the same lines, we may wish to compute the number of votes cast for a specific candidate without revealing who voted for whom.

What we are dealing with is known as **(secure) multiparty computations** (**MPC**). For a long time, it has been a field of cryptography that was difficult to put into practice for its low performance, but the last decade has shown impressively better implementations that make MPC a practical tool for secure distributed computations. At the same time, one may need to devise application-specific solutions for the simple reason that there are too many parties to make general tools practically feasible. We will discuss an example in which resource constraints demand an alternative solution.

It is beyond the scope of this chapter to go into any significant depth concerning MPC; the interested reader is referred to Evans et al. [2018] for a good introduction. Let us first consider an important building block for MPC, namely **oblivious transfer**. In this case, we assume Alice has n secret messages $\mathsf{m}_1, \ldots, \mathsf{m}_n$, while Bob is interested (and allowed) to know only one of those messages. Which specific message he wants to know should be kept secret to Alice, while at the same time all other messages should be kept secret to Bob. Assume Bob wishes to know the ith message. In that case, he generates a number Q that Alice, in turn, uses to generate a total of n different keys $\mathsf{PK}_1, \ldots, \mathsf{PK}_n$ to encrypt each message:

$$\mathsf{m}_i^* = PK_i(\mathsf{m}_i)$$

Bob, meanwhile, uses Q to generate a decryption key SK_i that matches *only* PK_i. Clearly, when Bob receives the set of encrypted messages $\mathsf{m}_1^*, \ldots, \mathsf{m}_n^*$ there will be only a single message that he can decrypt, namely m_i^*. Using SK_i any attempt to decrypt m_j^* (with $i \neq j$) will fail.

An important assumption for this scheme to work, is that having only Q, Bob cannot successfully generate any decryption key SK_j for PK_j. For the case that $n = 2$, there is an elegant, simple protocol that achieves this, based on a Diffie-Hellman key exchange [Chou and Orlandi, 2015]. The protocol is shown in Figure 9.6 (note that all computations are assumed to take place in $\mathbb{Z}_p$, that is, modulo p).

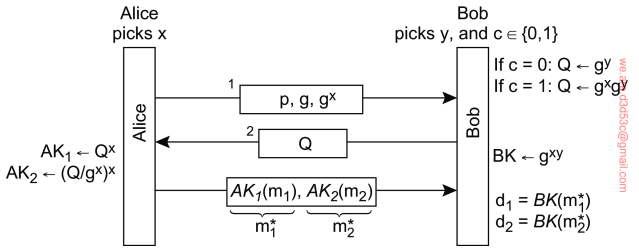


Figure 9.6: A simple 1-out-of-2 oblivious transfer.

In the protocol, besides having his own private key y, Bob also has a choice for an additional binary parameter, denoted here as c. The value of c determines what Bob will do in response to the initial message from Alice. Assume first that $c = 0$, in which case Bob will choose $\mathsf{Q} = g^y$. This is, actually, the regular Diffie-Hellman key exchange: Bob computes $\mathsf{BK} = g^{xy}$, while Alice also computes $\mathsf{AK}_1 = g^{xy}$, after which she encrypts m_1 using the now shared secret key $g^{xy} = \mathsf{BK}$. Alice also computes $\mathsf{AK}_2 = g^{xy-x^2}$, yet using BK is useless. Moreover, because Bob does not have x, computing g^{xy-x^2} is computationally infeasible.

Now assume that $c = 1$, so that Bob sets $\mathsf{Q} = g^{x+y}$. Alice computes $\mathsf{AK}_1 = g^{x^2+xy}$ and $\mathsf{AK}_2 = (g^x g^y / g^x)^x = g^{xy}$. In other words, we now see that $\mathsf{BK} = \mathsf{AK}_2$ allowing Bob to successfully decrypt *only* message m_2^*. He will never get to know m_1^*, while Alice in this setup will also never know which message Bob was able to decrypt.

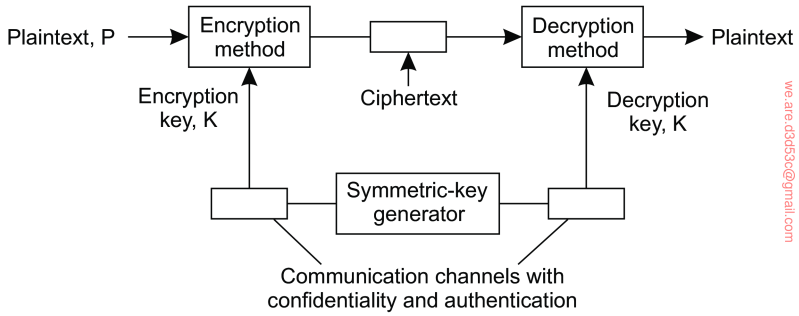
To illustrate how oblivious transfer is used in MPC, suppose we have two parties P_1 and P_2 who need to compute a function $F(a, b)$ on two input values: a secret a known only to P_1 and a secret b known only to P_2. For simplicity, let us also assume that $a \in \mathbf{X}$ and $b \in \mathbf{Y}$ and that both $\mathbf{X}$ and $\mathbf{Y}$ are finite, i.e., each contains only a finite number of elements. In that case, we can construct a matrix F of size $|\mathbf{X}| \cdot |\mathbf{Y}|$ with $\mathbf{F}[i, j] = F(x_i, y_j)$ for each pair $(x_i, y_j) \in \mathbf{X} \times \mathbf{Y}$. The problem we want to solve is to let both P_1 and P_2 compute $F(a, b)$ yet the other should not be able to discover b or a, respectively.

To this end, P_1 generates $|\mathbf{X}| \cdot |\mathbf{Y}|$ unique key pairs $(\mathsf{K}_\mathsf{i}, \mathsf{K}_\mathsf{j})$ and encrypts each entry of the table $\mathbf{F}$, leading to a table $\mathbf{F}^*[i, j] = K_i(K_j(F(x_i, x_j)))$. Assume that a corresponds to the ith element of $\mathbf{X}$ (i.e., $a = x_i$). P_1 then sends $\mathbf{F}^*$ and K_i to P_2, but not before having permuted all its elements. In addition, it sends Q using a 1-out-of-$|Y|$ oblivious transfer. Assume that $b = y_j$. Using Q, P_2 can construct the key K_j, and only the key K_j that allows it to decrypt $\mathbf{F}^*[i, j]$, corresponding to $F(a, b)$. All other entries will remain a secret to P_2, while at the same time, P_1 will not know about $b = y_j$.

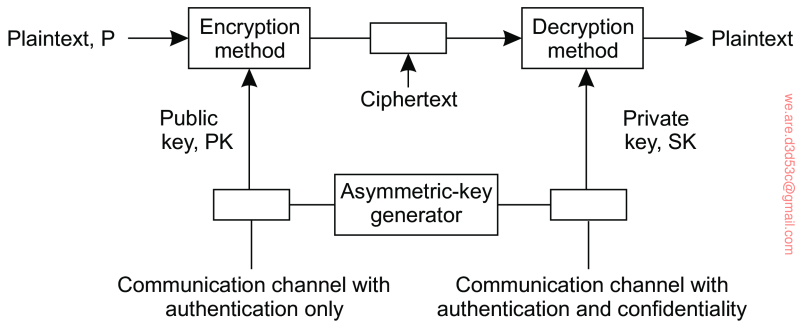
Note that permuting the table before sending it to P_2 is essential, for otherwise a and b could be looked up. Also, we assume that P_2 can recognize that it decrypted the correct entry, for how would it otherwise know that the decrypted value makes sense? This recognition can be done by adding some extra information, such as a specific bit string that pops up after successful decryption.

Key distribution

One of the more difficult parts in key management is the actual distribution of initial keys. In a symmetric cryptosystem, the initial shared secret key must be communicated along a **secure channel**: a communication channel that, in principle, provides mutual authentication of the two communicating parties, as well as confidentiality for the messages that are exchanged, as shown in Figure 9.7. We will return to secure channels later in this chapter. If there are no keys available to Alice and Bob to set up such a secure channel, it is necessary to distribute the key out-of-band. In other words, Alice and Bob will have to get in touch with each other using some other communication channel.



(a)



(b)

Figure 9.7: (a) Secret-key distribution. (b) Public-key distribution (see also [Menezes et al., 1996]).

In the case of a public-key cryptosystem, we need to distribute the public key in such a way that the receivers can be sure that the key is indeed paired to a claimed private key. In other words, as shown in Figure 9.7, although the public key itself may be sent as plaintext, it is necessary that the channel

through which it is sent can provide authentication. The private key, of course, needs to be sent across a secure channel, providing authentication as well as confidentiality.

When it comes to key distribution, the authenticated distribution of public keys is perhaps the most interesting. In practice, public-key distribution takes place by **public-key certificates**. Such a certificate consists of a public key together with a string identifying the entity to which that key is associated. The entity could be a user, but also a host or some special device. The public key and identifier have together been signed by a **certification authority**, and this signature has been placed on the certificate as well. (The identity of the certification authority is naturally part of the certificate.) Signing takes place by a private key $\mathsf{SK}_{\mathsf{CA}}$ that belongs to the certification authority. The corresponding public key $\mathsf{PK}_{\mathsf{CA}}$ is assumed to be well known. For example, the public keys of various certification authorities are built into most Web browsers and shipped with the binaries.

Using a public-key certificate works as follows. Assume that a client wishes to ascertain that the public key found in the certificate, indeed belongs to the identified entity. It then uses the public key of the associated certification authority to verify the certificate's signature. If the signature on the certificate matches the *(public key, identifier)*-pair, the client accepts that the public key indeed belongs to the identified entity.

It is important to note that by accepting the certificate as being in order, the client actually trusts that the certificate has not been forged. In particular, the client must assume that the public key $\mathsf{PK}_{\mathsf{CA}}$ indeed belongs to the associated certification authority. If in doubt, it should be possible to verify the validity of $\mathsf{PK}_{\mathsf{CA}}$ through another certificate coming from a different, possibly more trusted certification authority.

Note 9.3 (More information: Lifetime of certificates)
An important issue concerning certificates is their longevity. First, let us consider the situation in which a certification authority hands out lifelong certificates. Essentially, what the certificate then states is that the public key will always be valid for the entity identified by the certificate. Clearly, such a statement is not what we want. If the private key of the identified entity is ever compromised, no unsuspecting client should ever be able to use the public key (let alone malicious clients). In that case, we need a mechanism to **revoke** the certificate by making it publicly known that the certificate is no longer valid.

There are several ways to revoke a certificate. One common approach is with a **certificate revocation list** (**CRL**) published regularly by the certification authority. Whenever a client checks a certificate, it will have to check the CRL to see whether the certificate has been revoked or not. This means that the client will at least have to contact the certification authority each time a new CRL is published. Note that if a CRL is published daily, it also takes a day to revoke a certificate. Meanwhile, a

compromised certificate can be falsely used until it is published on the next CRL. Consequently, the time between publishing CRLs cannot be too long.

An alternative approach is to restrict the lifetime of each certificate. The validity of a certificate automatically expires after some time. If, for whatever reason, the certificate should be revoked before it expires, the certification authority can still publish it on a CRL. However, this approach will still force clients to check the latest CRL whenever verifying a certificate.

A final extreme case is to reduce the lifetime of a certificate to nearly zero. In effect, this means that certificates are no longer used; instead, a client will always have to contact the certification authority to check the validity of a public key. As a consequence, the certification authority must be continuously online and capable of handling potentially continuous streams of validity checks.

In practice, certificates are handed out with restricted lifetimes, after which they need to be renewed. Depending on the application, renewal may require manual intervention, akin to renewing automatically expired passwords. In other cases, such as identifying Web servers, renewal can also be done completely automatically as essentially a check of the existence of a specific file at a specific location suffices. The latter happens with the Let's Encrypt system for TLS certificates.

Using certification authorities is one way of ensuring that a public key is valid in the sense of claimed ownership. However, it can easily come with administrative hassle. Also, making use of certification authorities is no guarantee that certificates are correct: there have been cases in which the ownership of a public key was systematically not properly established, and also certification authorities are known to have been subject to successful attacks.

As an alternative, systems have been developed relying on developing a **web of trust**. A well-known example comes from its initiator, Phil Zimmerman, the developer of the **Pretty Good Privacy** (**PGP**) system, which has been around ever since the 1990s [Garfinkel, 1995]. The key idea, when developing a web of trust, is that groups of people get to together and exchange their public keys. At that point, one establishes the true ownership of a public key. In particular, Alice may digitally sign Bob's public key $\mathsf{PK}_{\mathsf{bob}}$, effectively endorsing that the public key belongs to Bob. Other people will also endorse Bob's public key (and that of many others during the meeting). The result is that anyone who trusts the ownership of Alice's public key, that is, trusts the signature attached to Bob's public key, has no reason not to believe that $\mathsf{PK}_{\mathsf{bob}}$ is indeed owned by Bob.

In a more sophisticated setup, one can also attach a weight to Alice's signature. For example, if Chuck believes Alice to be somewhat sloppy when it comes to signing other people's public key, he may decide to wait until he has seen at least $k > 1$ people in his web of trust who have signed Bob's public key as well before deciding that $\mathsf{PK}_{\mathsf{bob}}$ is genuinely owned by Bob. An important

question is whether Chuck, in turn, could ever endorse Bob's key without ever having truly established his identity. With automatic endorsements, we open the floor to digital attacks of all sorts, including ones in which an attacker behaves well for a long time to strike hard only after considerable trust has been established. For this reason, the big question with a web of trust is how quickly, and with which confidence, can we actually disseminate trustworthy certificates? It has been a topic of much research, yet after all these years there is no consensus to what extent webs of trust can operate on a large scale. In this sense, Ulrich et al. [2011] draw interesting conclusions from a practical analysis of that web: its core is formed by mutually connected (and endorsed) relationships, yet is relatively small. In particular, many users are connected to each other and that core through relatively few and often long paths. Such paths make it much harder to establish trust in a robust way.

9.3 Authentication

We continue our discussion by looking at authentication. After discussing some of the basics, we concentrate on various authentication protocols, the use of an authentication service, and how various aspects come together when setting up a secure communication channel.

9.3.1 Introduction to authentication

As we mentioned before, **authentication** is all about verifying the claimed identity of a person, a software component, a device, and so on. In the following, we shall use the generic term "client" for any of these types of entities. Following Stallings [2017], we distinguish four means of authentication:

1. Authentication based on what a client **knows**, such as a password or a personal identification number.
2. Authentication based on what a client **has**, such as an ID card, cell phone, or software token.
3. Authentication based on what a client **is**, i.e., static biometrics such as a fingerprint or facial characteristics.
4. Authentication based on what a client **does**, i.e., dynamic biometrics such as voice patterns or typing patterns.

In the case of **single-factor authentication**, only one of the four means of authentication is used. In practice, this means that clients are asked to enter something they know, such as a password. However, we have also come across single-factor authentication based on what a client has, namely a software token in the case of automatically logging in within the SSH framework using a *(private,public)*-key pair. With modern cell phones, it is by now also common

to make use of what a client is, typically, face recognition or fingerprints. Single-factor authentication based on dynamic biometrics is less common.

It is safe to say that humans are generally not good at all when it comes to managing passwords (see, for example, a study by Ur et al. [2016] that shows that users are generally not even aware of how bad their passwords are). In this light, we have been seeing increasingly more **multi-factor authentication** schemes being used. A common one is to use a cell phone that a user has registered with a service. When logging into a system using a password or PIN, the user is asked to acknowledge the login through that cell phone, or even provide additional information like a number sent by the service to that phone, a PIN code, or have a fingerprint scanned and verified (in which case we sometimes speak of additional **electronic authentication**).

An important observation is that, in general, authentication takes place only once: when a client wants access to a service. However, this one-time authentication for the duration of a session is often not good enough to guarantee that the service is dealing with the client it originally authenticated. The simplest example is when someone is helping out, but for a different account. Alice logs in, and grants Bob access from the same computer. Of course, this may be fine if Alice trusts Bob (and the service trusts Alice enough to allow for these type of take-overs to happen), but the situation becomes different if Alice is forced by Bob to let him into the system. In such cases, **continuous authentication** is needed: the client is not only authenticated when requesting access at the beginning of a session, but also during the entire session.

Continuous authentication is not new, yet with the availability of powerful internet-of-things devices, new scenarios have been considered [Ayeswarya and Norman, 2019; Gonzalez-Manzano et al., 2019]. For example, if a user can be continuously authenticated, it becomes possible to monitor different situations, such as safely driving a vehicle, or ensuring that a room is optimized for a specific person. When ensuring that logins are not handed off to unauthorized people, as just described, it may help to see if a personalized device is still close to the computer accessing the service. Jakubeit et al. [2022] describe a situation in which the location of a user is fingerprinted based on Wi-Fi signals to see whether the client is at a known location from which it was previously authenticated. If not, a second authentication factor may be explicitly required from the client.

9.3.2 Authentication protocols

Before going into the details of various authentication protocols, it is worthwhile noting that authentication and message integrity cannot do without each other. Consider, for example, a distributed system that supports authentication of two communicating parties, but does not provide mechanisms to ensure message integrity. In such a system, Bob may know for sure that Alice

is the sender of a message m. However, if Bob cannot be given guarantees that m has not been modified during transmission, what use is it to him to know that Alice sent (the original version of) m?

Likewise, suppose that only message integrity is supported, but no mechanisms exist for authentication. When Bob receives a message stating that he has just won $1,000,000 in the lottery, how happy can he be if he cannot verify that the message was sent by the organizers of that lottery?

Consequently, authentication and message integrity should go together. In many protocols, the combination works roughly as follows. Again, assume that Alice and Bob want to communicate, and that Alice takes the initiative in setting up a channel. Alice starts by sending a message to Bob, or otherwise to a trusted third party, who will help set up the channel. Once the channel has been set up, Alice knows for sure that she is talking to Bob, and Bob knows for sure he is talking to Alice, they can exchange messages.

To subsequently ensure integrity of the data messages that are exchanged after authentication has taken place, it is common practice to use secret-key cryptography through session keys. As we explained, a **session key** is a shared (secret) key that is used to encrypt messages for integrity and possibly also confidentiality. Such a key is generally used only for as long as the channel exists. When the channel is closed, its associated session key is destroyed. We return to session keys below.

Authentication based on a shared secret key

Let us start by taking a look at an authentication protocol based on a secret key that is already shared between Alice and Bob. How the two actually managed to obtain a shared key securely is not essential for now. In the description of the protocol, Alice and Bob are abbreviated by A and B, respectively, and their shared key is denoted as $K_{A,B}$. The protocol takes a common approach whereby one party challenges the other to a response that can be correct only if the other knows the shared secret key. Such solutions are also known as **challenge-response protocols**.

In the case of authentication based on a shared secret key, the protocol proceeds as shown in Figure 9.8. First, Alice sends her identity to Bob (message 1), indicating that she wants to set up a communication channel between the two. Bob subsequently sends a **challenge** R_B to Alice, shown as message 2. Such a challenge could take the form of a random number. Alice is required to encrypt the challenge with the secret key $K_{A,B}$ that she shares with Bob, and return the encrypted challenge to Bob. This response is shown as message 3 in Figure 9.8 containing $K_{A,B}(R_B)$.

When Bob receives the response $K_{A,B}(R_B)$ to his challenge R_B, he can decrypt the message using the shared key again to see if it contains R_B. If so, he then knows that Alice is on the other side, for who else could have encrypted R_B with $K_{A,B}$ in the first place? In other words, Bob has now

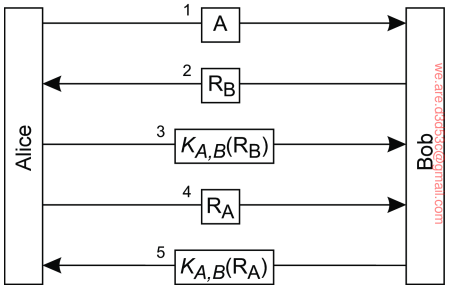


Figure 9.8: Authentication based on a shared secret key.

verified that he is indeed talking to Alice. However, note that Alice has not yet verified that it is indeed Bob on the other side of the channel. Therefore, she sends a challenge R_A (message 4), which Bob responds to by returning $K_{A,B}(R_A)$, shown as message 5. When Alice decrypts it with $K_{A,B}$ and sees her R_A, she knows she is talking to Bob.

Note 9.4 (Advanced: On the design of security protocols)
One of the difficult issues in security is designing protocols that actually work. To illustrate how easily things can go wrong, consider an "optimization" of the authentication protocol in which the number of messages has been reduced from five to three, as shown in Figure 9.9. The basic idea is that if Alice eventually wants to challenge Bob anyway, she might as well send a challenge along with her identity when setting up the channel. Likewise, Bob returns his response to that challenge, along with his own challenge, in a single message.

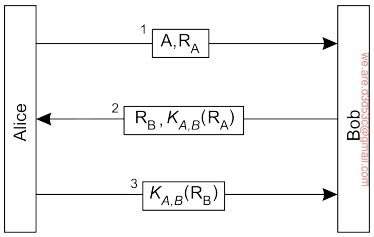


Figure 9.9: Authentication based on a shared secret key, but using three instead of five messages.

Unfortunately, this protocol no longer works. It can easily be defeated by what is known as a **reflection attack**. To explain how such an attack works, consider an intruder called Chuck, whom we denote as C in our protocols. Chuck's goal is to set up a channel with Bob so that Bob believes he is talking to Alice. Chuck can establish this if he responds correctly to a challenge sent by Bob, for instance, by returning the encrypted version of a number that Bob sent. Without knowledge

of $K_{A,B}$, only Bob can do such an encryption, and this is precisely what Chuck tricks Bob into doing.

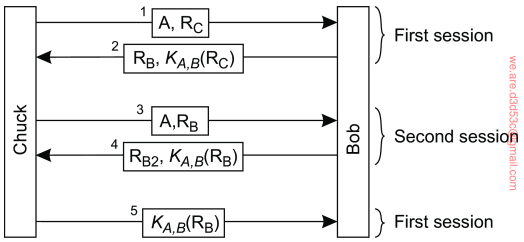


Figure 9.10: The reflection attack.

The attack is illustrated in Figure 9.10 Chuck starts out by sending a message containing Alice's identity A, along with a challenge R_C. Bob returns his challenge R_B and the response $K_{A,B}(R_C)$ in a single message. At that point, Chuck would need to prove he knows the secret key by returning $K_{A,B}(R_B)$ to Bob. Unfortunately, he does not have $K_{A,B}$. Instead, what he does is attempt to set up a second channel to let Bob do the encryption for him.

Therefore, Chuck sends A and R_B in a single message as before, but now pretends that he wants a second channel. This is shown as message 3 in Figure 9.10 Bob, not recognizing that he, himself, had used R_B before as a challenge, responds with $K_{A,B}(R_B)$ and another challenge R_{B2}, shown as message 4. At that point, Chuck has $K_{A,B}(R_B)$ and finishes setting up the first session by returning message 5 containing the response $K_{A,B}(R_B)$, which was originally requested from the challenge sent in message 2.

As explained by Kaufman et al. [2003], one of the mistakes made during the adaptation of the original protocol was that the two parties in the new version of the protocol were using the same challenge in two different runs of the protocol. A better design is to always use different challenges for the initiator and for the responder. For example, if Alice always uses an odd number and Bob an even number, Bob would have recognized that something fishy was going on when receiving R_B in message 3 in Figure 9.10. (Unfortunately, this solution is subject to other attacks, notably the one known as the "man-in-the-middle-attack," which is explained in [Ferguson et al., 2010].) In general, letting the two parties setting up a secure channel doing a number of things identically is not a good idea.

Another principle that is violated in the adapted protocol is that Bob gave away valuable information in the form of the response $K_{A,B}(R_C)$ without knowing for sure to whom he was giving it. This principle was not violated in the original protocol, in which Alice first needed to prove her identity, after which Bob was willing to pass her encrypted information.

There are many principles that developers of cryptographic protocols have gradually come to learn over the years. One important lesson is that designing security protocols that do what they are supposed to do is often much harder than it looks. Also, tweaking an existing protocol to improve its performance,

can easily affect its correctness. More on design principles for protocols can be found in [Abadi and Needham, 1996] with a more recent analysis in [Fiebig et al., 2018].

Authentication using a key distribution center

One of the problems with using a shared secret key for authentication is scalability. If a distributed system contains N hosts, and each host is required to share a secret key with each of the other $N-1$ hosts, the system as a whole needs to manage $N(N-1)/2$ keys, and each host has to manage $N-1$ keys. For large N, this will lead to problems. An alternative is to use a centralized approach by a **key distribution center** (**KDC**). This KDC shares a secret key with each of the hosts, but no pair of hosts is required to have a shared secret key as well. In other words, using a KDC requires that we manage N keys instead of $N(N-1)/2$, which is clearly an improvement.

If Alice wants to set up a secure channel with Bob, she can do so through a (trusted) KDC. The whole idea is that the KDC hands out a key to both Alice and Bob that they can use for communication, shown in Figure 9.11.

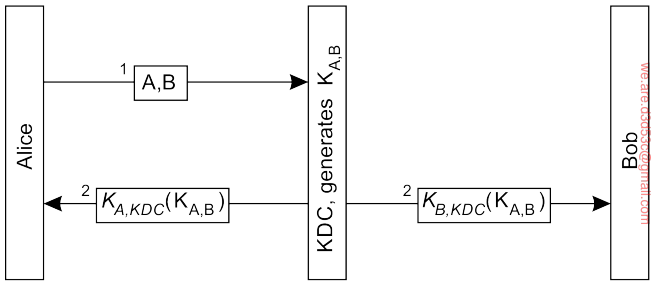


Figure 9.11: The principle of using a Key Distribution Center (KDC).

Alice first sends a message to the KDC, telling it that she wants to talk to Bob. The KDC returns a message containing a shared secret key $K_{A,B}$ that she can use. The message is encrypted with the secret key $K_{A,KDC}$ that Alice shares with the KDC. In addition, the KDC sends $K_{A,B}$ also to Bob, but now encrypted with the secret key $K_{B,KDC}$ it shares with Bob.

The main drawback of this approach is that Alice may want to start setting up a secure channel with Bob even before Bob had received the shared key from the KDC. In addition, the KDC is required to get Bob into the loop by passing him the key. These problems can be circumvented if the KDC just passes $K_{B,KDC}(K_{A,B})$ back to Alice, and lets her take care of connecting to Bob. This leads to the protocol shown in Figure 9.12. The message $K_{B,KDC}(K_{A,B})$ is also known as a **ticket**. It is Alice's job to pass this ticket to Bob. Note that Bob is still the only one who can make sensible use of the ticket, as he is the only one besides the KDC who knows how to decrypt the information it contains.

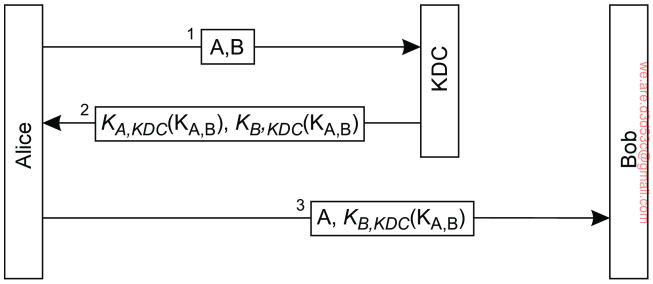


Figure 9.12: Using a ticket and letting Alice set up a connection to Bob.

Note 9.5 (Advanced: The Needham-Schroeder protocol)
The protocol shown in Figure 9.12 is actually a variant of a well-known example of an authentication protocol using a KDC, known as the **Needham-Schroeder authentication protocol**, named after its inventors [Needham and Schroeder, 1978]. The Needham-Schroeder protocol, shown in Figure 9.13 is a so-called **multiway challenge-response protocol** and works as follows.

When Alice wants to set up a secure channel with Bob, she sends a request to the KDC containing a challenge R_A, along with her identity A and, of course, that of Bob. The KDC responds by giving her the ticket $K_{B,KDC}(K_{A,B})$, along with the secret key $K_{A,B}$ that she can subsequently share with Bob.

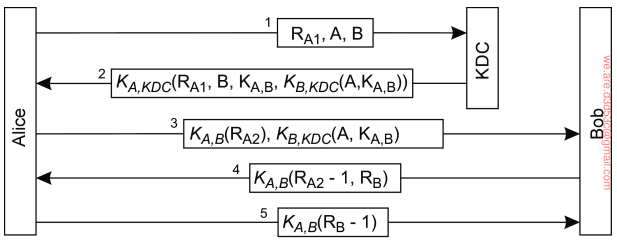


Figure 9.13: The Needham-Schroeder authentication protocol.

The challenge R_{A1} that Alice sends to the KDC along with her request to set up a channel to Bob is also known as a nonce. A **nonce** is a random number that is used only once, such as one chosen from a very large set. The main purpose of a nonce is to uniquely relate two messages to each other, in this case message 1 and message 2. In particular, by including R_{A1} again in message 2, Alice will know for sure that message 2 is sent as a response to message 1, and that it is not, for example, a replay of an older message.

To understand the problem at hand, assume that we did not use nonces, and that Chuck has stolen one of Bob's old keys, say $K^{old}_{B,KDC}$. In addition, Chuck has intercepted an old response $K_{A,KDC}(B, K_{A,B}, K^{old}_{B,KDC}(A, K_{A,B}))$ that the KDC had returned to a previous request from Alice to talk to Bob. Meanwhile, Bob will

have negotiated a new shared secret key with the KDC. However, Chuck patiently waits until Alice again requests to set up a secure channel with Bob. At that point, he replays the old response, and fools Alice into making her believe she is talking to Bob because he can decrypt the ticket and prove he knows the shared secret key $K_{A,B}$. Clearly this is unacceptable and must be defended against. By including a nonce, such an attack is impossible because replaying an older message (having a different nonce) will immediately be discovered.

Message 2 also contains B, the identity of Bob. By including B, the KDC protects Alice against the following attack. Suppose that B was left out of message 2. In that case, Chuck could modify message 1 by replacing the identity of Bob with his own identity, say C. The KDC would then think Alice wants to set up a secure channel to Chuck, and responds accordingly. As soon as Alice intends to contact Bob, Chuck intercepts the message and fools Alice into believing she is talking to Bob. By copying the identity of the other party from message 1 to message 2, Alice will immediately detect that her request had been modified.

After the KDC has passed the ticket to Alice, the secure channel between Alice and Bob can be set up. Alice starts with sending message 3, which contains the ticket to Bob, and a challenge R_{A2} encrypted with the shared key $K_{A,B}$ that the KDC had just generated. Bob then decrypts the ticket to find the shared key, and returns a response $R_{A2} - 1$ along with a challenge R_B for Alice.

The following remark regarding message 4 is in order. In general, by returning $R_{A2} - 1$ and not just R_{A2}, Bob not only proves he knows the shared secret key, but also that he has actually decrypted the challenge. Again, this ties message 4 to message 3 in the same way that the nonce R_A tied message 2 to message 1. The protocol is thus more protected against replays. However, in this special case, it would have been sufficient to just return $K_{A,B}(R_{A2}, R_B)$, for the simple reason that this message has not yet been used anywhere in the protocol before. $K_{A,B}(R_{A2}, R_B)$ already proves that Bob has been capable of decrypting the challenge sent in message 3. Message 4 as shown in Figure 9.13 is due to historical reasons.

The Needham-Schroeder protocol as presented here still has the weak point that if Chuck ever got a hold of an old key $K^{old}_{A,B}$, he could replay message 3 and get Bob to set up a channel. Bob will then believe he is talking to Alice, while, in fact, Chuck is at the other end. In this case, we need to relate message 3 to message 1, that is, make the key dependent on the initial request from Alice to set up a channel with Bob. The solution is shown in Figure 9.14.

The trick is to incorporate a nonce in the request sent by Alice to the KDC. However, the nonce has to come from Bob: this assures Bob that whoever wants to set up a secure channel with him, will have gotten the appropriate information from the KDC. Therefore, Alice first requests Bob to send her a nonce R_{B1}, encrypted with the key shared between Bob and the KDC. Alice incorporates this nonce in her request to the KDC, which will then subsequently decrypt it and put the result in the generated ticket. In this way, Bob will know for sure that the session key is tied to the original request from Alice to talk to Bob.

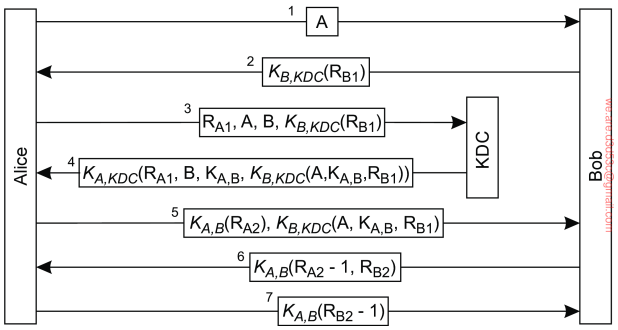


Figure 9.14: Protection against malicious reuse of a previously generated session key in the Needham-Schroeder protocol.

Authentication using public-key cryptography

Let us now look at authentication with a public-key cryptosystem that does not require a KDC. Again, consider the situation that Alice wants to set up a secure channel to Bob, and that both are in the possession of each other's public key. A typical authentication protocol based on public-key cryptography is shown in Figure 9.15 which we explain next.

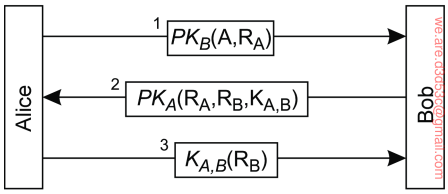


Figure 9.15: Mutual authentication in a public-key cryptosystem.

Alice starts with sending a **challenge** R_A to Bob encrypted with his public key PK_B. It is Bob's job to decrypt the message and return the challenge to Alice. Because Bob is the only person that can decrypt the message (using the private key that is associated with the public key Alice used), Alice will know that she is talking to Bob. Note again that it is important that Alice is guaranteed to be using Bob's public key, and not the public key of someone impersonating Bob.

When Bob receives Alice's request to set up a channel, he returns the decrypted challenge, along with his own challenge R_B to authenticate Alice. In addition, he generates a **session key** $K_{A,B}$ that can be used for further communication. Bob's response to Alice's challenge, his own challenge, and

the session key are put into a message encrypted with the public key PK_A belonging to Alice, shown as message 2 in Figure 9.15. Only Alice will be capable of decrypting this message using her private key SK_A.

Alice, finally, returns her response to Bob's challenge using the session key $K_{A,B}$ generated by Bob. In that way, she will have proven that she could decrypt message 2, and thus that she is actually Alice to whom Bob is talking.

The need for session keys

During the establishment of a secure channel, after the authentication phase has completed, the communicating parties generally use a unique shared session key for confidentiality. The session key is safely discarded when the channel is no longer used. An alternative would have been to use the same keys for confidentiality as those that are used for setting up the secure channel. However, there are a number of important benefits to using session keys [Kaufman et al., 2003].

First, when a key is used often, it becomes easier to reveal it. In a sense, cryptographic keys are subject to "wear and tear" just like ordinary keys. The basic idea is that if an intruder can intercept a lot of data that has been encrypted using the same key, it becomes possible to mount attacks to find certain characteristics of the keys used, and possibly reveal the plaintext or the key itself. For this reason, it is much safer to use the authentication keys as little as possible. In addition, such keys are often exchanged using some relatively time-expensive out-of-band mechanism, such as regular mail or telephone. Exchanging keys that way should be kept to a minimum.

Another important reason for generating a unique key for each secure channel is to ensure protection against replay attacks, as we have come across previously a number of times. By using a unique session key each time a secure channel is set up, the communicating parties are at least protected against replaying an entire session. To protect replaying individual messages from a previous session, additional measures are generally needed, such as including timestamps or sequence numbers as part of the message content.

Suppose that message integrity and confidentiality were achieved by using the same key used for session establishment. In that case, whenever the key is compromised, an intruder may be able to decrypt messages transferred during an old conversation, clearly not a desirable feature. Instead, it is much safer to use per-session keys because if such a key is compromised, at worst, only a single session is affected. Messages sent during other sessions stay confidential.

Related to this last point is that Alice may want to exchange some confidential data with Bob, but she does not trust him so much that she would provide him information in the form of data that have been encrypted with long-lasting keys. She may want to reserve such keys for highly confidential

messages that she exchanges with parties she really trusts. In such cases, using a relatively cheap session key to talk to Bob is sufficient.

By and large, authentication keys are often established in such a way that replacing them is relatively expensive. Therefore, the combination of such long-lasting keys with the much cheaper and more temporary session keys is often a good choice for implementing secure channels for exchanging data.

Example of an authentication service: Kerberos

It should be clear by now that incorporating security into distributed systems is not trivial. Problems are caused by the fact that the entire system must be secure; if some part is insecure, the whole system may be compromised. To assist the construction of distributed systems that can enforce a myriad of security policies, a number of supporting systems have been developed that can be used as a basis for further development. An important system that is widely used is **Kerberos** [Steiner et al., 1988; Kohl et al., 1994]

Kerberos was developed at M.I.T. and is based on the **Needham-Schroeder authentication protocol**. A detailed description of the Kerberos system can be found in [Neuman et al., 2005] whereas practical information on running Kerberos is described by Garman [2003]. A publicly available implementation of Kerberos, known as Shishi, is described in [Josefsson, 2015].

Kerberos can be viewed as a security system that assists clients in setting up a secure channel with any server that is part of a distributed system. Security is based on shared secret keys. There are two different components. The **authentication server** (**AS**) is responsible for handling a login request from a user. The AS authenticates a user and provides a key that can be used to set up secure channels with servers. Setting up secure channels is handled by a **ticket-granting service** (**TGS**). The TGS hands out special messages, known as **tickets**, that are used to convince a server that the client is really who it claims to be.

Let us take a look at how Alice logs onto a distributed system that uses Kerberos and how she can set up a secure channel with server Bob. We assume that Alice has previously been registered at the distributed system, so that there already exists a shared secret key $K_{A,AS}$ between her and the authentication server. That key is assumed to be derived from Alice's password. For example, assuming a character-string password, we can take a cryptographic hash of that password as the secret key. Of course, we do need to ensure that the secret key is protected against unauthorized access. For Alice to log onto the system, she can use any workstation available. The workstation sends her name in plaintext to the AS, which returns a session key $K_{A,TGS}$ and a ticket that she will need to hand over to the TGS.

The ticket that is returned by the AS contains the identity of Alice, along with a generated secret key that Alice and the TGS can use to communicate with each other. The ticket itself will be handed over to the TGS by Alice.

Therefore, it is important that no one but the TGS can read it. For this reason, the ticket is encrypted with the secret key $K_{AS,TGS}$ shared between the AS and the TGS.

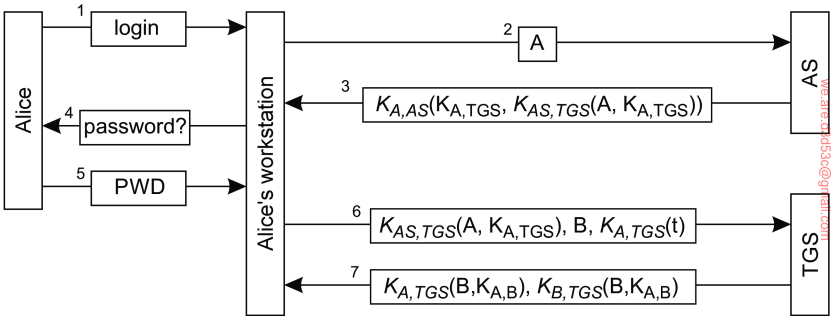


Figure 9.16: Authentication in Kerberos.

This part of the login procedure is shown as messages 1, 2, and 3 in Figure 9.16, respectively. Message 1 is not really a message, but corresponds to Alice typing in her login name at a workstation. Message 2 contains that name and is sent to the AS. Message 3 contains the session key $K_{A,TGS}$ and the ticket $K_{AS,TGS}(A, K_{A,TGS})$. To ensure privacy, message 3 is encrypted with the secret key $K_{A,AS}$ shared between Alice and the AS.

When the workstation receives the response from the AS, it prompts Alice for her password (shown as message 4), which it uses subsequently to generate the shared key $K_{A,AS}$. Note that this approach not only has the advantage that Alice's password is never sent as plaintext across the network, but also that the workstation does not even have to temporarily store it. Moreover, as soon as it has derived the shared key $K_{A,AS}$ from that password, the workstation will find the session key $K_{A,TGS}$, and can forget about Alice's password altogether (as well as the shared secret $K_{A,AS}$).

After this part of the authentication has taken place, Alice can consider herself logged into the system through the current workstation. The ticket received from the AS is stored temporarily (typically for 8–24 hours), and will be used for accessing remote services. Of course, if Alice leaves her workstation, she should destroy any cached tickets. If she wants to talk to Bob, she requests the TGS to generate a session key for Bob, shown as message 6 in Figure 9.16. The fact that Alice has the ticket $K_{AS,TGS}(A, K_{A,TGS})$ proves that she is Alice. The TGS responds with a session key $K_{A,B}$, again encapsulated in a ticket that Alice will later have to pass to Bob.

Message 6 also contains a timestamp, t, encrypted with the secret key shared between Alice and the TGS. This timestamp is used to prevent Chuck from maliciously replaying message 6 again, and trying to set up a channel to Bob. The TGS will verify the timestamp before returning a ticket to Alice. If it

differs more than a few minutes from the current time, the request for a ticket is rejected.

This scheme establishes what is known as **single sign-on**. As long as Alice does not change workstations, there is no need for her to authenticate herself to any other server that is part of the distributed system. This feature is important when having to deal with many services that are spread across multiple machines. In principle, servers in a way have delegated client authentication to the AS and TGS, and will accept requests from any client that has a valid ticket. Of course, services such as remote login will require that the associated user has an account, but this is independent of authentication through Kerberos.

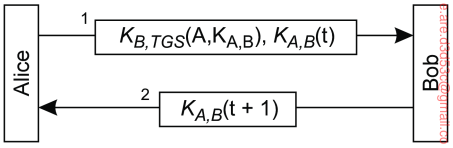


Figure 9.17: Setting up a secure channel in Kerberos.

Setting up a secure channel with Bob is now straightforward, and is shown in Figure 9.17. First, Alice sends to Bob a message containing the ticket she got from the TGS, along with an encrypted timestamp. When Bob decrypts the ticket, he notices that Alice is talking to him because only the TGS could have constructed the ticket. He also finds the secret key $K_{A,B}$, allowing him to verify the timestamp. At that point, Bob knows he is talking to Alice and not someone maliciously replaying message 1. By responding with $K_{A,B}(t+1)$, Bob proves to Alice that he is indeed Bob.

Example of a secure channel: HTTP over TLS

Let us now look at an often-used protocol suite for setting up a secure channel: the **Transport Layer Security** protocol, or simply **TLS**. The protocol suite comes in a number of flavors, originating from the **Secure Socket Layer** protocol that was designed in 1994. The most recent version is TLS 1.3, specified in [Rescorla et al., 2018]. TLS is generally best known for its use in securing the connection to a Web server when using HTTP, also known as **HTTPS**. In this case, HTTP is secured by setting up a secure channel between a client and the Web server using TLS in combination with TCP.

Understanding TLS can be quite difficult, mainly for two reasons. First, TLS 1.3 needs to be backward compatible with previous versions (at least the ones that have not been deprecated). Second, the two communicating parties need to go through a negotiation phase to decide precisely how they are actually going to use TLS. The latter starts with agreeing on the version, but also includes agreements on which cryptosystems (called cipher suites)

to use. TLS 1.3 simplifies matters in comparison to its predecessors in that it limits the options. We are going to ignore all kinds of extra information that is sent during the negotiation phase and stick to the core of the protocol.

As mentioned, TLS 1.3 limits the choices for client and server when it comes to using cipher suites. Let us assume that a client wants to set up a secure channel based on ephemeral Diffie-Hellman key exchange, using one of the possible Diffie-Hellman groups, say G. For example, G could be ffdhe2048 which corresponds to $g = 2$ and p equal to

$$p = 2^{2048} - 2^{1984} + (\lfloor 2^{1918} * e \rfloor + 560316) * 2^{64} - 1$$

where e denotes the base of the natural logarithm (i.e., $e \approx 2.718281$).

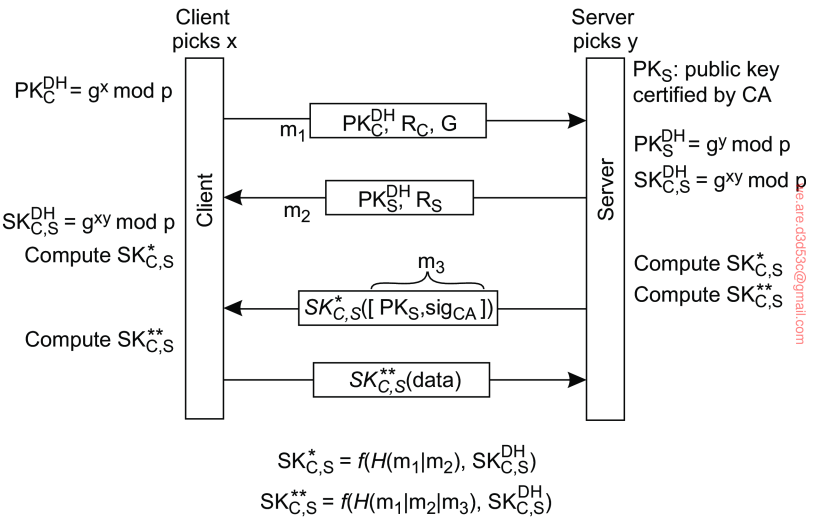


Figure 9.18: The principle of setting up a secure channel using TLS 1.3.

The client starts with choosing a private key x as explained when introducing Diffie-Hellman key exchange. By choice of G, the client computes a corresponding public key $PK_C^{DH} = g^x \bmod p$. The client continues by sending a `client-hello` message to the server. This message contains at least the public key PK_C^{DH}, a nonce R_C and G:

$$\text{Client: } \textit{sends } \texttt{client-hello} = [PK_C^{DH}, R_C, G]$$

The server, assuming it accepts G, also chooses a private key y and generates its associated public key $PK_S^{DH} = g^y \bmod p$. It subsequently returns a `server-hello` message:

$$\text{Server: } \textit{responds } \texttt{server-hello} = [PK_S^{DH}, R_S]$$

Note that at this point, the client has no guarantees that it is actually communicating with the intended server. The only thing the two have essentially communicated are generated public keys. These keys can be used to compute a shared secret key $\mathsf{SK}^{\mathsf{DH}}_{\mathsf{C,S}} = g^{xy} \bmod p$. As a next step, the client and the server first each compute the following hash:

$$\mathsf{MAC} = H([\mathsf{PK}^{\mathsf{DH}}_{\mathsf{C}}, \mathsf{R}_{\mathsf{C}}, \mathsf{G}, \mathsf{PK}^{\mathsf{DH}}_{\mathsf{S}}, \mathsf{R}_{\mathsf{S}}])$$

where H is the SHA256 hashing function [Eastlake and Hansen, 2011]. In other words, MAC is the 256-bit hash obtained by concatenating the `client-hello` and `server-hello` messages. MAC and $\mathsf{SK}^{\mathsf{DH}}$ are then used to generate the final shared secret session key $\mathsf{SK}^{*}_{\mathsf{C,S}} = f(\mathsf{MAC}, \mathsf{SK}^{\mathsf{DH}}_{\mathsf{C,S}})$ where f is a so-called **key-derivation function**. From this moment on, the communication between the client and the server is kept private, i.e., encrypted with the shared secret key $\mathsf{SK}^{*}_{\mathsf{C,S}}$. The server then continues with authenticating itself. To that end, we assume it can provide a certificate containing a public key PK_{S} signed by the certification authority CA. The server encrypts all its communication with the client using $\mathsf{SK}^{*}_{\mathsf{C,S}}$:

$$\text{Server } \textit{sends } SK^{*}_{C,S}([\mathsf{PK}_{\mathsf{S}}, \mathsf{sig}_{\mathsf{CA}}]) \text{ with } \mathsf{sig}_{\mathsf{CA}} = SK_{CA}(\mathsf{PK}_{\mathsf{S}})$$

At this point, the client can authenticate the server. Applications running on top of TLS could, in principle, rely entirely on the use of the shared secret key $\mathsf{SK}^{*}_{\mathsf{C,S}}$ while keeping the communication private. Better is to generate keys that the application-level protocol can use, independent of what is being used by the TLS connection. To that end, both client and server each compute a shared secret key from the information exchanged so far:

$$\mathsf{AMAC} = H([\mathsf{PK}^{\mathsf{DH}}_{\mathsf{C}}, \mathsf{R}_{\mathsf{C}}, \mathsf{G}, \mathsf{PK}^{\mathsf{DH}}_{\mathsf{S}}, \mathsf{R}_{\mathsf{S}}, \mathsf{PK}_{\mathsf{S}}, \mathsf{sig}_{\mathsf{CA}}])$$

that is, essentially the concatenation of the messages `client-hello` and `server-hello`, along with the certificate and again hashed using $H = \mathsf{SHA256}$. The final application-level shared secret key is computed as $\mathsf{SK}^{**}_{\mathsf{C,S}} = f(\mathsf{AMAC}, \mathsf{SK}^{*}_{\mathsf{C,S}})$. A summary of what we have just discussed is shown in Figure 9.18.

9.4 Trust in distributed systems

Fundamental to authentication is that a process can prove to be who it claims to be. Once that proof has been given, the question arises how good that proof actually is. One can argue that this is where trust starts to play: does the receiver of the proof accept that proof to be sufficient to provide the process access to the system? Yet trust as facilitated or implemented in distributed systems extends beyond just authentication, as witnessed by the vast interest in distributed ledger systems such as those implemented by blockchains. The issue of trust has also become more prominent with the increase of

(semi-)automated decision-making procedures that have been installed in numerous information systems and that directly affect the lives of many human beings. Making unjustified decisions does not help in building trust, and many such decisions have been made, and are presumably still being made. Perhaps most problematic are the situations in which one needs to trust decision-making without understanding the underlying process, as is generally inherently the case when applying machine-learning techniques.

In this section, we will zoom into matters of trust, yet will stay close to technical issues relevant for the design and implementation of distributed systems. Wierzbicki [2010] discusses trust in a broader context while still linking it strongly to information systems. A recent survey on trust and reputation models for information systems is provided by Braga et al. [2018].

9.4.1 Trust in the face of Byzantine failures

Before we take a more explicit look at trust, it is worthwhile considering its relation to failure models. Trust becomes an issue when there is a dependency of one process P on another process Q and a possibility that the latter no longer behaves according to what P expects. More specifically:

> *Trust is the assurance that one entity holds that another will perform particular actions according to a specific expectation.*

This definition, by Bursell [2022], can be refined in many ways, yet will do for now. The point is that when expectations have been made explicit, i.e., specified, there may be no need to talk about trust anymore. If we take a look at Byzantine failures within process groups, the underlying assumption is that there may be processes from which we cannot expect anything anymore: they may be behaving correctly or incorrectly, but we have no means of always correctly detecting whether they are living up to their specifications.

The interesting aspect of Byzantine failure models is that we can let go of trust and develop solutions in which there is no need to trust the individual processes. The only thing that matters is that the process group lives up to its specifications, that is, with a group size of n processes, at most $k \leq (n-1)/3$ will go rogue. Such a group can still reach a correct agreement among the nonfaulty processes. If the group cannot meet its specifications, for example, because of too many faulty processes, all bets are off.

9.4.2 Trusting an identity

Let us start by looking at a rather nasty issue when accepting incoming requests from a source without bothering about further authentication. This situation is not unusual: it constantly happens in many decentralized peer-to-peer systems. The problem we need to face is that of a so-called **Sybil attack** [Douceur, 2002]. The essence of this attack is that in distributed systems,

we generally rely on the fact that when presented with a logical identity, there is just a single associated physical entity. In other words, we rely on the following three properties, as also discussed in Section 6.1:

1. An identifier refers to at most one entity
2. Each entity is referred to by at most one identifier.
3. An identifier always refers to the same entity (i.e., it is never reused).

In the case of a Sybil attack, these assumptions are violated: an attacker simply creates multiple identities and joins the system separately with each of these identities to subsequently stage a specific attack. The general principle of a Sybil attack can be elegantly summarized as shown in Figure 9.19.

```
H = set of honest nodes
S = set of Sybil nodes
A = Attacker node
d = minimal fraction of Sybil nodes needed for an attack

while True:
    s = A.createNode()       # create a Sybil node
    S.add(s)                 # add it to the set S

    h = random.choice(H)     # pick an arbitrary honets node
    s.connectTo(h)           # connect the new sybil node to h

    if len(S) / len(H) > d:  # enough sybil nodes for...
        A.attack()           # ...an attack
```

Figure 9.19: The principle of a Sybil attack (adopted from [Iqbal and Matulevičius, 2021].

A typical example of a Sybil attack is when operating a decentralized peer-to-peer network such as Chord or Kademlia. An attacker simply creates multiple logical nodes that join the network. As the attacker has full control over these nodes, from a security perspective, they are viewed as a collection of colluders. In particular, there is no reason to assume that they behave as expected. For example, together, they can easily launch a denial-of-service attack by not forwarding lookup requests. If the attacker has control over a sufficient number of nodes, it is with high probability that any lookup request will need to go through a malicious node. Likewise, the colluding nodes can join in attacking the content stored in a network, if only to delete files they are collectively responsible for. Even in the face of using file replication, such a behavior may easily lead to permanent loss of data.

As another example of a Sybil attack, consider the aforementioned web of trust. If we allowed automatic endorsement of public keys, i.e., having Chuck endorse the public key of Bob without an out-of-band checking that the key

is really owned by Bob, we may find ourselves in the following situation. Note that the web of trust is essentially based on reputation: assuming Chuck does not entirely trust Alice when it comes to validating the public key of Bob, he may decide to also consider the validations from $k > 1$ others before accepting the validity of Bob's public key. Yet, with automatic endorsement, Alice could launch a Sybil attack by simply cloning her efforts by installing many nodes claiming they have validated Bob's public key. If k of these nodes are consulted by Chuck, then Chuck will believe the key indeed belongs to Bob. There is no harm in this, unless Alice had stolen Bob's private key and is now trying to act as Bob. Without automatic endorsement, such an attack would be much more difficult to realize.

Closely related to a Sybil attack is the so-called **eclipse attack**. In this case, a collection of colluding nodes will try to isolate a node from the network. We showed in Section 5.5.4, how with almost minimal effort a gossip-based service can be brought to a grinding halt before benevolent nodes may even suspect that something fishy is going on. That service is based on continuously exchanging randomly chosen links with neighbors in an attempt to maintain an overlay network that resembles a random graph. The links are chosen from a relatively small list that is local to each node in the network. An exchanged link replaces the link in this local list. In an eclipse attack, a number of colluding nodes never return randomly chosen links, but only links to their fellow colluders. The effect is devastating: within only relatively few exchanges by all nodes in the network, the malicious links will have contaminated the local lists to the extent that each local list contains only links to the colluding nodes. At that point, the colluders have full control over the network (see also Jesi et al. [2007]).

The simple way out in both cases is to use a centralized certification authority: whenever Alice connects to Bob, she will have to prove to be the holder of the digital identity associated with her. A certificate signed by a trusted authority should, in principle, do the job. Suppose the certification authority is not (sufficiently) trusted. This may happen, for example because the certification authority is not known enough. In that case, we may rely on a **trust chain**, by which the public key $\mathsf{PK}_{\mathsf{CA}}$ that is used to sign the certificate is accompanied by its own certificate, signed by another certification authority, say CA^*. If the public key $\mathsf{PK}_{\mathsf{CA}^*}$ is not trusted, then we can repeat the procedure by having that key be accompanied by another certificate signed by yet another certification authority CA^{**}, and so on. However, in the end, the recipient will have to trust the certification authority at the end of this chain.

Preventing Sybil attacks: blockchain solutions

Levine et al. [2006] and later Urdaneta et al. [2011] have looked at Sybil attacks in decentralized networks. Both come to the conclusion, as also already stated by Douceur [2002] that these attacks are almost impossible to prevent without

making use of a trusted authority. All other decentralized solutions up to that point could at best discourage Sybil attacks, but not prevent them. However, accepting a centralized, trusted party in an otherwise decentralized system is something that many find difficult to accept. Yet, if we could devise a system that would make having multiple identities unattractive, then maybe a trusted party may not be necessary. This is where **permissionless blockchains** come into play.

Launching multiple identities is attractive if the attacker knows that there is substantial gain in doing so compared to the price to pay for maintaining multiple identities. But the price is high when, for example, an identity is assumed to throw in considerable resources to be able to act. This is precisely what is expected in **proof-of-work** (**PoW**) blockchain systems. In PoW systems, validators of a block of transactions are engaged in a race, requiring considerable computational resources. There is no incentive for an attacker to launch another identity for the same physical node, as that identity will need to compete as well.

Having to run computational races is arguably a bad design choice for any distributed system. In an attempt to still prevent Sybil attacks but avoiding computational races, designers of blockchain systems have been investigating **proof-of-stake** (**PoS**) solutions [Nguyen et al., 2019]. In this case, those nodes for making decisions on validating transactions are selected according to how much stake they have in the system. Assuming stakes are expressed as certain tokens (such as digital coins), each token is indexed, after which a search is made for the owner of that indexed token. Obviously, the more tokens a node has, the higher the chance of being selected for decision-making. Again, we see that simply creating multiple identities is not rewarding, as each of these identities will need to obtain tokens before being able to act.

None of these blockchain solutions are without security problems (see Saad et al. [2020] and Li et al. [2020] for extensive overviews). Yet from the perspective of preventing Sybil attacks, one can argue that they do their job well.

Sybil-resistant accounting mechanisms

When trust in central authorities within a distributed system drops, we need to resort to decentralized solutions. The fundamental problem is that Alice has no reason whatsoever to trust Bob *a priori* whenever the two get in touch and one needs the other to engage in a transaction of some sort. A common solution is to build a reputation system by which Bob can prove to Alice that he is trustworthy. At the same time, such a system should be resistant to a Sybil attack, as we illustrated in the web-of-trust model.

As part of their work on building robust blockchain systems, Otte et al. [2020] introduced an accounting mechanism by which a node Q can show that it has already done considerable work to convince node P to let it do

some work for P. The benefit for Q, of course, is that its own performed work further increases so that it may later use that to ask others to work for it. The **NetFlow** accounting mechanism introduced by Otte et al. is based on earlier work by Seuken and Parkes [2014]. Interesting for our current discussion is that this decentralized accounting mechanism is resistant to Sybil attacks.

The basic idea is that each node P in the network maintains a list of nodes that are interested in doing work for P, called the **choice set** of P, denoted as $\mathsf{choice}(\mathsf{P})$. Which node is selected from $\mathsf{choice}(\mathsf{P})$ depends on that node's reputation, expressed in terms of work done for others.

To this end, P maintains a view of what nodes in the network have done so far for each other. The view is necessarily *subjective*: it may be incomplete (because P has no knowledge of all the participating nodes) and it may be inaccurate. In any case, P does know what it has done for others, and also what others have done for P. This information is enough for P to compute a *capacity* $\mathsf{cap}(\mathsf{Q})$ for any node $\mathsf{Q} \in \mathsf{choice}(\mathsf{P})$:

$$\mathsf{cap}(\mathsf{Q}) = \max\{MF(\mathsf{Q},\mathsf{P}) - MF(\mathsf{P},\mathsf{Q}), 0\}$$

where $MF(\mathsf{P},\mathsf{Q})$ is the value of the maximum flow from P to Q, i.e., the amount of work that P has, or could have contributed to work done for Q, including the work done by others. For example, if node R directly contributed 3 units of work for Q, and R had processed 7 units for P, then P indirectly may have contributed 3 units of work for Q, yet through R. The reasoning is that R may never have been able to work for Q if not for the work it did for P. The capacity of node Q from P's perspective, is thus the result of the work Q has directly and indirectly done for P, minus the work P has done for Q. When positive, one could say that P "owes" Q some work to do. These capacities are then used in maximum-flow computations that result in reputation scores for each node.

Now let us consider a Sybil attack by a node $\mathsf{Q} \in \mathsf{choice}(\mathsf{P})$. To that end, Q creates n Sybil nodes $\mathsf{Q}_1^*, \ldots, \mathsf{Q}_n^*$. We denote $\mathsf{Q} = \mathsf{Q}_0^*$ as just another Sybil. First, note that for any work contributed by a Sybil Q_i^* to another Sybil Q_j^* to cause an increase in $\mathsf{cap}(\mathsf{Q}_i^*)$, at least two conditions need to hold: (1) Q_j^* needs to have contributed work to some honest node R and (2) that node R needs to have contributed (indirectly) to P. What this means, is that Q can successfully launch an attack only if it had already performed work for honest nodes. Furthermore, the attack makes sense only if Q expects that other, honest nodes will contribute work to Q, and that this work is more than Q's total contributions to its Sybils. This can happen if the total capacity $\mathsf{Tcap}(\mathsf{Q})$ of the Sybils can grow, with

$$\mathsf{Tcap}(\mathsf{Q}) = \sum_{k=0}^{n} \mathsf{cap}(\mathsf{Q}_k^*)$$

Now assume that a unit of work is contributed by P to one of the Sybils, say Q_i^*. In that case, $MF(\mathsf{P},\mathsf{Q}_i^*)$ increases by 1 unit, so that $\mathsf{cap}(\mathsf{Q}_i^*)$ drops by 1

unit, and so does $\mathsf{Tcap}(\mathsf{Q})$. This may continue until $\mathsf{Tcap}(\mathsf{Q}) = 0$, at which point P will direct its attention to other nodes and ignore any of the Sybils until their joint capacity has grown, notably because other honest nodes have contributed work to those Sybils.

Having the Sybils perform work for each other does not help: if Q_i^* performs a unit of work for Q_j^*, then $\mathsf{cap}(\mathsf{Q}_i^*)$ goes up by 1 unit, yet $\mathsf{cap}(\mathsf{Q}_j^*)$ goes down by 1, leaving $\mathsf{Tcap}(\mathsf{Q})$ unaffected. The only way that $\mathsf{Tcap}(\mathsf{Q})$ can go up without P contributing work to one of the Sybils, is if some other honest node R contributes work. If P is unaware of R, then P will not know about this work and keep $\mathsf{Tcap}(\mathsf{Q})$ the same. In other words, Q cannot benefit from work it does for R to trick P into contributing work to one of its Sybils, including itself. However, if P does know about R, we can expect that both will have the same perspective on the value of $\mathsf{Tcap}(\mathsf{Q})$. As a consequence, R will act the same as P: it will contribute work to one of the Sybils until $\mathsf{Tcap}(\mathsf{Q})$ drops to 0. At that point, Q is where it stood before the attack: empty-handed and in need of a node willing to let it do some work.

This rather intuitive reasoning shows that a proper accounting mechanism (in the case of NetFlow based on maximum flows in a network) is actually Sybil resistant. A precise and formal reasoning can be found in [Stannat et al., 2021]. Also, [Otte, 2016] will be useful in understanding some underlying details of NetFlow that we have purposefully omitted here.

9.4.3 Trusting a system

An important claimed feature of many blockchain systems is that they can operate without the need for a trusted third party (i.e., they are completely decentralized), in addition to the fact that there is also no need to trust any of the individual participants. To understand this reasoning, we need to separate the blockchain from the way it is maintained. The latter is strongly determined by the consensus protocol that is executed to figure out which block of validated transactions can be appended to the current blockchain. The former has everything to do with the transparency and protection of essentially a read-only distributed ledger. This is where ensuring data integrity is crucial.

Recall that a blockchain is literally a chain of blocks, with each block consisting of a number of validated transactions. When it comes to ensuring data integrity, we need a means to be able to detect that the current blockchain has not been tampered with, i.e., ensuring that any change to the current chain cannot go unnoticed and thus be flagged as an attack.

The principle of a blockchain is shown in Figure 9.20. The first block, called the **genesis block**, contains the number `0x00000000` which is effectively a null pointer. Important is that each block contains a number of transactions and a hash value over the data contained in the block. That hash value is copied to the successor in the chain, and also serves as input to the hash value of the succeeding block. This cryptographic linking between two blocks is

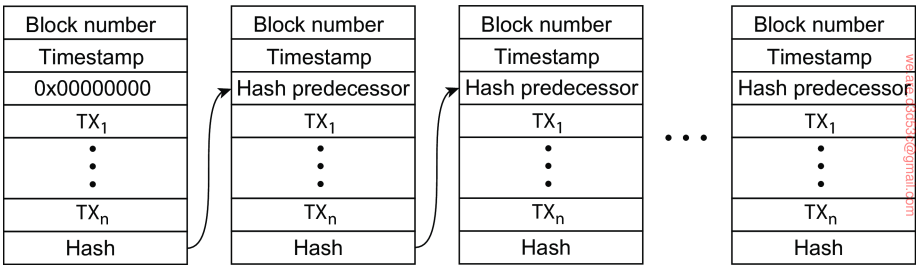


Figure 9.20: The principle of a blockchain.

important. Suppose that block B_i has hash value h_i and its successor, B_{i+1} has hash value h_{i+1}. Obviously, any change of block B_i will invalidate h_i. An attacker may compute a new hash value, say h_i^*, but this value will have to be propagated to block B_{i+1}, in turn invalidating h_{i+1}. In other words, an attack can be successful only if all blocks succeeding B_i are modified as well. Considering that a blockchain is assumed to be a read-only data structure, it will most likely have been massively replicated. Changing the chain while going unnoticed is deemed to be too difficult, to say the least.

A transaction can, in principle, take any form. Important is that a transaction has been validated, for example because signatures of the parties involved in the transaction have been verified as well as the content of the transaction. What this validation entails is thus specific to the application of a blockchain.

As a simple example, consider an e-voting system that is implemented using a blockchain. In such a system, voters are assumed to have been registered (and verified that they are entitled to vote). Registration entails that a voter receives a single token that is to be used for voting. When Alice wants to cast her vote, she can essentially engage in a transaction with her favorite candidate, say Bob, and transfer the token to Bob. The latter is the actual transaction. Validating that transaction means, for example, that the entire chain needs to be checked to see whether that token had not been used before. Of course, e-voting through blockchains violates some of its claimed advantages, such as being fully decentralized.

E-voting by making use of blockchains is also illustrative in the face of trust. Although one can set up a system with a tamper-proof and verifiable distributed ledger, may not be enough to warrant trust. In the case of e-voting, Park et al. [2021] are clear: blockchains suffer from the same problems as any electronic voting system. What it then boils down to is that one needs to trust that the distributed system has been properly implemented and secured against attacks.

As also argued by Werbach [2018], it is not just that one needs to trust the correctness of the implementation of a system, but also that it functions

as intended. In the case of blockchains, an upcoming phenomenon is the use of **smart contracts**: programs that are automatically executed when certain conditions are met within a block. This can be as simple as sending a notification to Alice and Bob when their transaction made it to the chain, but could also entail automatically engaging new transactions. Even if such contracts can be proven to be correct given a specification, the question is whether the specification meets intentions. Practice in law shows that humans are not always that good at transforming intentions into specifications (or regulations).

Finally, the claim that (permissionless) blockchains do not require trust in each participating entity, but that only the ledger needs to be trusted, may not stand so firm. In practice, there are intermediaries who offer services to end users for making use of blockchain applications, if only to make it easy for people to participate. Those intermediaries, and the systems they use, will need to be trusted. In many ways, blockchain systems and any other distributed system share a lot when it comes to trusting a system.

For an extensive and in-depth discussion on trust in computer systems, we refer the interested reader to Bursell [2022].

9.5 Authorization

We now concentrate on providing an entity proper access to a system's resources after that entity has been authenticated.

9.5.1 General issues in access control

To understand the various issues involved in access control, the simple model shown in Figure 9.21 is generally adopted. It consists of *subjects* that issue a request to access an *object*. An object is very much like the objects we have been discussing so far. It can be thought of as encapsulating its own state and implementing the operations on that state. The operations of an object that subjects can request to be carried out are made available through interfaces. Subjects can best be thought of as being processes acting on behalf of users, but can also be objects that need the services of other objects in order to carry out their work.

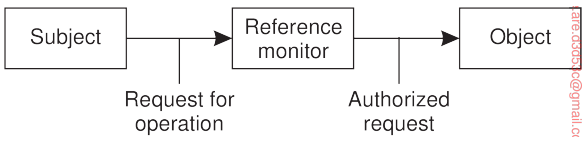


Figure 9.21: General model of controlling access to objects.

Controlling the access to an object is all about protecting the object against invocations by subjects that are not allowed to have specific (or even any) of the methods carried out. Also, protection may include object management issues, such as creating, renaming, or deleting objects. Protection is often enforced by a program called a **reference monitor**. A reference monitor records which subject may do what, and decides whether a subject is allowed to have a specific operation carried out. This monitor is called (e.g., by the underlying trusted operating system) each time an object is invoked. Consequently, it is critical that the reference monitor is itself tamperproof: an attacker must not be able to fool around with it.

Access control policies

When considering the protection of an application, there are essentially three approaches that can be followed, as shown in Figure 9.22. The first approach is to concentrate directly on the protection of the data that is associated with the application. By direct, we mean that irrespective of the various operations that can possibly be performed, the primary concern is to ensure data integrity. Typically, this type of protection occurs in database systems in which various integrity constraints can be formulated that are automatically checked each time a data item is modified (see, for example, Doorn and Rivero [2002]).

The second approach is to concentrate on protection by specifying exactly which operations may be performed, and by whom, when certain data or resources are to be accessed. In this case, the focus of control is strongly related to **access control mechanisms**, which we discuss extensively later in this chapter. For example, in an object-based system, it may be decided to specify for each method that is made available which specific clients are permitted to invoke that method. Alternatively, access control methods can be applied to an entire interface offered by an object, or to the entire object itself. This approach thus allows for various granularities of access control.

A third approach is to focus directly on users by taking measures by which only specific people, or their proxies in the case of, for example, delegation, have access to the application, irrespective of the operations they want to carry out. For example, Alice may have been given read access to a specific file, but no write access. On the other hand, Bob may be allowed to change the file, or even delete it. Again, combinations of these three approaches are possible and occur in practice.

In general, we can distinguish several **access control policies**. A common distinction is often made into the following four types:

1. Mandatory access control
2. Discretionary access control
3. Role-based access control

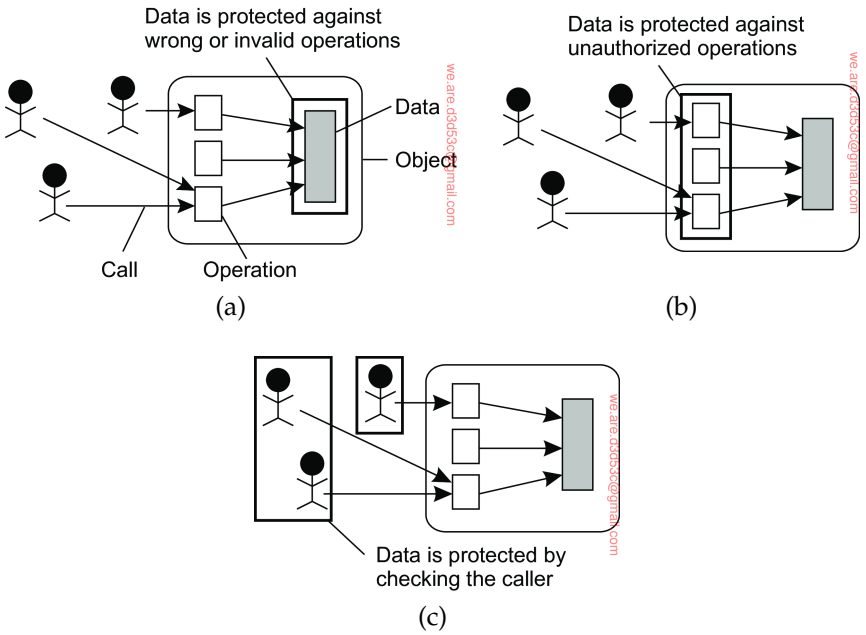


Figure 9.22: Three approaches for protection against security threats. (a) Protection against invalid operations, (b) Protection against unauthorized operations. (c) Protection against unauthorized callers.

4. Attribute-based access control

Perhaps conceptually the simplest access control policy is that of **Mandatory Access Control** (**MAC**). In this case, access control policies are decided beyond the control of an individual: a central administration defines who gets access to what. In the context of military security, data is typically labeled according to some level of secrecy, ranging from "public access" to "top secret." Only if Alice has been assigned the appropriate confidentiality level, will she get access to data up to that level. Although conceptually simple, mandatory access control can be quite difficult from a security-management perspective.

Easier and generally more common in many distributed systems is to make use of **Discretionary Access Control** (**DAC**). In this case, the owner of an object (such as a file) is entitled to change access rights (such as read, write, or execute), but also who may have access to that object. Typically, in Unix systems a distinction is made between the owner, a group, and the world. MAC and DAC are often seen to be combined.

As a next step, we can make objects in a distributed system less dependent on owners, but instead concentrate on how they are supposed to be used by looking at the roles of people. This leads to what is known as **Role-based**

Access Control (**RBAC**) [Ferraiolo et al., 2007]. The essence of RBAC is that users are not authorized based on their identity, but based on the role they have within an organization. For example, within a university, we can distinguish roles such as teacher, student, project controller, data steward, group leader, dean, etc. When Alice is authenticated, she will also be linked to a role, which may need to be indicated at the time she logs into the system. Then, in the associated role, she will have been granted various access rights to specific objects.

More recently, attention is being paid to **Attribute-based Access Control** (**ABAC**) [Hu et al., 2014; 2015]. In ABAC, attributes of users *and* of objects they want to access are considered for deciding on the specific access rule. ABAC allows for much finer-grained access control in comparison to RBAC. For example, not all teachers at a university may be allowed access to the exam results of all students. Instead, one may want to formulate the constraint that only teachers of the Distributed Systems course have read access to grades from students enrolled in that course. Likewise, only the formal leader of the group on Pervasive Systems may be entitled to read the yearly review reports from members in that group, except the ones of the full professors. We will return to ABAC later in this chapter.

Access control matrix

A common approach to modeling the access rights of subjects with respect to objects is to construct an **access control matrix**. Each subject is represented by a row in this matrix; each object is represented by a column. If the matrix is denoted M, then an entry $M[s, o]$ lists precisely which operations subject s can request to be carried out on object o. In other words, whenever a subject s requests to perform the operation m on object o, the reference monitor should check if m is listed in $M[s, o]$. If m is not listed in $M[s, o]$, the execution of the operation fails.

Considering that a system may easily need to support thousands of users and millions of objects that require protection, implementing an access control matrix as a true matrix, is not the way to go. Many entries in the matrix will be empty: a single subject will generally have access to relatively few objects. Therefore, other, more efficient ways are followed to implement an access control matrix.

One widely applied approach is to have each object maintain a list of the access rights of subjects that want to access the object. In essence, this means that the matrix is distributed column-wise across all objects, and that empty entries are left out. This type of implementation leads to what is called an **access control list** (**ACL**). Each object is assumed to have its own associated ACL.

Another approach is to distribute the matrix row-wise by giving each subject a list of **capabilities** it has for each object. A capability thus corresponds

to an entry in the access control matrix. Not having a capability for a specific object means that the subject has no access rights for that object.

A capability can be compared to a ticket: its holder is given certain rights that are associated with that ticket. It is also clear that a ticket should be protected against modifications by its holder. One approach that is particularly suited in distributed systems, is to protect (a list of) capabilities with a signature.

The difference between how ACLs and capabilities are used to protect the access to an object is shown in Figure 9.23. Using ACLs, when a client sends a request to a server, the server's reference monitor will check whether it knows the client and if that client is known and allowed to have the requested operation carried out, as shown in Figure 9.23(a).

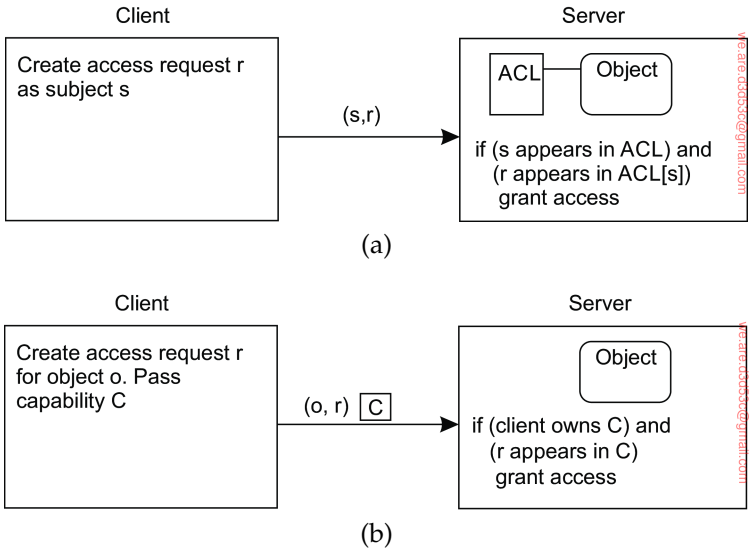


Figure 9.23: Comparison between ACLs and capabilities for protecting objects. (a) Using an ACL. (b) Using capabilities.

When using capabilities, a client simply sends its request to the server. The server is not necessarily interested in whether it knows the client; the capability may say enough. However, such an approach does not prevent a client to pass its capability to another program. This is precisely the problem of delegation that we discussed before. Therefore, in practice, the client will one way or the other have been registered at the server, while the capability is simply bound to that specific client. At that point, the server need only check whether the capability is valid (i.e., belongs to the client) and whether the requested operation is listed in the capability. This approach to protecting objects using capabilities is shown in Figure 9.23(b).

9.5.2 Attribute-based access control

Let us concentrate a bit more on an emerging type of access control, namely the one based on attributes. **Attribute-based access control** (**ABAC**) is a powerful access control policy that can viewed as superseding other policies. Yet, ABAC still has a long way to go when it comes to its adoption in distributed systems, as also explained by Servos and Osborn [2017] who investigated the various issues and challenges that ABAC is facing.

Basic model

In essence, ABAC assumes that access policies are described in terms of attributes and their values. Attributes can belong to many things, including users, objects, and also the current state of a distributed system. To make things concrete, let us consider ABAC for a university setting. We could then have the following:

- **User attributes** may include name, data of birth, current roles, home address, department, qualifiers obtained, contract status, etc. Depending on the role (e.g., teacher or student), a different set of attributes may be relevant.
- **Object attributes** can be anything, such as creator, last-modified time, version number, file type, file size, but also information related to its content, such as the student or course associated with the object.
- **Environmental attributes** describe the current state of the system in which access is requested, including date and time, current workload, maintenance status, storage properties, available services, and such.
- **Connection attributes** provide information on the current session, for example an IP address, session duration, available bandwidth and latency estimates, type and strength of security used, and so on.
- Finally, **administrative attributes** may reflect global policies, including minimal security settings, general access regulations, and maximum session durations, to name a few.

Example: The Policy Machine

To make matters concrete, let us look at an example system for attribute-based access control, namely the **Policy Machine**, described in [Ferraiolo et al., 2011; 2015]. The architecture of the Policy Machine is a simple client-server model, where a centralized server maintains a database of *(attribute,value)* pairs associated with users, applications, operations, and objects. At the client side, a small support program is used to communicate with the server, notably for handling the access privileges of client-side applications. Obviously, the server and the client-side support program need to be trusted to work as

specified. Essential to the Policy Machine is the expression of access control rules. To that end, a distinction is made between assignments, prohibitions, and obligations.

An **assignment** is simply telling the system that a user u has an associated attribute ua: u is said to be assigned to ua, denoted as $\mathsf{u} \rightarrow \mathsf{ua}$. Likewise, an object o can be assigned to an object attribute oa. In the same fashion, an attribute can be assigned to another attribute. For example, if $\mathsf{ua}_1 \rightarrow \mathsf{ua}_2$, then all users associated with ua_1 will also be associated with ua_2. Assignments thus essentially yield sets: if $\mathsf{u} \rightarrow \mathsf{ua}$, then $\mathsf{u} \in \mathsf{ua}$.

In a similar vein, user attributes can be assigned to operation sets, and operation sets can be assigned to object attributes. This leads to triplets of the form $\mathit{allowed}(\mathsf{ua}, \mathsf{ops}, \mathsf{oa})$ which expresses that a user u can perform operation op on object o only if $\mathsf{u} \in \mathsf{ua}, \mathsf{op} \in \mathsf{ops}, \mathsf{o} \in \mathsf{oa}$.

A **prohibition** expresses what users or applications are *not* allowed to do. In particular, a user can be denied certain operations of certain objects, expressed as $\mathit{denied}(\mathsf{u}, \mathsf{ops}, \mathsf{os})$ with ops a set of operations and os a set of objects. To ensure that operations are truly restricted to a given set of objects, that is, denied to all other objects, the Policy Machine supports expressions $\mathit{denied}(\mathsf{u}, \mathsf{ops}, \neg\mathsf{os})$. In other words, if u requests to perform operation $\mathsf{op} \in \mathsf{ops}$ on an object $\mathsf{o} \notin \mathsf{os}$, the Policy Machine will deny that request.

Finally, an **obligation** is used to automatically take actions when certain events happen. A simple example is denying a user to copy information from one set of files to another:

$$\mathsf{when\ u\ reads\ f} \in \mathsf{fs\ then}\ \mathit{denied}(\mathsf{u}, \{\mathsf{write}\}, \neg\mathsf{fs}).$$

This is an interesting example in the sense that it forms the basis for preventing information leakage. We have mentioned only a part of what can be expressed by the Policy Machine, yet it should be clear that attribute-based access control is by itself a powerful approach for expressing many kinds of access control policies. Interestingly, the Policy Machine can also be used for MAC, DAC, and RBAC.

The simplest case is perhaps RBAC. By simple assignments such as $\mathsf{Alice} \rightarrow \mathsf{Teachers}$ we effectively assign the role of teacher to Alice. Alice can be assigned other roles as well, but the important aspect is that the system will take those roles as its starting point for access control. We do note that advanced RBAC schemes require more sophisticated assignments in terms of the Policy Machine, yet the principle should be clear.

When it comes to mandatory access control (MAC), the basic idea is that one can introduce a notion of security levels $L_1, \ldots, L_n$ where L_1 denotes the lowest and L_n the highest level of security. More concrete, if Alice has access to "top secret" documents, she should also have access to "secret" documents. The idea is straightforward: each object is assigned to an appropriate object security-level attribute, such as OL_i. Likewise, each user is assigned to a

user security-level attribute, such as $\mathsf{UL_j}$. Moreover, each user security-level attribute $\mathsf{UL_i}$ is assigned to $\mathsf{UL_{i-1}}$ indicating that all users assigned to $\mathsf{UL_i}$ have the same rights as users at a less secure level, represented by the user attribute $\mathsf{UL_{i-1}}$.

The real issue with MAC is that we need to prevent information leaking from higher levels to lower levels. As before, this can be prevented by the obligation

$$\mathsf{when\ u\ reads\ f} \in \mathsf{OL_i\ then}\ \mathit{denied}(\mathsf{u}, \{\mathsf{write}\}, \neg\{\cup_{k=i}^{n}\mathsf{OL_k}\}).$$

So, for example, if we have five security levels, then anyone who reads a document from level $\mathsf{OL_3}$, cannot write to documents at level $\{\mathsf{OL_1}, \mathsf{OL_2}\}$ or equivalently cannot write to documents not in $\{\mathsf{OL_3}, \mathsf{OL_4}, \mathsf{OL_5}\}$.

Finally, let us have a look at discretionary access (DAC). The essence of DAC is that each object has an owner, and that this owner can actually set access rights for others. For example, in Unix systems, a user alice belonging to a group teachers can set read, write and execute access rights for herself, members of the group teachers, and anyone else. A user with administrative privileges (typically, the root user) can additionally assign objects to users and groups. Some simple examples illustrate these principles.

```
alice$ chmod u=rw,g=r,o= document
```

expresses that user alice changes the rights for herself to read (r) and write (w) document, yet allows members of the group to which document has been assigned to only read the file, while anyone else has no access rights whatsoever. Likewise,

```
root# chown alice:teachers document
```

tells us that the user root assigns ownership of document to user alice and group teachers.

Following Ferraiolo et al. [2011], one can create a unique user attribute per user. So, Alice would be known to the system by assigning her to the user attribute alice. Likewise, each group is represented by a user attribute, and we could, for example, perform the assignment alice $\rightarrow$ teachers. Following Unix conventions, we create an object attribute alice home which represents the home directory of user alice. The trick is to make sure that Alice is entitled to execute an appropriate set of administrative commands, but also to ensure that the results match the DAC model. For example, we would need Alice to allow to create objects within the home directory. This can be done by assigning a create operation to the object attribute alice home and installing the obligation

$$\mathsf{when\ alice\ performs\ create(o)\ at\ alice\ home\ then\ o} \rightarrow \mathsf{alice\ home}.$$

When considering the Unix DAC model, in which (almost) everything is a file, we also need to consider creating objects that represent a subdirectory. Again, being a directory can be expressed in terms of an object attribute, and typically each directory object will need to be associated with a parent directory. In general, we may thus have:

when alice performs create(o) at directory then o → directory.

These examples also illustrate that expressing a DAC model in an ABAC model requires careful consideration of all the operations and elements of the specific DAC model at hand. In fact, being able to accurately and completely expressing other models in terms of attribute-based access control is considered to be one of the open challenges [Servos and Osborn, 2017].

9.5.3 Delegation

Now consider the following problem. A user Alice makes use of an e-mail service provider who stores her mailbox somewhere within that provider's organization. Typically, she is required to log in to the provider through her Web browser, after which she gets to see an appropriate interface for handling her mail. However, Alice would prefer to use her own favorite mail client, which is installed on her laptop. The core question is how to allow that mail client to act on behalf of Alice? In other words, the mail client would need to get access to Alice's mailbox, and allow her to then retrieve new mail, read mail, send mail, and so on. In essence, Alice would like to **delegate** her access rights to the mail client, and in such a way that the mail client can also automatically log in and do its work.

Delegation of access rights is an important technique for implementing protection in computer systems and distributed systems, in particular. The basic idea is simple: by passing certain access rights from one process to another, it becomes easier to distribute work between several processes without adversely affecting the protection of resources. In the case of distributed systems, processes may run on different machines and even within different administrative domains. Delegation can avoid much overhead, as protection can often be handled locally.

There are several ways to implement delegation. The least favorable one is to hand over the user credentials directly to an application. In fact, doing so would be a bad idea: there is, in general, no reason to trust an application or the machine it is running on. A much better and general approach, as described by Neuman [1993] and implemented in the Kerberos system, is to make use of a proxy. A **proxy** in the context of security in computer systems is a token that allows its owner to operate with the same or restricted rights and privileges as the subject that granted the token. A process (or user for that matter) can create a proxy with at best the same rights and privileges it has itself. If a process creates a new proxy based on one it currently has, the

derived proxy will have at least the same restrictions as the original one, and possibly more.

Before considering a general scheme for delegation, consider the following two approaches. First, delegation is relatively simple if Alice knows everyone. If she wants to delegate rights to Bob, she merely needs to construct a certificate saying "Alice says Bob has rights R," such as $[A, B, R]_A$ (i.e., the message containing the identifiers A of Alice and B of Bob, along with the specified access right R, and subsequently signed by Alice). If Bob wants to pass some of these rights to Chuck, he will ask Chuck to contact Alice and ask her for an appropriate certificate.

In a second case, Alice can simply construct a certificate saying, "The bearer of this certificate has rights R." However, in this case, we need to protect the certificate against illegal copying. Neuman's scheme handles this case, as well as avoiding the issue that Alice needs to know everyone to whom rights have to be delegated. Note that in this case, she does trust Bob in delegating rights to Chuck. In practice, this means that the user Alice trusts the application to which she delegated some rights, to correctly and justifiably delegate some of those rights to another application (she may not even be aware of). One obvious way to prevent further delegation is to ensure that it is not specified as part of Bob's rights. The service that hands out a delegation certificate for Bob, will check whether he actually is authorized to do so.

A proxy in Neuman's scheme has two parts, as illustrated in Figure 9.24. Let A be the process that created the proxy. The first part of the proxy is a set $C = \{R, PK_{proxy}\}$, consisting of a set R of access rights that have been delegated by A, along with a publicly known part of a secret that is used to authenticate the holder of the certificate. We will explain the use of PK_{proxy} below. The certificate carries the signature $sig(A, C)$ of A, to protect it against modifications. The second part contains the other part of the secret, denoted as SK_{proxy}. It is essential that SK_{proxy} is protected against disclosure when delegating rights to another process.

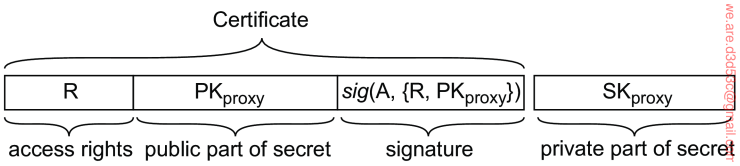


Figure 9.24: The general structure of a proxy as used for delegation.

Another way of looking at the proxy is as follows. If Alice wants to delegate some of her rights to Bob, she makes a list of rights (R) that Bob can exercise. By signing the list, she prevents Bob from tampering with it. However, having only a signed list of rights is often not enough. If Bob would like to exercise his rights, he may have to prove that he actually got the list

from Alice and did not, for example, steal it from someone else. Therefore, Alice comes up with a very nasty question ($\mathsf{PK}_{\mathsf{proxy}}$) that only she knows the answer to ($\mathsf{SK}_{\mathsf{proxy}}$). Anyone can easily verify the correctness of the answer when given the question. The question is appended to the list before Alice adds her signature.

When delegating some of her rights, Alice gives the signed list of rights, along with the nasty question, to Bob. She also gives Bob the answer, ensuring that no one can intercept it. Bob now has a list of rights, signed by Alice, which he can hand over to, say, Chuck, when necessary. Chuck will ask him the nasty question at the bottom of the list. If Bob knows the answer to it, Chuck will know that Alice had indeed delegated the listed rights to Bob.

An important property of this scheme is that Alice need not be consulted. In fact, Bob may decide to pass on (some of) the rights on the list to Dave. In doing so, he will also tell Dave the answer to the question, so that Dave can prove the list was handed over to him by someone entitled to it. Alice never needs to know about Dave at all.

A protocol for delegating and exercising rights is shown in Figure 9.25. Assume that Alice and Bob share a secret key $\mathsf{K}_{\mathsf{A,B}}$ that can be used for encrypting messages they send to each other. Then, Alice first sends Bob the certificate $[\mathsf{R}, \mathsf{PK}_{\mathsf{proxy}}]_{\mathsf{A}}$, signed by Alice. There is no need to encrypt this message: it can be sent as plaintext. Only the private part of the secret needs to be encrypted, shown as $K_{A,B}(\mathsf{SK}_{\mathsf{proxy}})$ in message 1.

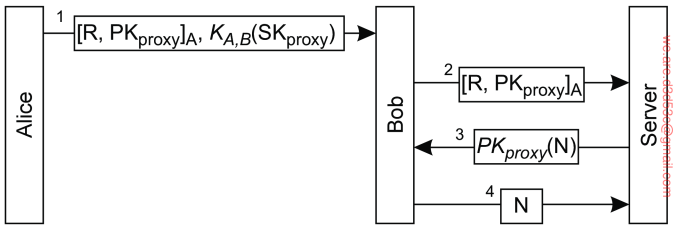


Figure 9.25: Using a proxy to delegate and prove ownership of access rights.

Now suppose that Bob wants to carry out an operation at a specific server. Also, assume that Alice is authorized to have that operation carried out, and that she has delegated those rights to Bob. Therefore, Bob hands over his credentials to the server in the form of the signed certificate $[\mathsf{R}, \mathsf{PK}_{\mathsf{proxy}}]_{\mathsf{A}}$.

At that point, the server will be able to verify that C has not been tampered with: any modification to the list of rights, or the nasty question will be noticed because both have been jointly signed by Alice. However, the server does not know yet whether Bob is the rightful owner of the certificate. To verify this, the server must use the secret that came with $[\mathsf{R}, \mathsf{PK}_{\mathsf{proxy}}]_{\mathsf{A}}$. To that end, the server sends Bob a **nonce** (which is just an arbitrarily chosen string) N, encrypted with $\mathsf{PK}_{\mathsf{proxy}}$. By decrypting $PK_{proxy}(\mathsf{N})$ and returning N, Bob proves he knows the secret and is thus the rightful holder of the certificate.

Note 9.6 (More information: OAuth (Open Authorization))
Delegation is, in principle, fairly straightforward. Yet, it has already taken many years to come to a practical scheme that was acceptable by many players. Also, solutions that are designed by multiple parties coming from different organizations tend not to be examples of simplicity, to say the least. This is certainly true for **Open Authorization**, generally known as **OAuth**. Nevertheless, OAuth is by now an important delegation protocol used by many organizations including Amazon, Google, Facebook, Microsoft, and Twitter. Its main purpose is to grant an application access to resources that are normally accessible to a user only through a Web interface. A typical example is that of using a local mail client, which acts on behalf of its owner, as described previously.

Let us take a closer look at the OAuth 2.1 framework, which distinguishes four different roles:

- A **resource owner**, which is typically an end user, such as Alice who normally accesses her mail through her provider's Web service.
- A **client**, which is an application that one would like to act on behalf of the resource owner, such as an e-mail client.
- A **resource server**, in practice forming some kind of Web interface through which a person like Alice would normally first authenticate herself after which she can access and manage her mail.
- An **authorization server**, forming the core entity when it comes to handing out certificates to a client on behalf of a resource owner.

The client application will have to be known at the authorization server through some separately executed registration process (which can also take place as part of the delegation process). The result is that the client receives its own, unique identifier, say cid. When Alice wants her application to operate on her behalf when it comes to a list R of rights, the application sends

$$\text{Client: } \textit{send } [\mathsf{cid}, \mathsf{R}, H(\mathsf{S})]$$

in which the hash of a temporary secret S is sent, computed with the hash function H. As its name suggests, the secret S is (for now) known only to the client application. In the message above, we omit some additional information, such as details of the hash function, as well as some other options.

If Alice is not yet logged in to the authorization server, she will be required to do so and confirm that the client application is entitled to the rights as expressed by R. At that point, the client receives a temporary **authorization code** AC from the server, which it should use to get the final **access token**. The client then sends a request for that token:

$$\text{Client: } \textit{sends } [\mathsf{cid}, \mathsf{AC}, \mathsf{S}].$$

By sending the secret S to the authorization server, the latter can check that it is dealing with the same client application as before, as it simply needs to compute $H(\mathsf{S})$ that was sent with the previous message from that application. Note that by now, the authorization server has verified that Alice indeed wants to delegate

access rights to the client. The client simply needs to prove that it is the same one that initiated the delegation request. If all is verified, the authorization server returns an access token AT to the client.

It should be clear that an access token is precious data: it gives the client application the same rights as Alice would have when going directly to the resource server. Therefore, the access token is communicated through a secure channel, and is also stored securely on the machine hosting the client (such as Alice's laptop). Likewise, one way or the other the client will have to authenticate itself, which is determined at registration time. There are different ways OAuth 2.1 supports authentication, a simple one being the scheme by which the client has a private key and the server keeps the associated public key, as we explained previously, when discussing SSH.

OAuth 2.1 distinguishes two types of access tokens. The first one is essentially just an identifier that the resource server can use to check the access rights of the client application. It does so by passing the token to the authorization server and retrieving those rights. This scheme requires that the resource server contacts the authorization server every time the client wants to perform an operation.

The second type takes the form of a (signed) certificate: it contains all the access rights of the client as granted by Alice. Those rights will hold until the certificate expires (and indeed, with this type of access token, there is an associated expiration time).

We have given a very concise description of OAuth 2.1, omitting many details and many of its options. The specification of OAuth 2.0 is found in [Hardt et al., 2012]. There are also several books that explain how to set up clients and servers so that they adhere to the framework (see, e.g., Parecki [2020] and Boyd [2012]).

9.5.4 Decentralized authorization: an example

So far, we have been discussing various authentication and authorization solutions, of which many have in common that there is a centralized component where permissions are checked and granted (or denied). For example, Kerberos deploys a centralized authentication server and a centralized ticket-granting service. Likewise, when dealing with delegation, a client application generally needs to be registered at an authorization server to receive a capability to perform certain operations at a service, or that the service can check the permissions at the authorization server. Each of these types of centralized services will have been made robust through replication using protocols such as Paxos or Byzantine Fault Tolerance. Also, using straightforward workload-balancing schemes, we can relatively easily scale out servers. For example, we may decide that all requests coming from applications with an ID less than some value K go to server #1, while the requests from other applications go to server #2.

The situation becomes more difficult when we decentralize a service across multiple administrative units, such as across different departments or

organizations. We then bump into the problem of administrative scalability [Neuman, 1994]. In this section, we take a look at a scalable decentralized authorization service, called **WAVE**, specifically developed to operate across multiple organizations. WAVE is a good example of some of the things we need to deal with when scaling out across administrative domains. Detailed information on WAVE can be found in [Andersen, 2019; Andersen et al., 2019].

Graph-based authorization

Fundamental to the WAVE framework is the application of **graph-based authorization**. In this scheme, a global, possibly worldwide directed graph is maintained. Each vertex represents an entity that is capable of granting or receiving permissions. For example, a vertex may represent Alice's home directory (where she has the right to, for example, create and delete files, and set various permissions, including giving Bob access to some files). A vertex could also represent a department in charge of a collection of applications, an online cloud service that offers various resources, and so on. Each entity has a collection of public keys, from which a hash is derived that serves as a means to look up those keys. The collection of associated private keys is, of course, kept secret by the party represented by the entity. The key pairs are used for various protections and signatures that we describe below.

Each directed edge represents a delegation of permissions from one entity (the **issuer**) to another (the **subject**). For example, Alice can, as an issuer, delegate specific file permissions to Bob, the subject. In WAVE, an edge represents an **attestation**. Permissions are always granted on **resources**, such as files, database (records), devices, and so on. An entity B gathers proof of its right to operate on a resource owned by A by showing that there is a directed path in the graph from A to B.

To this end, WAVE uses a policy mechanism called an **RTree**, which stands for *resource tree*. Keeping it simple, a resource is denoted by a URI such as `user-entity/music/albums/popular/band/`. The first element in the URI is particularly important: it identifies the entity responsible for the resource and the one which is responsible for all permissions. Each resource has an associated collection of permissions, which is part of an RTree. The identification of a resource, a specification of granted permissions on that resource, a duration for which the grant is valid, and possible restrictions on further delegation add up to an **RTree statement**. An attestation will then roughly consist of:

- The hash of the issuer and that of the subject.
- Identifiers of where information on issuer and subject can be found.
- A set of RTree statements.
- A signature of the issuer.

There is more related to an attestation, which we describe below. Andersen [2019] shows that with these basic ingredients, graph-based authorization can be used to emulate DAC and RBAC models.

There are two important issues that need to be dealt with. First, the graph needs to be stored and maintained in a scalable and fully decentralized fashion. Second, delegating permissions is one thing, but letting the whole world know that Alice delegated specific permissions to Bob is maybe not such a good idea. In other words, there is data that needs to be protected. Let us next look into these issues.

Decentralized storage

It is important to realize that the authorization service need, essentially, store information only on resources and attestations. Moreover, there should preferably be no reason at all to trust the storage: the less, the better. As we stated before, there is increasingly more interest in using distributed systems in which the individual participating parties need not be trusted. Instead, one relies on the fact that the system ensures that the provided information can be trusted. This is precisely what happens in distributed ledgers implemented as blockchains. A similar reasoning is applied in WAVE: assuming that each entity is responsible for its own resources, then the party represented by that entity need not be trusted, as long as the information it provides is reliable. In essence, only availability of relevant information is required. With this in mind, Andersen [2019] formulated the following minimal requirements for storage:

1. **Proof of monotonicity**: A storage operates as an unforgeable append-only log.
2. **Proof of inclusion**: When an object is added or retrieved, it must exist in storage.
3. **Proof of nonexistence**: When an object does not exist, the storage should state so.
4. **Proof of nonequivocation**: The storage offers the same view to every client.

Together, these requirements ensure the integrity of the global authorization graph. Note that these requirements come very close to what blockchain-based distributed ledgers offer.

Assuming that these requirements can be met, we may also assume that each entity offers its own local storage. In this way, scalability should, in principle, be easy to realize as long as an application can indeed identify the location where that entity's information is stored. Including global identifiers along with proper name resolution as provided through DNS should do the

trick. Of course, bundling multiple entities so that they make use of the same physical storage may be required for reaching certain efficiency goals, yet it should be clear that size scalability is not fundamentally hindered by letting each entity have its own storage. We will subsequently speak of an **entity store**.

Again, keeping things simple, every entity store essentially maintains an append-only **operation log**. Every time an object is appended to the log, it is cryptographically bound to its predecessor, very much like the secure linking in blockchains. In addition, the hash value of the appended object is securely appended to a separate **map log**, which is subsequently returned to the client performing the operation on the entity store.

When a client wants to retrieve on object (such as an attestation), it provides the entity store with the appropriate object identifier (namely its hash value) and its most recent version of the map log. Suppose the object has indeed been logged. The store will return (1) the object, (2) a proof that the object existed in the operation log (by returning a chain of hashes), (3) that the hash associated with the object is part of the map log, and (4) that the current map log is consistent with the one the client provided. Along the same lines, if the object does not exist, the store simply returns nil and proves that the object was not in the operation log by demonstrating that the hash value can never be part of the map log.

Meanwhile, separate processes regularly audit an entity store by reconstructing the operation log and checking its consistency with what is in the store. The scheme we just described is incomplete. More information can be found in [Andersen, 2019].

Data protection

Finally, let us see how data, and in particular attestations, can be protected. Again, we concentrate on only the simplest case of WAVE, called **structural security**. Assume A creates and attestation att for B. It appends the decryption key $\mathsf{SK}_{\mathsf{A}}^{\mathsf{att}}$ to the message, but not after encrypting it with an appropriate public key $\mathsf{PK}_{\mathsf{B}}^{\mathsf{att}}$ (as we shall see shortly, adding the decryption key by A, and only A, is strictly speaking not necessary):

$$\mathsf{A\ sends:}\ PK_B^{att}(\underbrace{[\mathsf{att}|\mathsf{SK}_{\mathsf{A}}^{\mathsf{att}}]}_{\mathsf{m}_1})$$

Let us now also assume that B creates an attestation att′ for C that is derived from att. For example, if att allowed B to write to a specific file and to pass on read permissions to others, B may allow C to read the file. No one but A, B, and C should know about this: it is simply no one else's business. However, in order for C to read the file, it will have to prove it may do so.

When B creates att′, it sends the following to C:

$$\text{B sends: } PK_C^{att'}(\underbrace{[\mathsf{att}'|\mathsf{m}_1|\mathsf{SK}_B^{\mathsf{att}}]}_{\mathsf{m}_2})$$

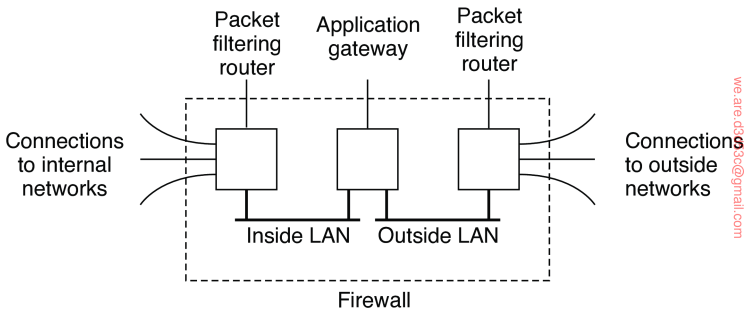
Being able to decrypt this message, C will find att′, m_1, and $\mathsf{SK}_B^{\mathsf{att}}$. Having $\mathsf{SK}_B^{\mathsf{att}}$, it can decrypt m_1 revealing att. Note that there is no reason for either A or B to be online: C has all the information it needs to prove it is allowed to read the file. It should now also be clear that having A also send $\mathsf{SK}_A^{\mathsf{att}}$ is not necessary. We include it to show the systematic approach of chaining attestations such that a node on a path can reveal all necessary attestations in a reverse order. Note, by the way, that this scheme strongly resembles the way that delegation is handled in Kerberos.

9.6 Monitoring

An important security aspect of any system is to check that policies are sufficient, correctly implemented, and enforced. It would be naive to assume that this is always the case. For this reason alone, it is important to keep track of what is going on in a distributed system so that proper measures can be taken when needed. A passive way of keeping track is to simply log special events: when do users sign in to a system, when and which applications access specific resources, and so on. Logging is important to understand what to do *after the fact*, yet will not help when the damage has been done. In this section, we take a closer look at an increasingly more important aspect of monitoring for security, namely **intrusion detection**, which is all about detecting unauthorized activities. As intrusion detection is often tied to a single computer, we only briefly discuss the basics to subsequently concentrate on distributed monitoring. We start our discussion by having a look at an almost ubiquitous intrusion-detection system, namely firewalls.

9.6.1 Firewalls

Before we take a look at the basics of intrusion detection, consider a common approach to ensuring that intruders stay out: using a **firewall** [Cheswick and Bellovin, 2000; Zwicky et al., 2000]. Firewalls form one of the most often used protection mechanisms in networked systems. Essentially, a firewall disconnects any part of a distributed system from the outside world, as shown in Figure 9.26. All outgoing, but especially all incoming packets are routed through a special computer and inspected before they are passed through. Unauthorized traffic is discarded and not allowed to continue. An important issue is that the firewall itself should be heavily protected against any kind of security threat: it should never fail. Equally important is that the rules that prescribe what can pass through are consistent and establish what is intended.



we.are.d0n3@gmail.com

Figure 9.26: A common implementation of a firewall.

As reported by Wool [2010] and Voronkov et al. [2017], properly configuring a firewall is a considerable challenge.

Firewalls essentially come in two different flavors that are often combined. An important type of firewall is a **packet-filtering gateway**. This type of firewall operates as a router and makes decisions whether to pass a network packet based on the source and destination address as contained in the packet's header. Typically, the packet-filtering gateway shown on the outside LAN in Figure 9.26 would protect against incoming packets, whereas the one on the inside LAN would filter outgoing packets. For example, to protect an internal Web server against requests from hosts that are not on the internal network, a packet-filtering gateway could decide to drop all incoming packets addressed to the Web server.

More subtle is the situation in which a company's network consists of multiple local-area networks. Each LAN can be protected using a packet-filtering gateway, which is configured to pass incoming traffic only if it originated from a host on one of the other LANs. In this way, a private virtual network can be set up.

The other type of firewall is an **application-level gateway**. In contrast to a packet-filtering gateway, which inspects only the header of network packets, this type of firewall actually inspects the content of an incoming or outgoing message. A typical example is a mail gateway that discards incoming or outgoing mail exceeding a certain size. More sophisticated mail gateways exist that are, for example, capable of filtering spam e-mail.

Another example of an application-level gateway is one that allows external access to a digital library server, but will supply only abstracts of documents. If an external user wants more, an electronic payment protocol is started. Users inside the firewall have direct access to the library service.

A special kind of application-level gateway is what is known as a **proxy gateway**. This type of firewall works as a front end to a specific kind of application, and ensures that only those messages are passed that meet certain criteria. Consider, for example, surfing the Web. Many Web pages contain

scripts or applets that are to be executed in a user's browser. To prevent such code to be downloaded to the inside LAN, all Web traffic could be directed through a Web proxy gateway. This gateway accepts regular HTTP requests, either from inside or outside the firewall. In other words, it appears to its users as a normal Web server. However, it filters all incoming and outgoing traffic, either by discarding certain requests and pages, or modifying pages when they contain executable code.

It should be clear that firewalls have important limitations when considered as a general-purpose security mechanism. As they operate on network traffic, they generally cannot handle authentication and authorization for specific resources. That also means that once an intruder has found a way to bypass the rules of a firewall, all bets are off that the firewall can protect the internal network. For example, if an attacker manages to take over an account, all unauthorized accesses will go unnoticed by the firewall. Instead, other detection mechanisms need to be in place to discover unusual or even unauthorized behavior.

9.6.2 Intrusion detection: basics

Intrusion detection systems generally come in two flavors. In **signature-based intrusion detection systems** (**SIDS**) patterns of known network-level intrusions have been collected against which a new pattern is matched. If there is a match, security operators are warned that an intrusion may be taking place. Patterns typically consist of a (collection of) specific network packets. What can easily complicate discovering patterns is when a pattern spans a series of network packets, intertwined with regular traffic. Most problematic with a SIDS is when no pattern is yet available, which typically happens with new, unknown attacks, such as zero-day attacks: attacks based on vulnerabilities that have not yet been made public [Bilge and Dumitraş, 2012]. The increase of zero-day attacks has rendered SIDS less useful.

More important are the **anomaly-based intrusion detection systems** (**AIDS**). As it name suggests, an AIDS assumes that we can model or extract typical behavior to subsequently detect nontypical, or anomalous behavior. Anomaly-based detection relies heavily on modern artificial-intelligence technologies, and notably machine learning. To this end, an AIDS will first have to go through a *training phase* and collect data that reflects typical, nonmalicious behavior. Once appropriate data sets have been collected and a detection model has been constructed based on that data (such as decision trees, specific neural networks, or classifiers), a *test phase* starts by taking new data and letting the model decide on anomalies.

An important problem with an AIDS is minimizing so-called *false negatives*: flagging an anomaly as typical behavior. Doing so means that the model is missing events that should have been reported to a security operator. To minimize false negatives, many systems follow a pessimistic approach and

flag behavior as being anomalous, in turn leading to many *false positives*. The result of many false positives is that security operators become weary of all the alerts [Hassan et al., 2019]. Combatting the number of false positives while still ensuring a very low number of false negatives has proven to be a difficult challenge requiring advanced techniques (see, for example, van Ede et al. [2022]).

There is much more to say about intrusion detection systems. Khraisat et al. [2019] provide a recent overview; a good introduction is provided by Bhuyan et al. [2014].

9.6.3 Collaborative intrusion detection

When considering large-scale distributed systems, it is clear that there cannot be a single point where monitoring can take place that allows to build a complete picture of all events that may indicate that the system is under attack. This is where a **collaborative intrusion detection system** (**CIDS**) becomes relevant. In essence, a CIDS is a collection of intrusion detection systems that share detections and analyses. Each of the IDS is considered to be positioned somewhere in the system. How a CIDS can be organized is discussed in [Vasilomanolakis et al., 2015], along with examples of existing systems. Here, we concentrate on the effects of such organizations when it comes to successfully detecting intrusions.

To this end, and inspired by Cordero et al. [2015], consider a collection of *sensors* that may be spread across a distributed system, each sensor measuring data potentially related to intrusions. For example, a sensor may be one that logs events related to a specific Web application, or one that captures network traffic at a specific node in the system. We assume that sensors can, in principle, not only collect data, but perform analyses and detect intrusions. Sensors are grouped into **communities**. Each community has a *community head*, which is responsible for collecting the data from the other members in its community, as well as conducting analyses on that data and detecting any intrusions. Communities may overlap: a sensor may be a member of multiple communities, in which case it will also report to multiple community heads. In a way, each community can be viewed as a single IDS.

The basic idea is that community heads exchange information with each other. If this does not happen, nor do any two communities have a sensor in common, then we are simply dealing with a collection of isolated IDSs. A common configuration is to let communities be disjunct, yet all community heads report to the same, single entity, leading to what is known as a *centralized CIDS*. More interesting is the configuration in which the community heads are organized into some kind of peer-to-peer network, i.e., a *distributed CIDS*.

Why is this setup interesting? As mentioned, from the perspective of intrusion detection, it is important to minimize false negatives, while not missing out on actual intrusions, that is, *true positives*. To be more specific, the

performance of an IDS is generally measured along the following metrics. We denote by *TP* the **true positives**: detections that have been correctly flagged as belonging to an intrusion. *TN* are **true negatives**: detections that have been correctly flagged as belonging to authorized behavior. **False positives** (*FP*) are detections that are incorrectly labeled as belonging to an intrusion, while **false negatives** (*FN*) denote missed intrusion detections. The whole idea is then to configure the sensors into communities such that **accuracy** and **precision** are maximized, with:

$$\textbf{Accuracy}: \quad ACC = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\textbf{Precision}: \quad PRE = \frac{TP}{TP + FP}$$

An aspect that was further investigated in [Cordero et al., 2015] is to have sensors be member of multiple communities. As a result, some sensors may report their findings to multiple community heads. The question then becomes how to configure the CIDS as a whole in terms of the number of communities, their sizes, and the sensors reporting to multiple community heads. One can expect that in a centralized setup (in particular, with just a single community and all sensors reporting to a single, centralized entity), accuracy and precision are best. This is indeed the case, but obviously, we may have a serious scalability problem when dealing with a large system.

In the experiments reported by Cordero et al. [2015], having large communities tends to lead to higher accuracy and precision, whereas it is also important to keep the total number of communities as low as scalability permits. Accuracy and precision also benefit from having overlapping communities, effectively meaning that sensor findings are considered by multiple community heads, to be later combined into the final detections.

To what extent the reported experiments are conclusive for other settings remains to be seen. However, the approach does give a means to think about how to organize a CIDS while taking scalability and performance into account. Typically, running various experiments with different datasets will be needed to find a satisfactory configuration.

9.7 Summary

Security plays an essential role in distributed systems. A distributed system should provide the mechanisms that allow a variety of different security policies to be enforced. Developing and properly applying those mechanisms generally makes security a difficult engineering exercise.

Secure distributed systems are built around at least five design principles: (1) having fail-safe defaults, (2) offering designs and implementations that are open, (3) providing a separation of privileges, (4) providing only those

privileges that are needed, and (5) minimizing redundancy of mechanisms and their implementations. Furthermore, we see that it is important to provide mechanisms at different layers in a distributed system, ranging from the lower levels of operating systems to layers in which applications are running. In this sense, understanding what exactly the trusted computing base entails of a system is important, as this base that needs to meet all security requirements.

Finally, and increasingly important, is that distributed systems provide privacy where needed. Guaranteeing that privacy is protected is easier said than done, but the fact that regulations are more firmly installed, such as the GDPR, contributes to the awareness that systems should be designed and developed with security and privacy built in from the start.

Essential to realizing secure distributed systems is the understanding and application of cryptography. Next to symmetric and asymmetric cryptosystems, we see increasingly more cryptographic mechanisms come into practical use due to efficient implementations. Notably homomorphic encryption schemes and multiparty computations are gradually seeing their way into distributed systems, mainly because they allow for untrusted third parties, such as operating in the cloud, to do computations without having to reveal sensitive data.

One could argue that security in distributed systems largely boils down to realizing authentication and authorization. Authentication deals with verifying claim identities. Authorization deals with protecting resources in such a way that only processes that have the proper access rights can actually access and use those resources. Access control always take place after a process has been authenticated. It is important to realize that having being able to verify an identity is not enough for establishing trust.

As distributed systems scale out to multiple organizations, we see a demand for solutions that need to rely less on trusting the individual parties involved in an application. This is one of the reasons why blockchain-based systems have grown in popularity, yet whether blockchain-based solutions can be taken as a general solution for handling lack of trust remains to be seen. Often, simpler and more efficient application-specific solutions may do the job, and perhaps even better.

Besides cryptography, authentication, and authorization, it is important to continuously monitor what is going on in a distributed system. Advances in intrusion detection systems have led to using often advanced machine-learning techniques for detecting anomalous behavior. An open question is how such systems can be effectively scaled out to distributed settings.

Many existing designs of distributed systems have incorporated security and privacy as an afterthought, often also because one may easily think that security and functionality demand a tradeoff to be made. However, there is ample evidence that such a tradeoff is not necessary, provided security is considered from the start of a design.

INDEX

2PC, *see* Distributed commit, two-phase protocol
3PC, *see* Distributed commit, three-phase protocol

ABAC, *see* Access control, attribute based
Abstract regions, 49
Acces control
  list, 596
Access control, 595
  attribute-based, 596, 598
  discretionary, 595, 599,600
  mandatory, 595, 599
  matrix, 596
  policy, 595
  role based, 595, 599
Access point, 9
Access transparency, *see* Distribution transparency
Accuracy, 613
ACL, *see* Access control, list
Activation policies, 155
Active Directory, 376, 379
Actuator, 43, 51
Adapter, 75
Adaptive redirection policy, 166
Address Resolution Protocol, 330
Advanced Message-Queuing Protocol, 227
  channel, 228
  connection, 228
  exchange, 231
  link, 228,229
  node, 229
  session, 228
AIDS, *see* Intrusion Detection System, anomaly based
Akamai, 6, 165, 453
  origin server, 165
Amazon
  Elastic Block Store, 141
  Elastic Compute Cloud, 140
  Machine Image, 140
  Simple Storage Service, 66,67, 75
Amazon AMI, *see* Amazon, Machine Image
Amazon S3, *see* Amazon, Simple Storage Service, 99
  bucket, 99
AMQP, *see* Advanced Message-Queuing Protocol
Apache, 122, 159, 552
  hook, 160
  module, 160
  Portable Runtime, 159
Apache APR, *see* Apache, Portable Runtime
API, *see* Application programming interface
Application layer, 188
Application programming interface, 129
Application-layer switch, 163
Architectural perspective, 8
Architectural style, 56
ARP, *see* Address resolution protocol
Asynchronous system, 469
Atomicity, 189

Authentication, 10, 21, 189
Authorization, 21, 189
  graph-based, 606
Autonomous System, 426
Availability, *see* Dependability, availability

Backup server, 485
Big endian, 198
Bitcoin, 291
Bits-per-second, 2
BitTorrent, 10, 96
  magnet link, 98
  torrent file, 97
  tracker, 97
Blockchain, 5, 22, 104
  genesis block, 591
  permissioned, 106, 501,502
  permissionless, 106, 290, 502, 589
  Proof of Stake, 292, 589
  Proof of Work, 290, 589
  smart contract, 593
Broadcasting, 236
Browser, 86
Byzantine agreement, 492
Byzantine failure, 468

Cache, 393, 407, 429
  cache hit, 429
  coherence detection, 443
  coherence enforcement, 444
  coherence protocol, 443
  content aware, 457
  content blind, 458
  cooperative cache, 451
  distributed cache, 451
  Network File System, 444
  Web, 451
  write back, 444
  write through, 444
Callback, 205
CAP theorem, 506
Capability, 596
Causal history, 267
Causality, 267
CDN, *see* Content Delivery Network
CGI, *see* Common Gateway Interface
Cgroups, 135
Challenge-response protocol, 573
  challenge, 573, 579
  multiway, 577
  nonce, 577
Checkpointing
  coordinated, 539
  distributed snapshot, 538
  domino effect, 539
  independent, 539,540
  recovery line, 538
Checksum, 186
Chord, 91, 233, 294, 333
  finger table, 333
  flooding, 239
  forwarder, 233
  successor, 91
Chrome OS, 145
CIDS, *see* Intrusion Detection System, collaborative
Client, 27, 37, 60, 79
  fat, 82
  thin, 82, 142
Client stub, 148, 194
Client-server computing, 27, 172
Clock
  accuracy, 254
  clock drift rate, 254
  drift, 254
  external synchronization, 254
  internal synchronization, 254
  precision, 254
  skew, 251
  synchronization, 254
  tick, 250
Closure mechanism, 348
  container, 348
Cloud computing, 12, 98

Cluster computing, 32, 34
Code segment, 171
Code-on-demand, 172
Coherence model, 406
Collaborative Intrusion Detection System, *see* Intrusion Detection System, collaborative
Common Gateway Interface, 87
Communication
  asynchronous, 191, 214, 301
  asysnchronous, 219
  persistent, 191, 231, 301
  synchronous, 27, 191
  transient, 191, 219
Communication pattern, 214
  pipeline, 217
  publish-subscribe, 216
  request-reply, 214
Communication perspective, 9
Communication protocol, 59, 183
  stack, 58
Communication service, 58, 183
Computational grid, 27
Computer timer, 250
  counter, 250
  holding register, 250
Concurrency transparency, *see* Distribution transparency
Concurrent operations, 260, 401
Confidentiality, 20, 313
Conflict-Free Replicated Data Type, 409
Conit, *see* Consistency unit
Connection-oriented service, 60, 183
Connectionless service, 184
Consensus, 475
  flooding, 475
  Paxos, 477, 479
  Paxos, accept message, 485
  Paxos, acceptor, 480
  Paxos, client, 480
  Paxos, leader, 480, 485
  Paxos, learned message, 485
  Paxos, learner, 480
  Paxos, proposal message, 491
  Paxos, proposal timestamp, 483
  Paxos, proposer, 480, 491
  Raft, 289, 477
  Raft, follower, 477
  Raft, leader, 477
  Raft, leader election, 289
  Raft, log, 477
  Raft, term, 289, 477
Consistency, 31
Consistency and replication perspective, 9
Consistency model, 406
  causal consistency, 401, 408
  client centric, 415,416, 448
  compositional, 400
  continuous consistency, 410, 446
  data centric, 395
  entry consistency, 404, 415, 434
  eventual consistency, 407
  linearizable, 400
  monotonic read, 417
  monotonic write, 418
  program consistency, 410
  read your writes, 420
  schedule, 405
  sequential consistency, 397
  serializable, 400, 405, 422
  strong eventual consistency, 409
  writes-follow-reads, 421
Consistency protocol, 437
  primary based, 438,439, 474
  primary-backup, 438
  quorum based, 441
  read quorum, 442
  replicated-write, 440
  write quorum, 442
Consistency unit, 411
  declaration, 414
  false share, 413
Contact address, 223

Container, 133, 348
Content Delivery Network, 6, 95, 165, 319, 425, 427, 453
Context awareness, 45
Cookie, 153
Coordinated Universal Time, 252,253
Coordination, 68, 248
  direct, 68
  event-based, 69
  mailbox, 69, 221
  referentially coupled, 68, 313
  temporally coupled, 68, 221, 313
Coordination perspective, 9
Copy-before-use, 15
CRDT, *see* Conflict-Free Replicated Data Type
Critical region, 264, 266, 275, 403
CRL, *see* Cryptosystem, certificate revocation list
Cryptosystem
  assymmetric, 21
  asymmetric, 557
  certificate revocation, 569
  certification authority, 569
  certification revocation list, 569
  public-key certificate, 569
  public-key system, 21, 557, 568
  secret-key system, 557
  session key, 564, 579,580
  shared-key system, 557
  symmetric, 21, 557
Cyclon, 304

DAC, *see* Access control, discretionary
Data link layer, 186
  frame, 186
Data store, 395
Data synchronization, 248
Deadlock, 272
Decentralization, 3, 7
Decentralized system, 4
Decryption, 555
Delegation, 601
Dependability, 18, 463
  availability, 19, 463,464
  availability, long-term, 464
  confidentiality, 547
  integrity, 547
  maintainability, 19, 463
  reliability, 19, 463,464
  safety, 19, 272, 463
Detection
  false negative, 613
  false positive, 613
  true negative, 613
  true positive, 613
DHCP, *see* Dynamic Host Configuration Protocol
DHT, *see* Distributed hash table
Differential GPS, 317
Diffie-Hellman group, 565
Diffie-Hellman key exchange, 565,566
  ephemeral, 565
Digital signature, 22, 561
Directed acyclic graph, 346
Directory node, 345
  location record, 338
  root, 338
Directory table, 345
Dispatcher, 125
Distribtion
  horizontal, 89
Distributed commit, 40, 189, 528
  blocking protocol, 531
  one-phase protocol, 529
  three-phase protocl, 534
  two-phase protocol, 529
Distributed consensus, 106
Distributed hash table, 90
Distributed ledger, 5, 104
Distributed object, 63
Distributed shared memory, 33
Distributed system, 4
Distributed transaction, 37
Distribution, 3
  vertical, 89

Distribution transparency, 6, 11, 148
access transparency, 11, 15, 67, 148
concurrency transparency, 13, 149
failure transparency, 13, 149
location transparency, 12, 148, 332
migration transparency, 12, 148
relocation transparency, 12, 148
replication transparency, 13, 149
DNS, *see* Domain Name System, 359
domain name, 189
DNSSEC, *see* Domain Name System, security extension
Document Object Model, 145
DOM, *see* Document Object Model
Domain, 338
Domain name, 165
Domain Name System, 6, 165, 189, 359,360, 472
canonical name, 361
domain, 30
key-signing key, 366
resource record, 360
security extension, 366
zone, 30, 354
zone transfer, 362
zone-signing key, 366
Downcall, 57
DSM, *see* Distributed shared memory
Dynamic Host Configuration Protocol, 46

EAI, *see* Enterprise application integration
EBS, *see* Amazon, Elastic Block Store
Eclipse attack, 344, 588
Edge computing, 100, 456
infrastructure, 100
orchestration, 103
Encapsulation, 63,64
End point, 150
End-to-end principle, 271, 551
Enterprise application integration, 38, 75, 226
Epidemic behavior, 240
anti-entropy, 241, 521
infected, 241
removed, 241
round, 241
rumor spreading, 243
susceptible, 241
Epidemic protocol, 240, 297
death certificate, 245
dormant death certificate, 245
Erdös-Rényi graph, 237
Error, 19, 465
Event bus, 70
Event counter, 261
Execution segment, 171
Extended markup language, 42, 224
Extensible, 16

FaaS, *see* Function-as-a-Service
Fail, 19
Failure, 465
arbitrary, 468
commission, 468
communication, 508
crash, 467
exception handling, 509
fail arbitrary, 469
fail noisy, 469
fail safe, 469
fail silent, 469
fail stop, 469
omission, 467,468
performance, 468
receive omission, 467
response, 468
send omission, 467
timing, 467
Failure detection, 506
eventually perfect, 507
suspect, 507

Failure rate function, 464
Failure transparency, *see* Distribution transparency
Farmer task, 217
Fault, 20, 465
  intermittent, 20, 466
  permanent, 20, 466
  transient, 20, 466
Fault tolerance, 5, 18, 20, 466
  Byzantine, Altruism, Rationality (BAR), 505
Fault-tolerance perspective, 9
Federated learning, 4, 168
FHE, *see* Security mechanism, full homomorphic encryption
File Transfer Protocol, 188
Firewall, 609
  application-level gateway, 610
  packet-filtering gateway, 610
  proxy gateway, 610
Fog computing, 101
Free riding, 96
Front end, 162
FTP, *see* File Transfer Protocol
Function-as-a-Service, 100

GDPR, *see* General Data Protection Regulation
General Data Protection Regulation, 553
Gentle reincarnation, 514
Global Positioning System, 316
Google Mail, 5
Gossiping, 240, 297, 311
  Data-injection attack, 306
  directional, 244, 297
  partial view, 298, 312
GPS, *see* Global Positioning System
Grid computer, 4
Grid computing, 32, 35
  application layer, 37
  collective layer, 37
  connectivity layer, 36
  fabric layer, 36
  resource layer, 36
Groupware, 10

Happens-before relation, 260
Hash function, 22, 290, 560
  digest, 291
  nonce, 291
  strong collision resistance, 560
  trapdoor, 560
  weak collision resistance, 560
Honest-but-curious, 315, 553
HTML, *see* HyperText Markup Language
HTTP, *see* HyperText Transfer Protocol
HTTPS, *see* HyperText Transfer Protocol, secure, 583
Hypercube, 90
Hyperspace, 312
HyperText Markup Language, 86
HyperText Transfer Protocol, 86, 188
  secure, 367, 551

IaaS, *see* Infrastructure-as-a-Service, 139
IANA, *see* Internet Assigned Numbers Authority
ICN, *see* Information-centric networking
Idempotent operation, 80, 513
Identifier-location split, 329
IDL, *see* Interface Definition Language
Implicit action, 45
In-network data processing, 50
Incremental snapshot, 539
Information-centric networking, *see* Named-data networking, 385
Infrastructure-as-a-Service, 99, 139
Instruction
  behavior-sensitive, 132
  control-sensitive, 132
  nonprivileged, 131

privileged, 131
Instruction set architecture, 129
Integrity, 20, 313
Interceptor, 76
message-level, 77
request-level, 77
Interface, 15, 184
Interface Definition Language, 15, 203
International Atomic Time, 252
Internet Assigned Numbers Authority, 150
Internet of Things, 44
Internet Protocol, 187
Internet Service Provider, 28, 100
Internet-of-Things, 100
Interoperability, 16
Interprocess communication, 115
Interrupt
direct overhead, 116
indirect overhead, 116
Intruder, 556
Intrusion detection, 609
Intrusion Detection System
anomaly-based, 611
collaborative, 612
community, 612
signature-based, 611
IoT, *see* Internet-of-Things
IP, *see* Internet Protocol
IPC, *see* Interprocess communication
ISA, *see* Instruction set architecture
ISP, *see* Internet Service Provider, *see* Internet Service Provider

JavaScript, 87, 146

KDC, *see* Key distribution center
Kerberos, 551, 581, 605
authentication service, 581
ticket-granting service, 581
Key distribution center, 576
single sign-on, 583
ticket, 576, 581
Key-derivation function, 585

LAMP, 140
LAN, *see* Local-area network
Landmark, 320
LDAP, *see* Lightweight Directory Access Protocol
Leader election, 96, 106, 283
bully algorithm, 283
ring algorithm, 285
Leaf domain, 338
Lease, 72, 433
age-based, 433
renewal-frequency based, 433
state-based, 434
Lightweight Directory Access Protocol, 376
directory information base, 377
directory information tree, 377
directory service agent, 379
directory user agent, 379
relative distinguished name, 377
Linda, 70, 307
Little endian, 198
Liveness property, 482, 490, 502, 505
Local-area network, 2
Location transparency, *see* Distribution transparency
Lock, 403, 434
Logical clock, 260, 263

MAC, *see* Access control, mandatory
Maintainability, *see* Dependability, maintainability
MANET, *see* Mobile ad hoc network
MCC, *see* Mobile Cloud Computing
Mean Time Between Failures, 19, 465
Mean Time To Failure, 19, 465
Mean Time To Repair, 19, 465

MEC, *see* Mobile Edge Computing
Mechanism, 17
Memory Management Unit, 113
Mesh network, 233
Message broker, 75, 95, 225, 308
Message digest, 562
Message logging, 537, 542
  optimistic, 543
  pessimistic, 543
  receiver based, 537
  sender based, 537
Message transfer time unit, 279
Message-oriented middleware, 41, 220
  router, 224
Message-Passing Interface, 218
Message-queuing system, 220
Microservice, 65
Middleware, 11, 74
  modifiable, 78
Migration
  receiver initiated, 172
  sender initiated, 172
Migration transparency, *see* Distribution transparency
MMU, *see* Memory Management Unit
Mobile ad hoc network, 47
Mobile agent, 168, 172
Mobile Cloud Computing, 48
Mobile computing, 47
Mobile Edge Computing, 48
Mobile host
  care-of-address, 331
  home agent, 331
MOM, *see* Message-oriented middleware
MPC, *see* Multiparty computation
MPI, *see* Message-Passing Interface
MPLS, *see* Multi-Protocol Label Switching
MTBF, *see* Mean Time Between Failures
MTTF, *see* Mean Time To Failure
MTTR, *see* Mean Time To Repair
MTTU, *see* Message transfer time unit
Multi-Protocol Label Switching, 454
Multicasting, 232, 236
  application-level, 232
  atomic, 522, 526
  causally ordered, 270, 526
  feedback implosion, 518
  feedback suppression, 519
  FIFO-ordered, 525
  flooding, 237
  group view, 523
  link stress, 234
  probabilistic flooding, 237
  relative delay penalty, 234
  stretch, 234
  totally ordered, 264, 523, 526
  tree cost, 235
  unordered, 524
  view change, 523
Multicomputer, 33
Multiparty computation, 23, 566
  oblivious transfer, 566
Multiprocessor, 33
Mutual exclusion, 189
  permission-based, 272
  token based, 272

Name
  alias, 349
  global, 345
  local, 345
Name resolution, 347
  automounting, 373
  file handle, 371,372
  iterative, 355
  recursive, 356
Name resolver, 355
Name space, 344
  absolute path name, 345
  administrational layer, 353
  directory node, 345

foreign, 349
global layer, 352
hard link, 349
leaf node, 344
managerial layer, 353
mount point, 349
mounting, 349, 369
mounting point, 349
path name, 345
relative path name, 345
root node, 345
Unix, 346, 348
Named-data networking, 329, 385
content store, 387
forwarding information base, 387
pending interest table, 387
Naming
access point, 327
address, 327
attribute-based, 375
directory service, 375
flat name, 329
home location, 331
human-friendly name, 328
identifier, 328
index server, 380
location independent, 328
naming system, 375
Naming graph
symbolic link, 349
Naming perspective, 9
Naming service, 30
Nano computers, 2
NAS, *see* Network-Attached Storage
NAT, *see* Network-address translation
NCS, *see* Network Coordinates System
NDN, *see* Named-data networking
Needham-Schroeder authentication protocol, 577, 581
NetFlow, 590
choice set, 590
Network Coordinates System, 318
Network File System, 83, 350, 369, 445
export directory, 370
NFS client, 84
NFS server, 85
open delegation, 445
Network layer, 186
packet, 187
Network Time Protocol, 256
reference clock, 256
startum server, 256
Network-address translation, 140, 163
Network-Attached Storage, 6
Networked computer system, 2
expansive view, 4
integrative view, 4
NFS, *see* Network File System
Node, 89
Nonce, 603
Notification, 69
Notification filtering, 306
NTP, *see* Network Time Protocol

OAuth, *see* Open Authorization
Object
interface, 63
method, 63
persistent, 204
state, 63
transient, 204
Object adapter, 75, 155
Object wrapper, 155
OGSA, *see* Open Grid Services Architecture
One-way function, 560
Open Authorization, 604
access token, 604
authorization code, 604
Open Grid Services Architecture, 37

Open Systems Interconnection Reference Model, 183
Openness, 15
Orchestration, 65
Order deviation, 412
Orphan computation, 513, 541
  expiration, 514
  extermination, 514
  grandorphan, 514
  reincarnation, 514
OSI, *see* Open Systems Interconnection Reference Model
Out-of-band data, 151
Overlay network, 89, 224, 231, 299, 308, 310,311, 315
  geometric, 318

PaaS, *see* Platform-as-a-Service, 139
Parallel processing, 33
Parameter marshaling, 197
Parameter passing
  call-by-copy/restore, 194
  call-by-reference, 193
  call-by-value, 193
  object reference, 200
Paravirtualization, 133
Partial failure, 7, 18
Partially synchronous, 469
Paxos, *see* Consensus, Paxos
PBFT, *see* Practical Byzantine Fault Tolerance
Peer-sampling service, 244, 298, 300, 303
Peer-to-peer search
  flooding, 93
  policy-based, 94
  random walk, 93,94, 311
Peer-to-peer system, 89
  structured, 90
  unstructured, 92
PEKS, *see* Public Key Encryption with Keyword Search
Pervasive system, 43
PGP, *see* Pretty Good Privacy
PHE, *see* Security mechanism, partial homomorphic encryption
Physical layer, 186
Piecewise deterministic execution model, 541
PlanetLab, 136
Platform-as-a-Service, 99, 139
Plugin, 225
Policy, 17
Policy Machine, 598
  assignment, 599
  obligation, 599
  prohibition, 599
Port, 150
Portability, 16
PoS, *see* Blockchain, proof of stake
Position-based routing, 319
PoW, *see* Blockchain, proof of work
Practical Byzantine Fault Tolerance, 498
  commit certificate, 500
  prepare certificate, 499
  view, 499
  view-change certificate, 501
Precision, 613
Presentation layer, 188
Pretty Good Privacy, 570
Primary server, 485
Problem monotonicity, 410
Process, 113
  context, 113
  process table, 113
Process group, 472
  flat, 472
  group server, 473
  hierarchical, 472
  k-fault tolerant, 474
Process migration, 167
Process perspective, 9
Process synchronization, 248
Processor context, 113
Progressive Web app, 148

Protocol stack, 185
Protocol suite, 185
Proximity neighbor selection, 338
Proximity routing, 337
PSS, *see* Peer-sampling service
Public key, 21
Public Key Encryption with Keyword Search, 314
Publish, 69, 71
Publish-subscribe, 41, 226, 306
  content-based, 72
  matching, 72
  message subject, 308
  routing filter, 310
  topic-based, 72
PWA, *see* Progressive Web app

QoS, *see* Quality of Service
Quality of Service, 424
Query containment check, 457
Queue manager, 222

RabbitMQ, 231
Raft, *see* Consensus, Raft
Random graph, 92, 237
RBAC, *see* Access control, role based
RBS, *see* Reference Broadcast Synchronization
RDF, *see* Resource Description Framework
RDP, *see* Relative delay penalty
Read-One, Write-All, 443
Read-write conflict, 407
Real-time Transport Protocol, 187
Recovery
  backward, 536
  checkpointing, 536
  erasure correction, 536
  forward, 536
Redundancy
  information, 470
  physical, 470
  time, 470
  triple modular, 470
Reference Broadcast Synchronization, 257
Reference monitor, 594
Reflection attack, 574
Reliability, *see* Dependability, reliability
Relocation transparency, *see* Distribution transparency
Remote desktop, 144
Remote evaluation, 172
Remote file service, 83
  remote access model, 84
  upload/download model, 84
Remote method invocation, 41, 205
Remote object, 63,64
  binding to, 63
  proxy, 63
  skeleton, 64
Remote procedure call, 38, 41,42, 84, 190, 192
  asynchronous, 205, 231
  at-least-once semantics, 511
  at-most-once semantics, 511
  client stub, 63
  deferred synchronous, 205
  exactly-once semantics, 511
  failure, 509
  multicast, 207
  one-way, 206
  synchronous, 231
Rendezvous node, 235, 308
Replicated object invocation, 436
Replication
  active, 431, 440, 474
  client initiated, 429
  invalidation protocol, 430
  mirror site, 426
  mirroring, 426
  partial, 457
  permanent replica, 426
  quorum based, 474
  sequencer, 441
  server initiated, 427

shared nothing, 427
update protocol, client-based, 431
update protocol, pull-based, 431
update protocol, push-based, 431
update protocol, server-based, 431
Replication transparency, *see* Distribution transparency
Representational State Transfer, 66
Request-reply, 79
Resource Description Framework, 375
Resource segment, 171
Resource sharing, 10
Resource virtualization, 127
REST, *see* Representational State Transfer
RESTful architecture, 66
Reverse proxy, 163
RMI, *see* Remote method invocation
Routing, 186
ROWA, *see* Read-One, Write-All
RPC, *see* Remote procedure call
RTP, *see* Real-Time Transport Protocol

SaaS, *see* Software-as-a-Service, 139
Safety, *see* Dependability, safety
Safety property, 482, 490, 502, 505
Sandbox, 147
Scalability
administrative, 24, 27
analysis of size scalability, 25
geographical, 24, 26
size, 24
Scalable Reliable Multicasting, 519
Scaling out, 28
Scaling technique
asynchronous communication, 29
caching, 31
hide latency, 29
partitioning and distribution, 30
replication, 31
Scaling up, 28
SCS, *see* Vserver, slice creation service
SCTP, *see* Streaming Control Transmission Protocol
Searchable encryption, 314
Secret key, 21
Secure channel, 22, 568
Secure Shell Protocol, 551
Secure Socket Layer, 583
Security
proxy, 601
Security key, 21
Security mechanism, 548
authentication, 548, 571
authentication, continuous, 572
authentication, electronic, 572
authentication, multi-factor, 572
authentication, single-factor, 571
authorization, 548
decryption, 21
encryption, 21, 548, 555
encryption, ciphertext, 555
encryption, plaintext, 555
full homomorphic encryption, 558
homomorphic encryption, 558
partial homomorphic encryption, 558
Security perspective, 10
Security policy, 547
Security principle, 549
fail-safe defaults, 549
least common mechanism, 550
least privilege, 549
open design, 549
separation of privilege, 549
Security threat, 547
Selective routing, 310

Self-certifying name, 343
Self-configurable system, 18
Sensor, 43
Sensor network, 49, 295
Servant, 89, 156
Server, 24, 27, 60, 79
  concurrent, 150
  iterative, 150
  permanent state, 153
  session state, 153
  soft state, 152
  stateful, 152
  stateless, 152
Server stub, 195
Server-side script, 88
Service-level agreement, 98
Service-oriented architecture, 37, 64
Session key, 573
Session layer, 188
Shard, 554
Shared data space, 46, 69
  attribute, 71
  tuple, 69
SIDS, *see* Intrusion Detection System, signature based
SLA, *see* Service-level agreement
Smartphone, 2
SOA, *see* Service-oriented architecture
Socket, 208
Socket interface, 208
Software architecture, 56
  component, 57
  connector, 57
  event based, 70
  interface, 57,58
  layered, 57, 61
  microservice, 63
  object, 63
  publish-subscribe, 70
  service, 63
  shared data space, 70
Software-as-a-Service, 99, 129, 139
Space-filling curve, 381
Spanner, 259
SRM, *see* Scalable Reliable Multicasting
SSH, *see* Secure Shell Protocol
Starvation, 272
State machine replication, 264
Stateless execution, 66
Streaming Control Transmission Protocol, 187
Strong mobility, 173
Subscribe, 69, 72
Successor, 333
Super peer, 95, 293
Superserver, 150
Switch tree, 235
Sybil attack, 344, 586
Synchronization variable, *see* Lock
Synchronous, 504
Synchronous system, 469
System architecture, 56, 79
  three-tiered, 83
  two-tiered, 81
System calls, 129

TAI, *see* International Atomic Time
TCB, *see* Trusted Computing Base
TCP, *see* Transmission Control Protocol, 187
TCP handoff, 163
Thread
  context, 114
  many-to-many model, 120
  many-to-one model, 120, 122
  one-to-one model, 120
Thread-level parallelism, 124
TIB/Rendezvous, 308
  rendezvous daemon, 308
Time
  leap second, 252
  mean solar second, 252
  solar day, 251
  solar second, 251
  transit sun, 251

Time-to-live, 93
TLB, *see* Translation Lookaside Buffer
TLP, *see* Thread-level parallelism
TLS, *see* Transport Layer Security
TMR, *see* Redundancy, triple modular
Topology-based identifier, 337
TP monitor, *see* Transaction-processing monitor
Transaction, 38
  ACID, 38
  nested, 39
Transaction-processing monitor, 40
Transactional RPC, 38
Transient object, 154
Translation Lookaside Buffer, 113
Transmission Control Protocol, 59, 187
Transport layer, 187
Transport Layer Security, 551, 583
Transport-layer switch, 163
Trap, 131
Trapdoor, 314
Triangle inequality, 320
TrueTime, 259
Trust, 10, 21, 304, 586
Trust chain, 366, 588
Trusted Computing Base, 552
Trusted third party, 104
TTL, *see* Time-to-live
Tunnel, 551
Tuple space, 70

Ubiquitous computing system, 44
UDP, *see* Universal Datagram Protocol
Uniform Resource Identifier, 66
Uniform Resource Locator, 12, 86
Universal Datagram Protocol, 187
Universal Plug and Play protocol, 46
Unix
  namespaces, 134
Upcall, 57
  handle, 58
UPnP, *see* Universal Plug and Play protocol
URI, *see* Uniform Resource Identifier
URL, *see* Uniform Resource Locator
UTC, *see* Coordinated Universal Time, 253
Utility computing, 98
Utilization, 26

Vector clock, 267,268
VFS, *see* Virtual File System
Virtual desktop environment, 82, 144
Virtual File System, 84
Virtual machine monitor
  hosted, 131
  native, 130
Virtual Network Computing, 144
Virtual organization, 35
Virtual private network, 550
Virtual synchrony, 523
  flush message, 528
  stable message, 527
Virtualization, 127
  guest operating system, 131
  host operating system, 131
  process virtual machine, 130
  virtual processor, 113
VM fork, 177
VNC, *see* Virtual Network Computing
VPN, *see* Virtual private network
Vserver, 136
  node manager, 137
  service provider, 137
  slice, 136
  slice authority, 137
  slice creation service, 137

WAN, *see* Wide-area network
War driving, 317
WAVE, 606

attestation, 606
entity store, 608
issuer, 606
map log, 608
operation log, 608
resource, 606
RTree, 606
RTree statement, 606
structural security, 608
subject, 606
Weak mobility, 172
Weak peer, 95
Web of trust, 570, 587
WebAssembly, 146
Wide-area network, 2
Window manager, 143
Worker task, 217
Worker thread, 125
Wrapper, 75
Write set, 416
Write-write conflict, 407

X kernel, 142
X protocol, 143
X Window System, 142
XML, *see* Extended markup language

Zeroconf, 46
ZeroMQ, 213
ZooKeeper, 280, 286, 422
ensemble, 286

BIBLIOGRAPHY

Abadi M. and Needham R. Prudent Engineering Practice for Cryptographic Protocols. *IEEE Transactions on Software Engineering*, 22(1):6–15, Jan. 1996. → 576.

Abbas N., Zhang Y., Taherkordi A., and Skeie T. Mobile Edge Computing: A Survey. *IEEE Internet of Things Journal*, 5(1):450–465, 2018. → 48.

Aberer K., Alima L. O., Ghodsi A., Girdzijauskas S., Hauswirth M., and Haridi S. The Essence of P2P: A Reference Architecture for Overlay Networks. In *5th International Conference on Peer-to-Peer Computing*, pages 11–20, Los Alamitos, CA., Aug. 2005. IEEE, IEEE Computer Society Press. → 89.

Acar A., Aksu H., Uluagac A. S., and Conti M. A Survey on Homomorphic Encryption Schemes: Theory and Implementation. *ACM Computing Surveys*, 51(4):1–35, 2018. → 558.

Adar E. and Huberman B. A. Free Riding on Gnutella. Hewlett Packard, Information Dynamics Lab, Jan. 2000. → 96.

Adelstein F., Gupta S., Richard G., and Schwiebert L. *Fundamentals of Mobile and Pervasive Computing*. McGraw-Hill, New York, NY, 2005. → 47.

Adve S. V. and Boehm H.-J. Memory Models: A Case for Rethinking Parallel Languages and Hardware. *Communications of the ACM*, 53(8):90–101, Aug. 2010. → 399.

Afanasyev A., Burke J., Refaei T., Wang L., Zhang B., and Zhang L. A Brief Introduction to Named Data Networking. In *IEEE Military Communications Conference (MILCOM)*, pages 1–6, 2018. → 386.

Agarwal A., George M., Jeyaraj A., and Schwarzkopf M. Retrofitting GDPR Compliance onto Legacy Databases. *Proceedings of the VLDB Endowment*, 15(4):958–970, Dec. 2022. → 554.

Ager B., Mühlbauer W., Smaragdakis G., and Uhlig S. Comparing DNS Resolvers in the Wild. In *10th Internet Measurement Conference*, pages 15–21, New York, NY, 2010. ACM Press. → 369.

Aguilera M. and Terry D. The Many Faces of Consistency. *Data Engineering*, page 3, 2016. → 395.

Ahlgren B., Dannewitz C., Imbrenda C., Kutscher D., and Ohlman B. A Survey of Information-centric Networking. *IEEE Communications Magazine*, 50(7):26–36, July 2012. → 385.

Aiyer A., Alvisi L., Clement A., Dahlin M., and Martin J.-P. BAR Fault Tolerance for Cooperative Services. In *20th Symposium on Operating System Principles*, pages 45–58, New York, NY, Oct. 2005. ACM, ACM Press. → 505.

Akgul F. *ZeroMQ*. Packt Publishing, Birmingham, UK, 2013. → 213.

Akyildiz I. F. and Kasimoglu I. H. Wireless Sensor and Actor Networks: Research Challenges. *Ad Hoc Networks*, 2:351–367, 2004. → 49.

Akyildiz I. F., Su W., Sankarasubramaniam Y., and Cayirci E. A Survey on Sensor Networks. *IEEE Communications Magazine*, 40(8):102–114, Aug. 2002. → 49.

Akyildiz I. F., Wang X., and Wang W. Wireless Mesh Networks: A Survey. *Computer Networks*, 47(4):445–487, Mar. 2005. → 49.

Albrecht J., Oppenheimer D., Vahdat A., and Patterson D. A. Design and Implementation Trade-Offs for Wide-Area Resource Discovery. *ACM Transactions on Internet Technology*, 8(4):1–44, 2008. → 383.

Alegre U., Augusto J. C., and Clark T. Engineering Context-Aware Systems and Applications: A Survey. *Journal of Systems and Software*, 117:55–83, 2016. → 45.

Ali W., Shamsuddin S. M., and Ismail A. S. A Survey of Web Caching and Prefetching. *International Journal of Advances in Soft Computing and Its Applications*, 3(1):18–44, 2011. → 453.

Allani M., Garbinato B., and Pedone F. Application Layer Multicast. In Garbinato B., Mirando H., and Rodrigues L., editors, *Middleware for Network Eccentric and Mobile Applications*, pages 191–214. Springer-Verlag, Berlin, 2009. → 233.

Allen R. and Lowe-Norris A. *Windows 2000 Active Directory*. O'Reilly & Associates, Sebastopol, CA., 2nd edition, 2003. → 379.

Allman M. Putting DNS in Context. In *Internet Measurement Conference*, page 309–316, 2020. → 369.

Alonso G., Casati F., Kuno H., and Machiraju V. *Web Services: Concepts, Architectures and Applications*. Springer-Verlag, Berlin, 2004. → 37.

Alvisi L. and Marzullo K. Message Logging: Pessimistic, Optimistic, Causal, and Optimal. *IEEE Transactions on Software Engineering*, 24(2):149–159, Feb. 1998. → 541, 542.

AMQP Working Group . AMQP, Protocol specification, Version 0-9-1, Nov. 2008. → 227.

Amza C., Cox A., Dwarkadas S., Keleher P., Lu H., Rajamony R., Yu W., and Zwaenepoel W. TreadMarks: Shared Memory Computing on Networks of Workstations. *Computer*, 29(2):18–28, Feb. 1996. → 33.

Andersen M. P. *Decentralized Authorization with Private Delegation*. Ph.D., University of California, Berkeley, 2019. → 606, 607, 608.

Andersen M. P., Kumar S., AbdelBaky M., Fierro G., Kolb J., Kim H.-S., Culler D. E., and Popa R. A. WAVE: A Decentralized Authorization Framework with Transitive Delegation. In *28th USENIX Security Symposium*, pages 1375–1392, 2019. → 606.

Andrews G. *Foundations of Multithreaded, Parallel, and Distributed Programming*. Addison-Wesley, Reading, MA., 2000. → 248.

Androutsellis-Theotokis S. and Spinellis D. A Survey of Peer-to-Peer Content Distribution Technologies. *ACM Computing Surveys*, 36(4):335–371, Dec. 2004. → 89.

Antonini M., Vecchio M., and Antonelli F. Fog Computing Architectures: A Reference for Practitioners. *IEEE Internet of Things Magazine*, 2(3):19–25, 2019. → 104.

Arkills B. *LDAP Directories Explained: An Introduction and Analysis*. Addison-Wesley, Reading, MA., 2003. → 376.

Attiya H. and Welch J. *Distributed Computing Fundamentals, Simulations, and Advanced Topics*. John Wiley, New York, 2nd edition, 2004. → 249.

Atxutegi E., Liberal F., Saiz E., and Ibarrola E. Toward Standardized Internet Speed Measurements for End Users: Current Technical Constraints. *IEEE Communications Magazine*, 54(9):50–57, Sept. 2016. → 454.

Avizienis A., Laprie J.-C., Randell B., and Landwehr C. Basic Concepts and Taxonomy of Dependable and Secure Computing. *IEEE Transactions on Dependable and Secure Computing*, 1(1):11–33, Jan. 2004. → 20, 466.

Awadallah A. and Rosenblum M. The vMatrix: A Network of Virtual Machine Monitors for Dynamic Content Distribution. In *7th Web Caching Workshop*, Aug. 2002. → 128.

Ayeswarya S. and Norman J. A Survey on Different Continuous Authentication Systems. *International Journal of Biometrics*, 11(1):67–99, 2019. → 572.

Azevedo L. G., Souza Soares E. F.de , Souza R., and Moreno M. F. Modern Federated Database Systems: An Overview. In *25th International Conference on Enterprise Information Systems*, pages 276–283, May 2020. → 427.

Babaoglu O. and Toueg S. Non-Blocking Atomic Commitment. In Mullender S., editor, *Distributed Systems*, pages 147–168. Addison-Wesley, Wokingham, 2nd edition, 1993. → 531.

Bader M. *Space-Filling Curves, An Introduction with Applications in Scientific Computing*. Springer-Verlag, Berlin, 2013. → 382.

Bailis P., Ghodsi A., Hellerstein J. M., and Stoica I. Bolt-on Causal Consistency. In *SIGMOD International Conference on Management Of Data*, pages 761–772, New York, NY, 2013. ACM, ACM Press. → 408.

Balakrishnan H., Kaashoek M. F., Karger D., Morris R., and Stoica I. Looking up Data in P2P Systems. *Communications of the ACM*, 46(2):43–48, Feb. 2003. → 90.

Baldauf M., Dustdar S., and Rosenberg F. A Survey on Context-aware Systems. *Int. J. Ad Hoc Ubiquitous Comput.*, 2:263–277, June 2007. → 46.

Baldoni R., Beraldi R., Quema V., Querzoni L., and Tucci-Piergiovanni S. TERA: Topic-based Event Routing for Peer-to-Peer Architectures. In *International Conference on Distributed Event-Based Systems*, pages 2–13, New York, NY, 2007. ACM Press. → 311.

Baldoni R., Querzoni L., Tarkoma S., and Virgillito A. Distributed Event Routing in Publish/Subscribe Communication Systems: a Survey. In Garbinato B., Miranda H., and Rodrigues L., editors, *Middleware for Network Eccentric and Mobile Applications*, pages 219–244. Springer-Verlag, Berlin, 2009. → 308.

Ballintijn G. *Locating Objects in a Wide-area System*. PhD thesis, Vrije Universiteit Amsterdam, 2003. → 338.

Banaei-Kashani F. and Shahab C. Criticality-based Analysis and Design of Unstructured Peer-to-Peer Networks as "Complex Systems". In *3rd International Symposium on Cluster Computing and the Grid*, pages 351–356, Los Alamitos, CA., May 2003. IEEE, IEEE Computer Society Press. → 237.

Baquero C. and Preguica N. Why Logical Clocks Are Easy. *Communications of the ACM*, 59(4):43–47, Mar. 2016. → 267.

Barborak M., Malek M., and Dahbura A. The Consensus Problem in Fault-Tolerant Computing. *ACM Computing Surveys*, 25(2):171–220, June 1993. → 504.

Barham P., Dragovic B., Fraser K., Hand S., Harris T., Ho A., Neugebar R., Pratt I., and Warfield A. Xen and the Art of Virtualization. In *19th Symposium on Operating System Principles*, pages 164–177, New York, NY, Oct. 2003. ACM, ACM Press. → 133.

Barron D. *Pascal – The Language and its Implementation*. John Wiley, New York, 1981. → 174.

Barroso L., Hölze U., and Ranganathan P. *The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines*. Synthesis Lectures on Computer Architectures. Morgan and Claypool, San Rafael, CA, 3rd edition, 2018. → 129.

Baset S. and Schulzrinne H. An Analysis of the Skype Peer-to-Peer Internet Telephony Protocol. In *25th INFOCOM Conference*, pages 1–11, Los Alamitos, CA., Apr. 2006. IEEE, IEEE Computer Society Press. → 28.

Basile C., Whisnant K., Kalbarczyk Z., and Iyer R. K. Loose Synchronization of Multithreaded Replicas. In *21st Symposium on Reliable Distributed Systems*, pages 250–255, Los Alamitos, CA., 2002. IEEE, IEEE Computer Society Press. → 436.

Basile C., Kalbarczyk Z., and Iyer R. K. A Preemptive Deterministic Scheduling Algorithm for Multithreaded Replicas. In *International Conference on Dependable Systems and Networks*, pages 149–158, Los Alamitos, CA., June 2003. IEEE Computer Society Press. → 436.

Bass L., Clements P., and Kazman R. *Software Architecture in Practice*. Addison-Wesley, Reading, MA., 4th edition, 2021. → 56, 57, 79.

Bavier A., Bowman M., Chun B., Culler D., Karlin S., Muir S., Peterson L., Roscoe T., Spalink T., and Wawrzoniak M. Operating System Support for Planetary-Scale Network Services. In *1st Symposium on Networked Systems Design and Implementation*, pages 245–266, Berkeley, CA, Mar. 2004. USENIX, USENIX. → 136.

Beckers K., Heisel M., and Hatebur D. *Pattern and Security Requirements*. Springer-Verlag, Berlin, 2015. → 547.

Ben-Ari M. *Principles of Concurrent and Distributed Programming*. Prentice Hall, Englewood Cliffs, N.J., 2nd edition, 2006. → 33.

Bernstein P. Middleware: A Model for Distributed System Services. *Communications of the ACM*, 39(2):87–98, Feb. 1996. → 37, 74.

Bernstein P. and Newcomer E. *Principles of Transaction Processing*. Morgan Kaufman, San Mateo, CA., 2nd edition, 2009. → 38, 529.

Bernstein P., Hadzilacos V., and Goodman N. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, Reading, MA., 1987. → 535.

Bershad B., Zekauskas M., and Sawdon W. The Midway Distributed Shared Memory System. In *COMPCON*, pages 528–537. IEEE, 1993. → 404.

Bharambe A. R., Agrawal M., and Seshan S. Mercury: Supporting Scalable Multi-Attribute Range Queries. In *SIGCOMM*, pages 353–366, New York, NY, Aug. 2004. ACM Press. → 384.

Bhuyan M. H., Bhattacharyya D. K., and Kalita J. K. Network Anomaly Detection: Methods, Systems and Tools. *IEEE Communications Surveys & Tutorials*, 16(1):303–336,

2014. → 612.

Bilal S. M., Bernardos C. J., and Guerrero C. Position-based Routing in Vehicular Networks: A Survey. *Journal of Network and Computer Applications*, 36(2):685–697, Mar. 2013. → 319.

Bilge L. and Dumitraş T. Before We Knew It: An Empirical Study of Zero-day Attacks in the Real World. In *Conference on Computer and Communications Security*, pages 833–844, 2012. → 611.

Birman K. *Guide to Reliable Distributed Systems: Building High-Assurance Applications and Cloud-Hosted Services*. Springer-Verlag, Berlin, 2012. → 152, 507.

Birman K. A Response to Cheriton and Skeen's Criticism of Causal and Totally Ordered Communication. *Operating Systems Review*, 28(1):11–21, Jan. 1994. → 271.

Birman K. and Joseph T. Reliable Communication in the Presence of Failures. *ACM Transactions on Computer Systems*, 5(1):47–76, Feb. 1987. → 523.

Birman K. and van Renesse R., editors. *Reliable Distributed Computing with the Isis Toolkit*. IEEE Computer Society Press, Los Alamitos, CA., 1994. → 271.

Birman K., Schiper A., and Stephenson P. Lightweight Causal and Atomic Group Multicast. *ACM Transactions on Computer Systems*, 9(3):272–314, Aug. 1991. → 526.

Birrell A. and Nelson B. Implementing Remote Procedure Calls. *ACM Transactions on Computer Systems*, 2(1):39–59, Feb. 1984. → 192.

Bishop M. *Computer Security: Art and Science*. Addison-Wesley, Reading, MA., 2nd edition, 2019. → 547.

Bittencourt L., Immich R., Sakellariou R., Fonseca N., Madeira E., Curado M., Villas L., DaSilva L., Lee C., and Rana O. The Internet of Things, Fog and Cloud continuum: Integration and challenges. *Internet of Things*, 3-4:134–155, 2018. → 103.

Blair G. and Stefani J.-B. *Open Distributed Processing and Multimedia*. Addison-Wesley, Reading, MA., 1998. → 16.

Blake G., Dreslinski R. G., Mudge T., and Flautner K. Evolution of Thread-Level Parallelism in Desktop Applications. *SIGARCH Computer Architecture News*, 38(3): 302–313, 2010. → 124.

Blaze M. *Caching in Large-Scale Distributed File Systems*. PhD thesis, Department of Computer Science, Princeton University, Jan. 1993. → 429.

Bloom B. H. Space/time Trade-offs in Hash Coding with Allowable Errors. *Communications of the ACM*, 13(7):422–426, 1970. → 559.

Boneh D., Crescenzo G. D., Ostrovsky R., and Persiano G. Public Key Encryption with Keyword Search. In *International Conference Theory and Applications of Cryptographic Techniques*, volume 3027 of *Lecture Notes in Computer Science*, pages 506–522, Berlin, 2004. Springer-Verlag. → 314.

Bonnet P., Gehrke J., and Seshadri P. Towards Sensor Database Systems. In *2nd International Conference on Mobile Data Management*, volume 1987 of *Lecture Notes in Computer Science*, pages 3–14, Berlin, Jan. 2002. Springer-Verlag. → 50.

Bosch C., Hartel P., Jonker W., and Peter A. A Survey of Provably Secure Searchable Encryption. *ACM Computing Surveys*, 47(2), Aug. 2014. → 314.

Boyd R. *Getting Started with OAuth 2.0*. O'Reilly & Associates, Sebastopol, CA., 2012.

→ 605.

Braga D. D. S., Niemann M., Hellingrath B., and Neto F. B. D. L. Survey on Computational Trust and Reputation Models. *ACM Computing Surveys*, 51(5):1–40, 2018. → 586.

Brewer E. CAP Twelve Years Later: How the "Rules" Have Changed. *Computer*, 45(2): 23–29, Feb. 2012. → 506.

Budhijara N., Marzullo K., Schneider F., and Toueg S. The Primary-Backup Approach. In Mullender S., editor, *Distributed Systems*, pages 199–216. Addison-Wesley, Wokingham, 2nd edition, 1993. → 438.

Budhiraja N. and Marzullo K. Tradeoffs in Implementing Primary-Backup Protocols. Technical Report TR 92-1307, Department of Computer Science, Cornell University, 1992. → 438.

Buford J. and Yu H. Peer-to-Peer Networking and Applications: Synopsis and Research Directions. In Shen et al. [2010], pages 3–45. → 89.

Buford J., Yu H., and Lua E. *P2P Networking and Applications*. Morgan Kaufman, San Mateo, CA., 2009. → 89.

Bugnion E., Nieh J., and Tsafrir D. *Hardware and Software Support for Virtualization*. Synthesis Lectures on Computer Architecture. Morgan and Claypool, San Rafael, CA, 2017. → 129.

Bursell M. *Trust in Computer Systems and The Cloud*. John Wiley, New York, 2022. → 586, 593.

Cabri G., Leonardi L., and Zambonelli F. Mobile-Agent Coordination Models for Internet Applications. *Computer*, 33(2):82–89, Feb. 2000. → 68.

Cachin C., Guerraoui R., and Rodrigues L. *Introduction to Reliable and Secure Distributed Programming*. Springer-Verlag, Berlin, 2nd edition, 2011. → 248, 469, 475, 476.

Callaghan B. *NFS Illustrated*. Addison-Wesley, Reading, MA., 2000. → 83, 373.

Cantin J., Lipasti M., and Smith J. The Complexity of Verifying Memory Coherence and Consistency. *IEEE Transactions on Parallel and Distributed Systems*, 16(7):663–671, July 2005. → 406.

Cao L. and Ozsu T. Evaluation of Strong Consistency Web Caching Techniques. *World Wide Web*, 5(2):95–123, June 2002. → 453.

Carriero N. and Gelernter D. Linda in Context. *Communications of the ACM*, 32(4): 444–458, 1989. → 70.

Carzaniga A., Rutherford M. J., and Wolf A. L. A Routing Scheme for Content-Based Networking. In *23rd INFOCOM Conference*, Los Alamitos, CA., Mar. 2004. IEEE, IEEE Computer Society Press. → 310.

Carzaniga A., Picco G. P., and Vigna G. Is Code Still Moving Around? Looking Back at a Decade of Code Mobility. In *29th International Conference on Software Engineering (ICSE) (companian)*, pages 9–20, Los Alamitos, CA., 2007. IEEE Computer Society Press. → 168.

Castro M. and Liskov B. Practical Byzantine Fault Tolerance and Proactive Recovery. *ACM Transactions on Computer Systems*, 20(4):398–461, Nov. 2002. → 498, 501.

Castro M., Druschel P., Hu Y. C., and Rowstron A. Topology-aware Routing in

Structured Peer-to-Peer Overlay Networks. Technical Report MSR-TR-2002-82, Microsoft Research, Cambridge, UK, June 2002a. → 337.

Castro M., Druschel P., Kermarrec A.-M., and Rowstron A. Scribe: A Large-Scale and Decentralized Application-Level Multicast Infrastructure. *IEEE Journal on Selected Areas in Communication*, 20(8):100–110, Oct. 2002b. → 233.

Castro M., Rodrigues R., and Liskov B. BASE: Using Abstraction to Improve Fault Tolerance. *ACM Transactions on Computer Systems*, 21(3):236–269, Aug. 2003. → 501.

Castro M., Costa M., and Rowstron A. Debunking Some Myths about Structured and Unstructured Overlays. In *2nd Symposium on Networked Systems Design and Implementation*, Berkeley, CA, Mar. 2005. USENIX, USENIX. → 299.

Chandra T., Griesemer R., and Redstone J. Paxos Made Live: An Engineering Perspective. In *26th Symposium on Principles of Distributed Computing*, pages 398–407, New York, NY, Aug. 2007. ACM, ACM Press. → 484.

Chaudhari S. S. and Biradar R. C. Survey of Bandwidth Estimation Techniques in Communication Networks. *Wireless Personal Communications*, 83(2):1425–1476, 2015. → 454.

Cheriton D. and Mann T. Decentralizing a Global Naming Service for Improved Performance and Fault Tolerance. *ACM Transactions on Computer Systems*, 7(2): 147–183, May 1989. → 352.

Cheriton D. and Skeen D. Understanding the Limitations of Causally and Totally Ordered Communication. In *14th Symposium on Operating System Principles*, pages 44–57. ACM, Dec. 1993. → 271.

Cheswick W. and Bellovin S. *Firewalls and Internet Security*. Addison-Wesley, Reading, MA., 2nd edition, 2000. → 609.

Chisnall D. *The Definitive Guide to the Xen Hypervisor*. Prentice Hall, Englewood Cliffs, N.J., 2007. → 133.

Chondros N., Kokordelis K., and Roussopoulos M. On the Practicality of Practical Byzantine Fault Tolerance. In *Middleware 2012*, volume 7662 of *Lecture Notes in Computer Science*, pages 436–455, Berlin, 2012. ACM/IFIP/USENIX, Springer-Verlag. → 501.

Chou T. and Orlandi C. The Simplest Protocol for Oblivious Transfer. In *International Conference Cryptology and Information Security in Latin America*, volume 9230 of *Lecture Notes in Computer Science*, pages 40–58, 2015. → 566.

Chow R. and Johnson T. *Distributed Operating Systems and Algorithms*. Addison-Wesley, Reading, MA., 1997. → 538.

Chu Y., Rao S. G., Seshan S., and Zhang H. A Case for End System Multicast. *IEEE Journal on Selected Areas in Communication*, 20(8):1456–1471, Oct. 2002. → 234.

Clark C., Fraser K., Hand S., Hansen J. G., Jul E., Limpach C., Pratt I., and Warfield A. Live Migration of Virtual Machines. In *2nd Symposium on Networked Systems Design and Implementation*, Berkeley, CA, May 2005. USENIX, USENIX. → 175, 176.

Clark D. The Design Philosophy of the DARPA Internet Protocols. In *SIGCOMM*, pages 106–114, New York, NY, Sept. 1989. ACM, ACM Press. → 152.

Clement A., Li H., Napper J., Martin J.-P., Alvisi L., and Dahlin M. BAR primer. In

International Conference on Dependable Systems and Networks, pages 287–296, 2008. → 505.

Cohen B. Incentives Build Robustness in Bittorrent. In *1st Workshop on Economics of Peer-to-Peer Systems*, June 2003. → 96.

Cohen E. and Shenker S. Replication Strategies in Unstructured Peer-to-Peer Networks. In *SIGCOMM*, pages 177–190, New York, NY, Aug. 2002. ACM, ACM Press. → 94.

Comer D. *Internetworking with TCP/IP, Volume I: Principles, Protocols, and Architecture*. Prentice Hall, Upper Saddle River, N.J., 6th edition, 2013. → 187.

Committee on National Security Systems . (CNSS) Glossary. Technical Report 4009, Apr. 2015. → 552.

Contavalli C., van der Gaast W., Lawrence D., and Kumari W. Client Subnet in DNS Queries. RFC 7871, May 2016. → 364.

Conti M., Gregori E., and Lapenna W. Content Delivery Policies in ReplicatedWeb Services: Client-Side vs. Server-Side. *Cluster Computing*, 8:47–60, Jan. 2005. → 456.

Corbett J. C., Dean J., Epstein M., Fikes A., Frost C., Furman J. J., Ghemawat S., Gubarev A., Heiser C., Hochschild P., Hsieh W., Kanthak S., Kogan E., Li H., Lloyd A., Melnik S., Mwaura D., Nagle D., Quinlan S., Rao R., Rolig L., Saito Y., Szymaniak M., Taylor C., Wang R., and Woodford D. Spanner: Google's Globally Distributed Database. *ACM Transactions on Computer Systems*, 31(3):8:1–8:22, Aug. 2013. → 259.

Cordero C. G., Vasilomanolakis E., Mühlhäuser M., and Fischer M. Community-based Collaborative Intrusion Detection. In *International Conference on Security and Privacy in Communication Systems*, pages 665–681. Springer, 2015. → 612, 613.

Cristian F. Probabilistic Clock Synchronization. *Distributed Computing*, 3:146–158, 1989. → 255.

Cristian F. Understanding Fault-Tolerant Distributed Systems. *Communications of the ACM*, 34(2):56–78, Feb. 1991. → 466.

Cui S., Belguith S., De Alwis P., Asghar M. R., and Russello G. Collusion Defender: Preserving Subscribers' Privacy in Publish and Subscribe Systems. *IEEE Transactions on Dependable and Secure Computing*, 18(3):1051–1064, 2021. → 314, 315.

Culkin J. and Zazon M. *AWS Cookbook, Recipes for Success on AWS*. O'Reilly & Associates, Sebastopol, CA., 2022. → 66, 99.

Dabek F., Cox R., Kaashoek F., and Morris R. Vivaldi: A Decentralized Network Coordinate System. In *SIGCOMM*, New York, NY, Aug. 2004a. ACM, ACM Press. → 321.

Dabek F., Li J., Sit E., Robertson J., Kaashoek M. F., and Morris R. Designing a dht for low latency and high throughput. In *1st Symposium on Networked Systems Design and Implementation*, pages 85–98, Berkeley, CA, Mar. 2004b. USENIX, USENIX. → 338.

Damas J., Graff M., and Vixie P. Extension Mechanisms for DNS (EDNS(0)). RFC 6891, Apr. 2013. → 366.

Dannewitz C., Golic J., Ohlman B., and Ahlgren B. Secure Naming for a Network of Information. In *29th INFOCOM Conference Workshops*, pages 1–6, Los Alamitos, CA., 2010. IEEE Computer Society Press. → 344.

Day J. and Zimmerman H. The OSI Reference Model. *Proceedings of the IEEE*, 71(12):

1334–1340, Dec. 1983. → 183.

Deering S. and Cheriton D. Multicast Routing in Datagram Internetworks and Extended LANs. *ACM Transactions on Computer Systems*, 8(2):85–110, May 1990. → 330.

Deering S., Estrin D., Farinacci D., Jacobson V., Liu C.-G., and Wei L. The PIM Architecture for Wide-Area Multicast Routing. *IEEE/ACM Transactions on Networking*, 4(2):153–162, Apr. 1996. → 330.

DeJonghe D. *NGINX Cookbook*. O'Reilly & Associates, Sebastopol, CA., 2nd edition, 2022. → 163.

Demers A., Greene D., Hauser C., Irish W., Larson J., Shenker S., Sturgis H., Swinehart D., and Terry D. Epidemic Algorithms for Replicated Database Maintenance. In *6th Symposium on Principles of Distributed Computing*, pages 1–12. ACM, Aug. 1987. → 240, 243, 245.

Demirbas M. and Kulkarni S. Beyond TrueTime: Using Augmented Time for Improving Google Spanner. In *7th International Workshop on Large Scale Distributed Systems and Middleware*, New York, NY, 2013. ACM Press. → 260.

Dey A. Context-Aware Computing. In Krumm J., editor, *Ubiquitous Computing Fundamentals*, pages 321–352. CRC Press, Boca Raton, FL, 2010. → 46.

Dey A. and Abowd G. Towards a Better Understanding of Context and Context-Awareness. In *Workshop on the What, Who, Where, When, Why and How of Context-Awareness*, New York, NY, Apr. 2000. ACM, ACM Press. → 45.

Diffie W. and Hellman M. New Directions in Cryptography. *IEEE Transactions on Information Theory*, IT-22(6):644–654, Nov. 1976. → 565.

Dilley J., Maggs B., Parikh J., Prokop H., Sitaraman R., and Weihl B. Globally Distributed Content Delivery. *IEEE Internet Computing*, 6(5):50–58, Sept. 2002. → 165.

Dimou A., Iliopoulos C., Polytidou E., Dhurandher S. K., Papadimitriou G., and Nicopolitidis P. A Comprehensive Review on Edge Computing: Focusing on Mobile Users. In Nicopolitidis P., Misra S., Yang L. T., Zeigler B., and Ning Z., editors, *Advances in Computing, Informatics, Networking and Cybersecurity*, number 289 in Lecture Notes in Networks and Systems, pages 121–152. Springer-Verlag, Berlin, 2022. → 48.

Diot C., Levine B., Lyles B., Kassem H., and Balensiefen D. Deployment Issues for the IP Multicast Service and Architecture. *IEEE Network*, 14(1):78–88, Jan. 2000. → 232.

Donnet B., Gueye B., and Kaafar M. A Survey on Network Coordinates Systems, Design, and Security. *IEEE Communications Surveys & Tutorials*, 12(4), Dec. 2010. → 318.

Donovan A. A. and Kernighan B. W. *The Go Programming Language*. Addison-Wesley, Reading, MA., 2015. → 122.

Doorn J. H. and Rivero L. C., editors. *Database Integrity: Challenges and Solutions*. Idea Group, Hershey, PA, 2002. → 594.

Douceur J. R. The Sybil Attack. In *1st International Workshop on Peer-to-Peer Systems*, volume 2429 of *Lecture Notes in Computer Science*, pages 251–260, Berlin, Mar. 2002. Springer-Verlag. → 586, 588.

Droms R. Dynamic Host Configuration Protocol. RFC 2161, Apr. 1997. → 46.

Dubois M., Scheurich C., and Briggs F. Synchronization, Coherence, and Event Ordering in Multiprocessors. *Computer*, 21(2):9–21, Feb. 1988. → 398.

Dunagan J., Harvey N. J. A., Jones M. B., Kostic D., Theimer M., and Wolman A. FUSE: Lightweight Guaranteed Distributed Failure Notification. In *6th Symposium on Operating System Design and Implementation*, Berkeley, CA, Dec. 2004. USENIX, USENIX. → 508.

Duvvuri V., Shenoy P., and Tewari R. Adaptive Leases: A Strong Consistency Mechanism for the World Wide Web. *IEEE Transactions on Knowledge and Data Engineering*, 15(5):1266–1276, Sept. 2003. → 433.

Eastlake D. and Hansen T. Us Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF), May 2011. → 585.

El-Sayed A., Roca V., and Mathy L. A Survey of Proposals for an Alternative Group Communication Service. *IEEE Network*, 17(1):46–51, Jan. 2003. → 233.

Elnozahy E., Alvisi L., Wang Y.-M., and Johnson D. A Survey of Rollback-Recovery Protocols in Message-Passing Systems. *ACM Computing Surveys*, 34(3):375–408, Sept. 2002. → 538, 543.

Elnozahy E. N. and Plank J. S. Checkpointing for Peta-Scale Systems: A Look into the Future of Practical Rollback-Recovery. *IEEE Transactions on Dependable and Secure Computing*, 1(2):97–108, Apr. 2004. → 540.

Elson J., Girod L., and Estrin D. Fine-Grained Network Time Synchronization using Reference Broadcasts. In *5th Symposium on Operating System Design and Implementation*, pages 147–163, Berkeley, CA, Dec. 2002. USENIX, USENIX. → 257, 258.

Erdös P. and Rényi A. On Random Graphs. *Publicationes Mathematicae*, 6:290–297, 1959. → 237.

Escriva R., Wong B., and Sirer E. G. HyperDex: A Distributed, Searchable Key-value Store. In *SIGCOMM*, pages 25–36, New York, NY, 2012. ACM Press. → 384.

Esposito C., Cotroneo D., and Russo S. On Reliability in Publish/Subscribe Services. *Computer Networks*, X(0):xxx, 2013. → 517.

Eugster P., Guerraoui R., Kermarrec A.-M., and Massoulié L. Epidemic Information Dissemination in Distributed Systems. *Computer*, 37(5):60–67, May 2004. → 240.

Evans D., Kolesnikov V., and Rosulek M. *A Pragmatic Introduction to Secure Multi-Party Computation*. NOW Publishers, Boston, MA; Delft, NL, 2018. → 566.

Feng B., Zhang H., Zhou H., and Yu S. Locator/Identifier Split Networking: A Promising Future Internet Architecture. *IEEE Communications Surveys & Tutorials*, 19 (4):2927–2948, 2017. → 329.

Ferguson N. and Schneier B. *Practical Cryptography*. John Wiley, New York, 2003. → 290.

Ferguson N., Schneier B., and Kohno T. *Cryptography Engineering: Design Principles and Practical Applications*. John Wiley, New York, 2010. → 557, 575.

Ferraiolo D., Kuhn D. R., and Chandramouli R. *Role-based Access Control*. Artech house, 2nd edition, 2007. → 596.

Ferraiolo D., Atluri V., and Gavrila S. The Policy Machine: A Novel Architecture and Framework for Access Control Policy Specification and Enforcement. *Journal of*

Systems Architecture, 57(4):412–424, 2011. → 598, 600.

Ferraiolo D., Gavrila S., and Jansen W. Policy Machine: Features, Architecture, and Specification. Technical Report NISTIR 7987, Revision 1, National Institute of Standards and Technology (NIST), Oct. 2015. → 598.

Ferrer A. J., Marquès J. M., and Jorba J. Towards the Decentralised Cloud: Survey on Approaches and Challenges for Mobile, Ad hoc, and Edge Computing. *ACM Computing Surveys*, 51(6):1–36, 2019. → 47.

Fiebig T., Lichtblau F., Streibelt F., Krüger T., Lexis P., Bush R., and Feldmann A. Learning from the Past: Designing Secure Network Protocols. In *Cybersecurity Best Practices*, pages 585–613. Springer-Verlag, Berlin, 2018. → 576.

Fielding R. *Architectural Styles and the Design of Network-based Software Architectures*. Ph.d., University of California, Irvine, 2000. → 66.

Fielding R. and Reschke J. Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing. RFC 7230, June 2014. → 188.

Fischer M., Lynch N., and Patterson M. Impossibility of Distributed Consensus with one Faulty Processor. *Journal of the ACM*, 32(2):374–382, Apr. 1985. → 503, 504.

Floyd S., Jacobson V., McCanne S., Liu C.-G., and Zhang L. A Reliable Multicast Framework for Light-weight Sessions and Application Level Framing. *IEEE/ACM Transactions on Networking*, 5(6):784–803, Dec. 1997. → 519, 520.

Fokkink W. *Distributed Algorithms: An Intuitive Approach*. MIT Press, Cambridge, MA., 2nd edition, 2018. → 248.

Foster I., Kesselman C., and Tuecke S. The Anatomy of the Grid, Enabling Scalable Virtual Organizations. *Journal of Supercomputer Applications*, 15(3):200–222, Fall 2001. → 36, 37.

Foster I. and others . The Open Grid Services Architecture, Version 1.5. GGF Informational Document GFD-I.080, June 2006. → 37.

Fowler R. *Decentralized Object Finding Using Forwarding Addresses*. Ph.D., University of Washington, Seattle, 1985. → 331.

Fox A. and Brewer E. Harvest, Yield, and Scalable Tolerant Systems. In *7th Worksop-Workshop on Hot Topics in Operating Systems (HotOS)*, pages 174–178, Los Alamitos, CA., Mar. 1999. IEEE, IEEE Computer Society Press. → 506.

Francis L. P. Privacy and Confidentiality: The Importance of Context. *The Monist*, 91 (1):52–67, 2008. → 553.

Franklin M. J., Carey M. J., and Livny M. Transactional Client-Server Cache Consistency: Alternatives and Performance. *ACM Transactions on Database Systems*, 22(3):315–363, Sept. 1997. → 443.

Fuggetta A., Picco G. P., and Vigna G. Understanding Code Mobility. *IEEE Transactions on Software Engineering*, 24(5):342–361, May 1998. → 171, 172.

Gamma E., Helm R., Johnson R., and Vlissides J. *Design Patterns, Elements of Reusable Object-Oriented Software*. Addison-Wesley, Reading, MA., 1994. → 75.

Garbacki P., Epema D., and van Steen M. The Design and Evaluation of a Self-Organizing Super-Peer Network. *IEEE Transactions on Computers*, 59(3):317–331, Mar. 2010. → 96.

Garcia-Molina H. Elections in a Distributed Computing System. *IEEE Transactions on Computers*, 31(1):48–59, Jan. 1982. → 283.

Garfinkel S. *PGP: Pretty Good Privacy*. O'Reilly & Associates, Sebastopol, CA., 1995. → 570.

Garman J. *Kerberos: The Definitive Guide*. O'Reilly & Associates, Sebastopol, CA., 2003. → 581.

Gazis A. and Katsiri E. Middleware 101: What to Know Now and for the Future. *ACM Queue*, 20(1):10–23, feb 2022. → 11.

Gelernter D. and Carriero N. Coordination Languages and their Significance. *Communications of the ACM*, 35(2):96–107, Feb. 1992. → 68.

Gentz R., Wu S. X., Wai H.-T., Scaglione A., and Leshem A. Data Injection Attacks in Randomized Gossiping. *IEEE Transactions on Signal and Information Processing over Networks*, 2(4):523–538, 2016. → 306.

Gerofi B., Ishikawa Y., Riesen R., and Wisniewski R. W., editors. *Operating Systems for Supercomputers and High Performance Computing*. Springer-Verlag, Berlin, 2019. → 34, 35.

Ghodsi A. Multicast and Bulk Lookup in Structured Overlay Networks. In Shen et al. [2010], pages 933–958. → 239.

Gifford D. Weighted Voting for Replicated Data. In *7th Symposium on Operating System Principles*, pages 150–162. ACM, Dec. 1979. → 441.

Gilbert S. and Lynch N. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-tolerant Web Services. *ACM SIGACT News*, 33(2):51–59, June 2002. → 505.

Gilbert S. and Lynch N. Perspectives on the CAP Theorem. *Computer*, 45(2):30–35, Feb. 2012. → 505.

Gkantsidis C., Mihail M., and Saberi A. Random Walks in Peer-to-Peer Networks: Algorithms and Evaluation. *Performance Evaluation*, 63:241–263, Mar. 2006. → 93.

Goel U., Wittie M. P., and Steiner M. Faster Web through Client-Assisted CDN Server Selection. In *14th International Conference on Computer Communications and Networks*, pages 1–10, 2015. → 166.

Goldreich O. *Foundations of Cryptography: Volume 2, Basic Applications*. Cambridge University Press, Cambridge, UK, 2009. → 553.

Gonzalez-Manzano L., Fuentes J. M. D., and Ribagorda A. Leveraging User-related Internet of Things for Continuous Authentication: A Survey. *ACM Computing Surveys*, 52(3):1–38, 2019. → 572.

Gray C. and Cheriton D. Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency. In *12th Symposium on Operating System Principles*, pages 202–210, New York, NY, Dec. 1989. ACM, ACM Press. → 433.

Gray J. Notes on Database Operating Systems. In Bayer R., Graham R., and Seegmuller G., editors, *Operating Systems: An Advanced Course*, volume 60 of *Lecture Notes in Computer Science*, pages 393–481. Springer-Verlag, Berlin, 1978. → 529.

Gray J. and Reuter A. *Transaction Processing: Concepts and Techniques*. Morgan Kaufman, San Mateo, CA., 1993. → 38.

Gray J., Helland P., O'Neil P., and Sashna D. The Dangers of Replication and a Solution.

In *SIGMOD International Conference on Management Of Data*, pages 173–182, Montreal, June 1996. ACM. → 395.

Gropp W., Lusk E., and Skjellum A. *Using MPI-2, Portable Parallel Programming with the Message-Passing Interface*. MIT Press, Cambridge, MA., 3rd edition, 2016. → 220.

Guerraoui R. and Schiper A. Software-Based Replication for Fault Tolerance. *Computer*, 30(4):68–74, Apr. 1997. → 472.

Guerraoui R., Knežević N., Quéma V., and Vukolić M. The next 700 bft protocols. In *5th EuroSys (European Conference on Computer Systems)*, pages 363–376, New York, NY, 2010. ACM Press. → 501.

Guichard J., Faucheur F. L., and Vasseur J.-P. *Definitive MPLS Network Designs*. Cisco Press, Indianapolis, IN, 2005. → 455.

Guttman E. Autoconfiguration for IP Networking: Enabling Local Communication. *IEEE Internet Computing*, 5:81–86, 2001. → 46.

Hadzilacos V. and Toueg S. Fault-Tolerant Broadcasts and Related Problems. In Mullender S., editor, *Distributed Systems*, pages 97–145. Addison-Wesley, Wokingham, 2nd edition, 1993. → 466, 526.

Hahmy H. *Concepts, Applications, Experimentation and Analysis of Wireless Sensor Networks*. Springer-Verlag, Berlin, 2nd edition, 2021. → 49.

Hardt D. and others . The OAuth 2.0 Authorization Framework. RFC 6746, Oct. 2012. → 605.

Hassan W. U., Guo S., Li D., Chen Z., Jee K., Li Z., and Bates A. Nodoze: Combatting Threat Alert Fatigue with Automated Provenance Triage. In *Symposium on Network and Distributed System Security*, 2019. → 612.

Haynes T. Network File System (NFS) Version 4 Protocol. RFC 7530, Mar. 2015. → 83, 350.

Helder D. A. and Jamin S. End-Host Multicast Communication Using Switch-Trees Protocols. In *2nd International Symposium on Cluster Computing and the Grid*, pages 419–424, Los Alamitos, CA., May 2002. IEEE, IEEE Computer Society Press. → 235, 236.

Hellerstein J. M. and Alvaro P. Keeping CALM: When Distributed Consistency is Easy. *Communications of the ACM*, 63(9):72–81, Sept. 2020. → 410.

Henning M. A New Approach to Object-Oriented Middleware. *IEEE Internet Computing*, 8(1):66–75, Jan. 2004. → 156.

Herlihy M. and Shavit N. *The Art of Multiprocessor Programming*. Morgan Kaufman, San Mateo, CA., 2008. → 113.

Herlihy M. and Wing J. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, July 1991. → 400.

Herlihy M., Shavit N., Luchangco V., and Spear M. *The Art of Multiprocessor Programming*. Morgan Kaufman, San Mateo, CA., 2nd edition, 2021. → 33, 400.

Hintjens P. *ZeroMQ*. O'Reilly & Associates, Sebastopol, CA., 2013. → 213.

Hohpe G. and Woolf B. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, Reading, MA., 2004. → 37, 41, 227.

Hong C.-H. and Varghese B. Resource Management in Fog/Edge Computing: A Survey on Architectures, Infrastructure, and Algorithms. *ACM Computing Surveys*, 52(5), 2019. → 103.

Hong J. The Privacy Landscape of Pervasive Computing. *IEEE Pervasive Computing*, 16 (3):40–48, 2017. → 103.

Horauer M. *Clock Synchronization in Distributed Systems*. Ph.D., University of Vienna, Department of Computer Science, Feb. 2004. → 253.

Horner L. J. Edge Strategies in Industry: Overview and Challenges. *IEEE Transactions on Network and Service Management*, 18(3):2825–2831, 2021. → 100.

Horowitz M. and Lunt S. FTP Security Extensions. RFC 2228, Oct. 1997. → 188.

Hosseini M., Ahmed D., Shirmohammadi S., and Georganas N. A Survey of Application-Layer Multicast Protocols. *IEEE Communications Surveys & Tutorials*, 9 (3):58–74, 2007. → 233.

Howard H., Schwarzkopf M., Madhavapeddy A., and Crowcroft J. Raft Refloated: Do We Have Consensus? *Operating Systems Review*, 49(1):12–21, jan 2015. → 289.

Hu V. C., Ferraiolo D., Kuhn R., Schnitzer A., Sandlin K., Miller R., and Scarfone K. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. Technical Report NIST 800-162, National Institute of Standards and Technology, Jan. 2014. → 596.

Hu V. C., Kuhn D. R., Ferraiolo D. F., and Voas J. Attribute-Based Access Control. *Computer*, 48(2):85–88, 2015. → 596.

Huffaker B., Fomenkov M., Plummer D. J., Moore D., and Claffy K. Distance Metrics in the Internet. In *International Telecommunications Symposium*, Los Alamitos, CA., Sept. 2002. IEEE, IEEE Computer Society Press. → 454.

Hunt G., Nahum E., and Tracey J. Enabling Content-Based Load Distribution for Scalable Services. Technical report, IBM T.J. Watson Research Center, May 1997. → 163.

Hunt P., Konar M., Junqueira F. P., and Reed B. ZooKeeper: Wait-free Coordination for Internet-scale Systems. In *USENIX Annual Technical Conference*, 2010. → 280, 422.

Hutto P. and Ahamad M. Slow Memory: Weakening Consistency to Enhance Concurrency in Distributed Shared Memories. In *10th International Conference on Distributed Computing Systems*, pages 302–311, Paris, France, May 1990. IEEE. → 401.

Iqbal M. and Matulevičius R. Exploring Sybil and Double-Spending Risks in Blockchain Systems. *IEEE Access*, 9:76153–76177, 2021. → 587.

ISO . Open Distributed Processing Reference Model - Part 2: Foundations. International Standard ISO/IEC IS 10746-2, 1995. → 12.

Jakubeit P., Peter A., and van Steen M. The Measurable Environment as Nonintrusive Authentication Factor on the Example of WiFi Beacon Frames. In *5th International Workshop on Emerging Technologies for Authorization and Authentication*, Sept. 2022. → 572.

Jalote P. *Fault Tolerance in Distributed Systems*. Prentice Hall, Englewood Cliffs, N.J., 1994. → 443, 463.

Janic M. *Multicast in Network and Application Layer*. Ph.d., Delft University of Technology,

The Netherlands, Oct. 2005. → 232.

Janiga M. J., Dibner G., and Governali F. J. Internet Infrastructure: Content Delivery. Goldman Sachs Global Equity Research, Apr. 2001. → 455.

Jelasity M. and Kermarrec A.-M. Ordered Slicing of Very Large-Scale Overlay Networks. In *6th International Conference on Peer-to-Peer Computing*, pages 117–124, Los Alamitos, CA., Sept. 2006. IEEE Computer Society Press. → 300.

Jelasity M., Montresor A., and Babaoglu O. Gossip-based Aggregation in Large Dynamic Networks. *ACM Transactions on Computer Systems*, 23(3):219–252, Aug. 2005. → 297.

Jelasity M., Voulgaris S., Guerraoui R., Kermarrec A.-M., and van Steen M. Gossip-based Peer Sampling. *ACM Transactions on Computer Systems*, 25(3), Aug. 2007. → 241, 298.

Jesi G., Montresor A., and van Steen M. Secure Peer Sampling. *Computer Networks*, 54 (12):2086–2098, Aug. 2010. → 304, 306.

Jesi G.-P., Hales D., and van Steen M. Identifying Malicious Peers Before It's Too Late: A Decentralized Secure Peer Sampling Service. In *1st International Conference on Self-Adaptive and Self-Organizing Systems*, Los Alamitos, CA., June 2007. IEEE, IEEE Computer Society Press. → 588.

Johnson B. An Introduction to the Design and Analysis of Fault-Tolerant Systems. In Pradhan D., editor, *Fault-Tolerant Computer System Design*, pages 1–87. Prentice Hall, Upper Saddle River, N.J., 1995. → 470.

Josefsson S. *Shishi – Kerberos 5 Implementation*. Samurai Media Limited, Wickford, UK, 2015. → 581.

Joseph J., Ernest M., and Fellenstein C. Evolution of grid computing architecture and grid adoption models. *IBM Systems Journal*, 43(4):624–645, Apr. 2004. → 37.

Junqueira F. and Reed B. *ZooKeeper*. O'Reilly & Associates, Sebastopol, CA., 2014. → 280, 281, 287.

Kahn D. *The Codebreakers*. Macmillan, New York, 1967. → 556.

Karsten M. and Barghi S. User-level Threading: Have Your Cake and Eat It Too. In *International Conference on Measurements and Modeling of Computer Systems*, volume 4. ACM, Mar. 2020. → 122.

Kasera S., Kurose J., and Towsley D. Scalable Reliable Multicast Using Multiple Multicast Groups. In *International Conference on Measurements and Modeling of Computer Systems*, pages 64–74, Seattle, WA, June 1997. ACM. → 520.

Kaufman C., Perlman R., and Speciner M. *Network Security: Private Communication in a Public World*. Prentice Hall, Englewood Cliffs, N.J., 2nd edition, 2003. → 575, 580.

Kemme B., Jimenez Peris R., and Patino-Martinez M. *Replicated Databases*. Synthesis Lectures on Computer Architectures. Morgan and Claypool, San Rafael, CA, 2010. → 426.

Kermarrec A.-M. and Triantafillou P. XL Peer-to-peer Pub/Sub Systems. *ACM Computing Surveys*, 46(2):16:1–16:45, Nov. 2013. → 309.

Khormali A., Park J., Alasmary H., Anwar A., Saad M., and Mohaisen D. Domain Name System Security and Privacy: A Contemporary Survey. *Computer Networks*,

185, 2021. → 365.

Khoshafian S. and Buckiewicz M. *Introduction to Groupware, Workflow, and Workgroup Computing*. John Wiley, New York, 1995. → 226.

Khraisat A., Gondal I., Vamplew P., and Kamruzzaman J. Survey of Intrusion Detection Systems: Techniques, Datasets and Challenges. *Cybersecurity*, 2(1):1–22, 2019. → 612.

Kim M., Fielding J. J., and Kotz D. Risks of Using AP Locations Discovered Through War Driving. In *4th International Conference Pervasive Computing*, volume 3968 of *Lecture Notes in Computer Science*, pages 67–82, Berlin, May 2006. Springer-Verlag. → 318.

Kirsch J. and Amir Y. Paxos for System Builders. Technical Report CNDS-2008-2, John Hopkins University, Mar. 2008. → 480, 484.

Kleiman S. Vnodes: an Architecture for Multiple File System Types in UNIX. In *Summer Technical Conference*, pages 238–247, Atlanta, GA, June 1986. USENIX. → 84.

Kleppmann M. and Beresford A. R. A Conflict-Free Replicated JSON Datatype. *IEEE Transactions on Parallel and Distributed Systems*, 28(10):2733–2745, Oct. 2017. → 410.

Kohl J., Neuman B., and T'so T. The Evolution of the Kerberos Authentication System. In Brazier F. and Johansen D., editors, *Distributed Open Systems*, pages 78–94. IEEE Computer Society Press, Los Alamitos, CA., 1994. → 581.

Kopetz H. and Verissimo P. Real Time and Dependability Concepts. In Mullender S., editor, *Distributed Systems*, pages 411–446. Addison-Wesley, Wokingham, 2nd edition, 1993. → 18, 463.

Koren I. and Krishna C. M. *Fault-Tolerant Systems*. Morgan Kaufman, San Mateo, CA., 2007. → 463, 497.

Kotla R., Alvisi L., Dahlin M., Clement A., and Wong E. Zyzzyva: Speculative Byzantine Fault Tolerance. *ACM Transactions on Computer Systems*, 27(4), Dec. 2009. → 501.

Krakowiak S. *Middleware Architecture with Patterns and Frameworks*. Creative Commons, 2009. → 58.

Kreitz G. and Niemelä F. Spotify – Large Scale, Low Latency, P2P Music-on-Demand Streaming. In *10th International Conference on Peer-to-Peer Computing*, pages 266–275, Los Alamitos, CA., Aug. 2010. IEEE, IEEE Computer Society Press. → 28.

Kshemkalyani A. and Singhal M. *Distributed Computing, Principles, Algorithms, and Systems*. Cambridge University Press, Cambridge, UK, 2008. → 272, 493, 495.

Kucharski A. *The Rules of Contagion, Why Things Spread – And Why They Stop*. Profile Books, 2020. → 62.

Kumar K., Liu J., Lu Y.-H., and Bhargava B. A Survey of Computation Offloading for Mobile Systems. *Mobile Networks and Applications*, 18(1):129–140, 2013. → 168.

Lagar-Cavilla H. A., Whitney J. A., Scannell A. M., Patchin P., Rumble S. M., Lara E.de , Brudno M., and Satyanarayanan M. SnowFlock: Rapid Virtual Machine Cloning for Cloud Computing. In *4th EuroSys (European Conference on Computer Systems)*, pages 1–12, New York, NY, 2009. ACM Press. → 177.

Lai A. and Nieh J. Limits of Wide-Area Thin-Client Computing. In *International Conference on Measurements and Modeling of Computer Systems*, pages 228–239, New

York, NY, June 2002. ACM, ACM Press. → 144.

LaMarca A. and Lara E.de . *Location Systems: An Introduction to the Technology Behind Location Awareness*. Morgan & Claypool, San Rafael, CA, 2008. → 317.

Lamport L. The Part-Time Parliament. *ACM Transactions on Computer Systems*, 16(2): 133–169, May 1998. → 479.

Lamport L. Paxos Made Simple. *ACM SIGACT News*, 32(4):51–58, Dec. 2001. → 479, 480.

Lamport L. Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7):558–565, July 1978. → 260, 441.

Lamport L. How to Make a Multiprocessor Computer that Correctly Executes Multiprocessor Programs. *IEEE Transactions on Computers*, C-29(9):690–691, Sept. 1979. → 397.

Lamport L., Shostak R., and Paese M. The Byzantine Generals Problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, July 1982. → 468.

Lampson B. How to Build a Highly Available System using Consensus. In Babaoglu O. and Marzullo K., editors, *12th International Workshop on Distributed Algorithms*, volume 1151 of *Lecture Notes in Computer Science*, pages 1–17, Berlin, Oct. 1996. Springer-Verlag. → 479.

Laprie J.-C. Dependability – Its Attributes, Impairments and Means. In Randell B., Laprie J.-C., Kopetz H., and Littlewood B., editors, *Predictably Dependable Computing Systems*, pages 3–24. Springer-Verlag, Berlin, 1995. → 547.

Laurie B. and Laurie P. *Apache: The Definitive Guide*. O'Reilly & Associates, Sebastopol, CA., 3rd edition, 2002. → 161.

Lawder J. and King P. Querying Multi-dimensional Data Indexed Using Hilbert Space-Filling Curve. *ACM Sigmod Record*, 30(1):19–24, Mar. 2000. → 381.

Levesque M. and Tipper D. A Survey of Clock Synchronization Over Packet-Switched Networks. *IEEE Communications Surveys & Tutorials*, 18(4):2926–2947, 2016. → 253, 257.

Levine B. and Garcia-Luna-Aceves J. A Comparison of Reliable Multicast Protocols. *ACM Multimedia Systems Journal*, 6(5):334–348, 1998. → 519.

Levine B. N., Shields C., and Margolin N. B. A Survey of Solutions to the Sybil Attack. Technical report, University of Massachusetts Amherst, Amherst, MA, 2006. → 588.

Lewis B. and Berg D. J. *Multithreaded Programming with Pthreads*. Prentice Hall, Englewood Cliffs, N.J., 2nd edition, 1998. → 113.

Li X., Jiang P., Chen T., Luo X., and Wen Q. A Survey on the Security of Blockchain Systems. *Future Generation Computer Systems*, 107:841–853, 2020. → 589.

Lilja D. Cache Coherence in Large-Scale Shared-Memory Multiprocessors: Issues and Comparisons. *ACM Computing Surveys*, 25(3):303–338, Sept. 1993. → 443.

Lin M.-J. and Marzullo K. Directional Gossip: Gossip in a Wide-Area Network. In *Proceedings 3rd European Dependable Computing Conf.*, volume 1667 of *Lecture Notes in Computer Science*, pages 364–379. Springer-Verlag, Berlin, Sept. 1999. → 244.

Lin S.-D., Lian Q., Chen M., , and Zhang Z. A Practical Distributed Mutual Exclusion Protocol in Dynamic Peer-to-Peer Systems. In *3rd International Workshop on Peer-to-Peer Systems*, volume 3279 of *Lecture Notes in Computer Science*, pages 11–21, Berlin,

Feb. 2004. Springer-Verlag. → 277, 278.

Ling B. C., Kiciman E., and Fox A. Session State: Beyond Soft State. In *1st Symposium on Networked Systems Design and Implementation*, pages 295–308, Berkeley, CA, Mar. 2004. USENIX, USENIX. → 153.

Liskov B. From viewstamped Replication to Byzantine Fault Tolerance. In Charron-Bost B., Pedone F., and Schiper A., editors, *Replication, Theory and Practice*, volume 5959 of *Lecture Notes in Computer Science*, chapter 5, pages 121–149. Springer-Verlag, Berlin, 2010. → 498.

Liu C. and Albitz P. *DNS and BIND*. O'Reilly & Associates, Sebastopol, CA., 5th edition, 2006. → 189, 359.

Liu C.-G., Estrin D., Shenker S., and Zhang L. Local Error Recovery in SRM: Comparison of Two Approaches. *IEEE/ACM Transactions on Networking*, 6(6):686–699, Dec. 1998. → 520.

Liu F. and Solihin Y. Understanding the Behavior and Implications of Context Switch Misses. *ACM Transactions on Architecture and Code Optimization*, 7(4):21:1–21:28, Dec. 2010. → 116.

Lo V., Zhou D., Liu Y., GauthierDickey C., and Li J. Scalable Supernode Selection in Peer-to-Peer Overlay Networks. In *2nd Workshop on Hot Topics in Peer-to-Peer Systems (HotP2P)*, pages 18–27, Los Alamitos, CA., July 2005. IEEE Computer Society Press. → 293.

Lua E., Crowcroft J., Pias M., Sharma R., and Lim S. A Survey and Comparison of Peer-to-Peer Overlay Network Schemes. *IEEE Communications Surveys & Tutorials*, 7 (2):22–73, Apr. 2005. → 32, 89.

Lui J., Misra V., and Rubenstein D. On the Robustness of Soft State Protocols. In *12th International Conference on Network Protocols*, pages 50–60, Los Alamitos, CA., Oct. 2004. IEEE, IEEE Computer Society Press. → 152.

Lv Q., Cao P., Cohen E., Li K., and Shenker S. Search and Replication in Unstructured Peer-to-Peer Networks. In *16th International Conference on Supercomputing*, pages 84–95, New York, NY, June 2002. ACM, ACM Press. → 93, 94.

Lynch N. *Distributed Algorithms*. Morgan Kaufman, San Mateo, CA., 1996. → 249, 283.

Lyu L., Yu H., Zhao J., and Yang Q. *Threats to Federated Learning*, volume 12500 of *Lecture Notes in Artificial Intelligence*, pages 3–16. Springer Nature, Cham, 2020. → 169.

Maassen J., Kielmann T., and Bal H. E. Parallel Application Experience with Replicated Method Invocation. *Concurrency & Computation: Practice and Experience*, 13(8-9): 681–712, 2001. → 436.

Madden S. R., Franklin M. J., Hellerstein J. M., and Hong W. TinyDB: An Acquisitional Query Processing System for Sensor Networks. *ACM Transactions on Database Systems*, 30(1):122–173, 2005. → 51.

Mahajan P., Alvisi L., and Dahlin M. Consistency, availability, and convergence. Technical Report TR-11-22, University of Texas at Austin, May 2011. → 408.

Malhotra A., Cohen I. E., Brakke E., and Goldberg S. Attacking the Network Time Protocol. In *Symposium on Network and Distributed System Security*, Feb. 2016. → 257.

Malone T. and Crowston K. The Interdisciplinary Study of Coordination. *ACM Computing Surveys*, 26(1):87–119, Mar. 1994. → 248.

Mao Z. M., Cranor C. D., Douglis F., Rabinovich M., Spatscheck O., and Wang J. A Precise and Efficient Evaluation of the Proximity between Web Clients and their Local DNS Servers. In *USENIX Annual Technical Conference*, pages 229–242, Berkeley, CA, June 2002. USENIX, USENIX. → 166.

Marzullo K. and Owicki S. Maintaining The Time in a Distributed System. In *2nd Symposium on Principles of Distributed Computing*, pages 295–305, New York, NY, 1983. ACM, ACM Press. → 259.

Mattern F. and Floerkemeier C. *From the Internet of Computers to the Internet of Things*, pages 242–259. Springer-Verlag, Berlin, 2010. → 44.

Mazouni K., Garbinato B., and Guerraoui R. Building Reliable Client-Server Software Using Actively Replicated Objects. In Graham I., Magnusson B., Meyer B., and Nerson J.-M., editors, *Technology of Object Oriented Languages and Systems*, pages 37–53. Prentice Hall, Englewood Cliffs, N.J., 1995. → 436.

Medina V. and Garcia J. A Survey of Migration Mechanisms of Virtual Machines. *ACM Computing Surveys*, 46(3):30, Jan. 2014. → 175.

Meling H. and Jehl L. Tutorial Summary: Paxos Explained from Scratch. In *17th International Conference on Principles of Distributed Systems*, pages 1–10. Springer, 2013. → 484.

Menasce D. and Almeida V. *Capacity Planning for Web Services*. Prentice Hall, Englewood Cliffs, N.J., 2002. → 25.

Menezes A. J., Oorschot P. C.van , and Vanstone S. A. *Handbook of Applied Cryptography*. CRC Press, Boca Raton, 3rd edition, 1996. → 568.

Merideth M. G. and Reiter M. K. *Selected Results from the Latest Decade of Quorum Systems Research*, pages 185–206. Springer-Verlag, Berlin, 2010. → 443.

Message Passing Interface Forum . MPI: A Message-Passing Interface Standard, version 4.0. Technical report, University of Tenness, Knoxville, June 2021. → 220.

Meyerovich L. A. and Bodik R. Fast and Parallel Webpage Layout. In *19th International World Wide Web Conference*, pages 711–720, New York, NY, 2010. ACM Press. → 124.

Mills D. Network Time Protocol (version 3): Specification, Implementation, and Analysis. RFC 1305, July 1992. → 257.

Mills D. L. *Computer Network Time Synchronization: The Network Time Protocol on Earth and in Space*. CRC Press, Boca Raton, FL, 2nd edition, 2011. → 257.

Milojicic D., Douglis F., Paindaveine Y., Wheeler R., and Zhou S. Process Migration. *ACM Computing Surveys*, 32(3):241–299, Sept. 2000. → 167.

Min S. L. and Baer J.-L. Design and Analysis of a Scalable Cache Coherence Scheme Based on Clocks and Timestamps. *IEEE Transactions on Parallel and Distributed Systems*, 3(1):25–44, Jan. 1992. → 443.

Mockapetris P. Domain Names - Concepts and Facilities. RFC 1034, Nov. 1987a. → 354, 359.

Mockapetris P. Domain Names - Implementation and Specification. RFC 1035, Nov. 1987b. → 354, 359.

Mohan C., Strong R., and Finkelstein S. Method for Distributed Transaction Commit and Recovery using Byzantine Agreement within Clusters of Processors. In *2nd Symposium on Principles of Distributed Computing*, pages 89–103, New York, NY, 1983. ACM Press. → 468.

Moll P., Shang W., Yu Y., Afanasyev A., and Zhang L. A Survey of Distributed Dataset Synchronization in Named Data Networking. Technical Report NDN-0053, UCLA, May 2021. → 386.

Mottola L. and Picco G. P. Programming Wireless Sensor Networks: Fundamental Concepts and State of the Art. *ACM Computing Surveys*, 43(3):19:1–19:51, Apr. 2011. → 49.

Moura G. C. M., Castro S., Hardaker W., Wullink M., and Hesselman C. Clouding up the Internet: How Centralized is DNS Traffic Becoming? In *Internet Measurement Conference*, page 42–49, 2020. → 365.

Mousazadeh M. and Ladani B. T. Gossip-based Data Aggregation in Hostile Environments. *Computer Communications*, 62:1–12, 2015. → 306.

Mühl G., Fiege L., and Pietzuch P. *Distributed Event-Based Systems*. Springer-Verlag, Berlin, 2006. → 69.

Muntz D. and Honeyman P. Multi-level Caching in Distributed File Systems. In *Winter Technical Conference*, pages 305–313, San Francisco, CA, Jan. 1992. USENIX. → 429.

Murty J. *Programming Amazon Web Services*. O'Reilly & Associates, Sebastopol, CA., 2008. → 66, 99.

Najafi A., Tai A., and Wei M. Systems Research is Running out of Time. In *Workshop on Hot Topics in Operating Systems (HotOS)*, page 65–71, 2021. → 249.

Naur P. and Randell B. Report on the NATO Software Engineering Conference 1968. Technical report, Scientific Affairs Division NATO, Brussels, Belgium, Oct. 1968. → 74.

Needham R. and Schroeder M. Using Encryption for Authentication in Large Networks of Computers. *Communications of the ACM*, 21(12):993–999, Dec. 1978. → 577.

Nelson B. *Remote Procedure Call*. Ph.D., Carnegie-Mellon University, 1981. → 514.

Neuman B. Proxy-Based Authorization and Accounting for Distributed Systems. In *13th International Conference on Distributed Computing Systems*, pages 283–291, Pittsburgh, May 1993. IEEE. → 601.

Neuman B. Scale in Distributed Systems. In Casavant T. and Singhal M., editors, *Readings in Distributed Computing Systems*, pages 463–489. IEEE Computer Society Press, Los Alamitos, CA., 1994. → 24, 28, 606.

Neuman C., Yu T., Hartman S., and Raeburn K. The Kerberos Network Authentication Service. RFC 4120, July 2005. → 581.

Ng E. and Zhang H. Predicting Internet Network Distance with Coordinates-Based Approaches. In *21st INFOCOM Conference*, Los Alamitos, CA., June 2002. IEEE, IEEE Computer Society Press. → 320.

Nguyen C. T., Hoang D. T., Nguyen D. N., Niyato D., Nguyen H. T., and Dutkiewicz E. Proof-of-Stake Consensus Mechanisms for Future Blockchain Networks: Fundamentals, Applications and Opportunities. *IEEE Access*, 7:85727–85745, 2019. → 292, 589.

Nissenbaum H. *Privacy in Context*. Stanford University Press, Stanford, CA, 2010. → 553.

Noble B., Fleis B., and Kim M. A Case for Fluid Replication. In *NetStore'99*, Seattle, WA, Oct. 1999. → 430.

Noveck D. and Lever C. Network File System (NFS) Version 4 Minor Verion 1 Protocol. RFC 8881, Aug. 2020. → 83, 350.

Nyers L. and Jelasity M. A Comparative Study of Spanning Tree and Gossip Protocols for Aggregation. *Concurrency & Computation: Practice and Experience*, 2015. → 522.

Nygren E., Sitaraman R. K., and Sun J. The Akamai Network: A Platform for High-Performance Internet Applications. *Operating Systems Review*, 44(3):2–19, July 2010. → 165.

OASIS . AMQP, Protocol specification, Version 1.0, Oct. 2012. → 227.

Obraczka K. Multicast Transport Protocols: A Survey and Taxonomy. *IEEE Communications Magazine*, 36(1):94–102, Jan. 1998. → 232.

Oikonomou K. and Stavrakakis I. Performance Analysis of Probabilistic Flooding Using Random Graphs. In *World of Wireless, Mobile and Multimedia Networks, 2007. WoWMoM 2007. IEEE International Symposium on a*, pages 1–6, June 2007. doi: 10.1109/WOWMOM.2007.4351694. → 237.

OMG . The Common Object Request Broker: Architecture and Specification, revision 2.4.2. OMG Document formal/00-02-33, Object Management Group, Framingham, MA, Feb. 2001. → 156.

OMG . UML 2.0 Superstructure Specification. OMG Document ptc/04-10-02, Object Management Group, Framingham, MA, Oct. 2004. → 57.

Ongaro D. *Consensus: Bridging Theory AND Practice*. Ph.D., Stanford University, Aug. 2014. → 479.

Ongaro D. and Ousterhout J. In Search of an Understandable Consensus Algorithm. In *USENIX Annual Technical Conference*, pages 305–319, Berkeley, CA, June 2014. USENIX, USENIX. → 289, 477, 479.

Onica E., Felber P., Mercier H., and Riviere E. Confidentiality-Preserving Publish/Subscribe: A Survey. *ACM Computing Surveys*, 49(2), June 2016. → 315.

Oram A., editor. *Peer-to-Peer: Harnessing the Power of Disruptive Technologies*. O'Reilly & Associates, Sebastopol, CA., 2001. → 32.

Otte P. Sybil-resistant Trust Mechanisms in Distributed Systems. Msc, Delft University of Technology, Dec. 2016. → 591.

Otte P., Vos M.de , and Pouwelse J. TrustChain: A Sybil-resistant Scalable Blockchain. *Future Generation Computer Systems*, 107:770–780, 2020. → 589, 590.

Özsu T. and Valduriez P. *Principles of Distributed Database Systems*. Springer-Verlag, Berlin, 4th edition, 2020. → 89, 426.

Pahl C., Brogi A., Soldani J., and Jamshidi P. Cloud Container Technologies: A State-of-the-Art Review. *IEEE Transactions on Cloud Computing*, 7(3):677–692, 2019. → 135.

Pai V., Aron M., Banga G., Svendsen M., Druschel P., Zwaenepoel W., and Nahum E. Locality-Aware Request Distribution in Cluster-Based Network Servers. In

8th International Conference on Architectural Support for Programming Languages and Operating Systems, pages 205–216, New York, NY, Oct. 1998. ACM, ACM Press. → 163.

Panzieri F. and Shrivastava S. Rajdoot: A Remote Procedure Call Mechanism with Orphan Detection and Killing. *IEEE Transactions on Software Engineering*, 14(1):30–37, Jan. 1988. → 515.

Pappas V., Massey D., Terzis A., and Zhang L. A Comparative Study of the DNS Design with DHT-Based Alternatives. In *25th INFOCOMConference*, Los Alamitos, CA., May 2006. IEEE, IEEE Computer Society Press. → 368.

Parecki A. *OAuth 2.0 Simplified: A Guide to Building OAuth 2.0 Servers*. Lulu Press, Inc, 2020. → 605.

Park S., Specter M., Narula N., and Rivest R. L. Going From Bad to Worse: From Internet Voting to Blockchain Voting. *Journal of Cybersecurity*, 7(1):1–15, 2021. → 592.

Parlavantzas N. and Coulson G. Designing and Constructing Modifiable Middleware using Component Frameworks. *IET Software*, 1(4):113–126, Aug. 2007. → 78.

Passarella A. A Survey on Content-centric Technologies for the Current Internet: CDN and P2P Solutions. *Computer Communications*, 35(1):1–32, 2012. → 165.

Pautasso C., Zimmermann O., and Leymann F. Restful Web Services vs. "Big" Web Services: Making the Right Architectural Decision. In *17th International World Wide Web Conference*, pages 805–814, New York, NY, Aug. 2008. ACM Press. → 66, 67.

Pease M., Shostak R., and Lamport L. Reaching Agreement in the Presence of Faults. *Journal of the ACM*, 27(2):228–234, Apr. 1980. → 468.

Perkins C. IP Mobility Support in IPv4, Revised. RFC 5944, Nov. 2010. → 47.

Perkins C., Johnson D., and Arkko J. Mobility Support in IPv6. RFC 6275, July 2011. → 47, 331.

Peterson L., Bavier A., Fiuczynski M. E., and Muir S. Experiences Building PlanetLab. In *7th Symposium on Operating System Design and Implementation*, pages 351–366, Berkeley, CA, Nov. 2006. USENIX, USENIX. → 136.

Pike R., Presotto D., Dorward S., Flandrena B., Thompson K., Trickey H., and Winterbottom P. Plan 9 from Bell Labs. *Computing Systems*, 8(3):221–254, Summer 1995. → 346.

Pinzari G. NX X Protocol Compression. Technical Report D-309/3-NXP-DOC, NoMachine, Rome, Italy, Sept. 2003. → 144.

Pitoura E. and Samaras G. Locating Objects in Mobile Computing. *IEEE Transactions on Knowledge and Data Engineering*, 13(4):571–592, July 2001. → 338.

Plummer D. An Ethernet Address Resolution Protocol. RFC 826, Nov. 1982. → 330.

Podling S. and Boszormenyi L. A Survey of Web Cache Replacement Strategies. *ACM Computing Surveys*, 35(4):374–398, Dec. 2003. → 453.

Popek G. J. and Goldberg R. P. Formal Requirements for Virtualizable Third Generation Architectures. *Communications of the ACM*, 17(7):412–421, July 1974. → 131, 132.

Popescu A., Constantinescu D., Erman D., and Ilie D. A Survey of Reliable Multicast Communication. In *3rd Conference on Next Generation Internet Networks*, pages 111–118, May 2007. → 517.

Popescu A. M., Tudorache G. I., Peng B., and Kemp A. H. Surveying Position Based Routing Protocols for Wireless Sensor and Ad-hoc Networks. *International Journal on Communication Networks and Information Security*, 4(1):41–67, Apr. 2012. → 319.

Poslad S. *Ubiquitous Computing: Smart Devices, Environments and Interactions*. John Wiley, New York, 2009. → 44, 46.

Postel J. Simple Mail Transfer Protocol. RFC 821, Aug. 1982. → 226.

Postel J. and Reynolds J. File Transfer Protocol. RFC 995, Oct. 1985. → 188.

Pourghassemi B. *Adaptive Tools for Performance Analysis of Large-scale Applications*. Phd, University of California at Irvine, 2021. → 146.

Pouwelse J. A., Garbacki P., Epema D. H. J., and Sips H. J. The Bittorrent P2P File-Sharing System: Measurements and Analysis. In *4th International Workshop on Peer-to-Peer Systems*, volume 3640 of *Lecture Notes in Computer Science*, pages 205–216, Berlin, Feb. 2005. Springer-Verlag. → 97.

Pradhan D. *Fault-Tolerant Computer System Design*. Prentice Hall, Englewood Cliffs, N.J., 1996. → 464.

Preguica N. Conflict-Free Replicated Data Types: An Overview. arXiv:1806.10254, 2018. → 410.

Prisco R. D., Lampson B., and Lynch N. Revisiting the Paxos Algorithm. In Mavronicolas M. and Tsigas P., editors, *11th International Workshop on Distributed Algorithms*, volume 1320 of *Lecture Notes in Computer Science*. Springer-Verlag, Berlin, Sept. 1997. → 479.

Qin H., Li Q., Speiser J., Kraft P., and Ousterhout J. Arachne: Core-Aware Thread Management. In *13th Symposium on Operating System Design and Implementation*, pages 145–161. USENIX, Oct. 2018. → 122.

Rabinovich M. and Spastscheck O. *Web Caching and Replication*. Addison-Wesley, Reading, MA., 2002. → 451.

Rabinovich M., Rabinovich I., Rajaraman R., and Aggarwal A. A Dynamic Object Replication and Migration Protocol for an Internet Hosting Service. In *19th International Conference on Distributed Computing Systems*, pages 101–113, Austin, TX, June 1999. IEEE. → 427, 428.

Radia S. *Names, Contexts, and Closure Mechanisms in Distributed Computing Environments*. Ph.D., University of Waterloo, Ontario, 1989. → 348.

Rajaraman V. Grid Computing. *Resonance*, 21(5):401–415, 2016. → 35.

Ramanathan P., Shin K., and Butler R. Fault-Tolerant Clock Synchronization in Distributed Systems. *Computer*, 23(10):33–42, Oct. 1990. → 253.

Ramirez W., Masip-Bruin X., Yannuzzi M., Serral-Gracia R., Martinez A., and Siddiqui M. A Survey and Taxonomy of ID/Locator Split Architectures. *Computer Networks*, 60:13–33, 2014. → 329.

Raynal M. and Singhal M. Logical Time: Capturing Causality in Distributed Systems. *Computer*, 29(2):49–56, Feb. 1996. → 262.

Rescorla E. and others . The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, Aug. 2018. → 583.

Reynolds J. and Postel J. Assigned Numbers. RFC 1700, Oct. 1994. → 150.

Ricart G. and Agrawala A. An Optimal Algorithm for Mutual Exclusion in Computer Networks. *Communications of the ACM*, 24(1):9–17, Jan. 1981. → 274.

Richards M. and Ford N. *Fundamentals of Software Architecture*. O'Reilly & Associates, Sebastopol, CA., 2020. → 56.

Richardson T., Stafford-Fraser Q., Wood K. R., and Hopper A. Virtual Network Computing. *IEEE Internet Computing*, 2(1):33–38, Jan. 1998. → 144.

Risson J. and Moors T. Survey of Research towards Robust Peer-to-Peer Networks: Search Methods. *Computer Networks*, 50(17):3485–3521, 2006. → 93.

Rizzo L. Effective Erasure Codes for Reliable Computer Communication Protocols. *SIGCOMM Computer Communications Review*, 27(2):24–36, Apr. 1997. → 536.

Robbins K. and Robbins S. *UNIX Systems Programming*. Prentice Hall, Englewood Cliffs, N.J., 2003. → 113, 122.

Robin J. S. and Irvine C. E. Analysis of the Intel Pentium's Ability to Support a Secure Virtual Machine Monitor. In *9th USENIX Security Symposium*, pages 129–144, Berkeley, CA, 2000. USENIX. → 132.

Rodrigues L., Fonseca H., and Verissimo P. Totally Ordered Multicast in Large-Scale Systems. In *16th International Conference on Distributed Computing Systems*, pages 503–510, Hong Kong, May 1996. IEEE. → 441.

Rodriguez P., Spanner C., and Biersack E. Analysis of Web Caching Architecture: Hierarchical and Distributed Caching. *IEEE/ACM Transactions on Networking*, 21(4): 404–418, Aug. 2001. → 452.

Rosenblum M. and Garfinkel T. Virtual Machine Monitors: Current Technology and Future Trends. *Computer*, 38(5):39–47, May 2005. → 131.

Roussos G., Marsh A. J., and Maglavera S. Enabling Pervasive Computing with Smart Phones. *IEEE Pervasive Computing*, 4(2):20–26, Apr. 2005. → 44.

Rowe F., Baskerville R., and Wolff F.-C. Functionality vs. Security in IS: Tradeoff or Equilibrium? In *33rd International Conference on Information Systems*, 2012. → 566.

Rowstron A. and Druschel P. Pastry: Scalable, Distributed Object Location and Routing for Large-Scale Peer-to-Peer Systems. In *Middleware 2001*, volume 2218 of *Lecture Notes in Computer Science*, pages 329–350, Berlin, Nov. 2001. Springer-Verlag. → 233, 338.

Roy G. *RabbitMQ in Depth*. Manning Publications, Shelter Island, NY, 2018. → 227, 230, 231, 232.

Saad M., Spaulding J., Njilla L., Kamhoua C., Shetty S., Nyang D., and Mohaisen D. Exploring the Attack Surface of Blockchain: A Comprehensive Survey. *IEEE Communications Surveys & Tutorials*, 22(3):1977–2008, 2020. → 589.

Sagan H. *Space-Filling Curves*. Springer-Verlag, Berlin, 1994. → 382.

Sahoo J., Salahuddin M. A., Glitho R., Elbiaze H., and Ajib W. A Survey on Replica Server Placement Algorithms for Content Delivery Networks. *IEEE Communications Surveys & Tutorials*, 19(2):1002–1026, 2017. → 424, 425, 426.

Salaht F. A., Desprez F., and Lebre A. An Overview of Service Placement Problem in Fog and Edge Computing. *ACM Computing Surveys*, 53(3), June 2020. → 104.

Saltzer J. and Kaashoek M. *Principles of Computer System Design, An Introduction*.

Morgan Kaufman, San Mateo, CA., 2009. → 79, 546, 547.

Saltzer J. and Schroeder M. The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9):1278–1308, Sept. 1975. → 549, 550.

Saltzer J., Reed D., and Clark D. End-to-End Arguments in System Design. *ACM Transactions on Computer Systems*, 2(4):277–288, Nov. 1984. → 271.

Sambra A. V., Mansour E., Hawke S., Zereba M., Greco N., Ghanem A., Zagidulin D., Aboulnaga A., and Berners-Lee T. Solid: A Platform for Decentralized Social Applications based on Linked Data. *MIT CSAIL & Qatar Computing Research Institute, Tech. Rep.*, 2016. → 553.

Santoro N. *Design and Analysis of Distributed Algorithms*. John Wiley, New York, 2007. → 249.

Saroiu S., Gummadi P. K., and Gribble S. D. Measuring and Analyzing the Characteristics of Napster and Gnutella Hosts. *ACM Multimedia Systems*, 9(2):170–184, Aug. 2003. → 96.

Saxena D., Raychoudhury V., Suri N., Becker C., and Cao J. Named data networking: A survey. *Computer Science Review*, 19:15–55, 2016. → 386.

Saxena P. and Rai J. A Survey of Permission-based Distributed Mutual Exclusion Algorithms. *Computer Standards and Interfaces*, 25(2):159–181, May 2003. → 272.

Schlosser M., Sintek M., Decker S., and Nejdl W. HyperCuP – Hypercubes, Ontologies, and Efficient Search on Peer-to-Peer Networks. In *1st International Workshop on Agents and Peer-to-Peer Computing*, volume 2530 of *Lecture Notes in Computer Science*, pages 112–124, Berlin, July 2002. Springer-Verlag. → 238.

Schmidt A. Implicit Human Computer Interaction Through Context. *Personal and Ubiquitous Computing*, 4(2-3):191–199, June 2000. → 45.

Schmidt C. and Parashar M. Squid: Enabling Search in DHT-based systems. *Journal of Parallel and Distributed Computing*, 68:962–975, 2008. → 383.

Schmidt D., Stal M., Rohnert H., and Buschmann F. *Pattern-Oriented Software Architecture – Patterns for Concurrent and Networked Objects*. John Wiley, New York, 2000. → 76.

Schneider F. Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial. *ACM Computing Surveys*, 22(4):299–320, Dec. 1990. → 264, 431.

Schulzrinne H., Casner S., Frederick R., and Jacobson V. RTP: A Transport Protocol for Real-Time Applications. RFC 3550, July 2003. → 187.

Schwarzkopf M., Kohler E., Frans Kaashoek M., and Morris R. Position: GDPR Compliance by Construction. In Gadepally V., Mattson T., Stonebraker M., Wang F., Luo G., Laing Y., and Dubovitskaya A., editors, *Heterogeneous Data Management, Polystores, and Analytics for Healthcare (DMAH 2019, Poly 2019)*, volume 11721 of *Lecture Notes in Computer Science*, pages 39–53, Berlin, 2019. Springer-Verlag. → 554.

Sebesta R. *Programming the World Wide Web*. Addison-Wesley, Reading, MA., 8th edition, 2015. → 87.

Sereno M. and Gaeta R. Generalized Probabilistic Flooding in Unstructured Peer-to-Peer Networks. *IEEE Transactions on Parallel and Distributed Systems*, 22(12):2055–2062, 2011. ISSN 1045-9219. → 238.

Servos D. and Osborn S. L. Current Research and Open Problems in Attribute-based Access Control. *ACM Computing Surveys*, 49(4):1–45, 2017. → 598, 601.

Seuken S. and Parkes D. C. Sybil-proof Accounting Mechanisms with Transitive Trust. In *International Joint Conference on Autonomous Agents and Multiagent Systems*, pages 205–212. ACM, 2014. → 590.

Shahrad M., Balkind J., and Wentzlaff D. Architectural Implications of Function-as-a-Service Computing. In *Annual IEEE/ACM International Symposium on Microarchitecture*, page 1063–1075, New York, NY, USA, 2019. Association for Computing Machinery. → 100.

Shapiro M., Preguiça N., Baquero C., and Zawirski M. Conflict-Free Replicated Data Types. In *13th International Conference Stabilization, Safety, and Security of Distributed Systems*, page 386–400, Berlin, 2011. Springer-Verlag. → 409.

Sharma P., Chaufournier L., Shenoy P., and Tay Y. Containers and Virtual Machines at Scale: A Comparative Study. In *Middleware 2016*, pages 1–13. ACM/IFIP/USENIX, Dec. 2016. → 138.

Shastri S., Banakar V., Wasserman M., Kumar A., and Chidambaram V. Understanding and Benchmarking the Impact of GDPR on Database Systems. *Proceedings of the VLDB Endowment*, 13(7):1064–1077, mar 2020. → 554, 555.

Shen X., Yu H., Buford J., and Akon M., editors. *Handbook of Peer-to-Peer Networking*. Springer-Verlag, Berlin, 2010. → 636, 642.

Sheth A. P. and Larson J. A. Federated Database Systems for Managing Distributed, Heterogeneous, and Autonomous Databases. *ACM Computing Surveys*, 22(3):183–236, Sept. 1990. → 427.

Shin M., Park M., Oh D., Kim B., and Lee J. Survey on the Clock Synchronization Schemes for Propagation Delay Measurement. *International Journal of Advanced Science and Technology*, 35:139–140, Oct. 2011. → 253.

Shoch J. Internetwork Naming, Addressing, and Routing. In *17th International Computer Conference*, pages 72–79, Los Alamitos, CA., 1978. IEEE, IEEE Computer Society Press. → 329.

Shooman M. L. *Reliability of Computer Systems and Networks: Fault Tolerance, Analysis, and Design*. John Wiley, New York, 2002. → 463.

Shriram A. and Kaur J. Empirical Evaluation of Techniques for Measuring Available Bandwidth. In *26th INFOCOM Conference*, pages 2162–2170, Los Alamitos, CA., 2007. IEEE, IEEE Computer Society Press. → 454.

Silberschatz A., Galvin P., and Gagne G. *Operating System Concepts*. John Wiley, New York, 10th edition, 2019. → 346.

Singhal M. and Shivaratri N. *Advanced Concepts in Operating Systems: Distributed, Database, and Multiprocessor Operating Systems*. McGraw-Hill, New York, 1994. → 537.

Sivasubramanian S., Pierre G., and van Steen M. Replicating Web Applications On-Demand. In *1st International Conference on Services Computing*, pages 227–236, Los Alamitos, CA., Sept. 2004a. IEEE, IEEE Computer Society Press. → 457.

Sivasubramanian S., Szymaniak M., Pierre G., and van Steen M. Replication for Web Hosting Systems. *ACM Computing Surveys*, 36(3):1–44, Sept. 2004b. → 453.

Sivasubramanian S., Pierre G., van Steen M., and Alonso G. Analysis of Caching and Replication Strategies for Web Applications. *IEEE Internet Computing*, 11(1):60–66, Jan. 2007. → 456.

Sivrikaya F. and Yener B. Time Synchronization in Sensor Networks: A Survey. *IEEE Network*, 18(4):45–50, July 2004. → 257.

Skeen D. Nonblocking Commit Protocols. In *SIGMOD International Conference on Management Of Data*, pages 133–142. ACM, 1981. → 534.

Skeen D. and Stonebraker M. A Formal Model of Crash Recovery in a Distributed System. *IEEE Transactions on Software Engineering*, SE-9(3):219–228, Mar. 1983. → 534.

Sletten B. *WebAssembly: The Definitive Guide*. O'Reilly & Associates, Sebastopol, CA., 2022. → 146.

Smart N. P. *Cryptography Made Simple*. Springer-Verlag, Berlin, 2016. → 557.

Smith J. and Nair R. The Architecture of Virtual Machines. *Computer*, 38(5):32–38, May 2005a. → 129, 130.

Smith J. and Nair R. *Virtual Machines: Versatile Platforms for Systems and Processes*. Morgan Kaufman, San Mateo, CA., 2005b. → 131, 132.

Smith M. and Howes T. Lightweight Directory Access Protocol (LDAP): String Representation of Search Filters. RFC 4515, June 2006. → 379.

Smith R. E. A Contemporary Look at Saltzer and Schroeder's 1975 Design Principles. *IEEE Security & Privacy*, 10(6):20–25, 2012. → 549, 550.

Soltesz S., Pötzl H., Fiuczynski M. E., Bavier A., and Peterson L. Container-Based Operating System Virtualization: A Scalable, High-Performance Alternative to Hypervisors. In *2nd EuroSys (European Conference on Computer Systems)*, pages 275–287, New York, NY, Mar. 2007. ACM, ACM Press. → 137.

Spector A. Performing Remote Operations Efficiently on a Local Computer Network. *Communications of the ACM*, 25(4):246–260, Apr. 1982. → 510.

Srinivasan S. *Kilim: A Server Framework with Lightweight Actors, Isolation Types and Zero-Copy Messaging*. Ph.d., University of Cambridge, Computer Laboratory, Feb. 2010. → 122.

Srisuresh P. and Holdrege M. IP Network Address Translator (NAT) Terminology and Considerations. RFC 2663, Aug. 1999. → 163.

Stallings W. *Crypotgraphy and Network Security*. Pearson Education, Englewood Cliffs, N.J., 7th edition, 2017. → 571.

Stanciu V., van Steen M., Peter A., and Dobre C. Privacy-Preserving Crowd Sensing and Analytics. In *17th International Conference Mobile and Ubiquitous Systems: Computing, Networking, and Services*, 12 2020. → 558.

Stankovic J. A. Research Directions for the Internet of Things. *IEEE Internet of Things Journal*, 1(1):3–9, Feb. 2014. → 44.

Stannat A., Ileri C. U., Gijswijt D., and Pouwelse J. Achieving Sybil-proofness in Distributed Work Systems. In *International Joint Conference on Autonomous Agents and Multiagent Systems*, pages 1263–1271, 2021. → 591.

Steiner J., Neuman C., and Schiller J. Kerberos: An Authentication Service for Open Network Systems. In *Winter Technical Conference*, pages 191–202. USENIX, 1988.

→ 581.

Steinmetz R. and Nahrstedt K. *Multimedia Systems*. Springer-Verlag, Berlin, 2004. → 162.

Stevens W. *TCP/IP Illustrated, Volume 1: The Protocols*. Addison-Wesley, Reading, MA., 1994. → 59.

Stevens W. *UNIX Network Programming – Networking APIs: Sockets and XTI*. Prentice Hall, Englewood Cliffs, N.J., 2nd edition, 1998. → 123, 210.

Stevens W. *UNIX Network Programming – Interprocess Communication*. Prentice Hall, Englewood Cliffs, N.J., 2nd edition, 1999. → 113, 122.

Stevens W. and Rago S. *Advanced Programming in the UNIX Environment*. Addison-Wesley, Reading, MA., 2nd edition, 2005. → 115.

Stewart R. Stream Control Transmission Protocol. RFC 4960, Sept. 2007. → 188.

Stocker V., Smaragdakis G., Lehr W., and Bauer S. The Growing Complexity of Content Delivery Networks: Challenges and Implications for the Internet Ecosystem. *Telecommunications Policy*, 41(10):1003–1016, 2017. → 165.

Stoica I., Morris R., Liben-Nowell D., Karger D. R., Kaashoek M. F., Dabek F., and Balakrishnan H. Chord: A Scalable Peer-to-peer Lookup Protocol for Internet Applications. *IEEE/ACM Transactions on Networking*, 11(1):17–32, Feb. 2003. → 91, 333, 336.

Stratan C., Sacha J., Napper J., Costa P., and Pierre G. The XtreemOS Resource Selection Service. *ACM Transactions of Autonomous and Adaptive Systems*, 7(4), Dec. 2012. → 385.

Strauss J., Katabi D., and Kaashoek F. A Measurement Study of Available Bandwidth Estimation Tools. In *3rd Internet Measurement Conference*, pages 39–44, New York, NY, 2003. ACM Press. → 454.

Su A.-J., Choffnes D. R., Kuzmanovic A., and Bustamante F. E. Drafting Behind Akamai (Travelocity-Based Detouring). In *SIGCOMM*, page 435–446, 2006. → 166.

Sugerman J., Venkitachalam G., and Lim B.-H. Virtualizing I/O Devices on VMware Workstations Hosted Virtual Machine Monitor. In *USENIX Annual Technical Conference*, pages 1–14, Berkeley, CA, June 2001. USENIX, USENIX. → 132.

Sundararaman B., Buy U., and Kshemkalyani A. D. Clock Synchronization for Wireless Sensor Networks: A Survey. *Ad-Hoc Networks*, 3(3):281–323, May 2005. → 257.

Swamidass S. J. and Baldi P. Mathematical Correction for Fingerprint Similarity Measures to Improve Chemical Retrieval. *Journal of Chemical Information and Modeling*, 47(3):952–964, 2007. → 559.

Szymaniak M., Pierre G., and van Steen M. Scalable Cooperative Latency Estimation. In *10th International Conference on Parallel and Distributed Systems*, pages 367–376, Los Alamitos, CA., July 2004. IEEE, IEEE Computer Society Press. → 321.

Szymaniak M., Presotto D., Pierre G., and van Steen M. Practical Large-Scale Latency Estimation. *Computer Networks*, 52(7):1343–1364, May 2008. → 321.

Taiani F., Fabre J.-C., and Killijian M.-O. A Multi-Level Meta-Object Protocol for Fault-Tolerance in Complex Architectures. In *International Conference on Dependable Systems and Networks*, pages 270–279, Los Alamitos, CA., June 2005. IEEE Computer Society Press. → 436.

Taleb T., Samdanis K., Mada B., Flinck H., Dutta S., and Sabella D. On Multi-Access Edge Computing: A Survey of the Emerging 5G Network Edge Cloud Architecture and Orchestration. *IEEE Communications Surveys & Tutorials*, 19(3):1657–1681, 2017. → 103.

Tan S.-W., Waters G., and Crawford J. A Survey and Performance Evaluation of Scalable Tree-based Application Layer Multicast Protocols. Technical Report 9-03, University of Kent, UK, July 2003. → 235.

Tanenbaum A. and Bos H. *Modern Operating Systems*. Prentice Hall, Upper Saddle River, N.J., 5th edition, 2022. → 346.

Tanenbaum A., , Feamster N., and Wetherall D. *Computer Networks*. Prentice Hall, Upper Saddle River, N.J., 6th edition, 2021. → 183, 508.

Tanisch P. Atomic Commit in Concurrent Computing. *IEEE Concurrency*, 8(4):34–41, Oct. 2000. → 528.

Tarkoma S. *Overlay Networks, Toward Information Networking*. CRC Press, Boca Raton, FL, 2010. → 89.

Tarkoma S. and Kangasharju J. *Mobile Middleware: Supporting Applications and Services*. John Wiley, New York, 2009. → 47.

Tartalja I. and Milutinovic V. Classifying Software-Based Cache Coherence Solutions. *IEEE Software*, 14(3):90–101, May 1997. → 443.

Tel G. *Introduction to Distributed Algorithms*. Cambridge University Press, Cambridge, UK, 2nd edition, 2000. → 249, 283.

Terry D. *Replicated Data Management for Mobile Computing*. Synthesis Lectures on Data Management. Morgan and Claypool, San Rafael, CA, 2008. → 416.

Terry D., Demers A., Petersen K., Spreitzer M., Theimer M., and Welsh B. Session Guarantees for Weakly Consistent Replicated Data. In *3rd International Conference on Parallel and Distributed Information Systems*, pages 140–149, Los Alamitos, CA., Sept. 1994. IEEE, IEEE Computer Society Press. → 416, 420, 421.

Terry D., Petersen K., Spreitzer M., and Theimer M. The Case for Non-transparent Replication: Examples from Bayou. *IEEE Data Engineering*, 21(4):12–20, Dec. 1998. → 416.

Thomas R. A Majority Consensus Approach to Concurrency Control for Multiple Copy Databases. *ACM Transactions on Database Systems*, 4(2):180–209, June 1979. → 441.

TIBCO . *TIBCO Rendezvous Concepts, Release 8.3.0*. TIBCO Software Inc., Palo Alto, CA, July 2010. → 308.

Tourani R., Misra S., Mick T., and Panwar G. Security, Privacy, and Access Control in Information-Centric Networking: A Survey. *IEEE Communications Surveys & Tutorials*, 20(1):566–600, 2018. → 389.

Towsley D., Kurose J., and Pingali S. A Comparison of Sender-Initiated and Receiver-Initiated Reliable Multicast Protocols. *IEEE Journal on Selected Areas in Communication*, 15(3):398–407, Apr. 1997. → 518.

Trivedi K. *Probability and Statistics with Reliability, Queuing and Computer Science Applications*. John Wiley, New York, 2nd edition, 2002. → 26.

Tsafrir D. The Context-Switch Overhead Inflicted by Hardware Interrupts (and the

Enigma of Do-nothing Loops). In *2007 Workshop on Experimental Computer Science*, New York, NY, 2007. ACM Press. → 116, 117.

Tsui A. W., Lin W.-C., Chen W.-J., Huang P., and Chu H.-H. Accuracy Performance Analysis between War Driving and War Walking in Metropolitan Wi-Fi Localization. *IEEE Transations on Mobile Computing*, 9(11):1551–1562, 2010. → 318.

Turek J. and Shasha S. The Many Faces of Consensus in Distributed Systems. *Computer*, 25(6):8–17, June 1992. → 503, 504.

Ulrich A., Holz R., Hauck P., and Carle G. Investigating the OpenPGP Web of Trust. In *European Symposium on Research in Computer Security*, pages 489–507. Springer, 2011. → 571.

Umar A. *Object-Oriented Client/Server Internet Environments*. Prentice Hall, Upper Saddle River, N.J., 1997. → 81.

UPnP Forum . UPnP Device Architecture Version 1.1, Oct. 2008. → 46.

Ur B., Bees J., Segreti S. M., Bauer L., Christin N., and Cranor L. F. Do Users' Perceptions of Password Security Match Reality? In *2016CHI Conference on Human Factors in Computing Systems*, pages 3748–3760. ACM, 2016. → 572.

Urdaneta G., Pierre G., and van Steen M. A Survey of DHT Security Techniques. *ACM Computing Surveys*, 43(2), June 2011. → 344, 588.

Uzunov A. V. A Survey of Security Solutions for Distributed Publish/Subscribe Systems. *Computers & Security*, 61:94–129, 2016. → 315.

van der Toorn O., Müller M., Dickinson S., Hesselman C., Sperotto A., and van Rijswijk-Deij R. Addressing the Challenges of Modern DNS: A Comprehensive Tutorial. *Computer Science Review*, 45:100469, 2022. → 359, 364, 365.

van Ede T., Aghakhani H., Spahn N., Bortolameotti R., Cova M., Continella A., van Steen M., Peter A., Kruegel C., and Vigna G. DeepCASE: Semi-Supervised Contextual Analysis of Security Events. In *International Symposium on Security and Privacy*. IEEE, 2022. → 612.

van Renesse R. and Altinbuken D. Paxos Made Moderately Complex. *ACM Computing Surveys*, 47(3):42:1–42:36, Feb. 2015. → 479.

van Rijn V. and Rellermeyer J. S. A Fresh Look at the Architecture and Performance of Contemporary Isolation Platforms. In *Middleware 2021*, pages 323–336. ACM/IFIP/USENIX, Dec. 2021. → 139.

van Steen M. and Ballintijn G. Achieving Scalability in Hierarchical Location Services. In *26th International Computer Software and Applications Conference*, pages 899–905, Los Alamitos, CA., Aug. 2002. IEEE, IEEE Computer Society Press. → 342.

van Steen M., Hauck F., Homburg P., and Tanenbaum A. Locating Objects in Wide-Area Systems. *IEEE Communications Magazine*, 36(1):104–109, Jan. 1998. → 338.

Vaquero L. M., Rodero-Merino L., Caceres J., and Lindner M. A Break in the Clouds: Towards a Cloud Definition. *SIGCOMM Computer Communications Review*, 39(1): 50–55, Dec. 2008. → 98.

Vasilomanolakis E., Karuppayah S., Mühlhäuser M., and Fischer M. Taxonomy and Survey of Collaborative Intrusion Detection. *ACM Computing Surveys*, 47(4):1–33, 2015. → 612.

Vasudevan S., Kurose J. F., and Towsley D. F. Design and Analysis of a Leader Election Algorithm for Mobile Ad Hoc Networks. In *12th International Conference on Network Protocols*, pages 350–360, Los Alamitos, CA., Oct. 2004. IEEE, IEEE Computer Society Press. → 294, 297.

Velazquez M. A Survey of Distributed Mutual Exclusion Algorithms. Technical Report CS-93-116, University of Colorado at Boulder, Sept. 1993. → 272.

Viotti P. and Vukolic M. Consistency in Nontransactional Distributed Storage Systems. *ACM Computing Surveys*, 29(1), June 2016. → 395.

Vixie P. What DNS Is Not. *Communications of the ACM*, 52(12):43–47, Dec. 2009. → 369.

Vixie P. Rate-Limiting State. *Communications of the ACM*, 57(4):40–43, Apr. 2014. → 369.

Vogels W. Tracking Service Availability in Long Running Business Activities. In *1st International Conference on Service Oriented Computing*, volume 2910 of *Lecture Notes in Computer Science*, pages 395–408, Berlin, Dec. 2003. Springer-Verlag. → 507.

Vogels W. Eventually consistent. *Communications of the ACM*, 52(1):40–44, Jan. 2009. → 407.

Voigt P. and Bussche A.Von dem . *The EU General Data Protection Regulation (GDPR), A Practical Guide*. Springer-Verlag, Berlin, 2017. → 553.

Voorsluys W., Broberg J., Venugopal S., and Buyya R. Cost of Virtual Machine Live Migration in Clouds: A Performance Evaluation. In *1st International Conference on Cloud Computing*, volume 5931 of *Lecture Notes in Computer Science*, pages 254–265, Berlin, Dec. 2009. Springer-Verlag. → 176, 177.

Voronkov A., Iwaya L. H., Martucci L. A., and Lindskog S. Systematic Literature Review on Usability of Firewall Configuration. *ACM Computing Surveys*, 50(6):1–35, 2017. → 610.

Voulgaris S. and van Steen M. VICINITY: A Pinch of Randomness Brings out the Structure. In *Middleware 2013*, volume 8275 of *Lecture Notes in Computer Science*, pages 21–40, Berlin, Dec. 2013. ACM/IFIP/USENIX, Springer-Verlag. → 301.

Voulgaris S., Gavidia D., and van Steen. M. CYCLON: Inexpensive Membership Management for Unstructured P2P Overlays. *Journal of Network and Systems Management*, 13(2):197–217, June 2005. → 299, 304.

Voulgaris S., Rivière E., Kermarrec A.-M., and van Steen M. Sub-2-Sub: Self-Organizing Content-Based Publish and Subscribe for Dynamic and Large Scale Collaborative Networks. In *5th International Workshop on Peer-to-Peer Systems*, Feb. 2006. → 311.

Vu Q., Lupu M., and Ooi B. *Peer-to-Peer Computing, Principles and Applications*. Springer-Verlag, Berlin, 2010. → 89.

Vukolić M. *Quorum Systems: With Applications to Storage and Consensus*, volume 3 of *Synthesis Lectures on Distributed Computing Theory*. Morgan & Claypool Publishers, 2012. → 443.

Waldo J., Wyant G., Wollrath A., and Kendall S. A Note on Distributed Computing. In *2nd Workshop on Mobile Object Systems*, volume 1222 of *Lecture Notes in Computer Science*, pages 1–10, Berlin, July 1997. Springer-Verlag. → 15.

Walfish M., Balakrishnan H., , and Shenker S. Untangling the Web from DNS. In *1st Symposium on Networked Systems Design and Implementation*, pages 225–238, Berkeley,

CA, Mar. 2004. USENIX, USENIX. → 368.

Wams J. *Unified Messaging and Micro-Objects*. PhD thesis, VU University Amsterdam, 2012. → 15.

Welsh M. and Mainland G. Programming Sensor Networks Using Abstract Regions. In *1st Symposium on Networked Systems Design and Implementation*, Berkeley, CA, Mar. 2004. USENIX, USENIX. → 49.

Wendell P. and Freedman M. J. Going Viral: Flash Crowds in an Open CDN. In *11th Internet Measurement Conference*, pages 549–558, New York, NY, 2011. ACM Press. → 452.

Werbach K. *The Blockchain and the New Architecture of Trust*. MIT Press, Cambridge, MA., 2018. → 592.

Wessels D. *Squid: The Definitive Guide*. O'Reilly & Associates, Sebastopol, CA., 2004. → 452.

Wieringa R. and Jonge W.de . Object Identifiers, Keys, and Surrogates–Object Identifiers Revisited. *Theory and Practice of Object Systems*, 1(2):101–114, 1995. → 328.

Wierzbicki A. *Trust and Fairness in Open, Distributed Systems*, volume 298 of *Studies in Computational Intelligence*. Springer-Verlag, Berlin, 2010. → 586.

Wiesmann M., Pedone F., Schiper A., Kemme B., and Alonso G. Understanding Replication in Databases and Distributed Systems. In *20th International Conference on Distributed Computing Systems*, pages 264–274, Taipei, Taiwan, Apr. 2000. IEEE. → 395.

Wolff E. *Micrservices: Flexible Software Architecture*. Addison-Wesley, Reading, MA., 2017. → 65.

Wollrath A., Riggs R., and Waldo J. A Distributed Object Model for the Java System. *Computing Systems*, 9(4):265–290, Fall 1996. → 201.

Wolman A., Voelker G., Sharma N., Cardwell N., Karlin A., and Levy H. On the Scale and Performance of Cooperative Web Proxy Caching. In *17th Symposium on Operating System Principles*, pages 16–31, Kiawah Island, SC, Dec. 1999. ACM. → 451.

Wool A. Trends in Firewall Configuration Errors: Measuring the Holes in Swiss Cheese. *IEEE Internet Computing*, 14(4):58–65, 2010. → 610.

Wright G. and Stevens W. *TCP/IP Illustrated, Volume 2: The Implementation*. Addison-Wesley, Reading, MA., 1995. → 59.

Xiao Y., Zhang N., Lou W., and Hou Y. T. A Survey of Distributed Consensus Protocols for Blockchain Networks. *IEEE Communications Surveys & Tutorials*, 22(2):1432–1465, 2020. → 502, 503.

Xu X., Weber I., Staples M., Zhu L., Bosch J., Bass L., Pautasso C., and Rimba P. A Taxonomy of Blockchain-based Systems for Architecture Design. In *International Conference on Software Architecture (ICSA)*, pages 243–252. IEEE, Apr. 2017. → 108.

Yang B. and Garcia-Molina H. Designing a Super-Peer Network. In *19th International Conference on Data Engineering*, pages 49–60, Los Alamitos, CA., Mar. 2003. IEEE, IEEE Computer Society Press. → 95.

Yang K., Zhang K., Jia X., Hasan M. A., and Shen X. Privacy-preserving Attribute-Keyword Based Data Publish-Subscribe Service on Cloud Platforms. *Information*

Sciences, 387:116–131, 2017. → 314, 315.

Yang M., Zhang Z., Li X., and Dai Y. An Empirical Study of Free-Riding Behavior in the Maze P2P File-Sharing System. In *4th International Workshop on Peer-to-Peer Systems*, Lecture Notes in Computer Science, Berlin, Feb. 2005. Springer-Verlag. → 96.

Yellin D. Competitive Algorithms for the Dynamic Selection of Component Implementations. *IBM Systems Journal*, 42(1):85–97, Jan. 2003. → 78.

Yin M., Malkhi D., Reiter M. K., Gueta G. G., and Abraham I. Hotstuff: BFT Consensus with Linearity and Responsiveness. In *Symposium on Principles of Distributed Computing*, pages 347–356, 2019. → 502, 503.

Yousefpour A., Fung C., Nguyen T., Kadiyala K., Jalali F., Niakanlahiji A., Kong J., and Jue J. P. All One Needs to Know About Fog Computing and Related Edge Computing Paradigms: A Complete Survey. *Journal of Systems Architecture*, 98: 289–330, 2019. → 52, 100.

Yu H. and Vahdat A. Efficient Numerical Error Bounding for Replicated Network Services. In Abbadi A. E., Brodie M. L., Chakravarthy S., Dayal U., Kamel N., Schlageter G., and Whang K.-Y., editors, *26th International Conference on Very Large Data Bases*, pages 123–133, San Mateo, CA., Sept. 2000. Morgan Kaufman. → 446.

Yu H. and Vahdat A. Design and Evaluation of a Conit-Based Continuous Consistency Model for Replicated Services. *ACM Transactions on Computer Systems*, 20(3):239–282, 2002. → 410, 411, 455.

Zarrin J., Aguiar R. L., and Barraca J. P. Resource Discovery for Distributed Computing Systems: A Comprehensive Survey. *Journal of Parallel and Distributed Computing*, 113: 127–166, 2018. → 385.

ZeroC . *Distributed Programming with Ice*. ZeroC Inc., Brisbane, Australia, 2022. → 158, 159.

Zhang C., Xie Y., Bai H., Yu B., Li W., and Gao Y. A Survey on Federated Learning. *Knowledge-Based Systems*, 216:106775, 2021a. → 169.

Zhang F., Liu G., Fu X., and Yahyapour R. A Survey on Virtual Machine Migration: Challenges, Techniques, and Open Issues. *IEEE Communications Surveys & Tutorials*, 20(2):1206–1243, 2018. → 175, 176.

Zhang J., Ma M., Wang P., and Sun X.dong . Middleware for the Internet of Things: A Survey on Requirements, Enabling Technologies, and Solutions. *Journal of Systems Architecture*, 117, 2021b. → 49.

Zhang Q., Cheng L., and Boutaba R. Cloud Computing: State of the Art and Research Challenges. *Journal of Internet Services and Applications*, 1(1):7–18, May 2010. → 99.

Zhang Y., Xia Z., Afanasyev A., and Zhang L. A Note on Routing Scalability in Named Data Networking. In *International Conference Communications Workshops (ICC Workshops)*, pages 1–6, 2019. → 387.

Zhao F. and Guibas L. *Wireless Sensor Networks*. Morgan Kaufman, San Mateo, CA., 2004. → 49.

Zhu Z. and Afanasyev A. Let's ChronoSync: Decentralized Dataset State Synchronization in Named Data Networking. In *21st International Conference on Network Protocols*, pages 1–10, 2013. → 386.

Zhuang S. Q., Geels D., Stoica I., and Katz R. H. On Failure Detection Algorithms in Overlay Networks. In *24th INFOCOM Conference*, Los Alamitos, CA., Mar. 2005. IEEE, IEEE Computer Society Press. → 507.

Zogg J.-M. GPS Basics. Technical Report GPS-X-02007, UBlox, Mar. 2002. → 317.

Zolfaghari B., Srivastava G., Roy S., Nemati H. R., Afghah F., Koshiba T., Razi A., Bibak K., Mitra P., and Rai B. K. Content Delivery Networks: State of the Art, Trends, and Future Roadmap. *ACM Computing Surveys*, 53(2), 2021. → 165.

Zwicky E., Cooper S., Chapman D., and Russell D. *Building Internet Firewalls*. O'Reilly & Associates, Sebastopol, CA., 2nd edition, 2000. → 609.

GLOSSARY

BitTorrent: A distributed system for file sharing by which a file is partitioned into blocks and subsequently physically distributed among participants. A participant needs to copy and swap blocks with others (called tit-for-tat) to eventually gather all blocks to reconstruct the original file.

Blockchain: An append-only list of blocks of validated transactions. Each transaction and block can be replicated across all processes participating in the blockchain as they cannot be changed once published. The list is secured in the sense that *any* change to *any* transaction or block cannot go unnoticed.

Confidentiality: The property that information is disclosed only to authorized parties.

Consistency model (data centric): Describes what processes can expect concerning read and write operations from and to a logically shared data store that is physically distributed among the processes. In essence, a consistency model tells if and when local write operations are propagated to other processes, and the effects for local read operations.

Container: A special type of virtual machine, generally tailored to support only a specific operating system. A container provides an environment to an application suite, essentially mimicking the situation that this suite is being executed in isolation on a single machine. Where a virtual machine can host an operating system, a container can host only a suite of applications.

Critical region: A series of instructions to be executed by a process, which requires mutually exclusive access to specified resources.

Decentralized system: A networked computer system in which processes and resources are *necessarily* spread across multiple computers, usually caused by business constraints, lack of mutual trust, or geographical restrictions.

Dependable system: A system that provides availability, reliability, safety, maintainability, confidentiality, and integrity.

Distributed system: A networked computer system in which processes and resources are *sufficiently* spread across multiple computers, usually to meet performance and dependability requirements.

Distribution transparency: The phenomenon by which a distributed system attempts to hide the fact that its processes and resources are physically distributed across multiple computers, possibly separated by large distances.

Event matching: Typically occurs in publish-subscribe systems, where the publication of an event needs to be matched with all relevant subscriptions. A major problem is to support matching in a scalable manner.

Extensibility: Characterizes the extent to which a system can be extended (often without interrupting operations) with new functionality or components while avoiding affecting parts that are independent of those extensions.

Failure, error, fault: A system *fails* when it does not meet its specifications. A failure happens due to some *error*, such as a programming bug. The cause of an error is called a *fault*.

Fault tolerance: A system is fault tolerant when it can continue to provide its services according to specifications despite the presence of faults that influence its design, implementation, and execution.

Faults, errors, failures: A system is said to fail when it cannot meet its specifications. An error is a mistake that may need lead to a failure, such as a programming bug. A fault is the cause of an error.

Groupware: Distributed software to allow multiple users to collaborate from different locations through shared whiteboards, shared documents, etc.

Honest-but-curious: A server that behaves according to some protocol, but in the worst case keeps track of all the things it does.

Integrity: Ensures that alterations to the various assets of a system can be made only in an authorized way.

Interface Definition Language: A formal language for specifying the interfaces to various components of a distributed system. An IDL is used to generate code for different programming langauges that can subsequently be used to build applications making use of a component.

Interoperability: Characterizes the extent by which two implementations of systems or components from different manufacturers or development teams can co-exist and work together by merely relying on each other's services as specified by their respective interfaces.

Leader-election algorithm: A distributed algorithm that is executed among a group of processes such that, in the end, one of these processes can be designated as the leader of the group.

Logical clock: A system in which a group of processes keeps account of internal events, and also when messages are sent and received, to globally determine a consistent ordering of events.

Middleware: Software that constitutes a distributed system, implementing a myriad of mechanisms independent of any underlying system, as well as generally applicable for a wide range of applications.

Multiparty computation: The means for two or three parties to compute a value for which the data of those parties is needed, but without the need to actually share that data.

Open distributed system: A system that offers components that can easily be used by, or integrated into other systems. An open distributed system itself will often consist of components that originate from elsewhere.

Partial failure: A type of failure characteristic for a distributed system. Some process or resource, is not operating according to expectations, which, in turn, may have effects on other parts of the system. However, the system as a whole will continue to operate, albeit perhaps in unexpected ways.

Peer-sampling service: A service that operates in a potentially large distributed system, returning a (seemingly) randomly chosen peer from all available peers.

Perspectives on distributed systems: Due to the fact that a distributed system must meet so many different functional and nonfunctional requirements, which, in general, are mututal dependent, taking different perspectives on distributed systems allows for a more focused study. We

distinguish perspectives on architecture, processes, communication, coordination, naming, consistency and replication, fault tolerance, and security.

Pervasive system: Systems consisting of a myriad of devices that are intended to blend into our environment naturally. Typical devices include smartphones, smart watches, specific sensors and actuators, camera's, and so forth.

Portability: Characterizes to what extent an application developed for a distributed system A can be executed, without modification, on a different distributed system B that implements the same interfaces as A.

Remote procedure call: A communication mechanism that essentially mimicks local procedure calls, yet where the execution of the call takes place at a remote server. In general, jsut as with local procedure calls, the caller waits until the procedure has been carried out fully.

Scalability: Refers to the extent that a system can scale in terms of its size, in terms of how geographically dispersed its components can be without seriously negatively affecting performance, or the extent to which a system can span multiple administrative organizations.

Scaling up or out: Scaling up is the process by which a machine is equipped with more and often more powerful resources so that it can better accommodate performance-demanding applications. Scaling out is all about extending a networked computer system with more computers and subsequently distributing workloads across the extended set of computers.

Separating policy from mechanism: Where mechanisms in a distributed system facilitate basic functionalities, such as storage, communication, processes, and so on, policies describe *how* those facilities are used. In general, separating facilities from the way how they are used is a good design principle, yet may overly complicate the configuration of a system.

Software architecture: The organization of distributed systems in terms of software components and their interaction.

System architecture: The actual realization of a distributed system requires that software components are instantiated and placed on real machines. A system architecture is the final instantiation of a software architecture.

Transaction processing monitor: A TP monitor is one of the first general-purpose mechanisms built into middleware. It is service that follows a standard protocol for executing a number of subtransactions, such that (1) each subtransaction adheres to the ACID properties, but (2) also the collection of subtransactions meets the ACID requirements.

Trusted Computing Base: The set of all security mechanisms in a (distributed) computer system that are necessary and sufficient to enforce a security policy.

Vector clocks: A system implementing logical clocks such that the logical time at which an event took place can be used to conclude that an event indeed took place before another event, thus capturing a potential causality between events.

Virtual machine: In general, a system providing its own interface to operating systems and applications, and implementing that interface for a specific instruction set or operating system. As a consequence, migrating or porting code across different architectures or operating systems becomes easier as long as the target environment provides the same interface the virtual machine is offering.

Virtual processor: A counterpart of a physical processor developed in software, providing a context in which the set of instructions of the physical processor are executed. A thread provides a minimal context for concurrent execution of instructions. An (operating system) process, in addition provides much stronger isolation guarantees between processes. For example, a process has its own, protected address space.